Les Typologies de Deep Learning

Pr. Abdelalim SADIQ

Faculté des Sciences, Université Ibn Tofail
a.sadiq@uit.ac.ma

2024 - 2025

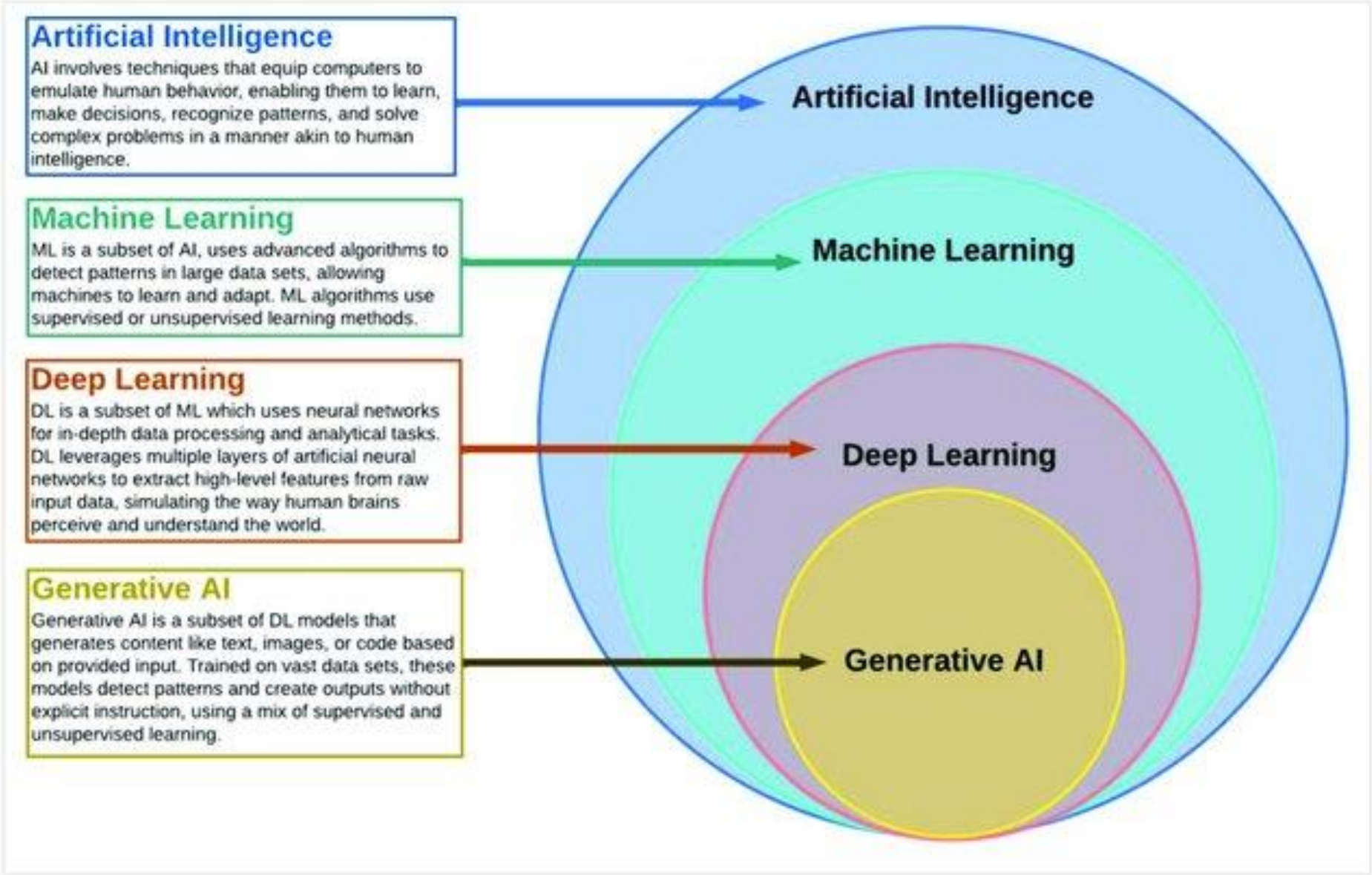
Plan de la présentation

- Introduction
- Le perceptron , Le Perceptron multicouche
- Le Réseaux de neurones convolutifs (CNN)
- Les Réseaux de neurones récurrents (RNN) et Mémoire à court terme et mémoire à long terme (LSTM)
- Unité récurrente fermée (Gated recurrent units (GRU)
- Réseau Antagoniste Génératif (Generative Adversarial Network (GAN))
- Autoencodeurs et réduction de dimension
- Transformers et mécanismes attention

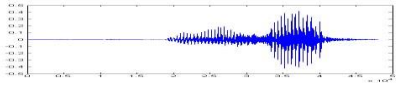
Mode d'évaluation

- La présence
- TP à rendre chaque semaine sur Classroom.
  - Implémentation des algorithmes from scratch.
  - Code utilisant les bibliothèques TensorFlow, Keras, etc.
- Projet sur une thématique spécifique.
- Examen final.

Introduction (1)



Introduction (2)

- **L'intelligence artificielle** est un domaine scientifique concerné par le développement d'algorithmes permettant aux ordinateurs d'apprendre sans être explicitement programmés.
- **Le machine learning** est un domaine de l'intelligence artificielle, qui consiste à programmer une machine pour que celle ci apprenne à réaliser des tâches en étudiant des exemples de ces dernières.
- Ces Exemples sont représentés par des données que la machine utilise pour développer un modèle, par exemple une fonction du type : $f\ (x)\ =\ ax\ +\ b$ (cas d'un modèle linéaire)
- Le machine learning consiste essentiellement à trouver cette fonction f :
  - Speech Recognition $f($  $) =$ "كيف حالك"
  - Image Recognition $f($  $) =$ "Chat"
  - Text Translation $f($ "How are you" $) =$ "كيف حالك"

Introduction (3)

- Image recognition

$f($  $) =$ “Chat”

A set of function

Model

$f_1, f_2 \cdots$

$f_1($  $) =$ “Chat”

$f_2($  $) =$ “Singe”

$f_1($  $) =$ “Chien”

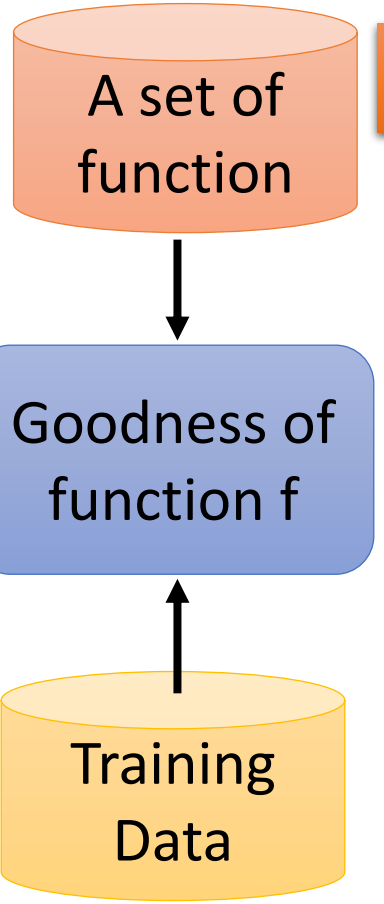
$f_2($  $) =$ “Tigre”

Slide from : Hung-yi Lee, Deep Learning Tutorial

Introduction (4)

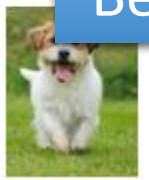
- Image recognition

$f($  $) =$ "Chat"



$f_1($  $) =$ "Chat" $f_2($  $) =$ "Singe"

Better!

$f_1($  $) =$ "Chien" $f_2($  $) =$ "Tigre"

Supervised Learning

function input:   

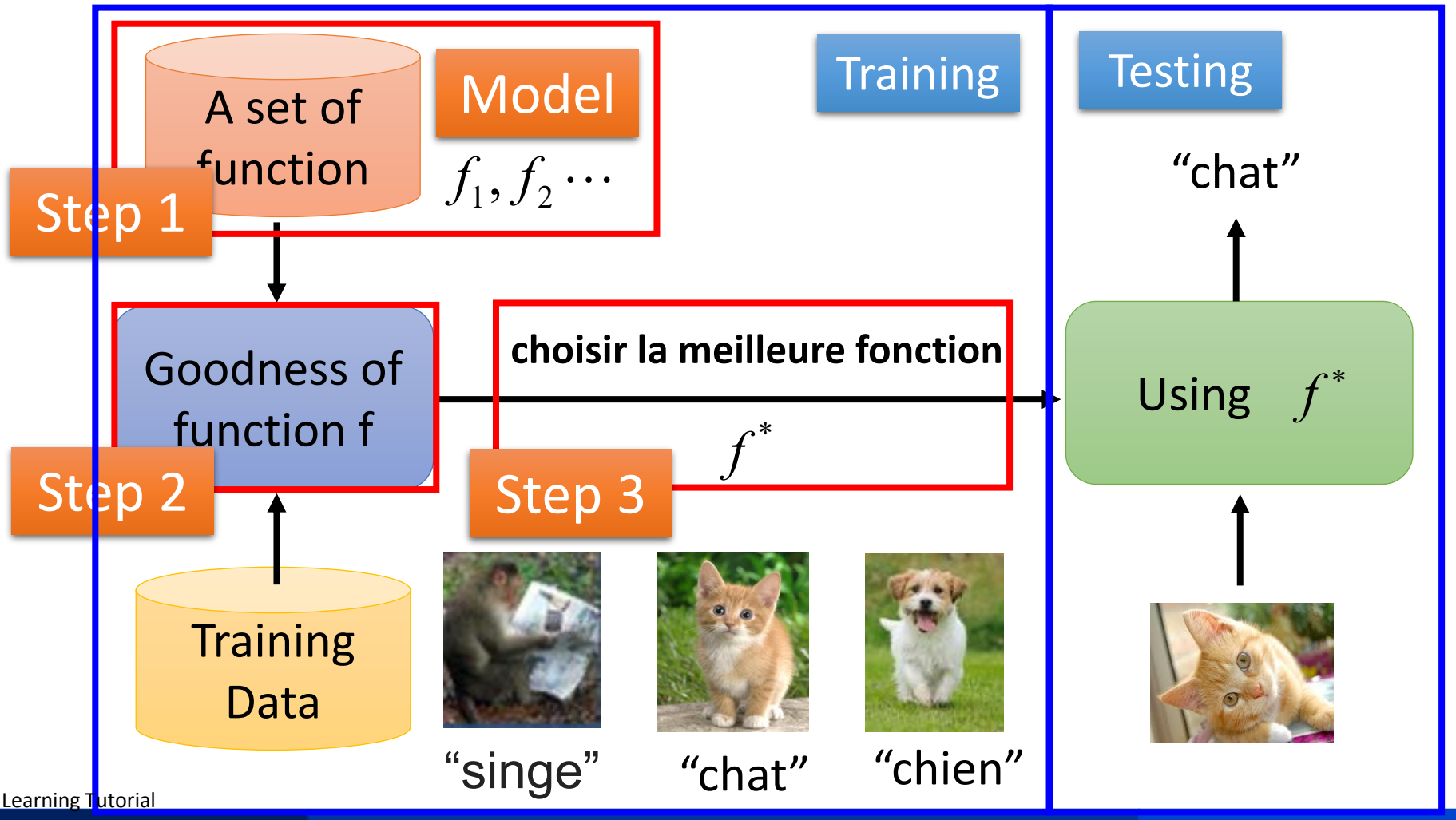
function output: "Singe" "chat" "Chien"

Slide from : Hung-yi Lee, Deep Learning Tutorial

Machine Learning (4)

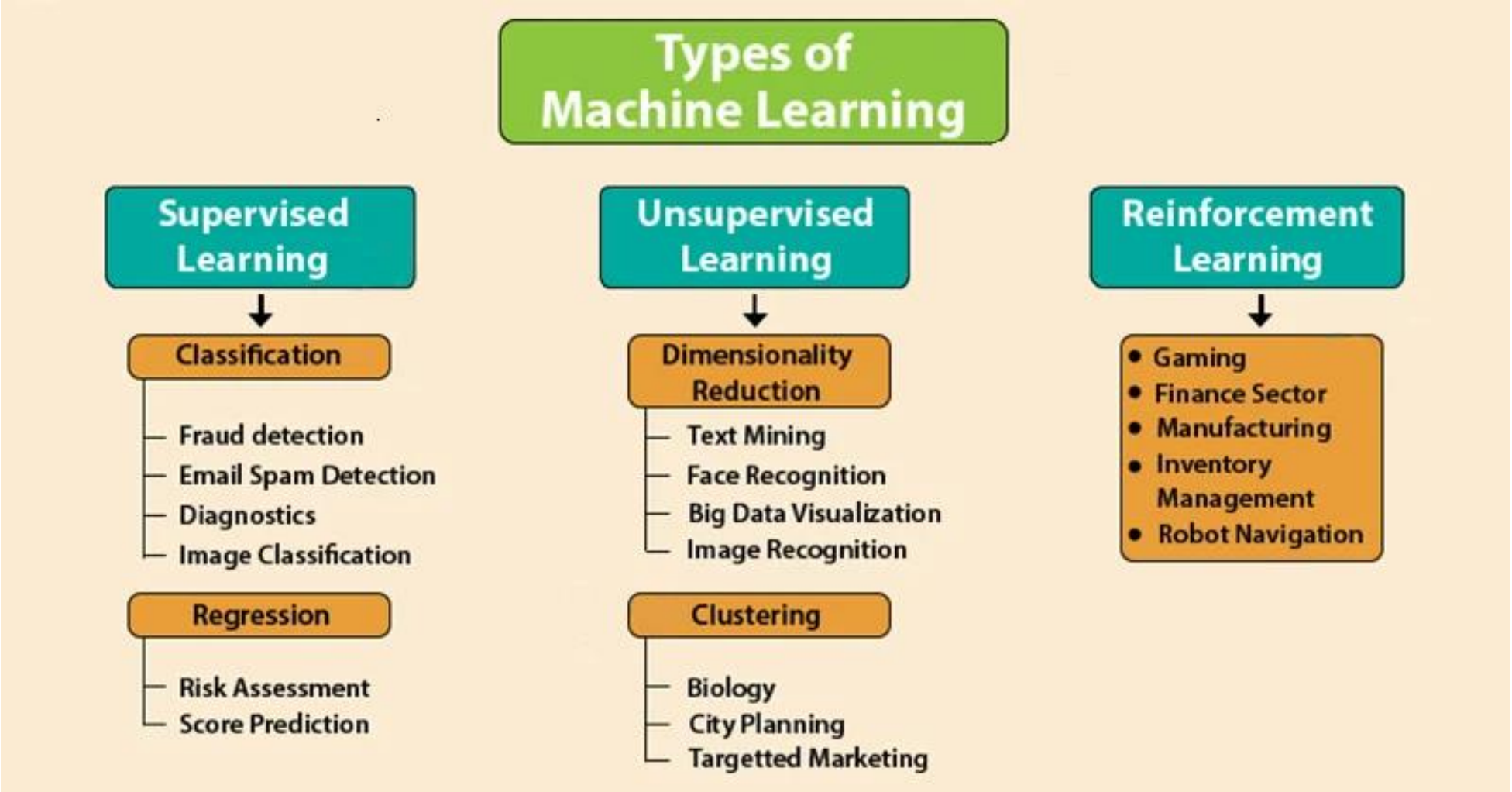
- Image recognition

$f($  $) =$ “Chat”



Slide from : Hung-yi Lee, Deep Learning Tutorial

Rappel : Machine Learning (1)



Rappel : Machine Learning (2)

- Machine learning supervisé peut se subdiviser en deux types :
  - **Classification :** la variable de sortie est une catégorie.
  - **Régression :** la variable de sortie est une valeur spécifique.
- Machine learning non supervisé
  - **Clustering** : l'objectif consiste à trouver des regroupements dans les données.
  - **Association** : l'objectif consiste à identifier les règles qui permettront de définir de grands groupes de données.
- Machine learning par renforcement
  - **Monte Carlo** : le programme reçoit ses récompenses à la fin de l'état « terminal ».
  - **Machine learning par différence temporelle (TD)** : les récompenses sont évaluées et accordées à chaque étape.

Apprentissage Supervisé (1)

- **L'apprentissage supervisé** consiste à fournir à une machine un grand nombre d'exemples pour qu'elle puisse les analyser et apprendre à partir d'eux. Pour maîtriser cette approche, il est essentiel de bien comprendre les quatre notions suivantes :

  - **Jeu de Données (Data Set)**

    Ensemble d'exemples utilisés pour entraîner le modèle.

  - **Le Modèle et ses Paramètres**

    Structure algorithmique qui apprend à partir des données en ajustant ses paramètres.

  - **Fonction de Coût**

    Mesure de l'erreur entre les prédictions du modèle et les résultats attendus, utilisée pour optimiser les paramètres.

  - **Algorithme d'Apprentissage**

    Procédure utilisée pour minimiser la fonction de coût en ajustant les paramètres du modèle.

Apprentissage Supervisé (2)

Jeu de Données (Dataset)

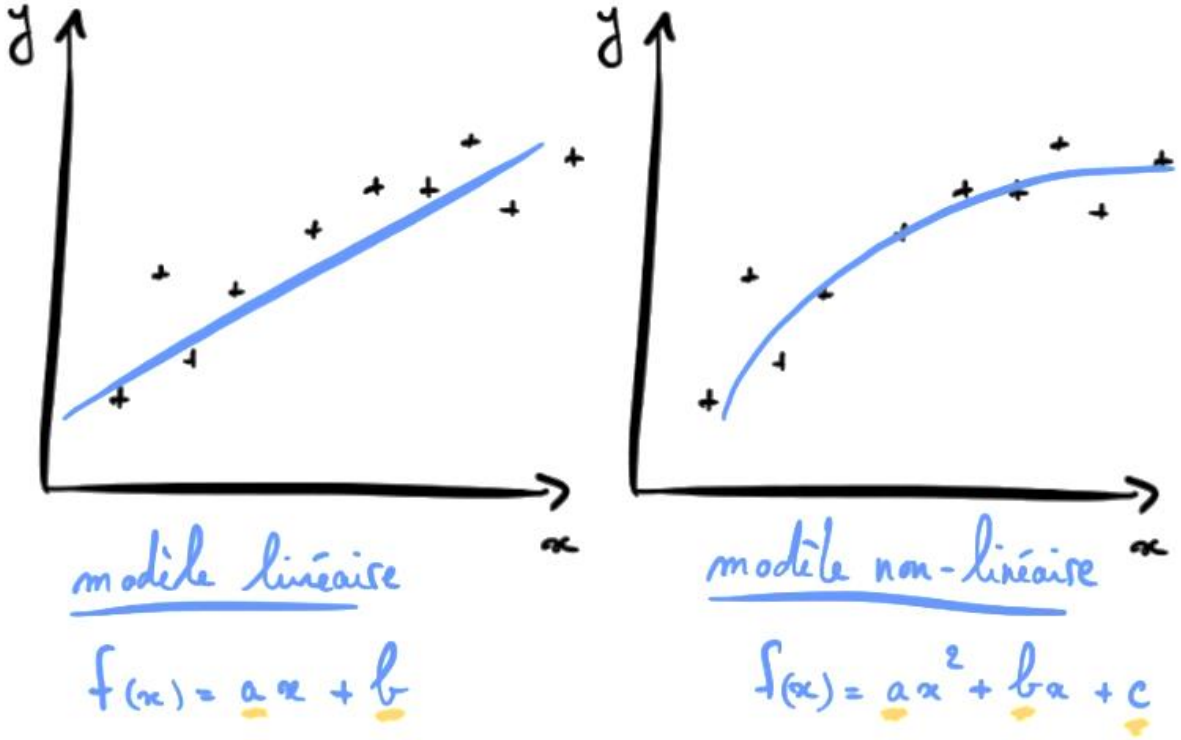
- En Machine Learning, les exemples (x,y) sont regroupés dans une structure appelé Dataset :
- **La variable y** est appelée *cible* (*target*). C'est la valeur que l'on cherche à prédire.
- **La variable x** est appelée *caractéristique* (*feature*). Un facteur influence la valeur de y. En général, un Dataset contient plusieurs caractéristiques (x_1,x_2,...), qui sont regroupées dans une matrice **X**

Target y	Features x_1	x_2	x_3
Prix	Surface m2	N chambres	Qualité
€ 313,000.00	124	3	1.5
€ 2,384,000.00	339	5	2.5
€ 342,000.00	179	3	2
€ 420,000.00	186	3	2.25
€ 550,000.00	180	4	2.5
€ 490,000.00	82	2	1
€ 335,000.00	125	2	2

Apprentissage Supervisé (3)

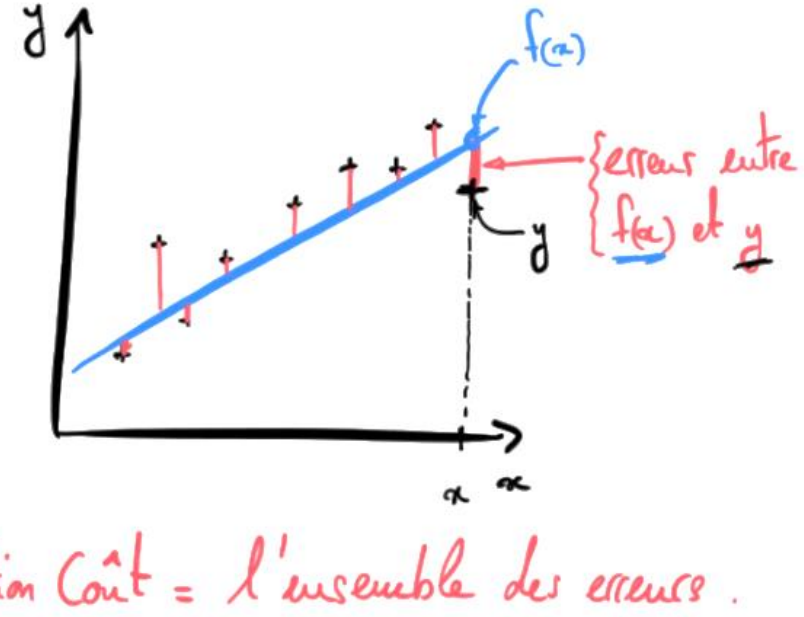
Modèle

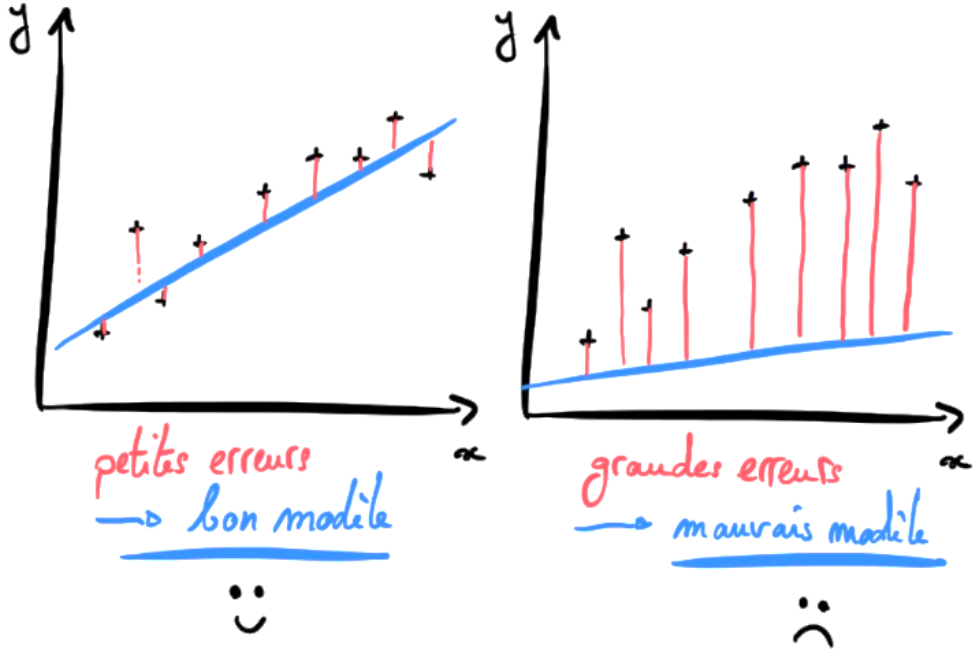
- En Machine Learning, un modèle est développé à partir du Dataset. Ce modèle peut être linéaire, comme illustré à gauche, ou non-linéaire, comme montré à droite. les critères pour choisir l'un ou l'autre de ces modèles dépendent de plusieurs facteurs, notamment la nature des données, la complexité des relations entre les variables, etc...



Fonction coût

- Il est important de noter qu'un modèle génère des erreurs par rapport au Dataset. Ces erreurs sont regroupées sous ce que l'on appelle la *Fonction Coût*.
- Avoir un bon modèle, c'est avoir un modèle qui nous donne de petites erreurs, donc une **petite Fonction Coût**.





Apprentissage Supervisé (5)

Algorithme d'apprentissage

- Apprendre, c'est minimiser la Fonction Coût
- L'objectif principal en apprentissage supervisé est de trouver les **paramètres** du **modèle** qui minimisent la **Fonction Coût**.
- Pour ce faire, on utilise **un algorithme d'apprentissage** ou (algorithme d'optimisation), le plus courant étant l'algorithme de **Descente de Gradient.**

Modèle	Algorithme d'optimisation
Les modèles linéaires	la descente de gradient
Les arbres de décision	l'algorithme CART
Les SVM	la marge maximum

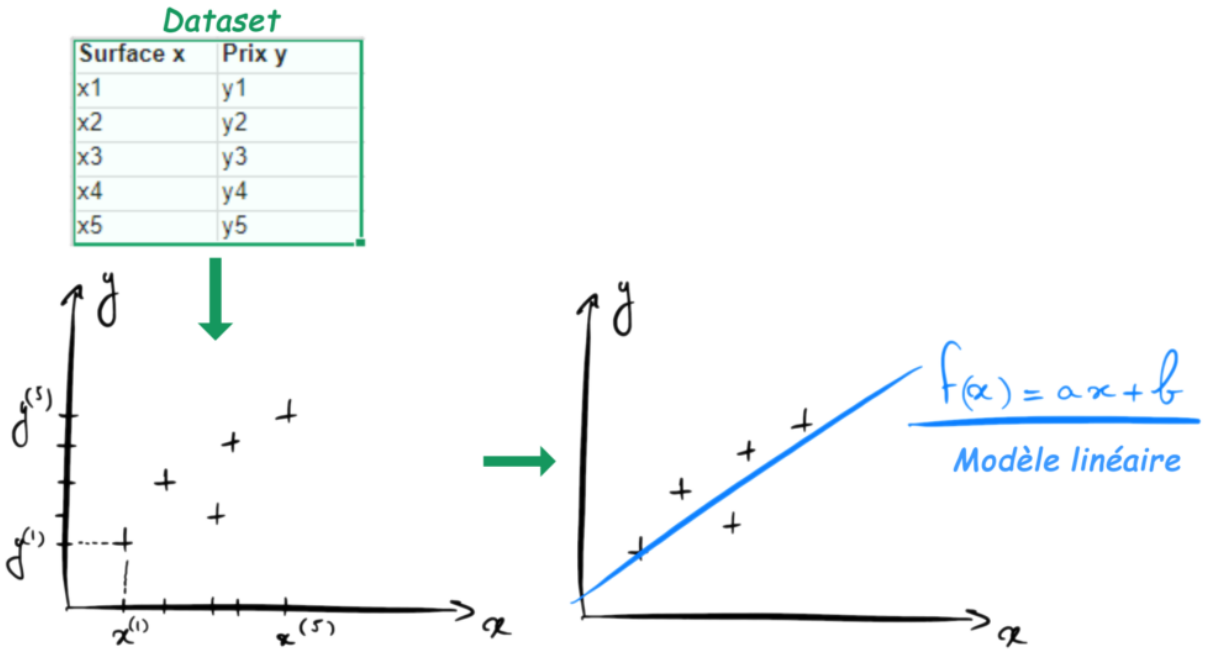
Régression Linéaire

- La régression linéaire est l'algorithme fondamental de machine Learning utiliser pour prédire des valeur linéaire.

  $f(x)=ax+b$

- Le but du jeu en régression linéaire est de trouver les paramètres **a** et **b** qui donnent le meilleur modèle possible, c'est à dire, le modèle qui s'ajuste le mieux à nos données.
- la formule pour exprimer l'erreur i entre le prix $y^{(i)}$ et la prédiction faites en utilisant la surface $x^{(i)}$:

  $erreur^{(i)} = (f(x^{(i)}) - y^{(i)})^2$

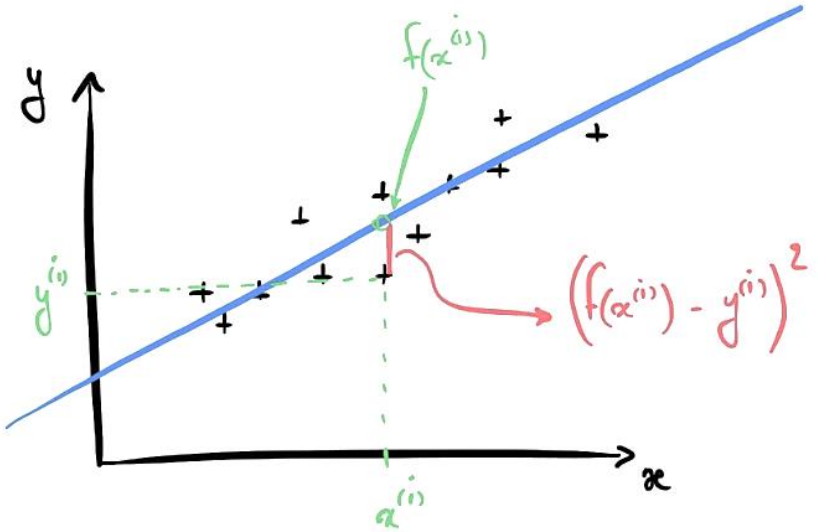


Régression Linéaire

- Chaque prédiction s'accompagne d'une erreur, on a donc m erreurs.
- On définit la **Fonction Coût** $J(a.b)$ comme étant la **moyenne** de toutes les erreurs :

$$J(a,b) = \frac{1}{2m}\sum_{i=1}^{m} erreur^i$$

$$J(a,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(f\left(x^{(i)}\right) - y^{(i)}\right)^2$$



- Cette fonction est connue sous le nom

d'**Erreur Quadratique Moyenne** (*Mean Squared Error*).

$$J(a,b) = \frac{1}{2m}\sum_{i=1}^{m}\left(ax^{(i)} + b - y^{(i)}\right)^2$$

Fonction Coût.

Apprentissage Supervisé (8)

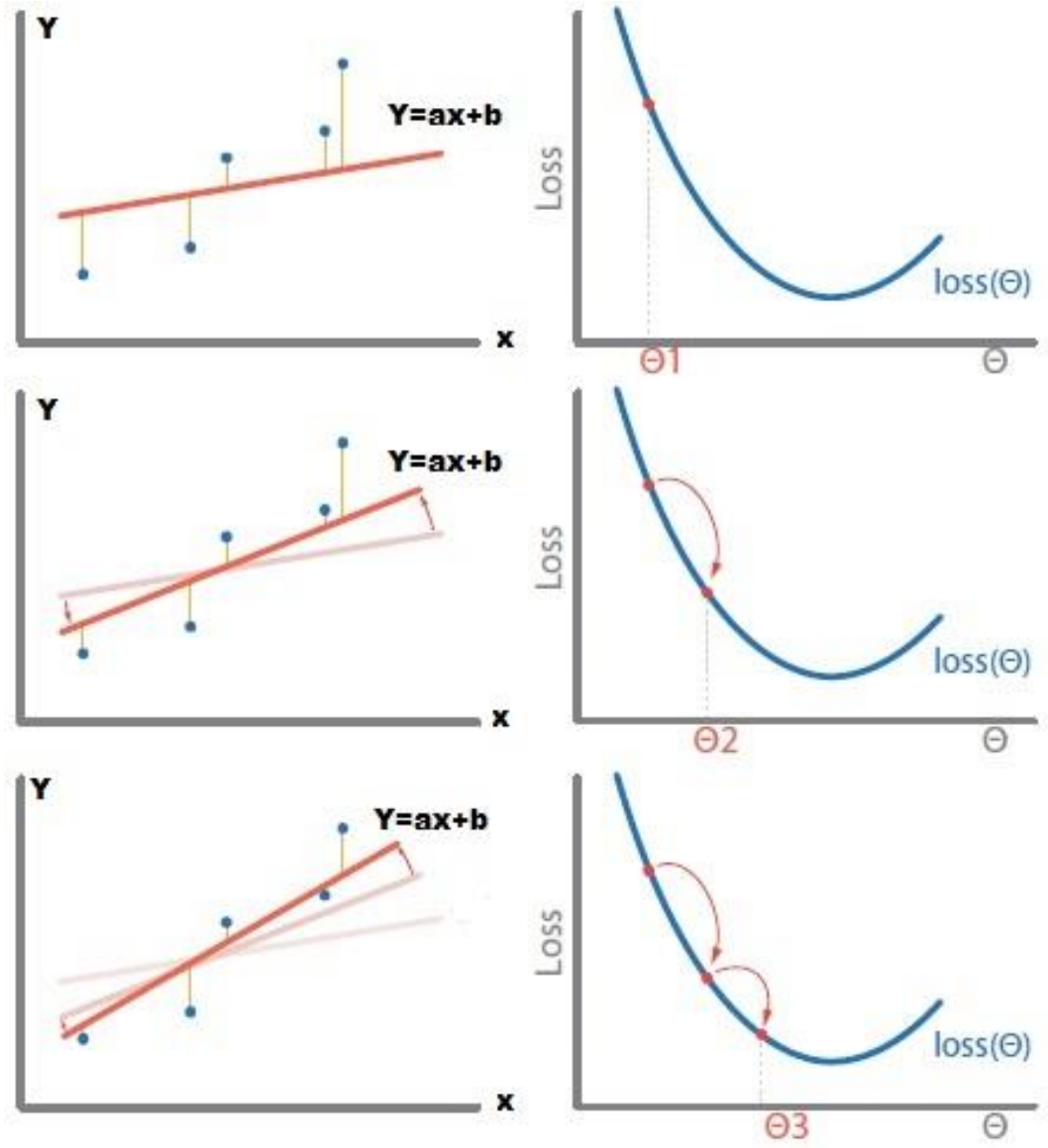
Régression Linéaire

Pour cela, on programme dans la machine un algorithme d'optimisation qui va venir tester différentes valeurs de **a** et **b** jusqu'à obtenir la combinaison qui minimise la distance entre le modèle et les points.

Il existe de nombreux modelés : comme les modèles linéaires, les arbres de décision, ou encore les Support Vector Machines.

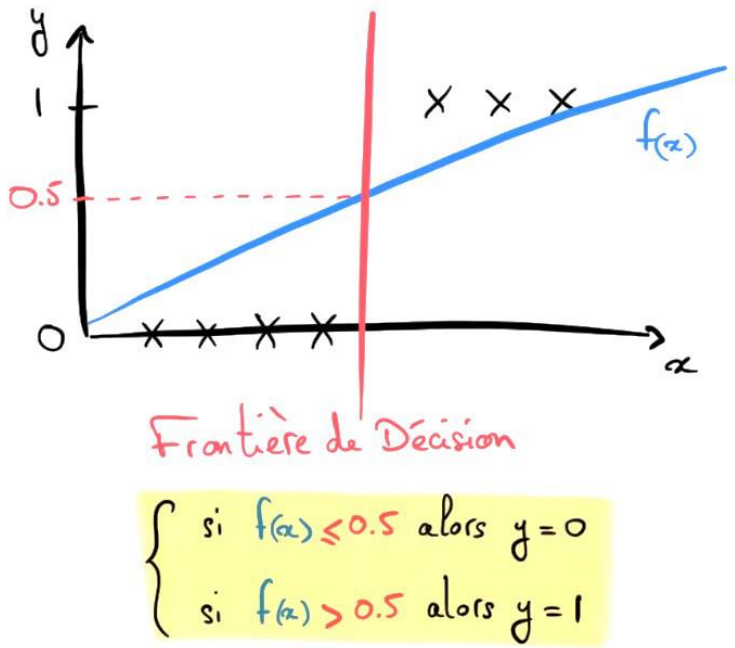
Voici quelques algorithmes d'optimisation selon le modèle :

Modèle	Algorithme d'optimisation
Les modèles linéaires	la descente de gradient
Les arbres de décision	l'algorithme CART
Les SVM	la marge maximum



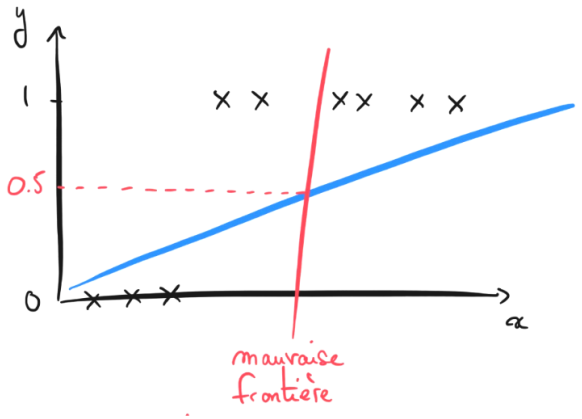
Régression Logistique

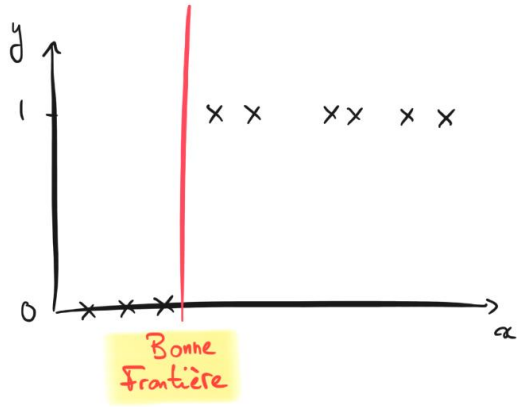
- **La classification**, qui consistent par exemple, à classer un email en tant que ‘spam’ ou ‘non spam’.
- Le Dataset contenant une variable target y pouvant prendre 2 valeurs seulement, par exemple 0 ou 1
  - si y = 0, alors l’email n’est pas un spam
  - si y = 1, alors l’email est un spam
- On dit également que l’on a **2 classes**, c’est une classification **binaire**.
- Pour ces problèmes, on ajoute au modèle une **frontière de décision** qui permet de classer un email dans la $classe$ 0 ou la $classe$ 1.

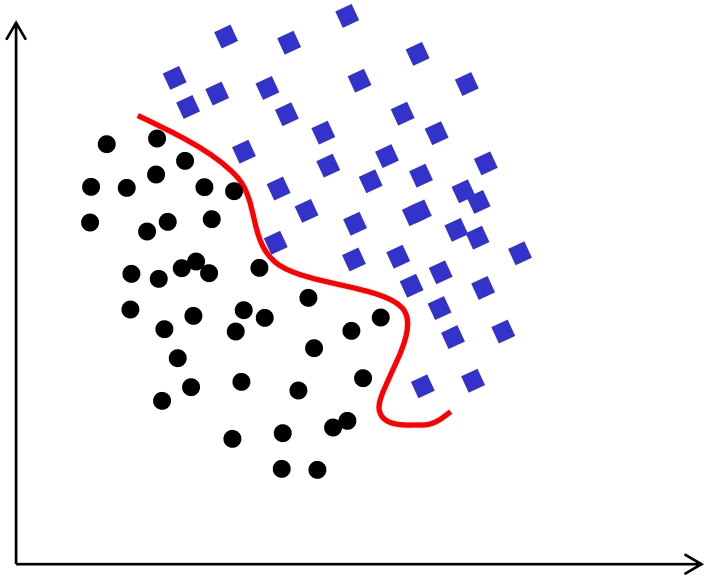


Régression Logistique

- Pour ces problèmes de classification binaire, le modèle linéaire $\boldsymbol{F} = \boldsymbol{X}.\boldsymbol{\theta}$, ne convient pas

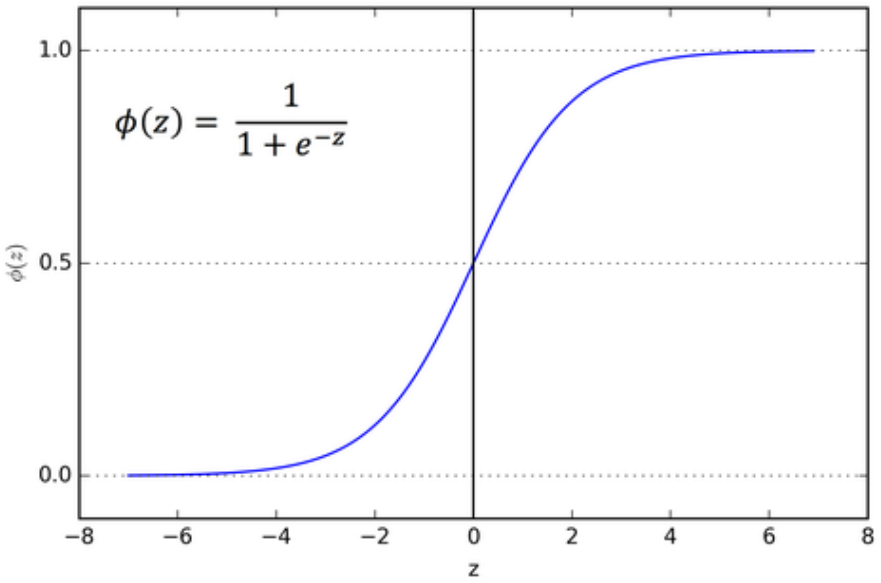






Cas de deux Classes non linéairement séparables

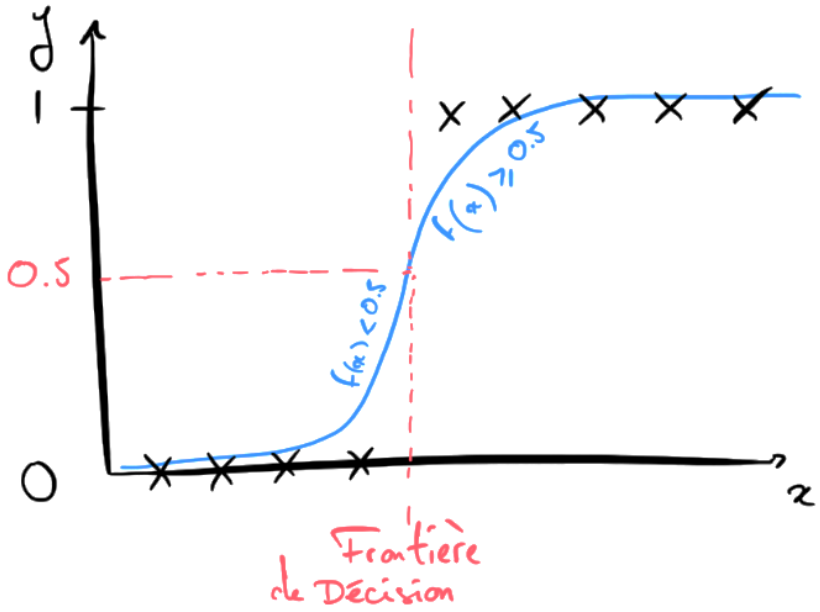
Régression Logistique



- Dans les problèmes de classification binaire, on développe une nouvelle fonction appelée **fonction logistique** (également connue sous le nom de **fonction sigmoïde** ou simplement σ). Cette fonction est particulière en ce qu'elle est toujours comprise entre 0 et 1.

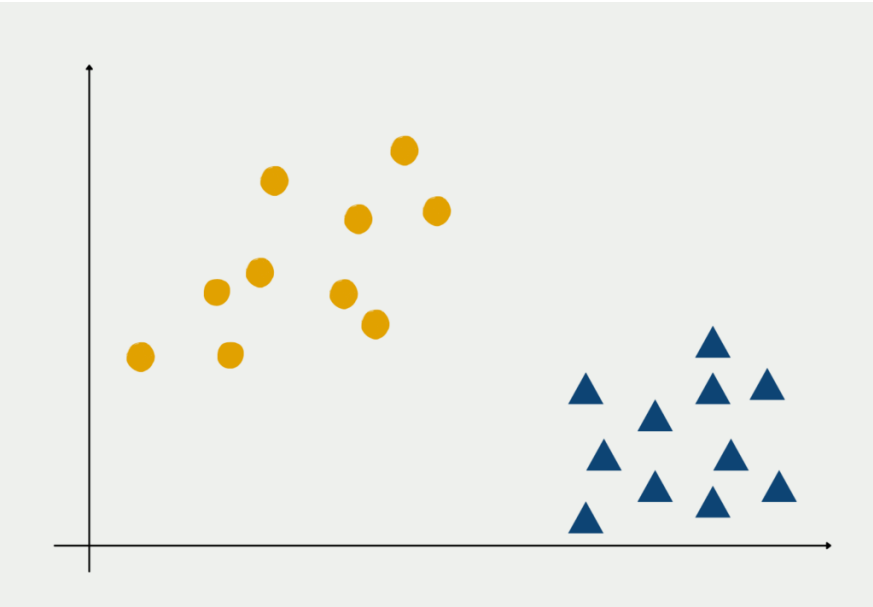
- on définit un seuil à **0.5** comme ceci :

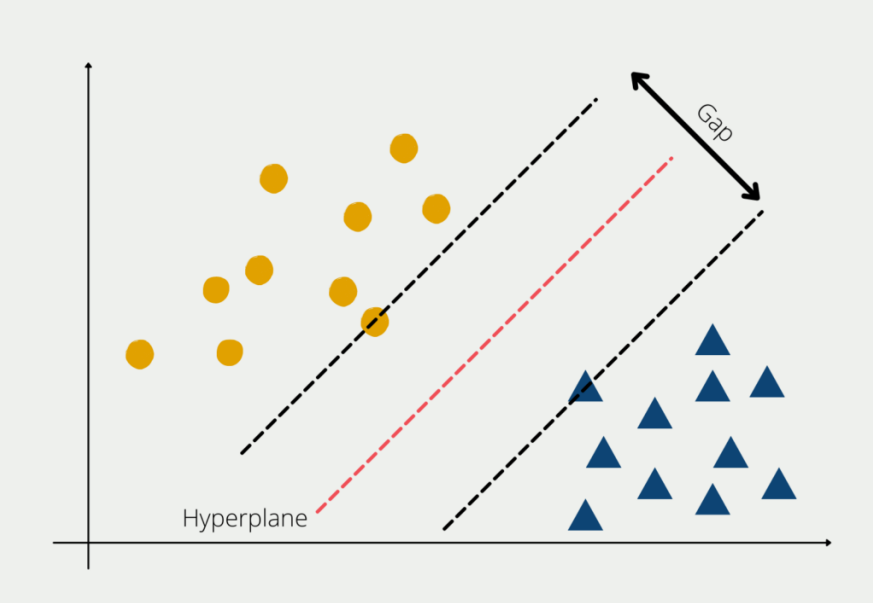
$$\begin{cases} y=0 \ si \ \sigma(X.\theta)<0.5 \\ y=1 \ si \ \sigma(X.\theta)\geq 0.5 \end{cases}$$



Machine à Vecteurs de Support (SVM)

- Les Machines à vecteurs de support (SVM) sont des algorithmes de classification supervisés utilisés pour séparer les données en différentes catégories.
- L'objectif de l'algorithme SVM est de trouver un hyperplan qui, du mieux possible, sépare les points de données d'une classe de ceux d'une autre classe

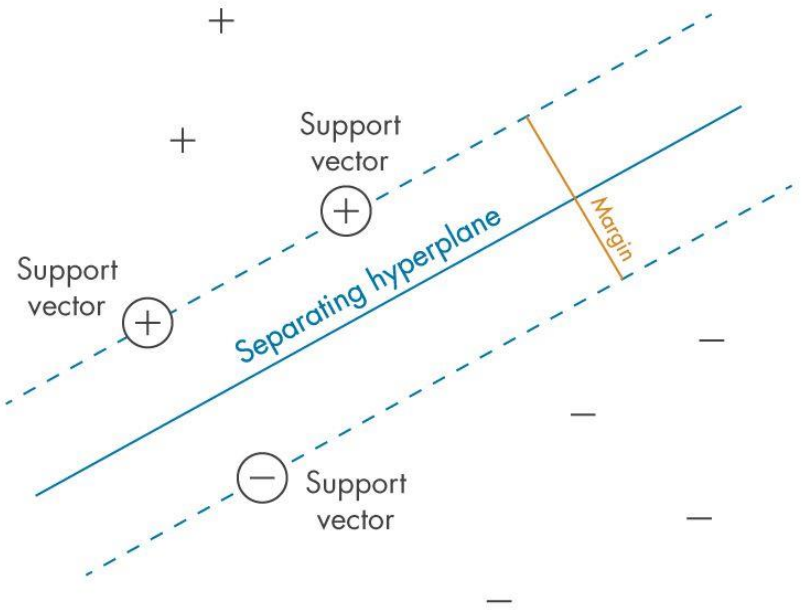




Machine à Vecteurs de Support (SVM)

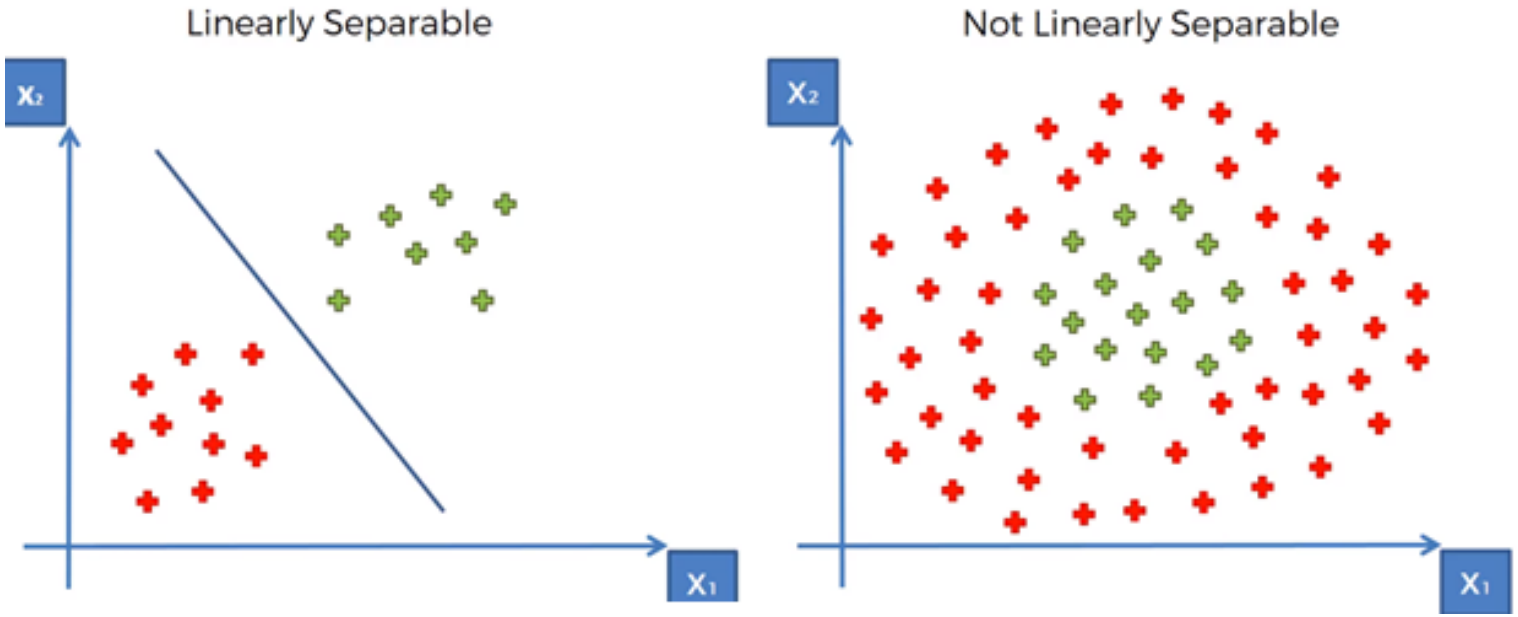
- Un hyperplan est une ligne (en 2D) ou une surface (en 3D ou plus) qui sépare les classes de données.
- La marge désigne la largeur maximale de l'espace parallèle à l'hyperplan qui ne contient aucun point de données
- SVM cherche à maximiser cette marge pour

assurer une bonne séparation entre les classes.



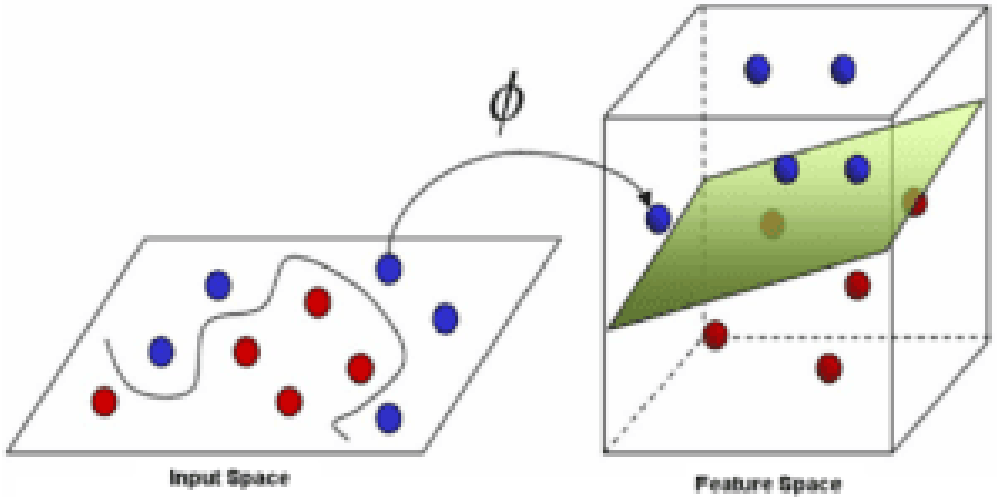
Machine à Vecteurs de Support (SVM)

- SVM Linéaire : Fonctionne bien lorsque les données peuvent être séparées par une ligne droite (ou un hyperplan dans un espace de dimension supérieure).
- SVM Non-Linéaire : Utilise des techniques comme le noyau (kernel trick) pour séparer des données non-linéaires en les transformant dans un espace de dimension supérieure.



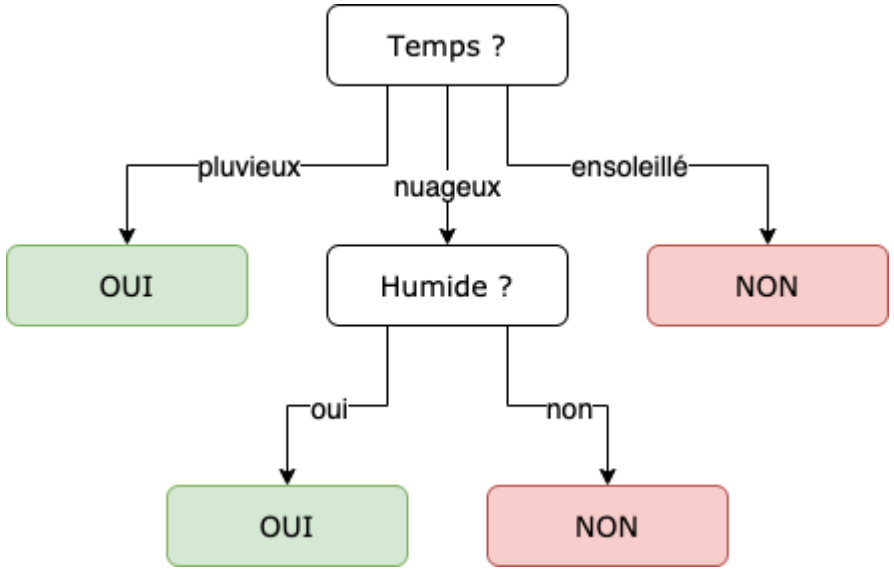
Machine à Vecteurs de Support (SVM)

- Pour traiter les cas où les données ne sont pas linéairement séparables, l'idée clé des SVM est de transformer l'espace de représentation des données en un espace de dimension plus élevée (potentiellement infinie). Dans cet espace de dimension supérieure, il est plus probable que les données soient linéairement séparables
- Ceci est réalisé grâce à une fonction Noyau (Kernel)
- Types de noyaux courants :
  - Noyau linéaire
  - Noyau polynomial
  - Noyau RBF (Radial Basis Function)
  - Le noyau RBF est souvent utilisé pour capturer des relations complexes non-linéaires entre les données.



Arbres de Décision

- Les arbres de décision sont des modèles de classification et de régression qui utilisent une structure en forme d'arbre pour prendre des décisions.
- Contenu :
  - **Noeud Racine** : Le noeud initial de l'arbre, où commence la décision.
  - **Noeuds Internes** : Points où les décisions sont prises, basés sur les valeurs des attributs.
  - **Feuille**s : Les noeuds finaux qui donnent la décision ou la prédiction.
  - **Diagramm**e : Schéma détaillé montrant les différents types de noeuds (racine, internes, et feuilles) avec des exemples de décisions.



Apprentissage Supervisé (17)

Autres Algorithmes supervisé

- k-Nearest Neighbors (k-NN)
- Naive Bayes
- Forêts Aléatoires (Random Forests)
- Gradient Boosting Machines (GBM)
  - XGBoost
  - LightGBM
  - CatBoost
- Neural Networks (réseaux de neurones)
  - Perceptrons Multicouches (MLP)

Apprentissage non Supervisé (1)

- Algorithmes de Clustering
- Algorithmes de Réduction de Dimensionnalité
- Algorithmes d'Association
- Algorithmes de Détection d'Anomalies

Apprentissage non Supervisé (2)

Autres Algorithmes de Clustering

- k-Means Clustering
- Hierarchical Clustering
  - Agglomerative Hierarchical Clustering
  - Divisive Hierarchical Clustering
- DBSCAN (Density-Based Spatial Clustering of Applications with Noise)
- Mean Shift Clustering
- Spectral Clustering
- Gaussian Mixture Models (GMM)

Apprentissage non Supervisé (3)

Algorithmes de Réduction de Dimensionnalité

- Principal Component Analysis (PCA)
- t-Distributed Stochastic Neighbor Embedding (t-SNE)
- Linear Discriminant Analysis (LDA)
- Isomap
- Locally Linear Embedding (LLE)
- Autoencoders (réseaux de neurones pour réduction de dimensionnalité)

Apprentissage non Supervisé (4)

Algorithmes d'Association

- Apriori Algorithm
- Eclat Algorithm
- FP-Growth (Frequent Pattern Growth)

Apprentissage non Supervisé (5)

Algorithmes de Détection d'Anomalies

- Isolation Forest
- One-Class SVM
- Local Outlier Factor (LOF)
- Elliptic Envelope

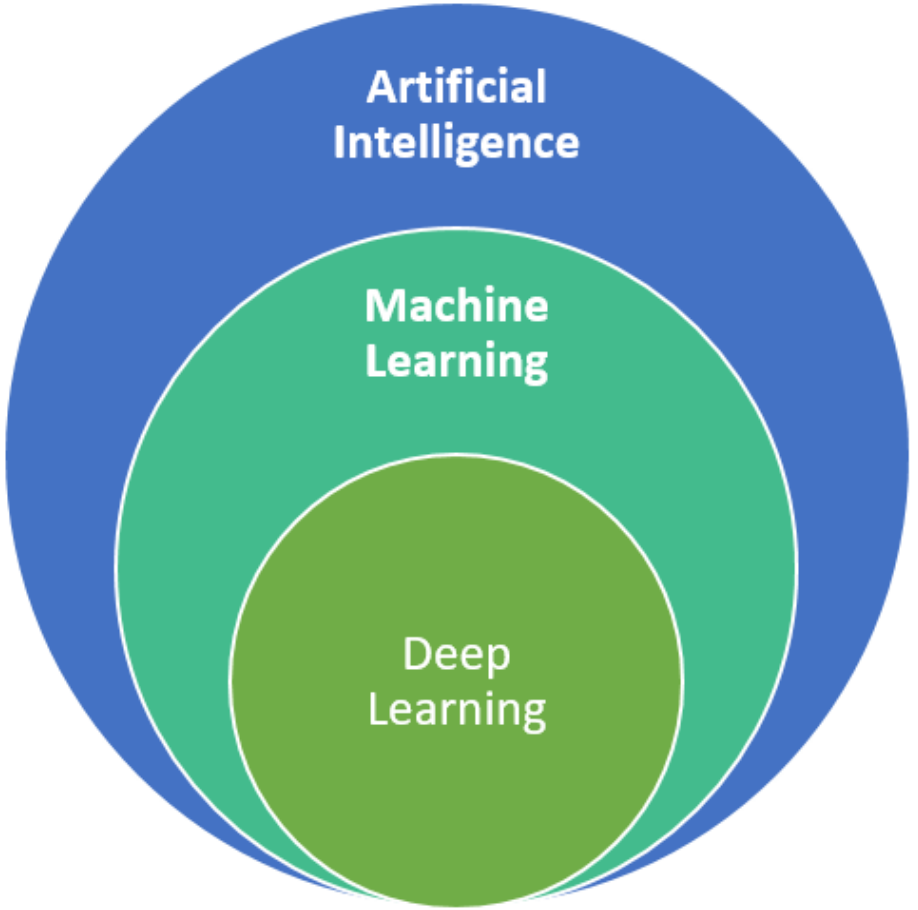
Apprentissage non Supervisé (6)

Autres Algorithmes non supervisé

- Self-Organizing Maps (SOM)
- Matrix Factorization (comme les Décompositions en Valeurs Singulières - SVD)
- Clustering basés sur les graphes (Graph-based Clustering)

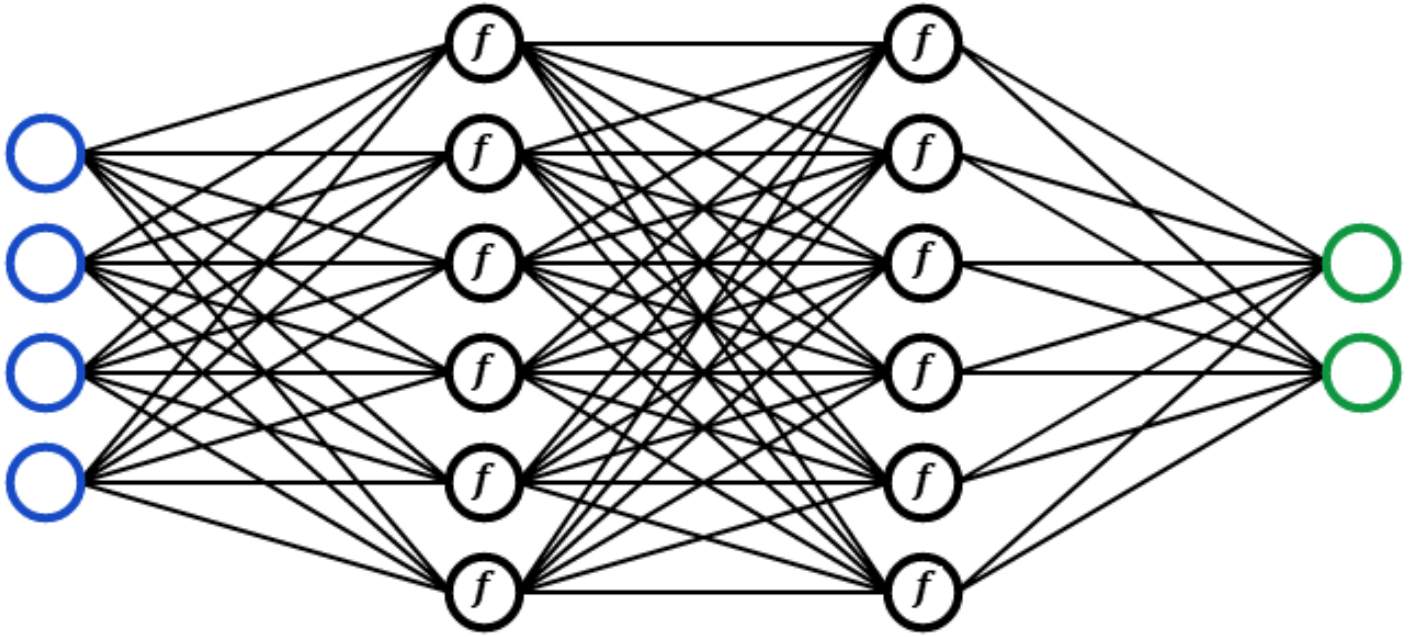
Deep Learning

- Le Deep Learning est un domaine du Machine Learning dans lequel, au lieu de développer un des modèles que l'on vient de citer, on développe à la place ce qu'on appelle des réseaux de neurones artificiels.
- Le principe reste exactement le même :
  - On fournit à la machine des données
  - Développer un modèle
  - Utilise un algorithme d'optimisation
  - Minimiser l'erreur



Deep Learning

- Le modèle ici n'est pas une simple fonction du type $f(x) = ax + b$ mais plutôt un réseau de fonctions connectées les unes aux autres. on l'appelle un réseau de neurone

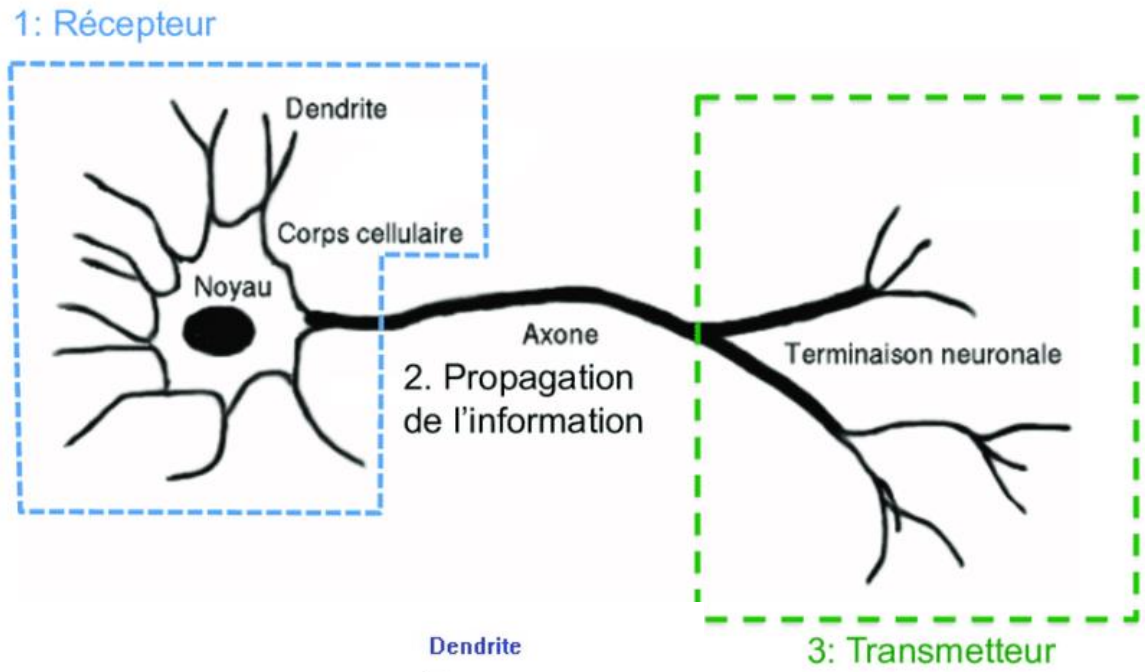


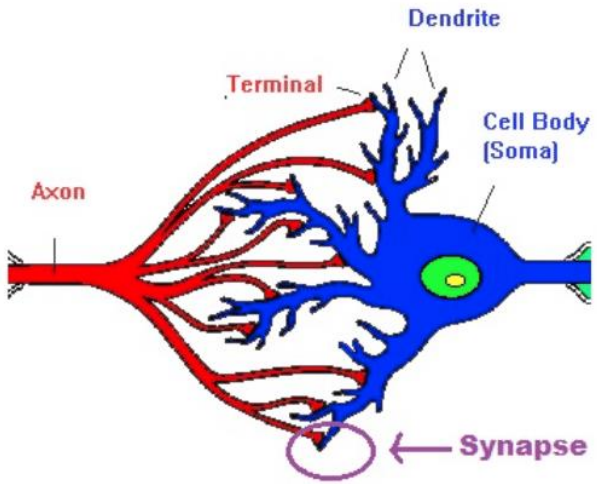
Histoire de Deep Learning

- **Années 1940-1950 :** Les bases du Deep Learning ont été jetées dans les années 1940-1950 par **Warren McCulloch et Walter Pitts**, qui ont crée les premiers modelés de neurones artificiels, ainsi que par **Donald Hebb**, qui a introduit le concept de plasticité synaptique dans le cerveau. Il s'agit d'un neurone binaire, c'est-à-dire dont la sortie vaut 0 ou 1.
- **Années 1950-1960** : Des chercheurs comme **Frank Rosenblatt** ont développe le Perceptron, un modèle de réseau de neurones artificiels à une seule couche. Le Perceptron avait la capacité d'apprendre à résoudre des problèmes de classification simples.
- **Années 1970-1990 :** L'intérêt pour l'intelligence artificielle (IA) a grandi, et les réseaux de neurones ont suscité l'attention des chercheurs. Cependant, dans les années 1980, l'IA a traversé une période appelée "l'hiver de l'IA" en raison des limitations technologiques et des approches de l'époque.
- **Années 2000** : Des progrès dans le matériel informatique, les algorithmes d'optimisation et les architectures de réseaux de neurones renouvellent l'intérêt pour le Deep Learning. Des modèles tels que les machines de Boltzmann restreintes (RBM) ou les réseaux de neurones profonds (DNN) commencent à montrer des résultats prometteurs dans des applications pratiques.
- **Années 2010 :** L'utilisation de vastes ensembles de données (big data) et de puissants processeurs graphiques (GPU) permet d'entraîner des réseaux de neurones profonds à plusieurs couches, appelés réseaux de neurones profonds convolutionnels (CNN) et réseaux de neurones profonds récurrents (RNN). Ces architectures sont appliquées avec succès à la vision par ordinateur, au traitement du langage naturel, à la reconnaissance vocale, etc.

Neurone biologique

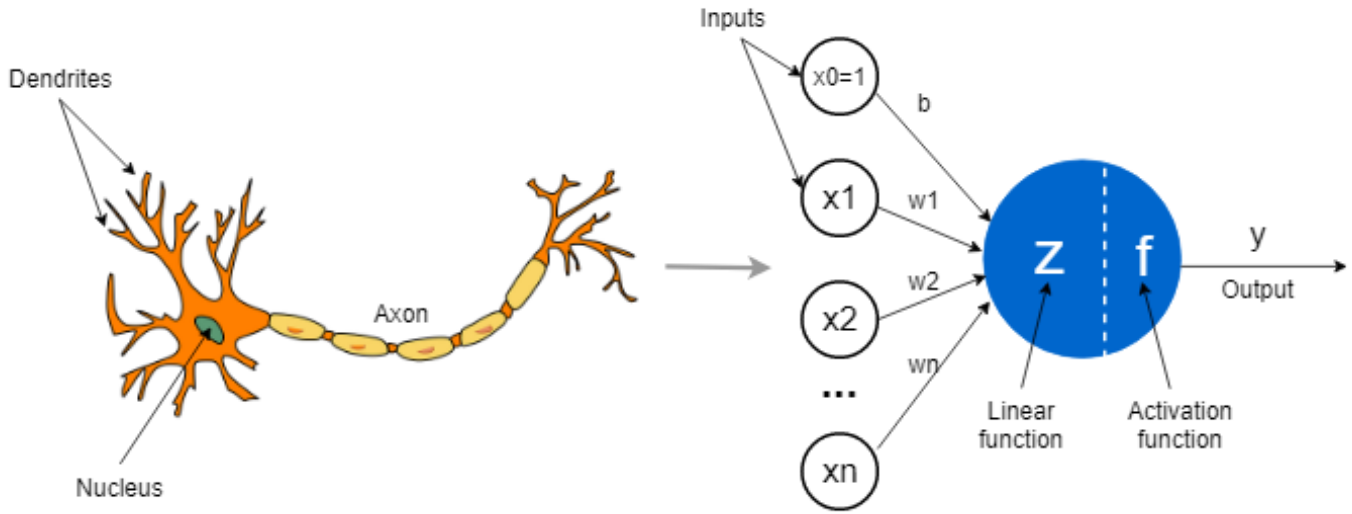
- Le neurone biologique sont des cellules excitables connectées les uns aux autres et ayant pour rôle de transmettre des informations dans notre système nerveux.
- Il est composé de trois parties principales :
  - Le **corps cellulaire** composé du **centre de contrôle traitant les informations** reçues par les dendrites.
  - Les **dendrites** sont les principaux **fils conducteurs** par lesquels **transitent l'information** venue de l'extérieur.
  - L'**axone** est un fil conducteur qui **conduit le signal de sortie du corps cellulaire vers d'autres neurones**.
- Quant aux **synapses**, elles font **effet de liaison et de pondération entre neurones** et permettent donc aux neurones de communiquer entre eux.





Neurone biologique

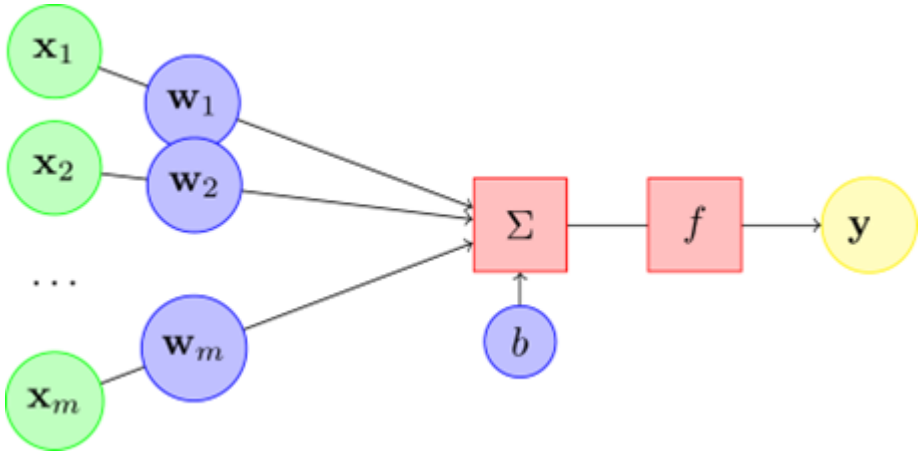
- Comparaison simple des étapes principales de l'**algorithme du perceptron (neurone artificiel)** avec les **éléments constitutifs** des neurones biologiques :
  - Les **synapses/dendrites** : **pondération de chaque élément** en entrée $w_i . xi$
  - **Corps cellulaires** : **application d'une fonction d'activation** f à la somme des entrées pondérées
  - **Axone** : **sortie** de notre modèle $\mathbf{y} = f(\sum_{i=1}^{n} w_i . x_i)$



Le perceptron (Neurone Artificiel)

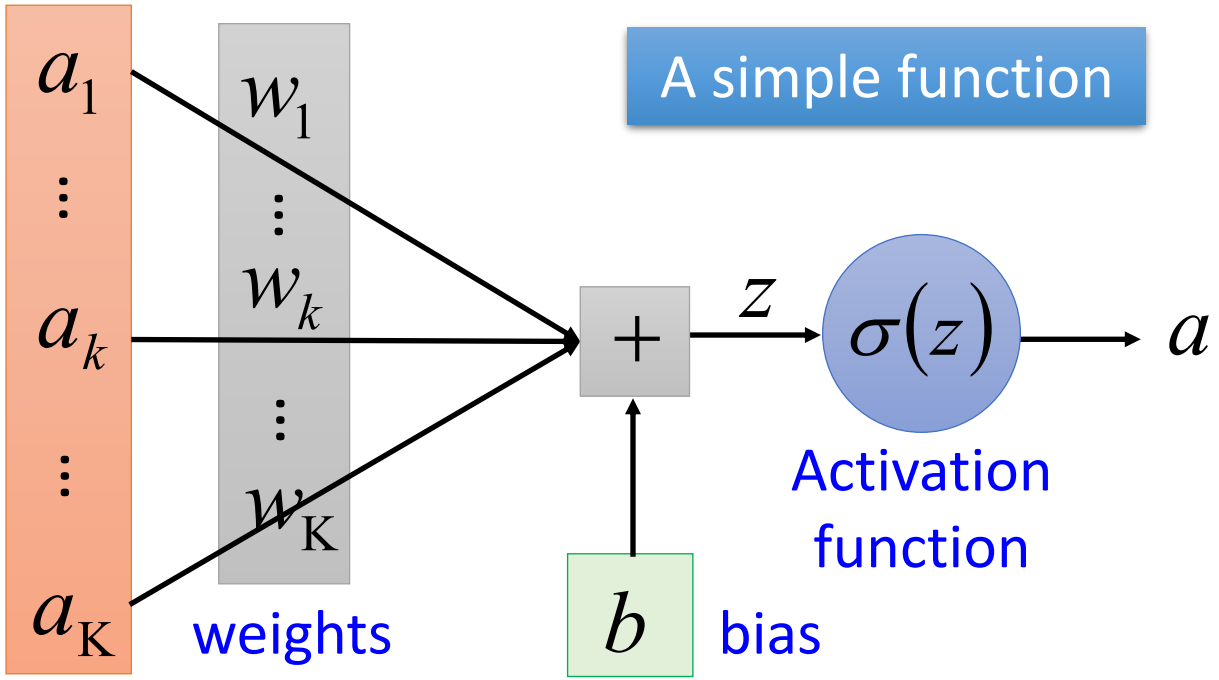
- Le perceptron, encore appelé neurone artificiel ou neurone formel, est l'unité de base des réseaux de neurones
- C'est un algorithme d'apprentissage supervisé de classifieurs binaires (c'est-à-dire séparant deux classes)
- Nous allons considérer les entités suivantes :
  - Des **entrées**, notées x sous formes de vecteurs, représentant les dendrites ;
  - Une **sortie**, notée y, représentant l'axone ;
  - des **paramètres**, notés w et b , influençant le fonctionnement du neurone
    - W : le poids (weight)
    - b : le biais
    - Σ : la somme ou
    - f : activation
- L'expression mathématique d'un perceptron est définie comme suit :

$$Y = f(\langle W, X \rangle + b)$$



Neuron

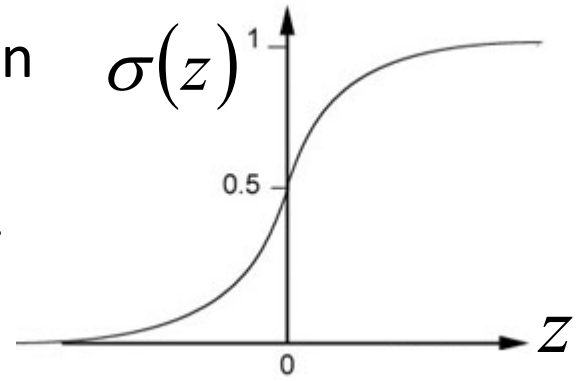
$$z = a_1 w_1 + \cdots + a_k w_k + \cdots + a_K w_K + b$$

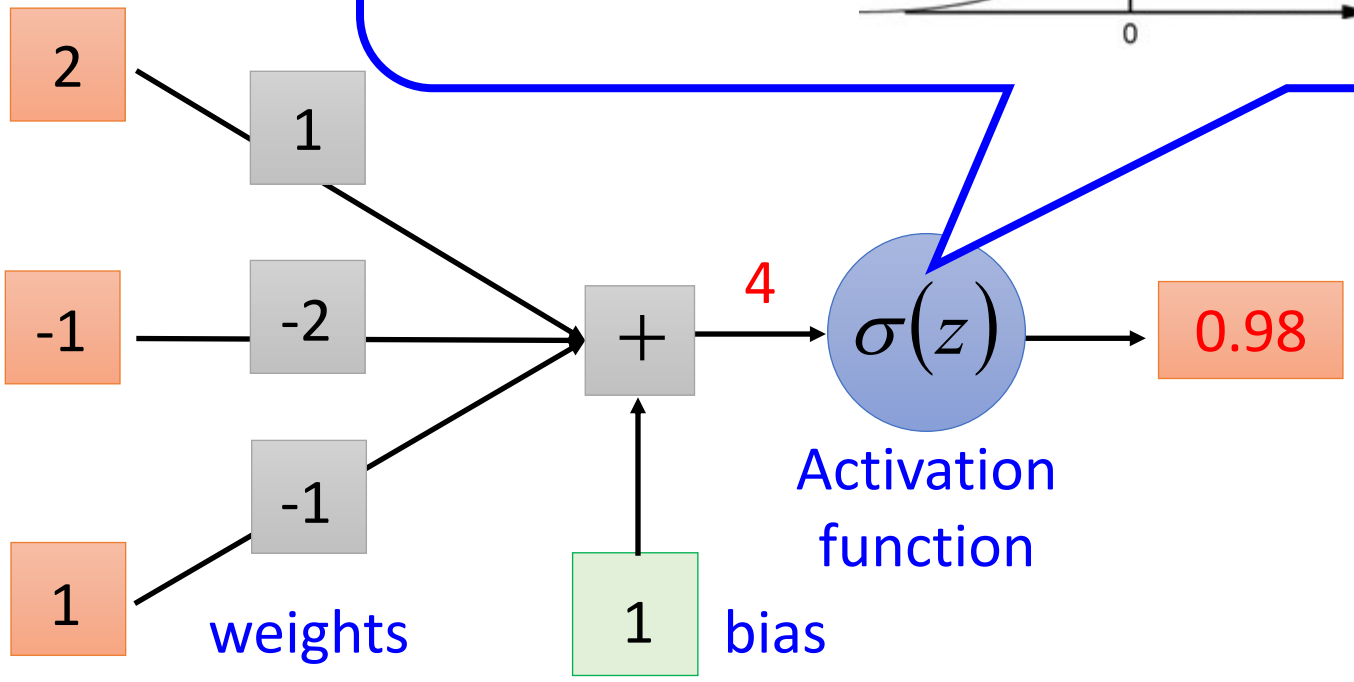


Neuron

Sigmoid Function

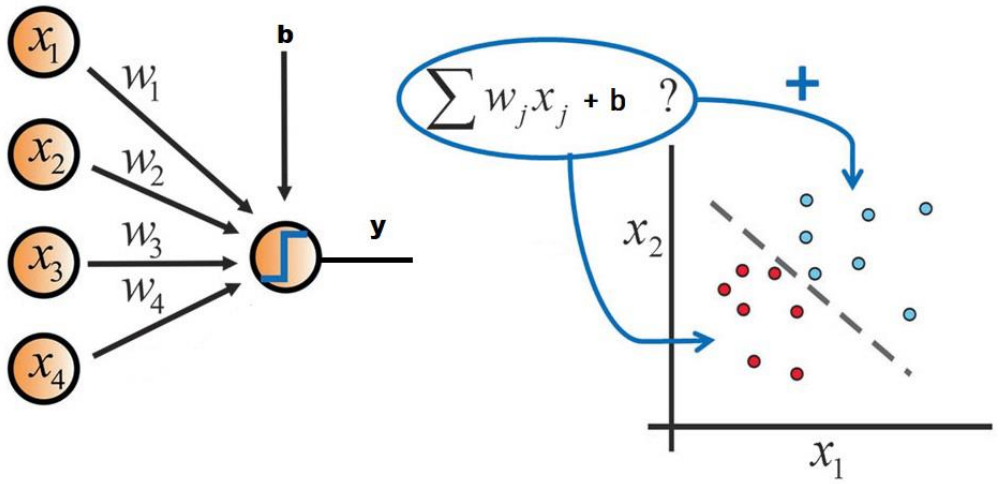
$$\sigma(z) = \frac{1}{1+e^{-z}}$$





Le perceptron

- L'équation d'un perceptron est donnée par : $z = w_1.x_1 + w_2.x_2 + b$.
- Pour prédire à quelle classe appartient un élément on vérifier le signe de z.
  - Si $z > 0$ c'est la classe bleue,
  - si $z < 0$ c'est la classe rouge.
  - La frontière de décision correspond à $z = 0$.

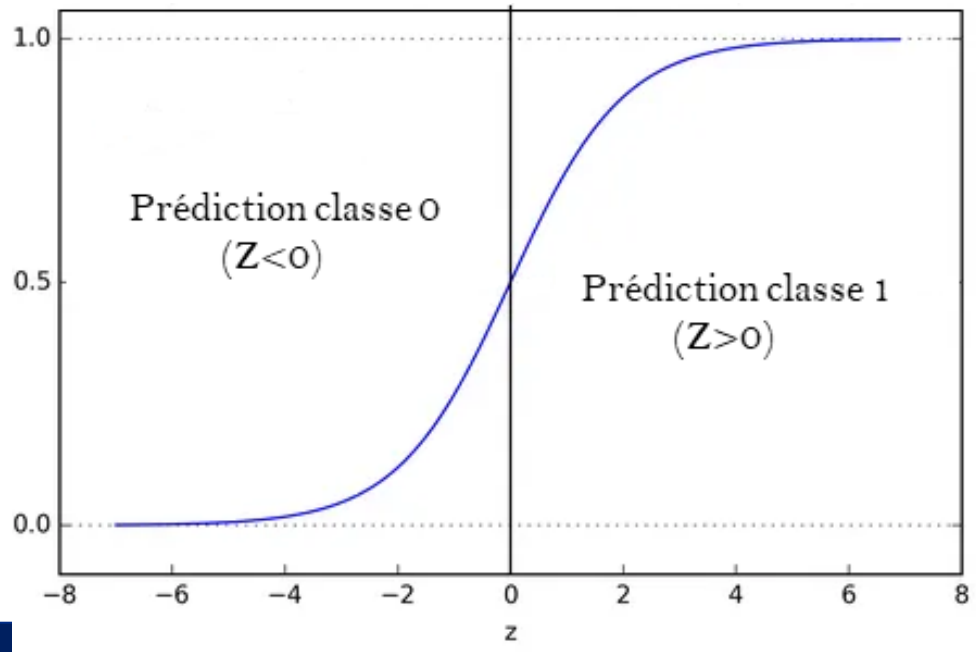


Le perceptron

- Pour améliorer ce modèle on va accompagner chaque prédiction d'une probabilité, plus une observation sera éloignée de la frontière de décision plus il sera évident qu'elles appartiennent bien à sa classe.
- Pour ce faire, nous pourrions employer une fonction d'activation qui produit une sortie se rapprochant progressivement de zéro ou de un à mesure que l'on s'éloigne de la frontière de décision, là où la fonction f est égale à zéro.
- La fonction qui nous permet de faire ca, c'est la fonction **sigmoïde** également appelé fonction logistique dont l'expression est :

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

- Cette fonction permet de convertir la sortie z en une probabilité $\boldsymbol{\sigma(z)}$.



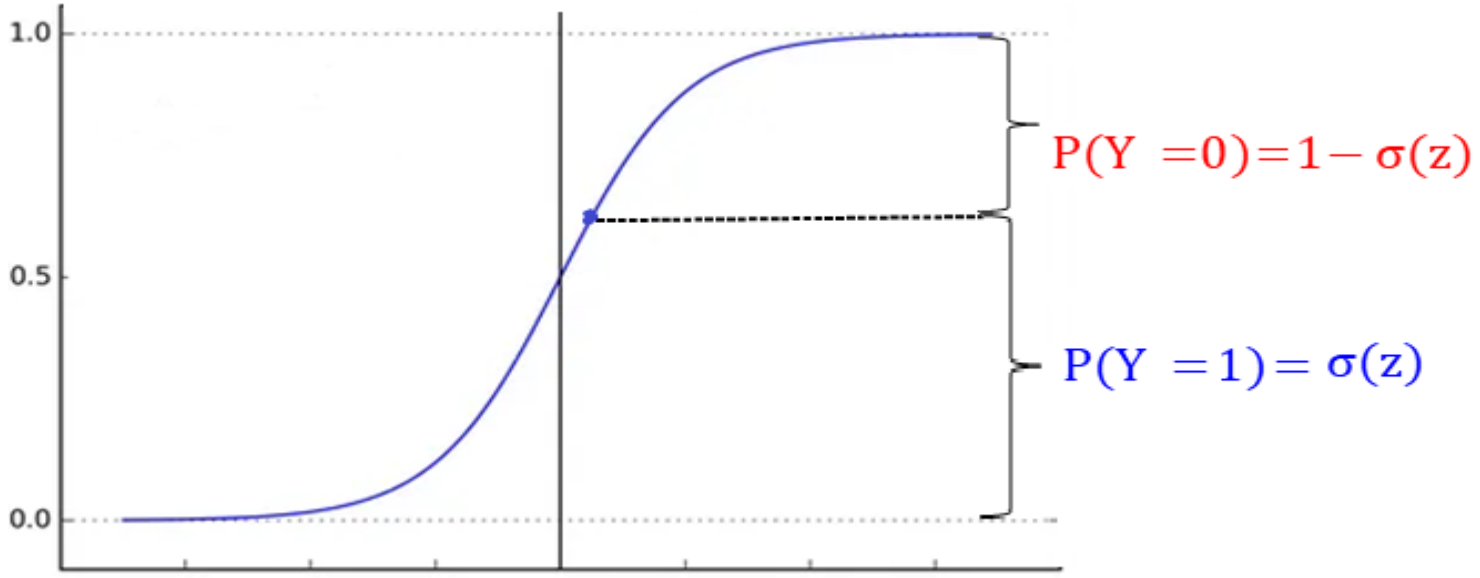
Le perceptron

- Ces probabilités suivent en fait une loi de Bernoulli : $P(Y = y) = \sigma(z)^y \times \left(1 - \sigma(z)\right)^{1-y}$
- Sachant que :

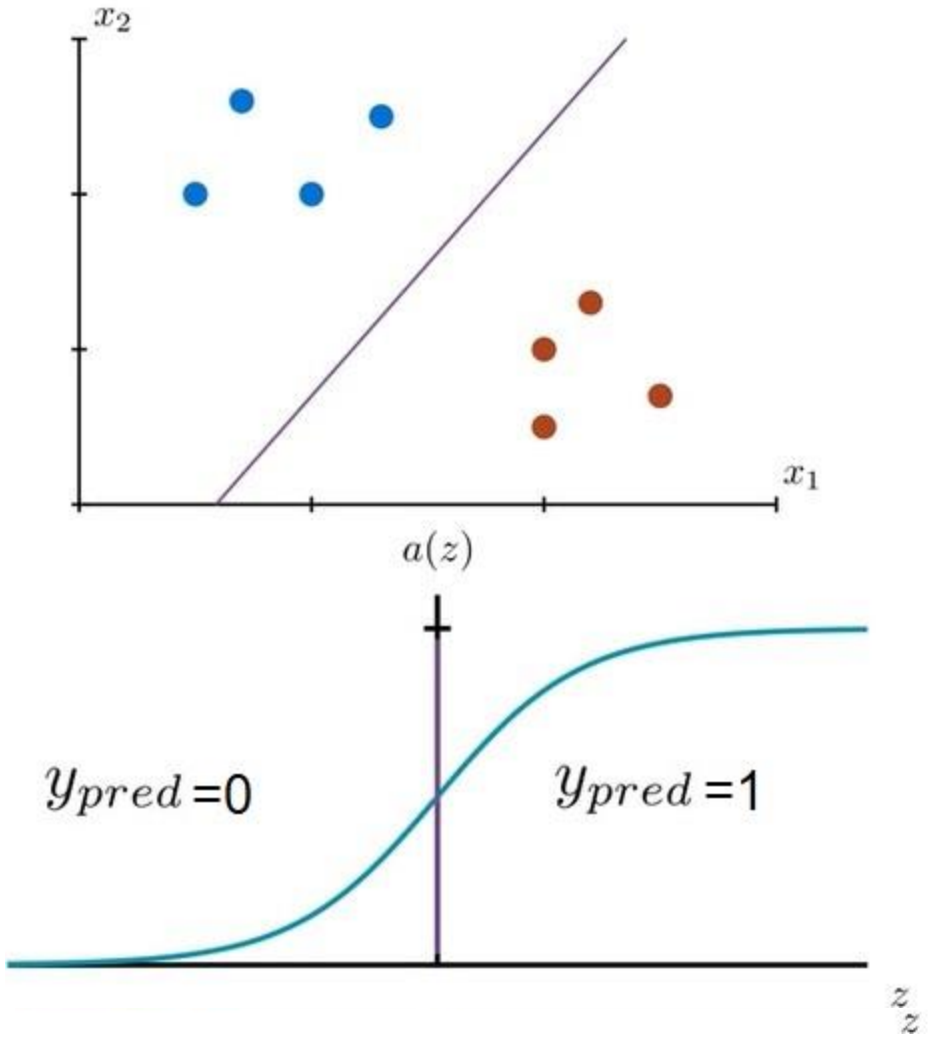
$$P(Y = 0) = \sigma(z)^0 \times (1 - \sigma(z))^{1-0} = (1 - \sigma(z))$$

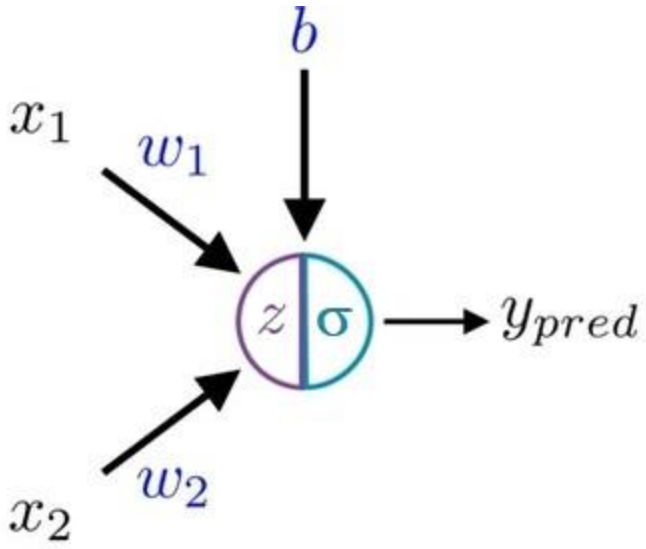
$$P(Y = 1) = \sigma(z)^1 \times (1 - \sigma(z))^{1-1} = \sigma(z)$$

- (Rappel : $x^0 = 1$)



Résumé



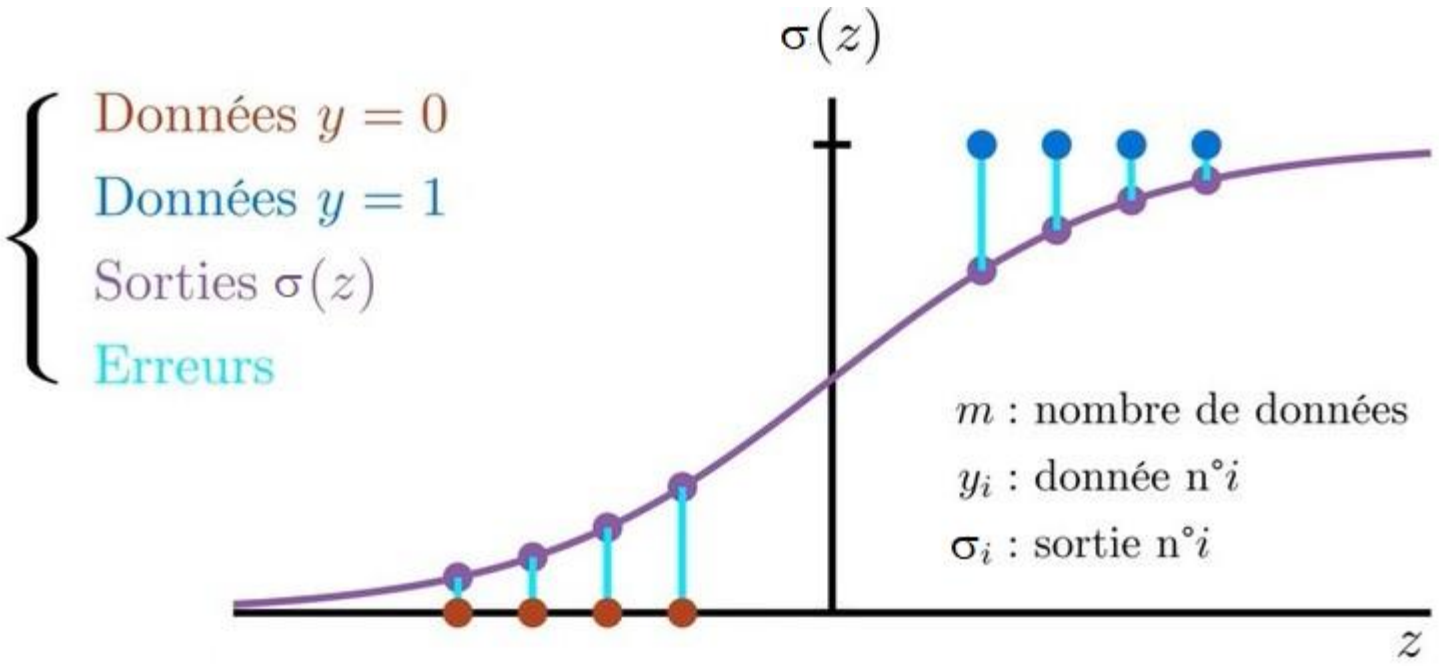


$$z = w_1 x_1 + w_2 x_2 + b,$$

$$\sigma = \frac{1}{1 + e^{-z}}$$

La Fonction coût

- A présent, notre objectif consiste à optimiser les paramètres W_1, W_2 *et* b afin d'obtenir le modèle le plus performant possible, c'est-à-dire celui qui minimise au maximum les divergences entre les sorties $\sigma(z)$ et les données réelles y. Cette optimisation s'appuie sur l'utilisation d'une **fonction de coût**, également appelée "**Loss Function**" en anglais, qui sert à évaluer de manière quantitative les erreurs commises par le modèle.



La Fonction coût

- Une façon d'évaluer la performance de notre modèle c'est de calculer sa vraisemblance (Likelihood en englais) en statistiques. Pour ce faire nous allons calculer simplement le produit de toutes ces probabilités en utilisant la loi de Bernoulli.

$$L = \prod_{i=1}^{m} P(Y = y_i) = \prod_{i=1}^{m} \boldsymbol{\sigma}_i^{y_i} \times (1 - \boldsymbol{\sigma}_i)^{1-y_i}$$

- Si le résultat de cette évaluation approche à 100%, cela indiquera que le modèle est vraisemblable, tandis qu'une proximité avec 0% traduira une forte improbabilité de notre modèle.

La Fonction coût

- Comme en calcul un produit de probabilité, des valeurs entre 0 et 1, on se rend compte que on tombe sur un résultat proche de zéro. Plus on a de nombreux plus le résultat tend vers zéro.
- La solution est d'utiliser le Logarithme car le logarithme d'un produit nous donne la somme des logarithmes.

$$\log(ab) = \log(a) + \log(b)$$

- on aura donc :

$$Log(L) = Log(\prod_{i=1}^{m} \sigma_i^{y_i} \times (1 - \sigma_i)^{1-y_i})$$

- Or le logarithme est fonction monotone croissante alors elle conserve l'ordre de nos termes. cela signifie que lorsqu'on cherchera le maximum de notre vraisemblance il nous suffira de chercher le maximum du log de la vraisemblance.

$$\boldsymbol{Argmax(Likelihood) = Argmax(\log(Likelihood))}$$

La Fonction coût

- Avant de commencer le calcul rappelons que : $\log(ab) = \log(a) + \log(b)$ et $\log(a^y) = ylog\,(a)$

$$LL = Log\left(\prod_{i=1}^{m} \boldsymbol{\sigma}_i^{y_i} \times (1-\boldsymbol{\sigma}_i)^{1-y_i}\right)$$

$$= \sum_{i=1}^{m} \left(\log\left(\boldsymbol{\sigma}_i^{y_i} \times (1-\boldsymbol{\sigma}_i)^{1-y_i}\right)\right)$$

$$= \sum_{i=1}^{m} \left(\log\left(\boldsymbol{\sigma}_i^{y_i}\right) + Log\left((1-\boldsymbol{\sigma}_i)^{1-y_i}\right)\right)$$

$$\boxed{LL = \sum_{i=1}^{m} (y_i\, Log(\boldsymbol{\sigma}_i) + (1-y_i) Log(1-\boldsymbol{\sigma}_i))}$$

La Fonction coût

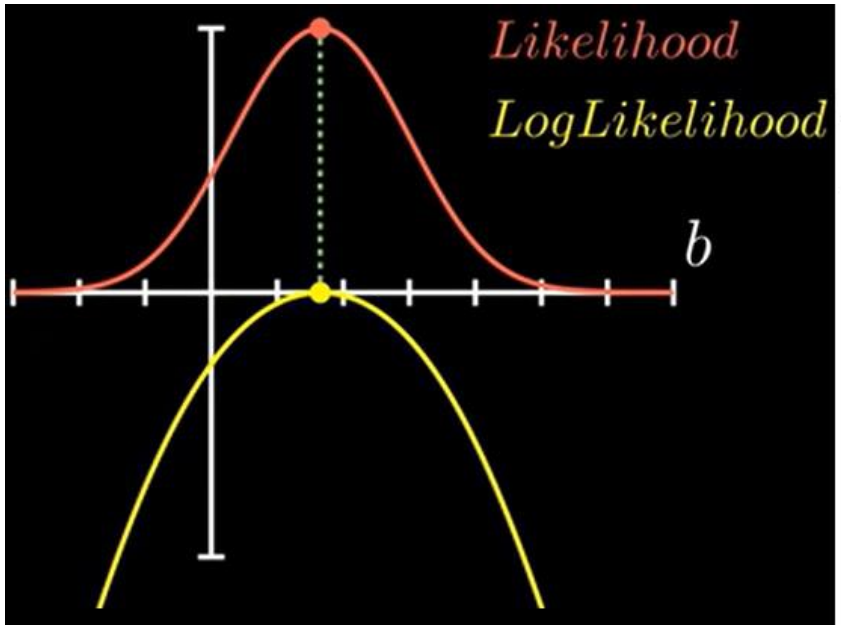
- Ce que nous avons accompli jusqu'à présent consiste à calculer le logarithme de la vraisemblance et notre objectif est de le maximiser afin d'obtenir le modèle le plus plausible.
- Cependant, en mathématiques, il n'existe pas d'algorithmes de maximisation typiques, on opte plutôt généralement pour des algorithmes de minimisation.
- Ainsi, pour maximiser une fonction $f(x)$, l'approche courante consiste à minimiser la fonction $(-f(x))$.
- Dans ce contexte, pour optimiser la vraisemblance de notre modèle, nous visons en réalité à minimiser la version négative de sa fonction.

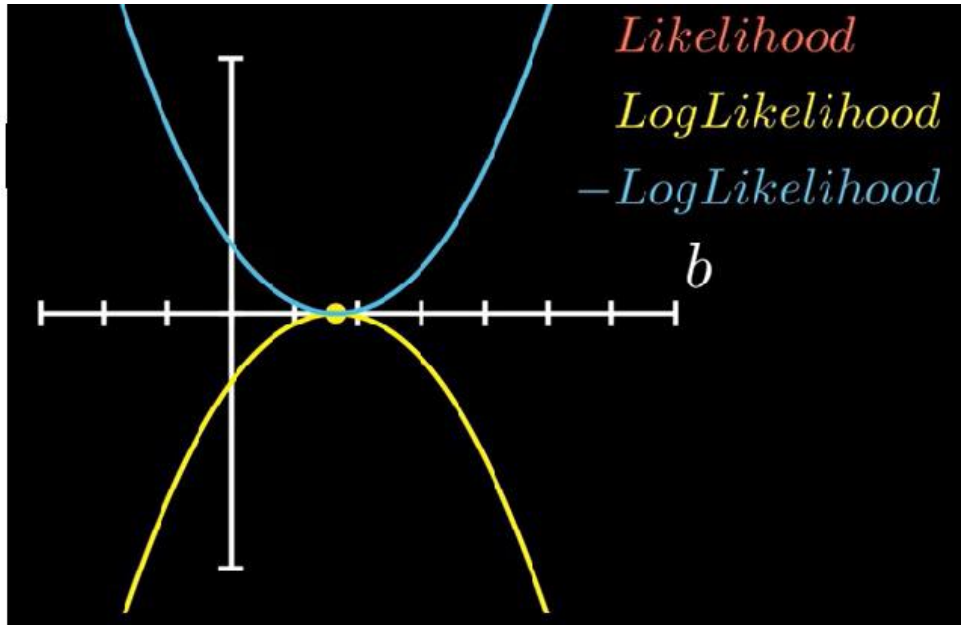
La Fonction coût

- $Maximiser(f\ (x)) = Minimiser(-f\ (x))$ ce qui va nous donner la Fonction **Log Loss**

$$L = -\frac{1}{m}\sum_{i=1}^{m}(y_i\, Log(\sigma_i) + (1 - y_i)Log(1 - \sigma_i))$$

- le facteur $\frac{1}{m}$ c'est juste un facteur pour normaliser le résultat





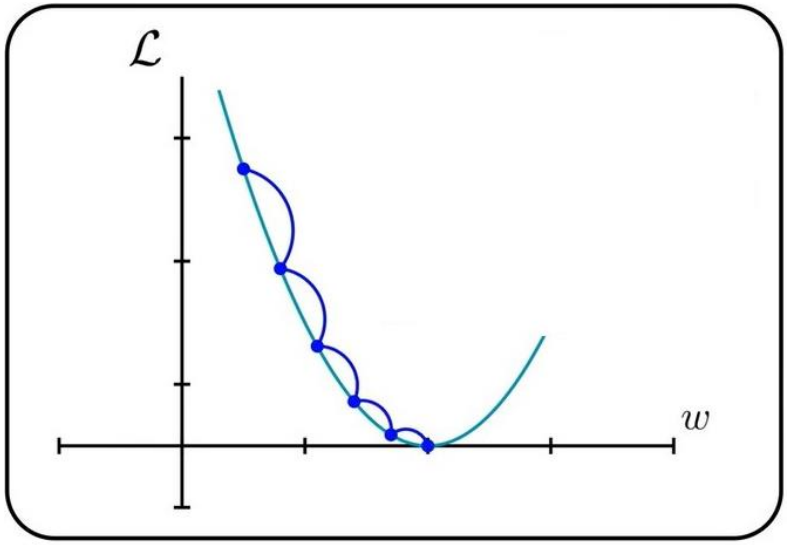
Descente de Gradient

- Consiste à ajuster les paramètres W et b de façon à minimiser les erreurs du modèle, c'est-à-dire à minimiser le Fonction Coût (Log Loss).
- Pour ça, il faut déterminer comment est-ce que cette fonction varie en fonction des différentes paramètres.
- C'est pourquoi on calcule le Gradient (ou la dérivée) de le Fonction Coût.
- En Mathématique, la dérivée d'une fonction indique comment cette fonction varie :
  - Si la dérivée $\frac{\partial L}{\partial w} < 0$ est négative, la fonction diminue quand w augmente, donc il faut augmenter w pour réduire l'erreur
  - Si la dérivée $\frac{\partial L}{\partial w} > 0$ est positive, la fonction augmente quand w augmente, donc il faut réduire w pour réduire l'erreur

Descente de Gradient

- Pour cela en utilise la fonction suivante :

$$w_{t+1} = w_t - \alpha \frac{\partial L}{\partial w_t}$$



ou

- w_{t+1}: Paramètre w à l'instant t+1
- w_t : Paramètre w à l'instant t+1
- α : Pas d'apprentissage positive
- $\frac{\partial L}{\partial w_t}$: Gradient à l'instant t

- En répétant cette formule en boucle, on dit ainsi capable d'atteindre le minimum de la fonction coût en descendant progressivement sa courbe d'où le terme de **descente de gradient.**
- Pour que cela marche, il faut que la fonction soit convexe (pas de minimum local).
- La fonction Log Loss est une fonction convexe.

Descente de Gradient

- Le gradient peut être décomposé en une chaine de dérivées partielles
  - $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \sigma} \times \frac{\partial \sigma}{\partial z} \times \frac{\partial z}{\partial w_1}$
  - $\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \sigma} \times \frac{\partial \sigma}{\partial z} \times \frac{\partial z}{\partial w_2}$
  - $\frac{\partial L}{\partial b} = \frac{\partial L}{\partial \sigma} \times \frac{\partial \sigma}{\partial z} \times \frac{\partial z}{\partial b}$

Rappel des fonctions de classification, activation et coût

$$z = w_1 . x_1 + w_2 . x_2 + b.$$

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

$$L = -\frac{1}{m} \sum_{i=1}^{m} (y_i \, Log(\sigma_i) + (1 - y_i) Log(1 - \sigma_i))$$

Descente de Gradient

- Dérivée de L par rapport à σ

$$\frac{\partial L}{\partial \sigma} = -\frac{1}{m}\sum_{i=1}^{m}\frac{y_i}{\sigma_i} - \frac{1-y_i}{1-\sigma_i}$$
$$= -\frac{1}{m}\sum_{i=1}^{m}\frac{y_i(1-\sigma_i) - (1-y_i)\sigma_i}{\sigma_i(1-\sigma_i)}$$
$$= -\frac{1}{m}\sum_{i=1}^{m}\frac{y_i-\sigma_i}{\sigma_i(1-\sigma_i)}$$

- Dérivée de z par rapport à w1, w2 et b

$$\frac{\partial z}{\partial w_1} = x_1$$
$$\frac{\partial z}{\partial w_2} = x_2$$
$$\frac{\partial z}{\partial b} = 1$$

Dérivée de a par rapport à z

$$\frac{\partial \sigma}{\partial z} = \frac{-e^{-z}}{-\left(1+e^{-z}\right)^2}$$
$$= \frac{e^{-z}}{\left(1+e^{-z}\right)^2}$$
$$= \frac{1}{1+e^{-z}} \times \frac{e^{-z}}{1+e^{-z}}$$
$$= \sigma \times \frac{1-1+e^{-z}}{1+e^{-z}}$$
$$= \sigma \times \left(\frac{1+e^{-z}}{1+e^{-z}} - \frac{1}{1+e^{-z}}\right)$$
$$= \sigma(1-\sigma)$$

Descente de Gradient

- Calcul du gradient par rapport à w_1

$$\frac{\partial L}{\partial w_1} = -\frac{1}{m}\sum_{i=1}^{m}\frac{y_i - \sigma_i}{\sigma_i(1-\sigma_i)} \times \sigma_i(1-\sigma_i) \times x_1$$
$$= -\frac{1}{m}\sum_{i=1}^{m}(y_i - \sigma_i)x_1$$
$$= \frac{1}{m}\sum_{i=1}^{m}(\sigma_i - y_i)x_1$$

- On obtient le gradient par rapport à w2 et b en remplaçant x1 par x2 et 1

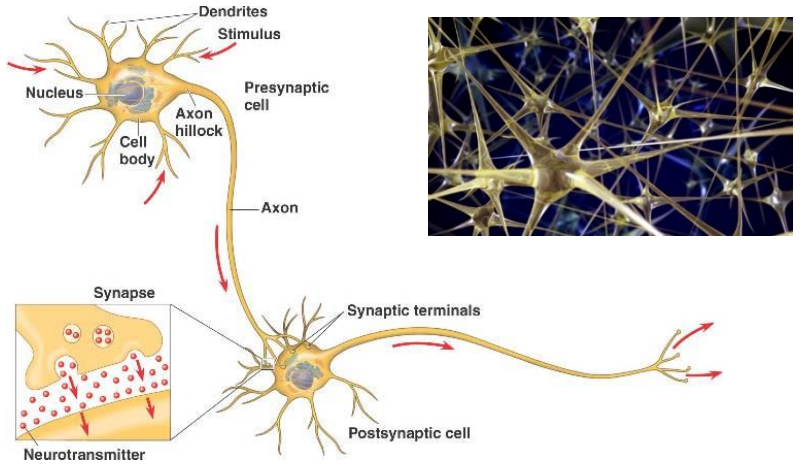
- Gradients

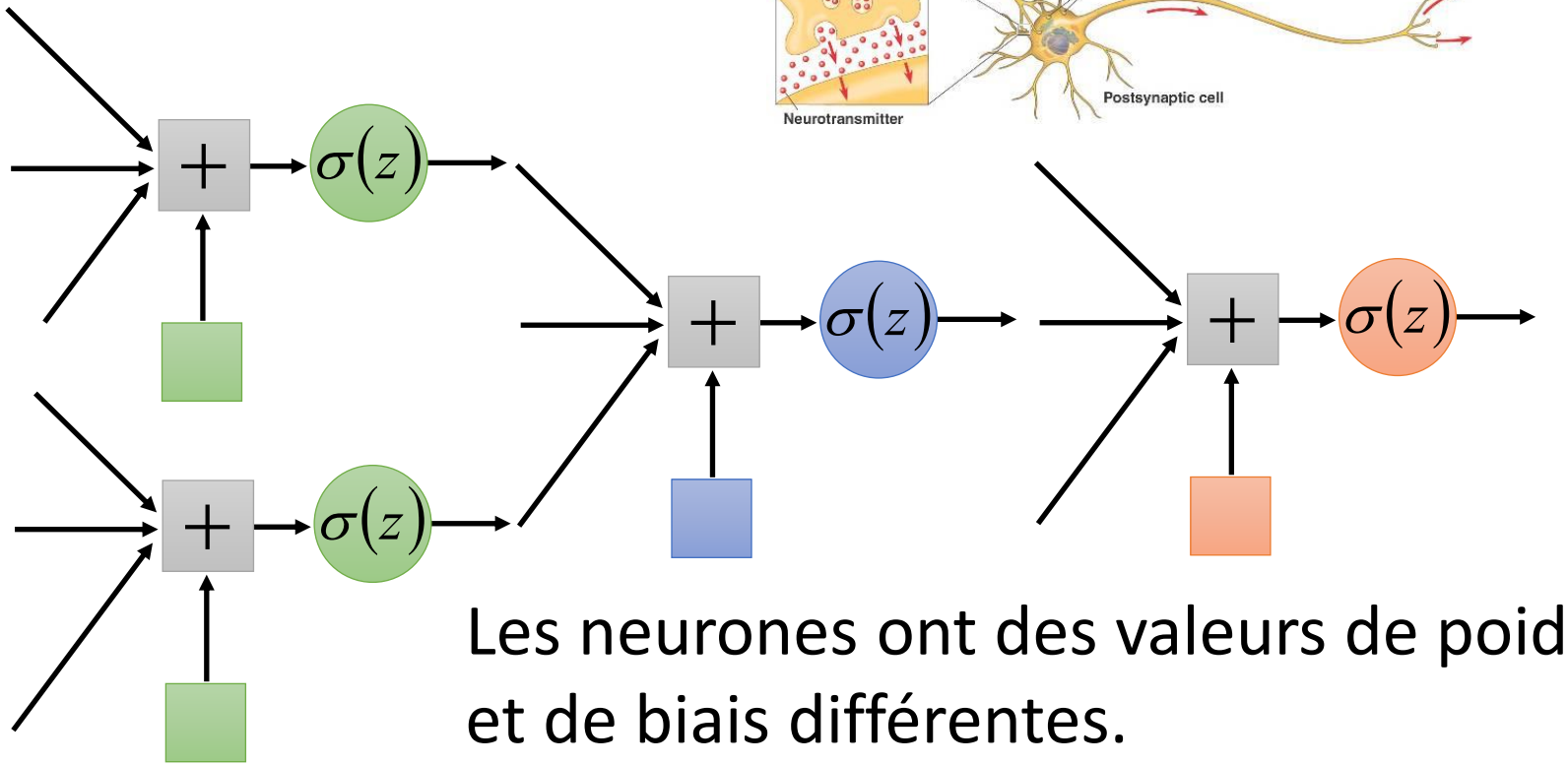
$$\frac{\partial L}{\partial w_1} = \frac{1}{m}\sum_{i=1}^{m}(\sigma_i - y_i)x_1$$
$$\frac{\partial L}{\partial w_2} = \frac{1}{m}\sum_{i=1}^{m}(\sigma_i - y_i)x_2$$
$$\frac{\partial L}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(\sigma_i - y_i)$$

Perceptrons multicouches

Perceptrons multicouches

Différentes connexions conduisent à différentes structures de réseau

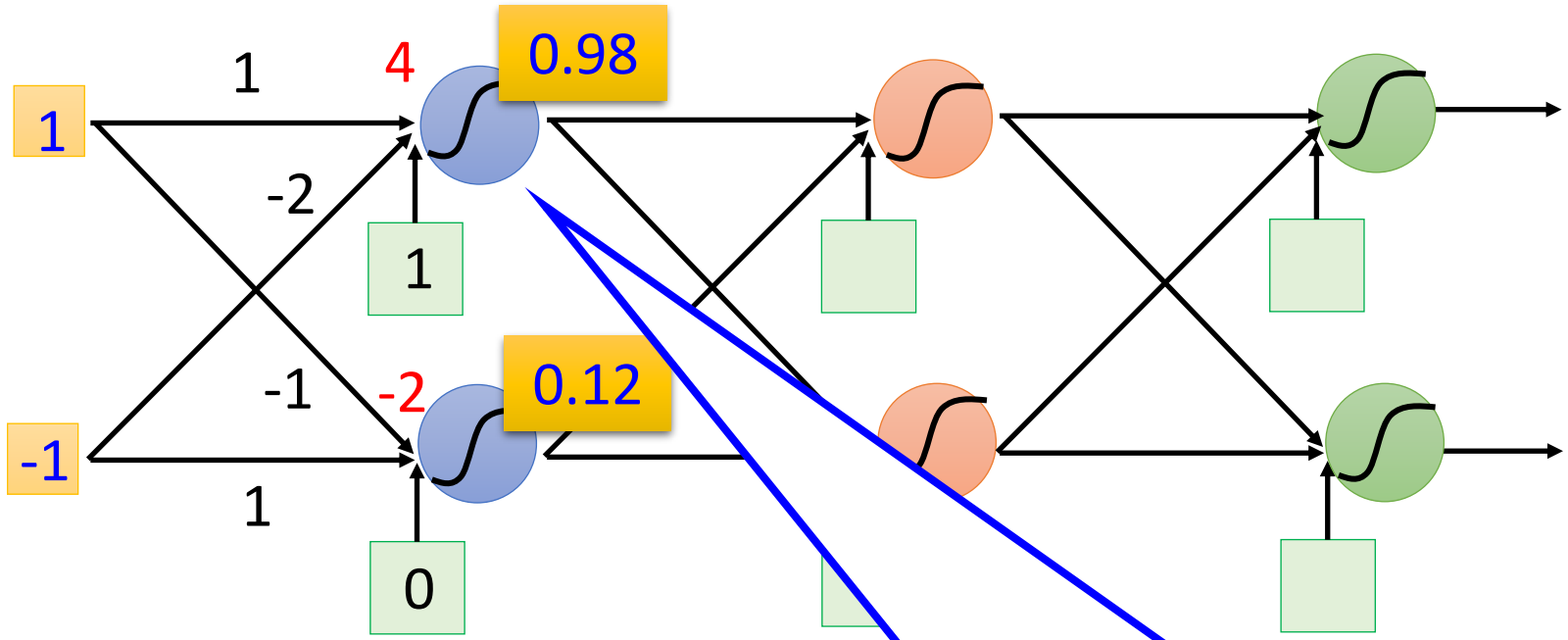




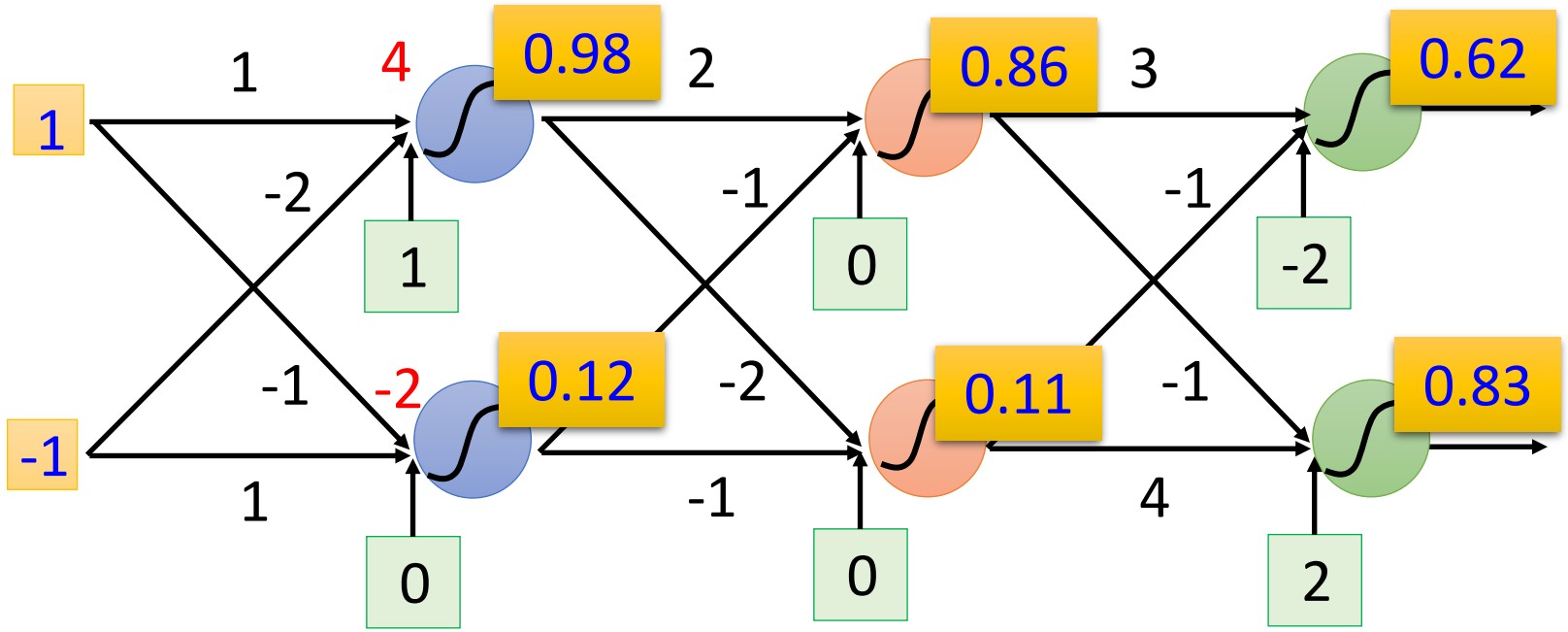
Les neurones ont des valeurs de poids et de biais différentes.

Les poids et les biais sont des paramètres de réseau θ

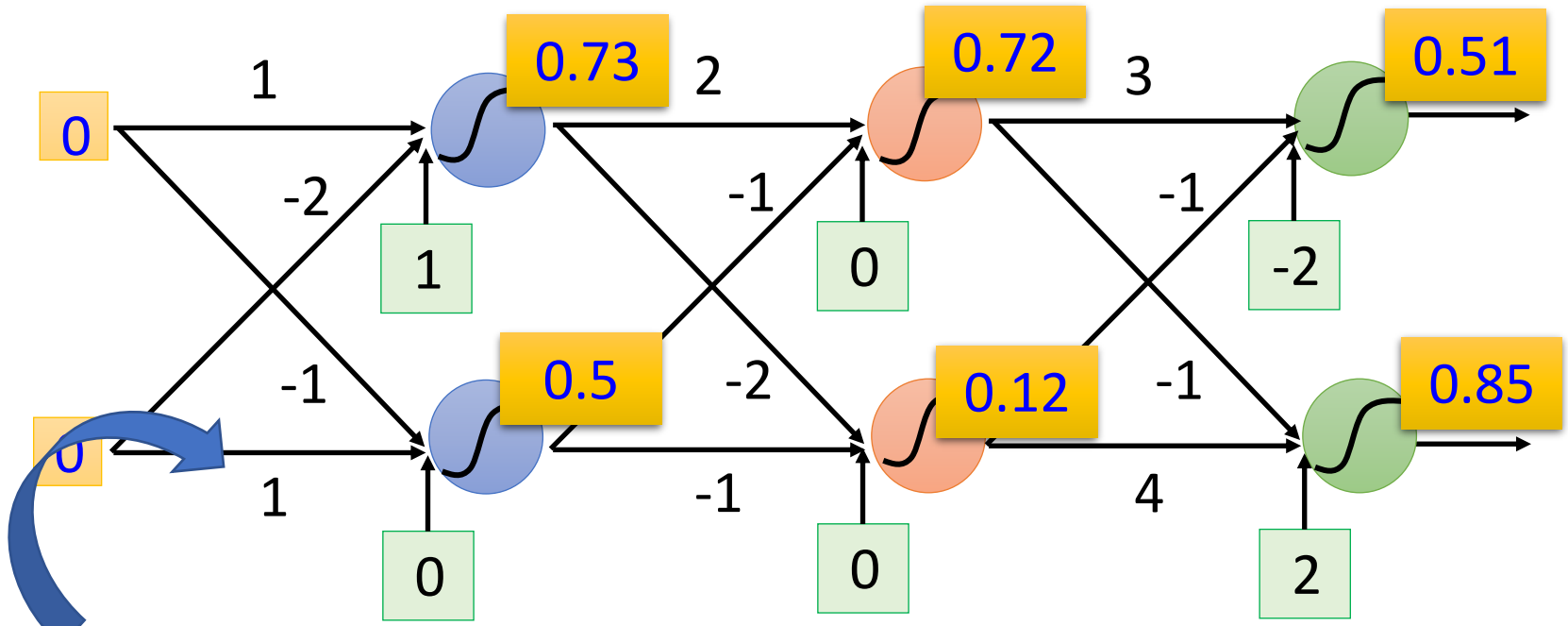
Perceptrons multicouches



Perceptrons multicouches



Perceptrons multicouches



C'est une function f

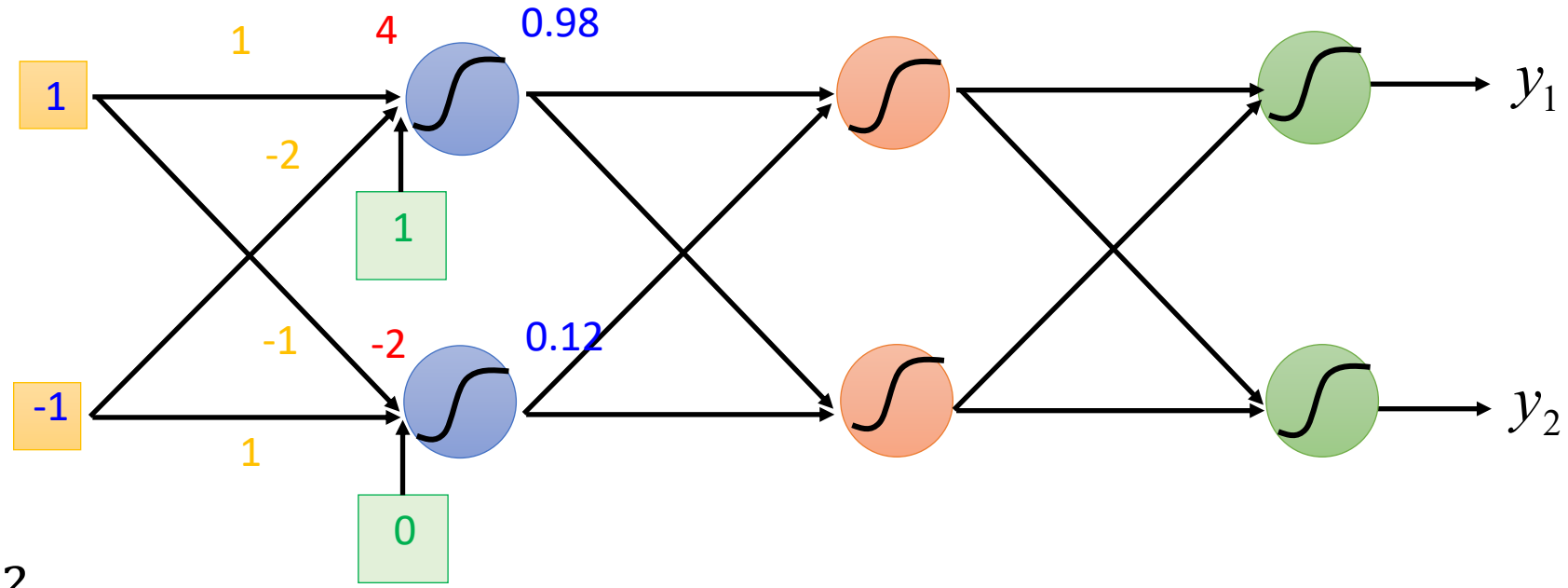
Vecteur d'entrée, vecteur de sortie

$$f\left(\begin{bmatrix}1\\-1\end{bmatrix}\right)=\begin{bmatrix}0.62\\0.83\end{bmatrix} \quad f\left(\begin{bmatrix}0\\0\end{bmatrix}\right)=\begin{bmatrix}0.51\\0.85\end{bmatrix}$$

Différents paramètres mènent à différentes fonctions, comment définir ces paramètres?

Vectorisation des calculs

- Les opérations matricielles sont utiles lorsque vous travaillez avec des entrées et des sorties multidimensionnelles



$$f: R^2 \rightarrow R^2$$

$$\sigma\left(\begin{bmatrix} 1 & -2 \\ -1 & 1 \end{bmatrix}\begin{bmatrix} 1 \\ -1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \end{bmatrix}\right) = \begin{bmatrix} 0.98 \\ 0.12 \end{bmatrix}$$

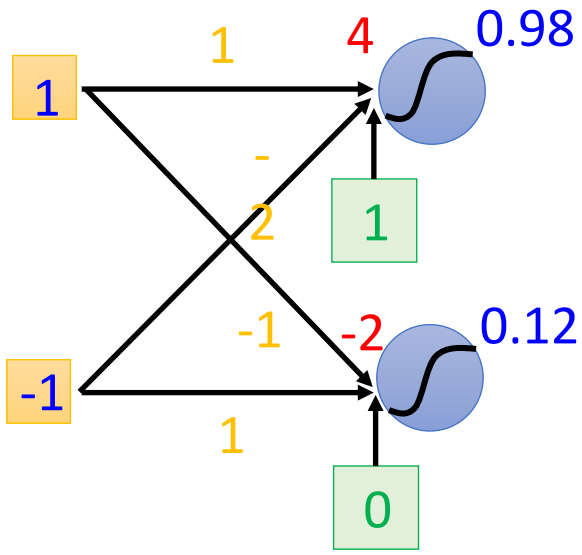
Sigmoide

$$\begin{bmatrix} 4 \\ -2 \end{bmatrix}$$

Calcul à répéter pour toutes les couches!

Perceptrons multicouches

- Les opérations matricielles sont utiles lorsque vous travaillez avec des entrées et des sorties multidimensionnelles

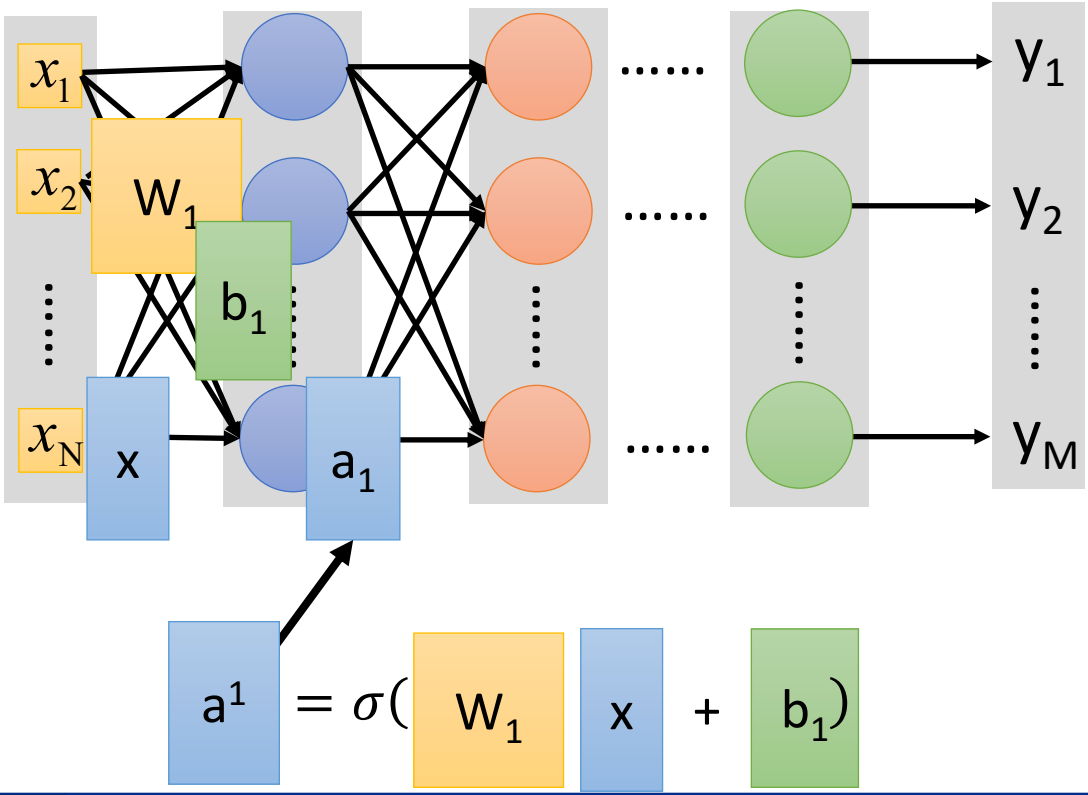


$$\sigma\left(\begin{bmatrix}1 & -2\\ -1 & 1\end{bmatrix}\begin{bmatrix}1\\ -1\end{bmatrix}+\begin{bmatrix}1\\ 0\end{bmatrix}\right)=\begin{bmatrix}0.98\\ 0.12\end{bmatrix}$$

$$\begin{bmatrix}4\\ -2\end{bmatrix}$$

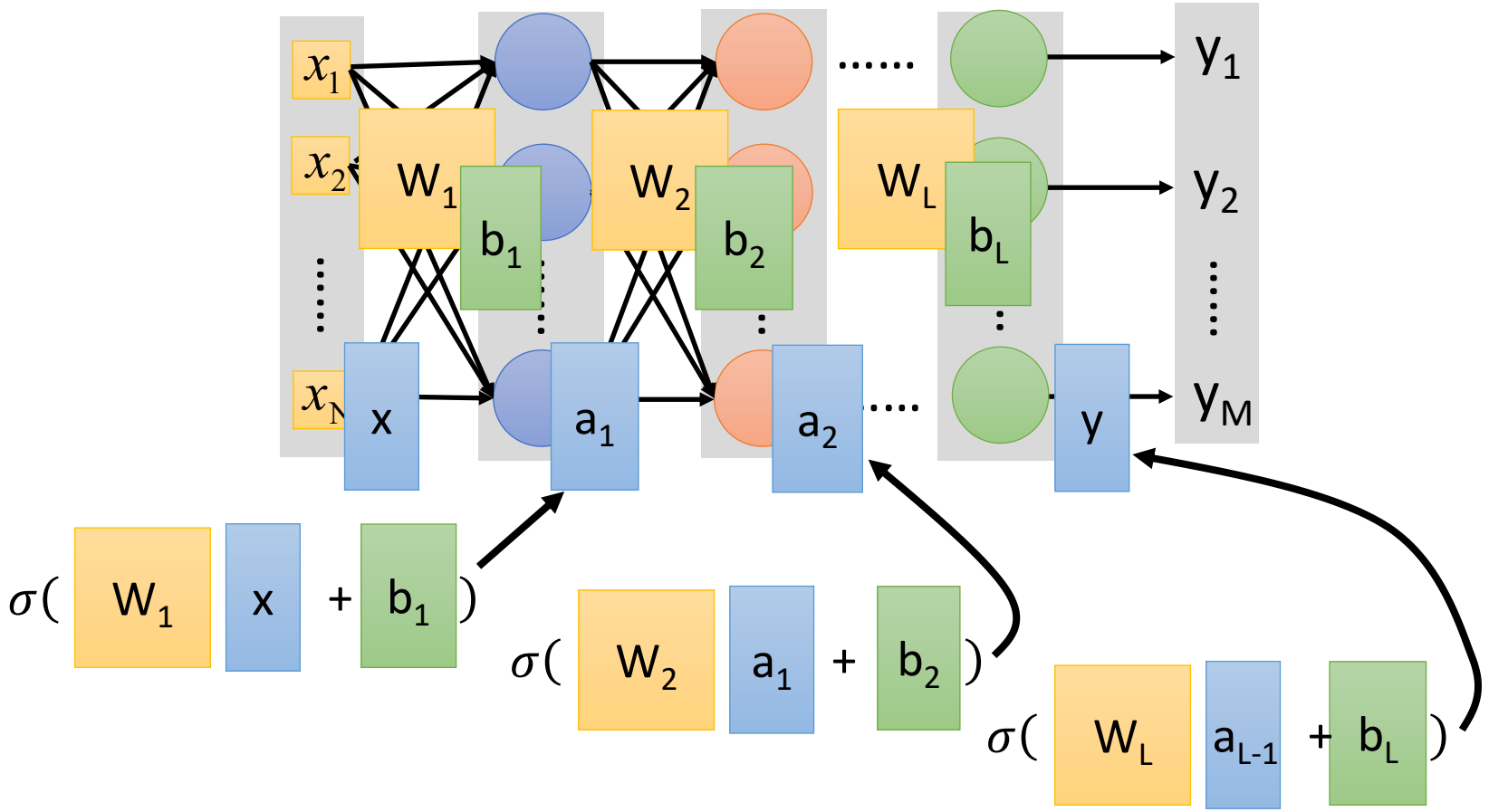
Vectorisation des calculs

- Réseau neuronal multicouche, calculs matriciels pour la première couche
  - Input vector x, weights matrix W_1, bias vector b_1, output vector a_1



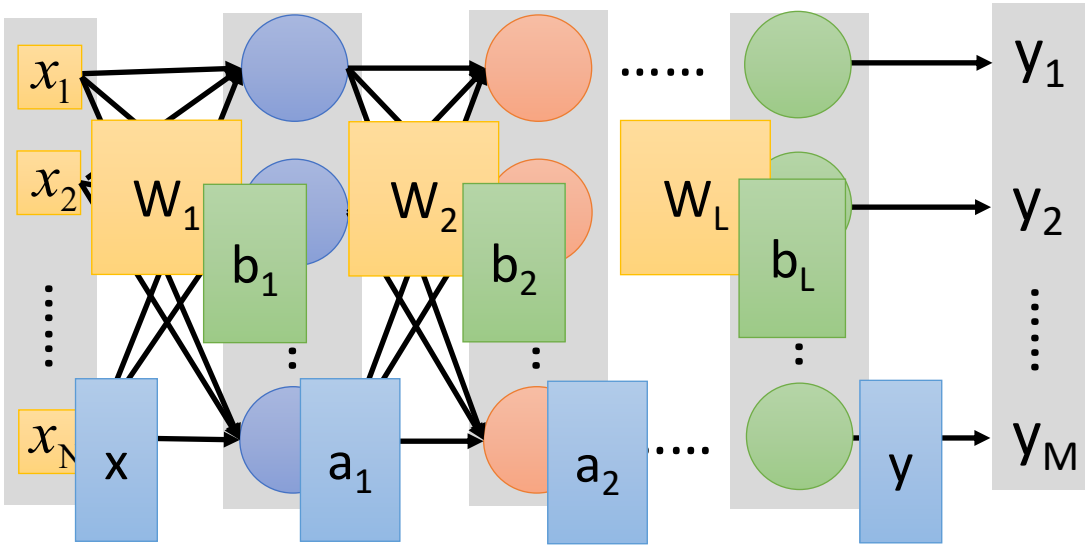
Vectorisation des calculs

- Réseau neuronal multicouche, calculs matriciels pour la première couche



Vectorisation des calculs

- Réseau neuronal multicouche, la fonction f mappe les entrées x aux sorties y, c'est-à-dire $y=f(x)$

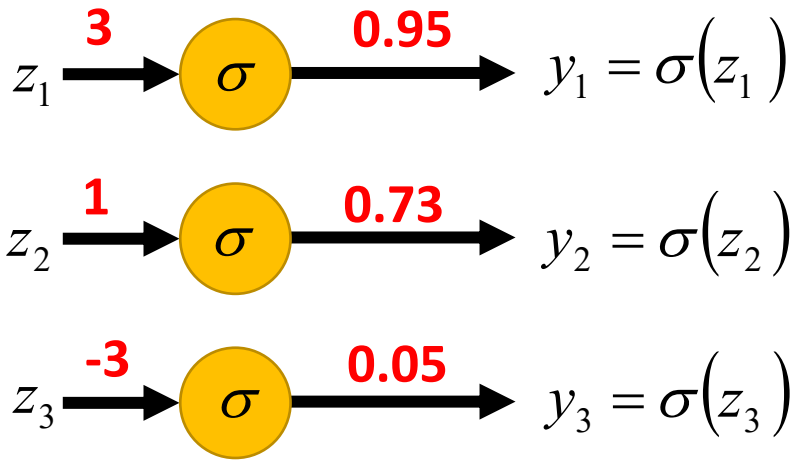


$$y = f(x) = \sigma(W_L \dots \sigma(W_2\, \sigma(W_1 x + b_1) + b_2) \dots + b_L)$$

Softmax Layer

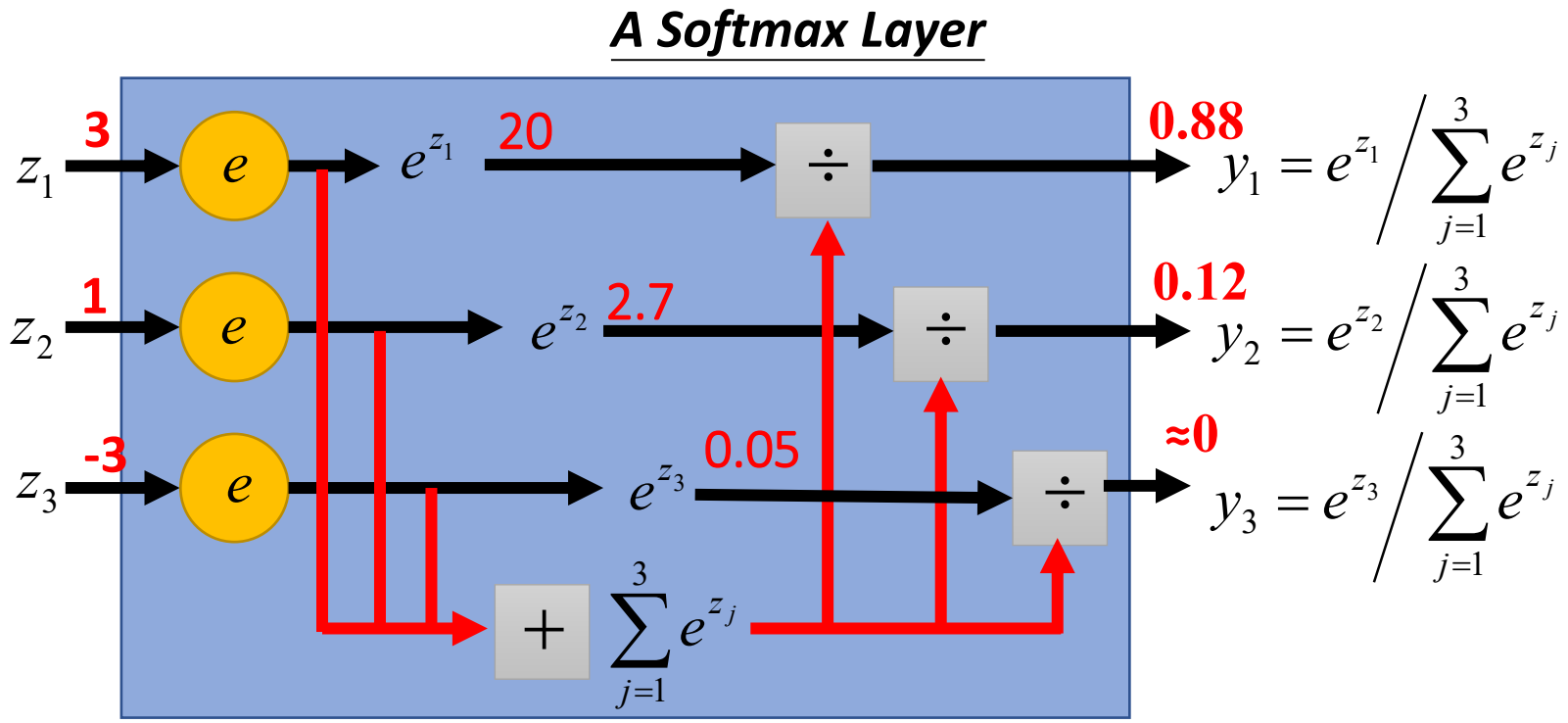
- Dans les tâches de **classification multi-classes**, la couche de sortie est généralement **une couche softmax**
  - On emploie une fonction d'**activation *softmax***
  - Si une couche avec une fonction d'activation sigmoïde est utilisée comme couche de sortie, les prédictions peuvent ne pas être faciles à interpréter
  - Notez qu'une couche de sortie avec des activations sigmoïdes peut toujours être utilisée pour la **classification binaire**

Une couche avec des activations sigmoïdes



Softmax Layer

- La **couche softmax** applique des **activations softmax** pour générer une valeur de probabilité dans la plage [0, 1]
- la fonction softmax transforme un vecteur $Z=(Z_1,Z_2,\ldots,Z_k)$ en un vecteur Y de K nombres réels strictement positifs et de somme 1. Les valeurs z entrées dans la couche softmax sont appelées logits



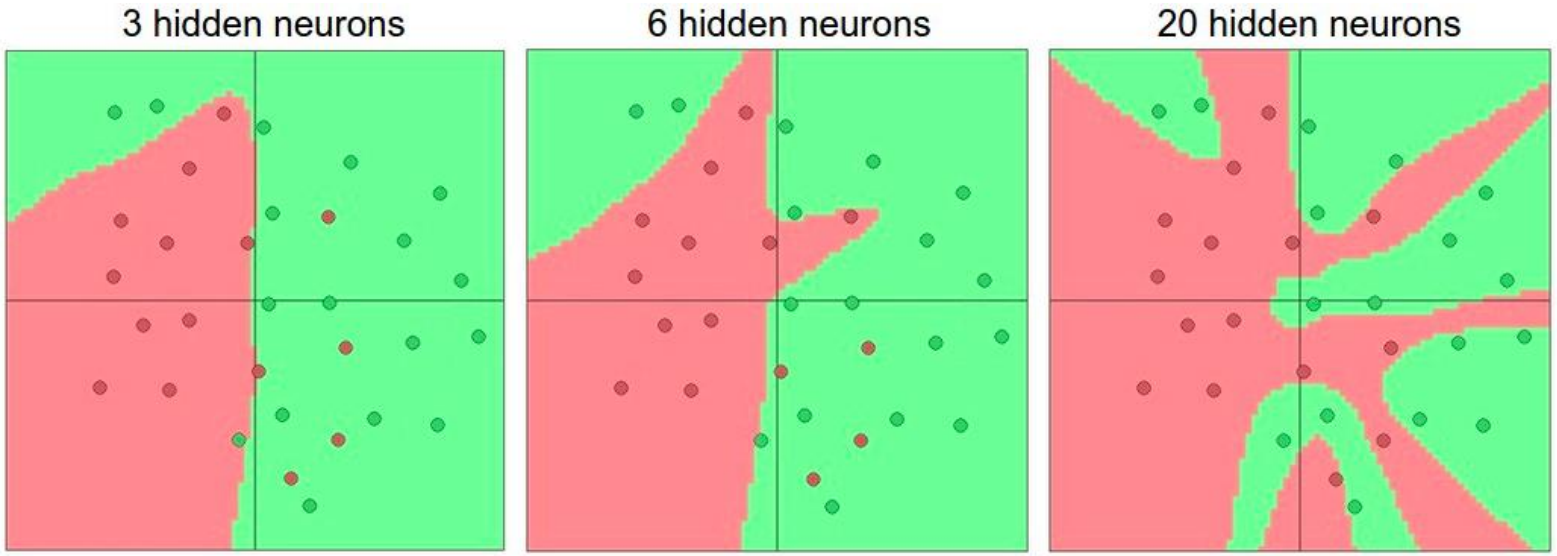
Probability:

- $0 < y_i < 1$
- $\sum_i y_i = 1$

0.88 +0.12 =1

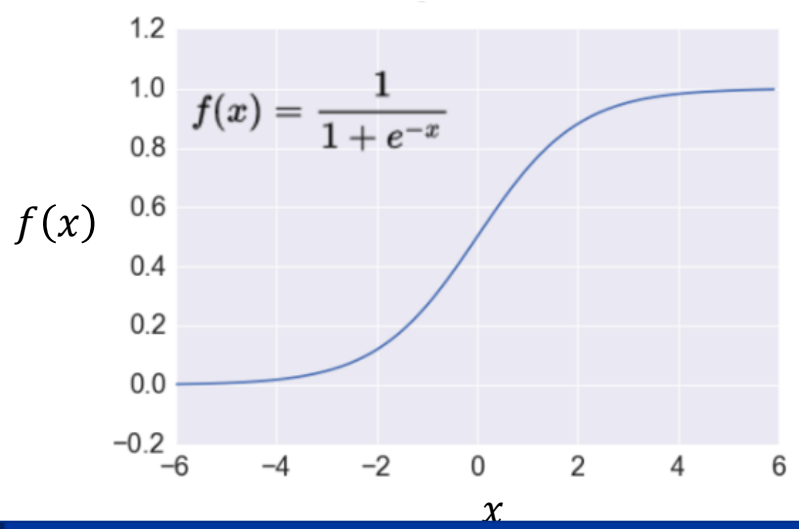
Fonctions d'activation

- Des activations non linéaires sont nécessaires pour apprendre des représentations de données complexes (non linéaires)
  - Sinon, les réseaux neuronaux ne seraient qu'une fonction linéaire(comme $W_1 W_2 x = Wx$)
  - Les réseaux neuronaux avec un grand nombre de couches (et de neurones) peuvent approximer des fonctions plus complexes
    - Figure : plus de neurones améliorent la représentation (mais peuvent être surajustés)



Activation: Sigmoid

- **Fonction Sigmoid σ: prend un nombre réel et le « réduit » dans la plage comprise entre 0 et 1**
  - La sortie peut être interprétée comme le taux de déclenchement d'un neurone biologique
    - Pas de déclenchement = 0 ; Déclenchement complet = 1
  - Lorsque l'activation du neurone est de 0 ou 1, les neurones sigmoïdes saturent
    - Les gradients dans ces régions sont presque nuls (presque aucun signal ne circule)
  - Les activations sigmoïdes sont moins courantes dans les réseaux de neurones modernes

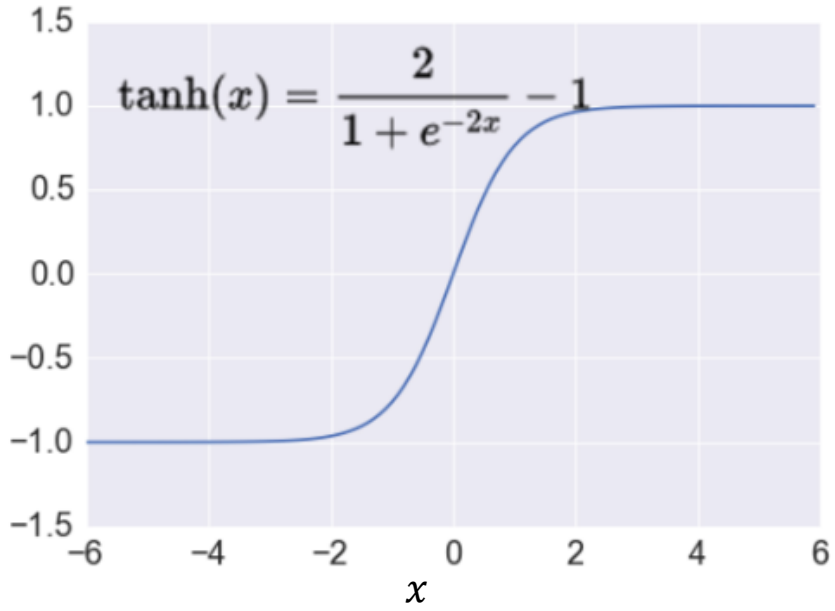


$\mathbb{R}^n \rightarrow [0,1]$

Activation: Tanh

- **Fonction Tanh (tangente hyperbolique) : prend un nombre réel et le « réduit » dans une plage comprise entre -1 et 1**
  - Comme la fonction sigmoïde, la fonction Tanh saturent les neurones
  - Contrairement à la fonction sigmoïde, la sortie est centrée sur zéro
  - Elle est donc préférée que la fonction sigmoïde
    - Tanh est une fonction sigmoïde mise à l'échelle : $\tanh(x)=2\cdot\sigma(2x)-1$

$f(x)$



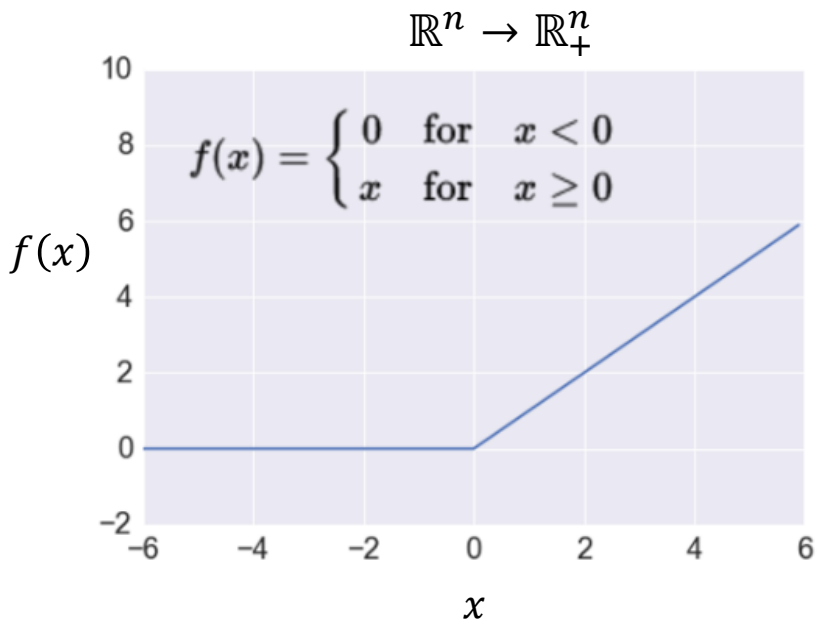
$\mathbb{R}^n \rightarrow [-1,1]$

Activation: ReLU

- ***ReLU*** (Rectified Linear Unit): **prend un nombre à valeur réelle et le limite à zéro**

$$f(x) = \max(0, x)$$

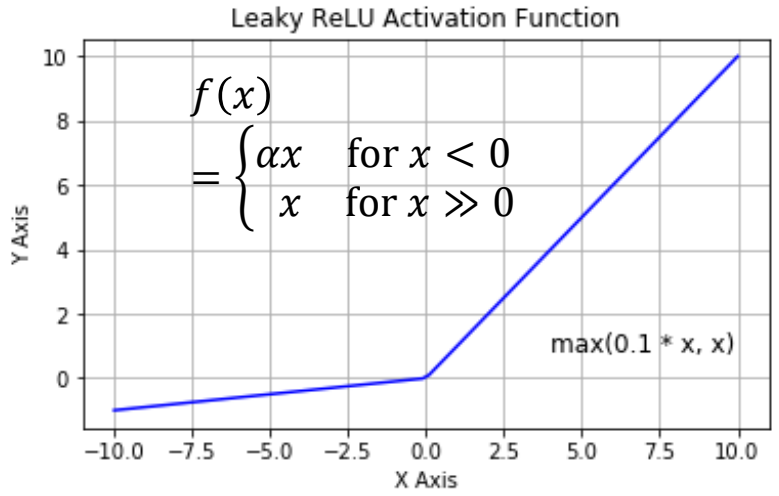
- La plupart des réseaux neuronaux profonds modernes utilisent des activations ReLU
- ReLU est rapide à calculer
  - Comparé à sigmoïde, tanh
  - Il suffit de seuiller une matrice à zéro
- Accélère la convergence de la descente de gradient
  - En raison de la forme linéaire et non saturante
- Évite le problème de disparition du gradient



Activation: Leaky ReLU

- Le problem de l'activation ReLU : ils peuvent « mourir ».
- ReLU pourrait provoquer une mise à jour des poids de telle sorte que les gradients puissent devenir nuls et que le neurone ne s'active plus sur aucune donnée
- ***Leaky ReLU :*** **activation function est une variante de ReLU**
  - Au lieu que la fonction soit égale à 0 lorsque x<0, une ReLU qui fuit a une petite pente négative (par exemple, α = 0,01 ou similaire)

- Cela résout le problème de la mort de ReLU
- La plupart des travaux actuels utilisent encore ReLU
  - Avec un réglage approprié du taux d'apprentissage, le problème de la mort de ReLU peut être évité



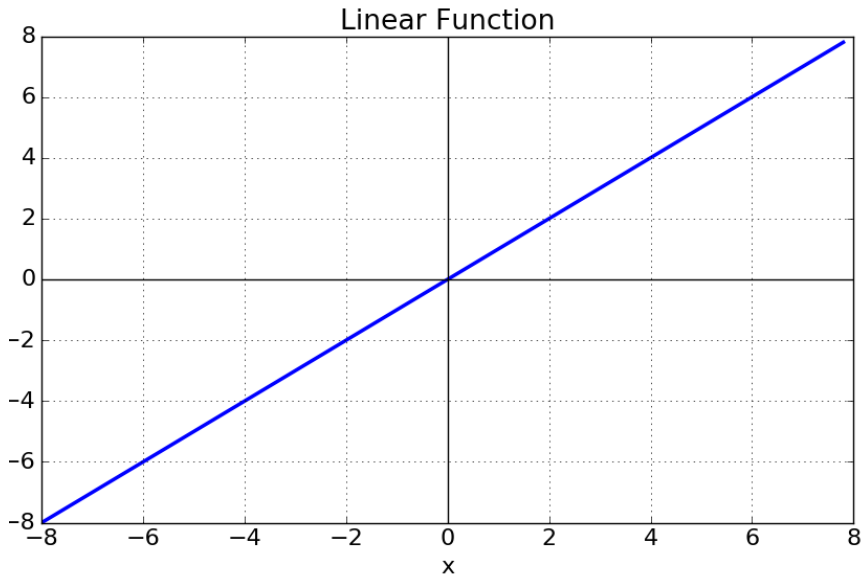
Activation: Linear Function

- ***Linear function*** **signifie que le signal de sortie est proportionnel au signal d'entrée du neurone**

$$\mathbb{R}^n \rightarrow \mathbb{R}^n$$

$$f(x) = cx$$

  - Si la valeur de la constante c est 1, elle est également appelée identity activation function
  - Ce type d'activation est utilisé dans les problèmes de régression
    - Par exemple, la dernière couche peut avoir une fonction d'activation linéaire, afin de générer un nombre réel (et non une appartenance à une classe)



Mini batch

Descente de gradient par lots

- La descente de gradient par lots utilise l'ensemble de l'entraînement pour calculer le gradient à chaque étape et mettre à jour les poids en conséquence, ce qui conduit à un minimum global de la fonction de perte.
- Inconvénients:
  - Il peut être lent et coûteux en calcul lorsqu'il s'agit de grands ensembles de données et de modèles complexes.
  - Il peut rester coincé dans des minima locaux peu profonds ou des points de selle, où la pente est très faible ou nulle.
  - Il ne permet pas l'apprentissage en ligne ou incrémental, ce qui signifie que de nouvelles données ne peuvent pas être ajoutées à l'ensemble d'apprentissage sans redémarrer l'algorithme.

Mini batch

Descente de gradient par mini-lots (mini batch)

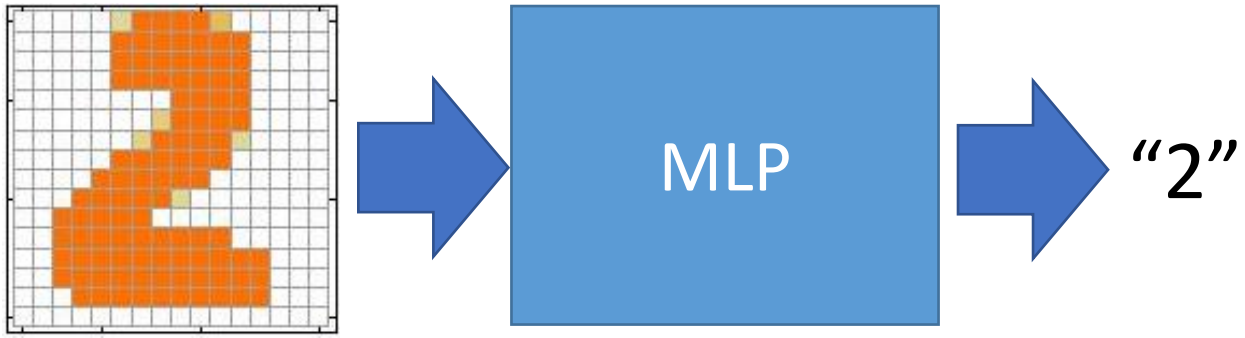
- La descente de gradient par mini-lot est une méthode d'optimisation qui utilise un sous-ensemble de l'ensemble d'apprentissage, appelé mini-lot, pour calculer le gradient à chaque étape et mettre à jour les poids en conséquence.
- La taille du mini-lot peut varier, mais elle se situe généralement entre 16 et 256 exemplaires.
- Cette approche présente plusieurs avantages, tels que la réduction du nombre de passages sur les données, l'introduction de bruit et de caractère aléatoire dans le gradient et la possibilité d'un apprentissage en ligne ou incrémentiel,
- elle peut également être moins stable et plus sensible au choix du taux d'apprentissage et de la taille des mini-lots.
- il peut osciller autour du point optimal au lieu de l'atteindre exactement, et nécessite plus de réglage pour trouver les meilleurs paramètres et hyperparamètres.

Variantes de la Descente de Gradient

- **Descente de gradient par batch (Batch Gradient Descent)** : Utilise l'ensemble complet des données d'entraînement pour calculer le gradient de la fonction de coût par rapport aux paramètres du modèle.
  - **Avantages** : Convergence vers le minimum global dans des problèmes convexes.
  - **Inconvénients** : Requiert beaucoup de mémoire pour traiter l'ensemble de données complet en une seule itération. Peut être lent pour de grandes quantités de données.
- **Descente de gradient stochastique (Stochastic Gradient Descent, SGD)** : La mise à jour des paramètres est effectuée pour chaque exemple de données individuellement.
  - **Avantages** : Réduit la charge de calcul par itération. Peut converger plus rapidement en raison des mises à jour fréquentes des paramètres.
  - **Inconvénients** : Variabilité accrue dans la direction de la mise à jour des paramètres, ce qui peut ralentir la convergence. Moins stable que la descente de gradient classique.
- **Descente de gradient par mini-lots (Mini-Batch Gradient Descent)** : L'ensemble de données est divisé en petits lots, et la mise à jour des paramètres est effectuée sur chaque lot.
  - **Avantages :** Combine les avantages du batch gradient descent (stabilité) et du stochastic gradient descent (efficacité de calcul). Adapté pour une mise à jour fréquente des paramètres tout en gérant efficacement la mémoire
  - **Inconvénients :** Nécessite un réglage du taux d'apprentissage plus délicat pour optimiser la convergence

Example d’application

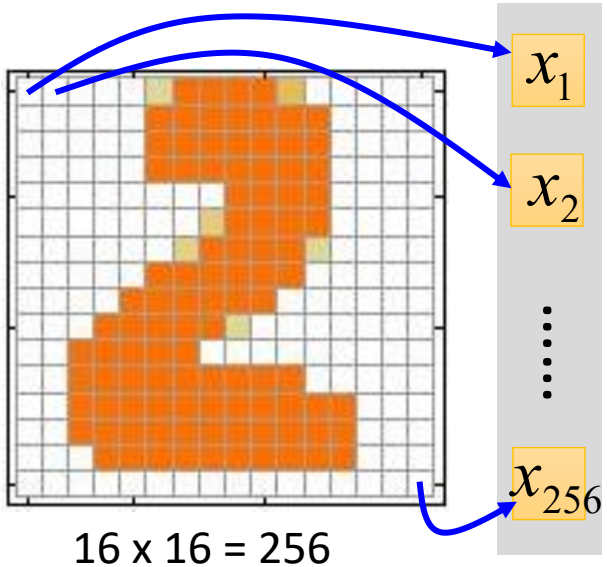
- Reconnaissance des chiffres manuscripts (Yann LeCun)



- MLP :Perceptrons Multicouches

Exemple d'application : Reconnaissance des chiffres manuscrits

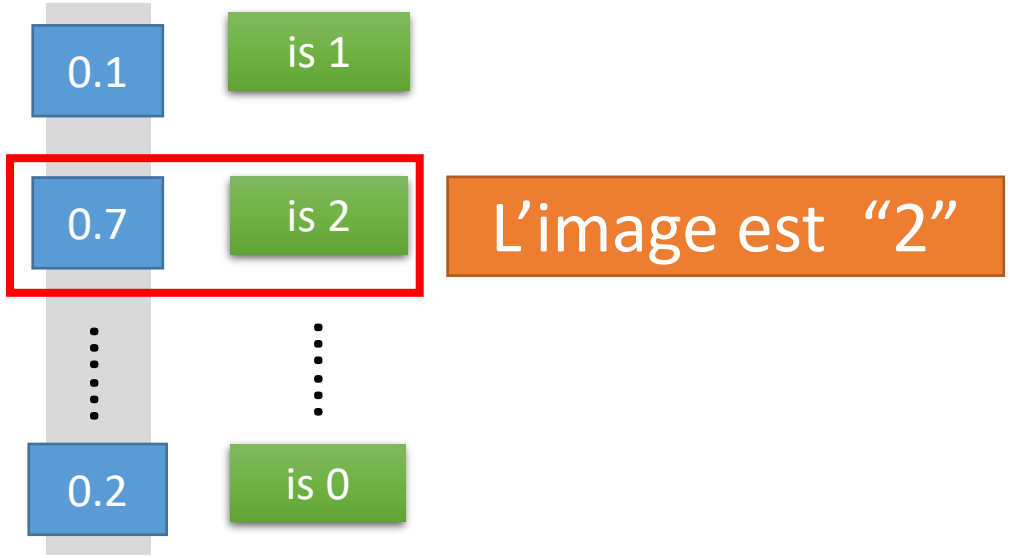
- **Entrées**



16 x 16 = 256

Pixel non vide → 1
Pixel vide → 0

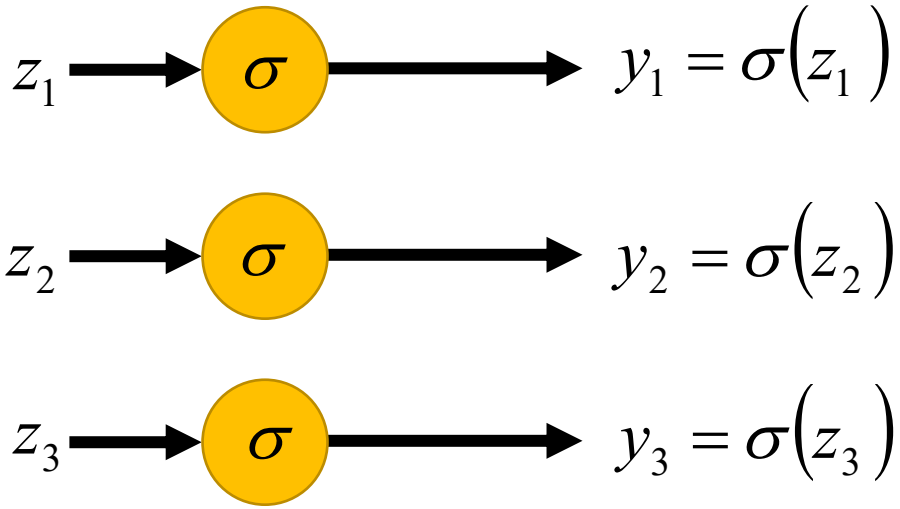
- **Sorties**



Chaque dimension donne la confiance en un résultat

Softmax

- Couche de sortie ordinaire

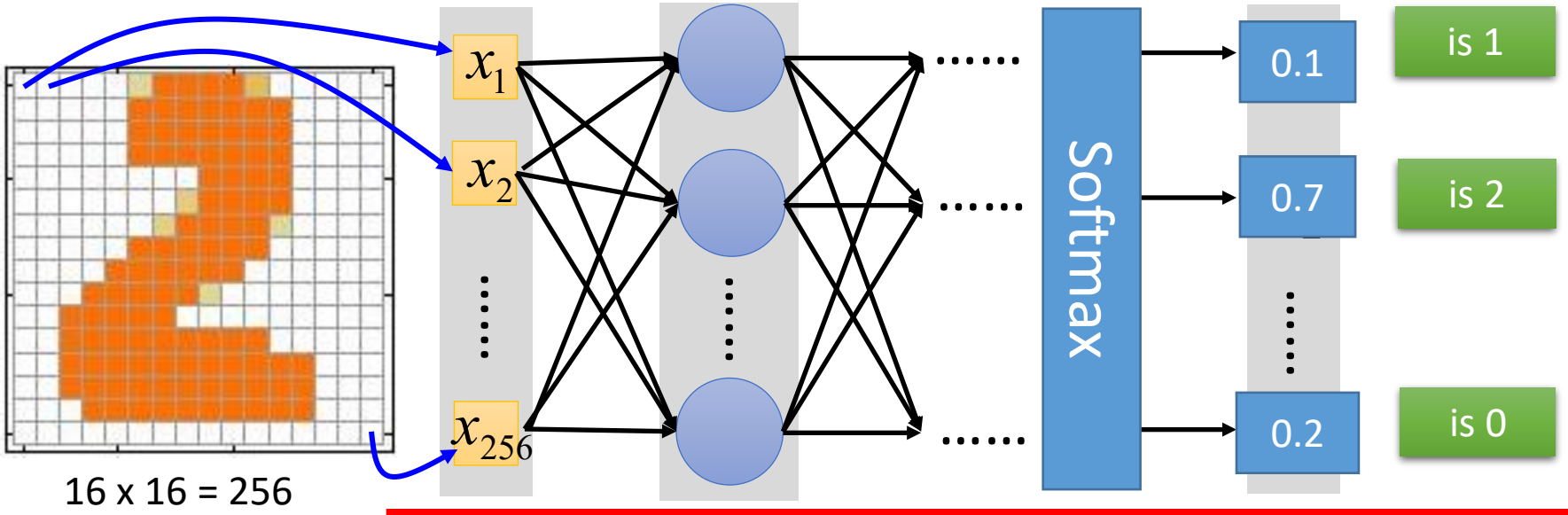


- En général, la somme des sorties du MLP est arbitraire
- Les valeurs de sortie peuvent être difficiles à interpréter

- Une couche de sortie *Softmax* peut aider

Réglage des paramètres du MLP

$$\theta = \{W^1, b^1, W^2, b^2, \cdots W^L, b^L\}$$



16 x 16 = 256

Pixel non vide → 1

Pas vide → 0

Il faut règler θ tel que …

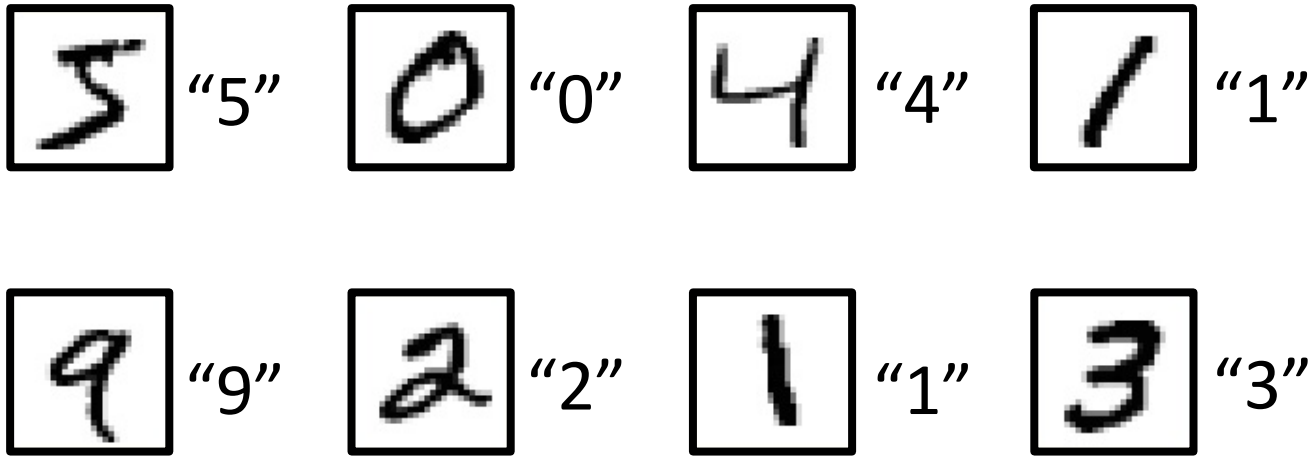
Entrée:

Entrée :

Comment ?

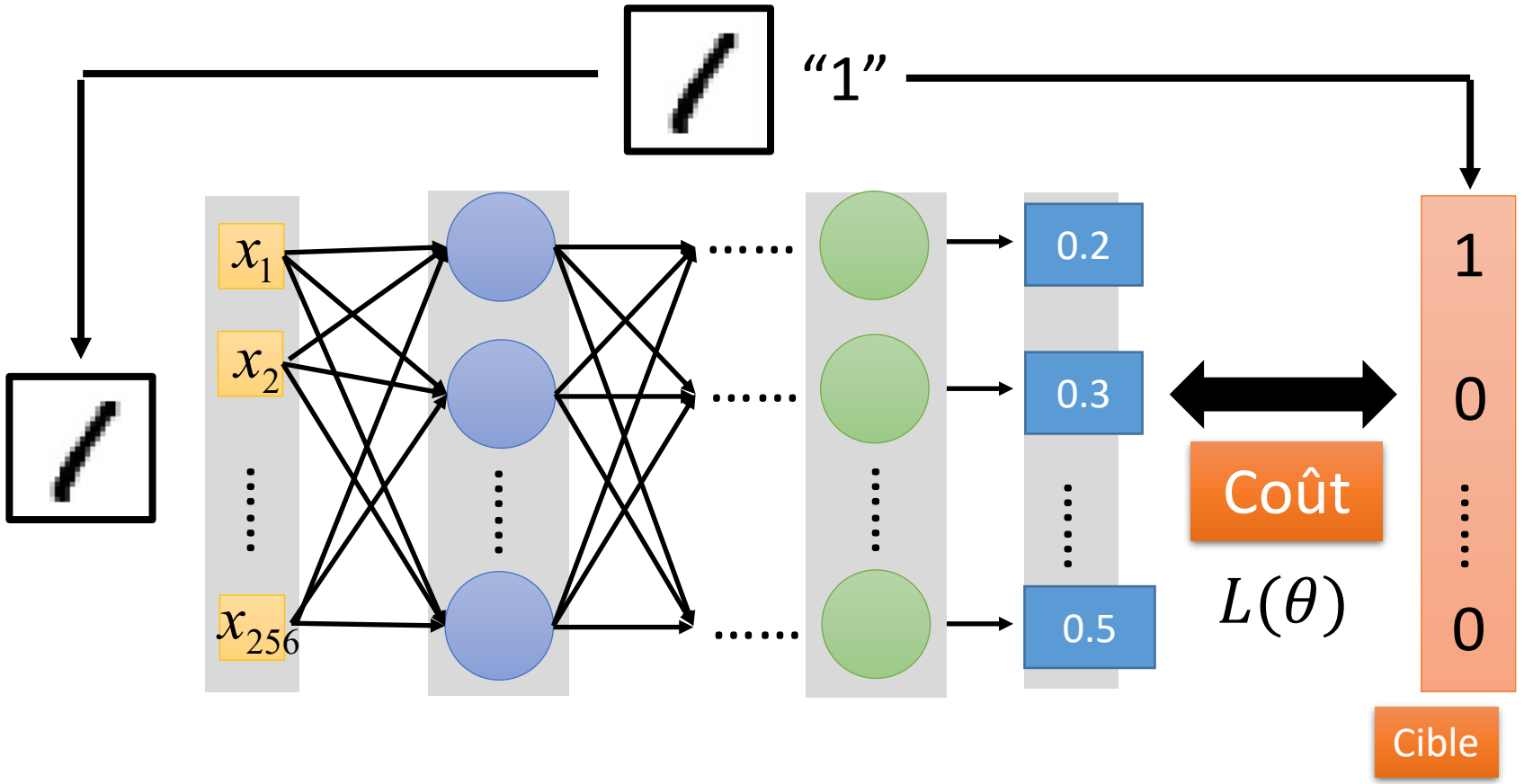
Préparation de l'apprentissage

- On convertit les images en vecteurs de pixels représentatifs et on associe un label à chaque vecteur



Coût

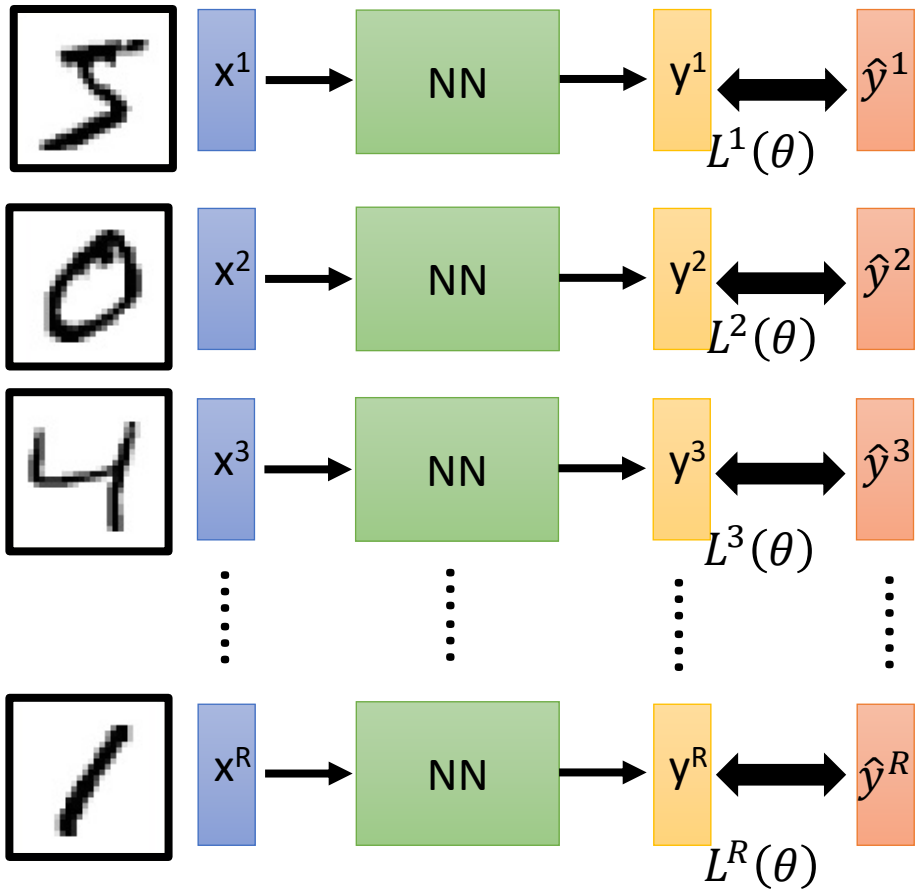
Chaque ensemble de paramètres θ mène à différents coûts de classification



Le coût d'une classification peut être toute mesure de l'écart entre les vecteurs de sortie voulu et obtenu

Coût total

Calculé pour l'ensemble des données …



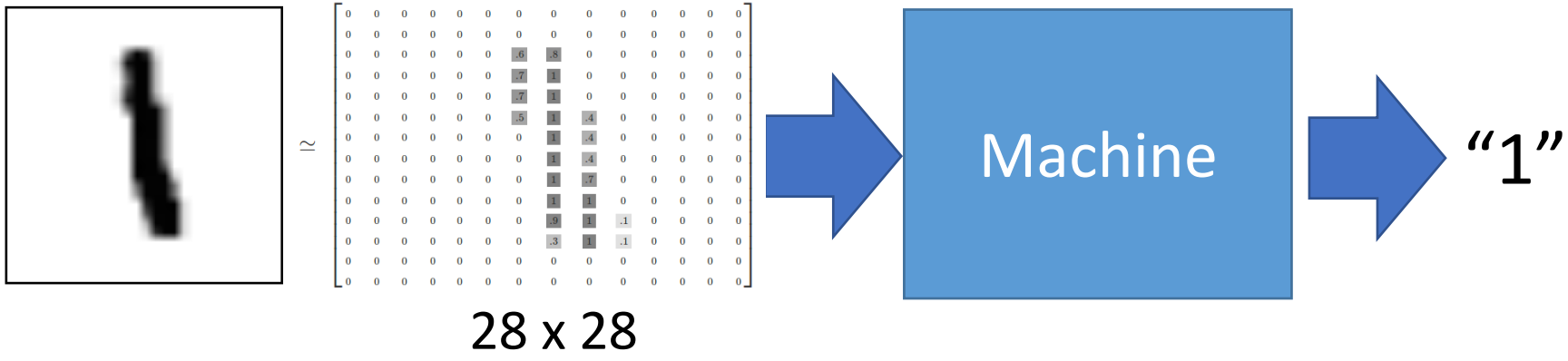
Coût total :

$$C(\theta) = \sum_{r=1}^{R} L^r(\theta)$$

Comment trouver l'ensemble de paramètres θ qui minimise $C(\theta)$?

Example Application

- Reconnaissance des chiffres manuscrits

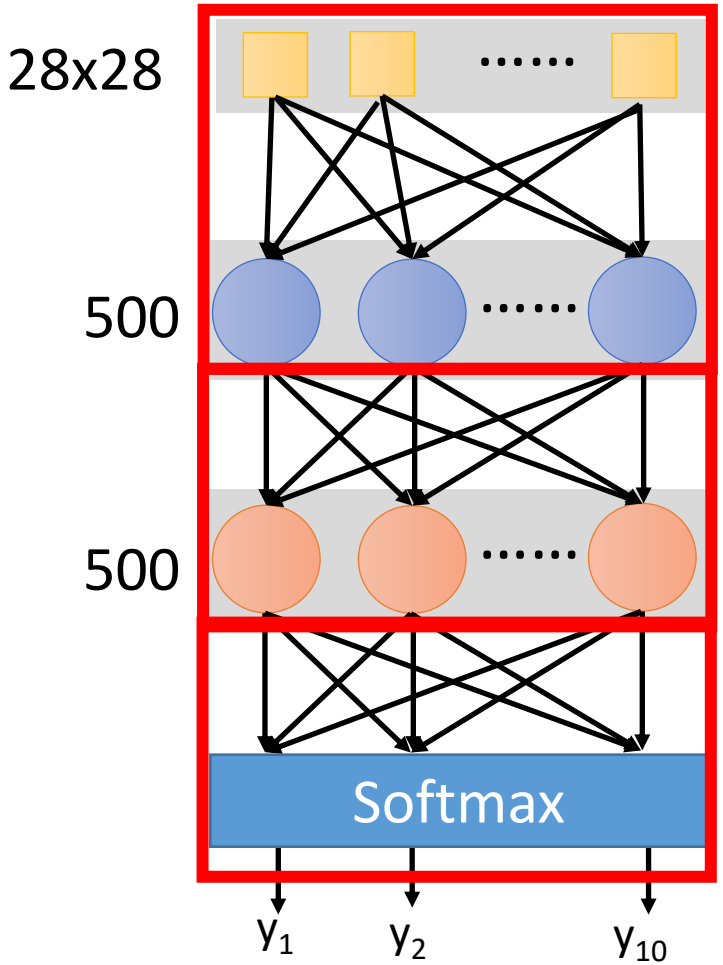


MNIST Data: http://yann.lecun.com/exdb/mnist/

Keras fournit une fonction de chargement de jeux de données :
http://keras.io/datasets/

Keras

Step 1: define a set of function → Step 2: goodness of function → Step 3: pick the best function



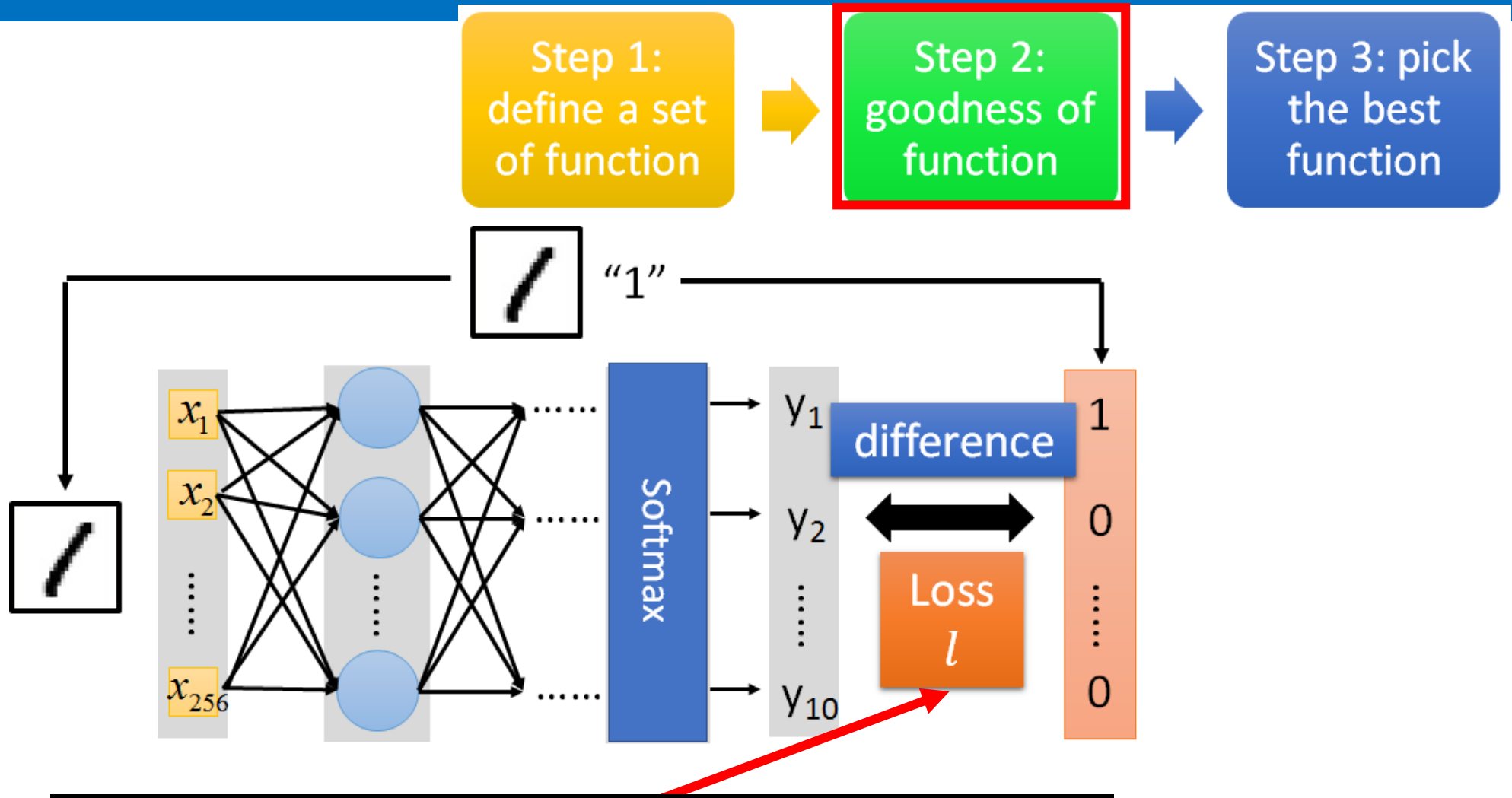

```
model = Sequential()
```

```
model.add( Dense( input_dim=28*28,
                  output_dim=500 ))
model.add( Activation('sigmoid') )
```

```
model.add( Dense( output_dim=500 ) )
model.add( Activation('sigmoid') )
```

```
model.add( Dense(output_dim=10 ) )
model.add( Activation('softmax') )
```

Keras


Step 1: define a set of function
Step 2: goodness of function
Step 3: pick the best function
"1"
x_1
x_2
x_{256}
Softmax
y_1
y_2
y_{10}
difference
Loss
l
1
0
0
model.compile(loss='mse',
optimizer=SGD(lr=0.1),
metrics=['accuracy'])

Keras



Step 3.1: Configuration

```
model.compile(loss='mse',
              optimizer=SGD(lr=0.1),
              metrics=['accuracy'])
```

$$w \leftarrow w - \eta \partial L / \partial w$$

0.1

Step 3.2: Find the optimal network parameters

```
model.fit(x_train, y_train, batch_size=100, nb_epoch=20)
```

Training data (Images)

Labels (digits)

Keras

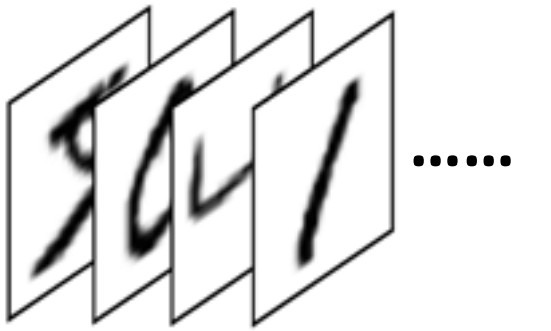


Step 3.2: Find the optimal network parameters

```
model.fit(x_train, y_train, batch_size=100, nb_epoch=20)
```

numpy array

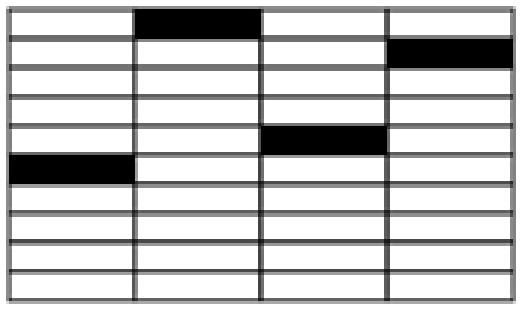
28 x 28 =784



Number of training examples

numpy array

10



Number of training examples

Keras



Save and load models

http://keras.io/getting-started/faq/#how-can-i-save-a-keras-model

How to use the neural network (testing):

case 1:

```
score = model.evaluate(x_test,y_test)
print('Total loss on Testing Set:', score[0])
print('Accuracy of Testing Set:', score[1])
```

case 2:

```
result = model.predict(x_test)
```

Réseaux de neurones convolutifs

Réseau de neurone convolutif

- *Les réseaux convolutifs sont une forme particulière de réseau neuronal multicouches dont l'architecture des connexions est inspirée de celle du cortex visuel des mammifères.*
- Leur conception suit la découverte de mécanismes visuels dans les organismes vivants. Ces réseaux de neurones artificiels (réseau de neurones à convolution, ou CNN) sont capables de catégoriser les informations des plus simples aux plus complexes.
- Ils consistent en un empilage multicouche de neurones, des fonctions mathématiques à plusieurs paramètres ajustables, qui pré-traitent de petites quantités d'informations.
- Les réseaux convolutifs sont caractérisé par leurs premières couches convolutionelles (généralement une à trois). Une couche convolutive, est basée comme son nom l'indique sur le principe mathématique de convolution, et cherche à repérer la présence d'un motif (dans un signal ou dans une image par exemple).

Réseau de neurone convolutif

- Pour une image,
  - la première couche convolutionelle peut détecter les contours des objets (par exemple un cercle),
  - la seconde couche convolutionelle peut combiner les contours en objets (par exemple une roue),
  - et les couches suivantes (non nécessairement convolutionelles) peuvent utiliser ces informations pour distinguer une voiture d'une moto.
- Une phase d'apprentissage sur des objets connus permet de trouver les meilleurs paramètres en montrant par exemple à la machine des milliers d'images d'un chien, d'une voiture ou d'un homme.
- L'un des enjeux est de trouver des méthodes pour ajuster ces paramètres le plus rapidement et le plus efficacement possible. Les réseaux neuronaux convolutifs ont de nombreuses applications dans la reconnaissance d'images, de vidéos ou le traitement du langage naturel.

Réseau de neurone convolutif

- Some patterns are much smaller than the whole image

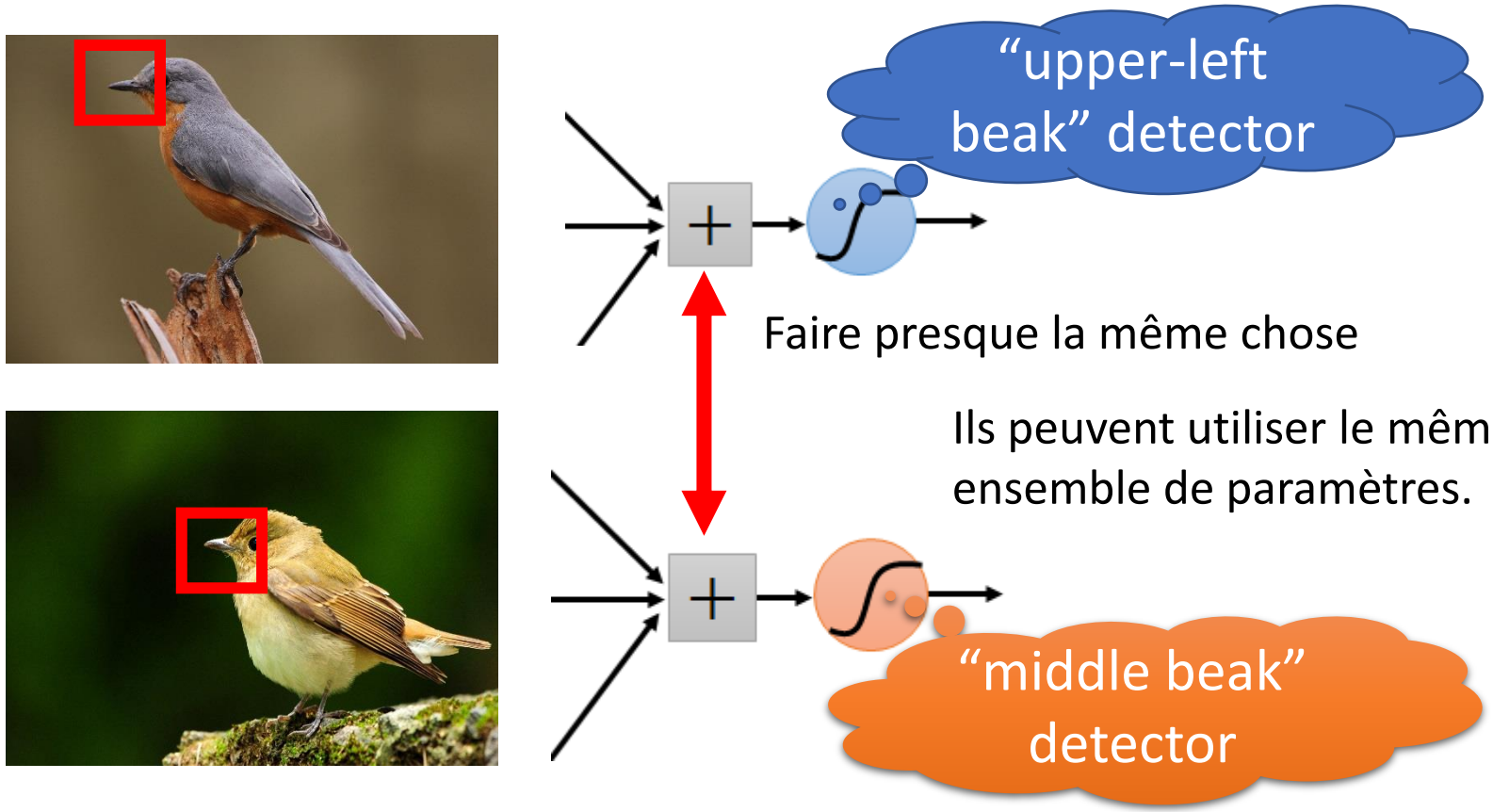
Un neurone n'a pas besoin de voir l'image entière pour découvrir le motif.

Connexion à une petite région avec moins de paramètres



Réseau de neurone convolutif

- Les mêmes schémas apparaissent dans différentes régions.



Réseau de neurone convolutif

- Le sous-échantillonnage des pixels ne modifiera pas l'objet

bird



subsampling

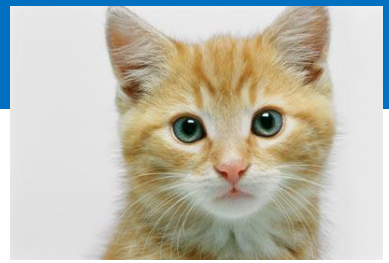
bird



Nous pouvons sous-échantillonner les pixels pour rendre l'image plus petite

Moins de paramètres pour que le réseau traite l'image

Réseau de neurone convolutif



Convolution

Max Pooling

Convolution

Max Pooling

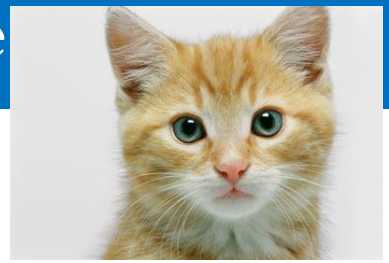
Peut se répéter plusieurs fois

Flatten

Fully Connected Feedforward network

cat dog ……

Réseau de neurone



Propriété 1

- Certains motifs sont beaucoup plus petits que l'image entière

Propriété 2

- Les mêmes schémas apparaissent dans différentes régions.

Propriété 3

- Le sous-échantillonnage des pixels ne modifiera pas l'objet

Convolution

Max Pooling

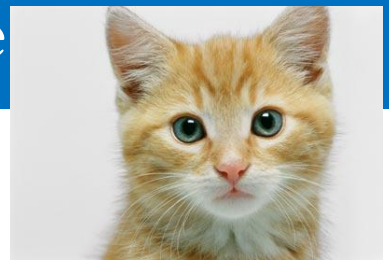
Convolution

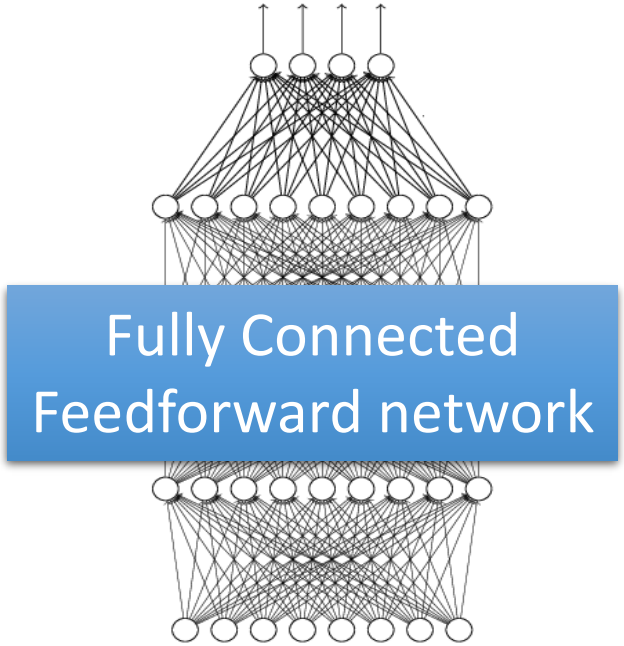
Max Pooling

Peut se répéter plusieurs fois

Flatten

Réseau de neurone





CNN – Convolution

Ce sont les paramètres réseau à apprendre.

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Matrix

-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

Matrix

⋮

Propriété 1

Chaque filtre détecte un petit motif(3 x 3).

CNN – Convolution

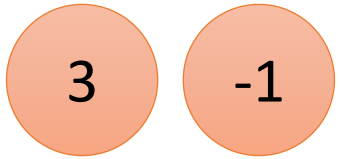
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



CNN – Convolution

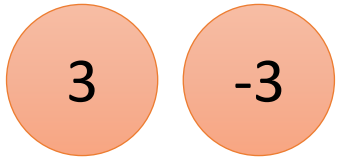
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

si stride=2

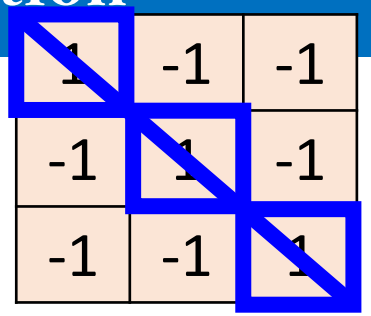
1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image



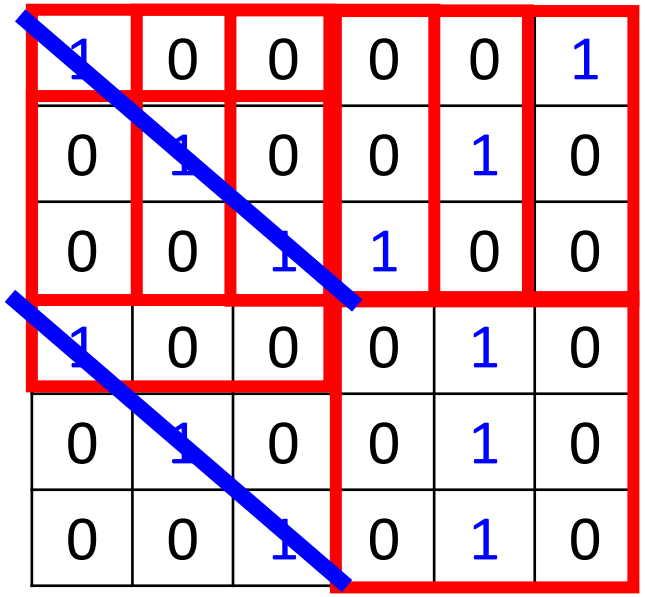
Prennent Stride =1

CNN – Convolution

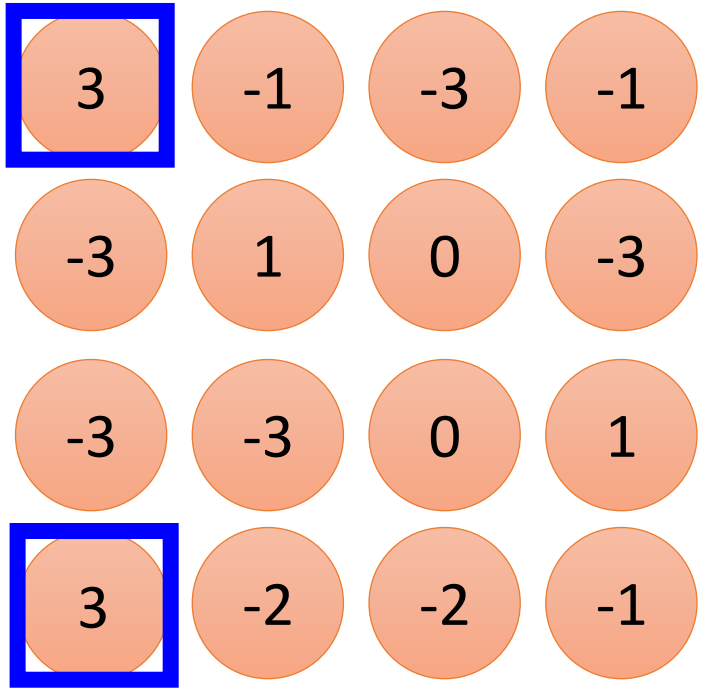


Filter 1

stride=1



6 x 6 image



Propriété 2

CNN – Convolution

-1	1	-1
-1	1	-1
-1	1	-1

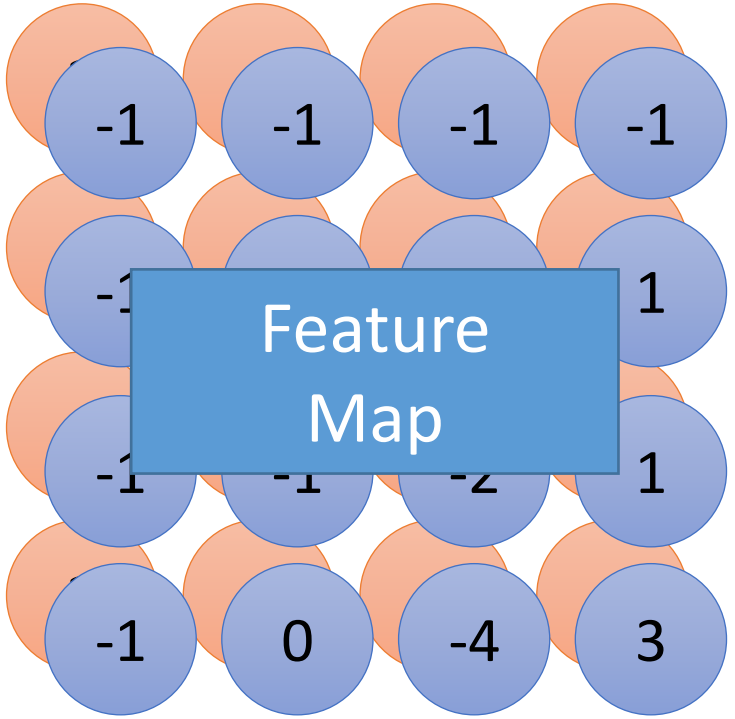
Filter 2

stride=1

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

6 x 6 image

Effectuons le même processus pour chaque filtre

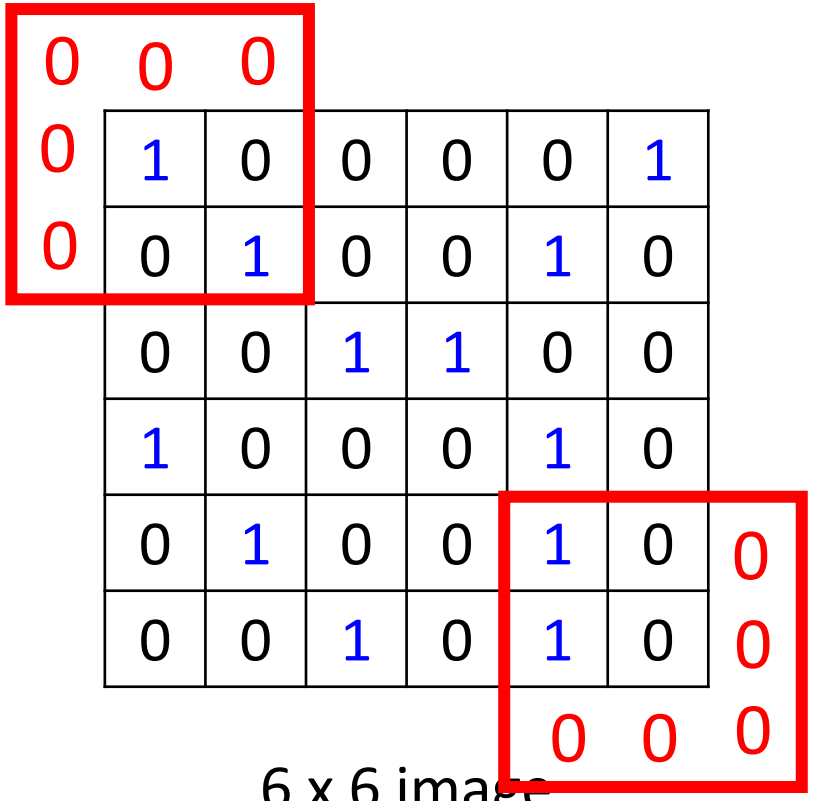


4 x 4 image

CNN – Zero Padding

1	-1	-1
-1	1	-1
-1	-1	1

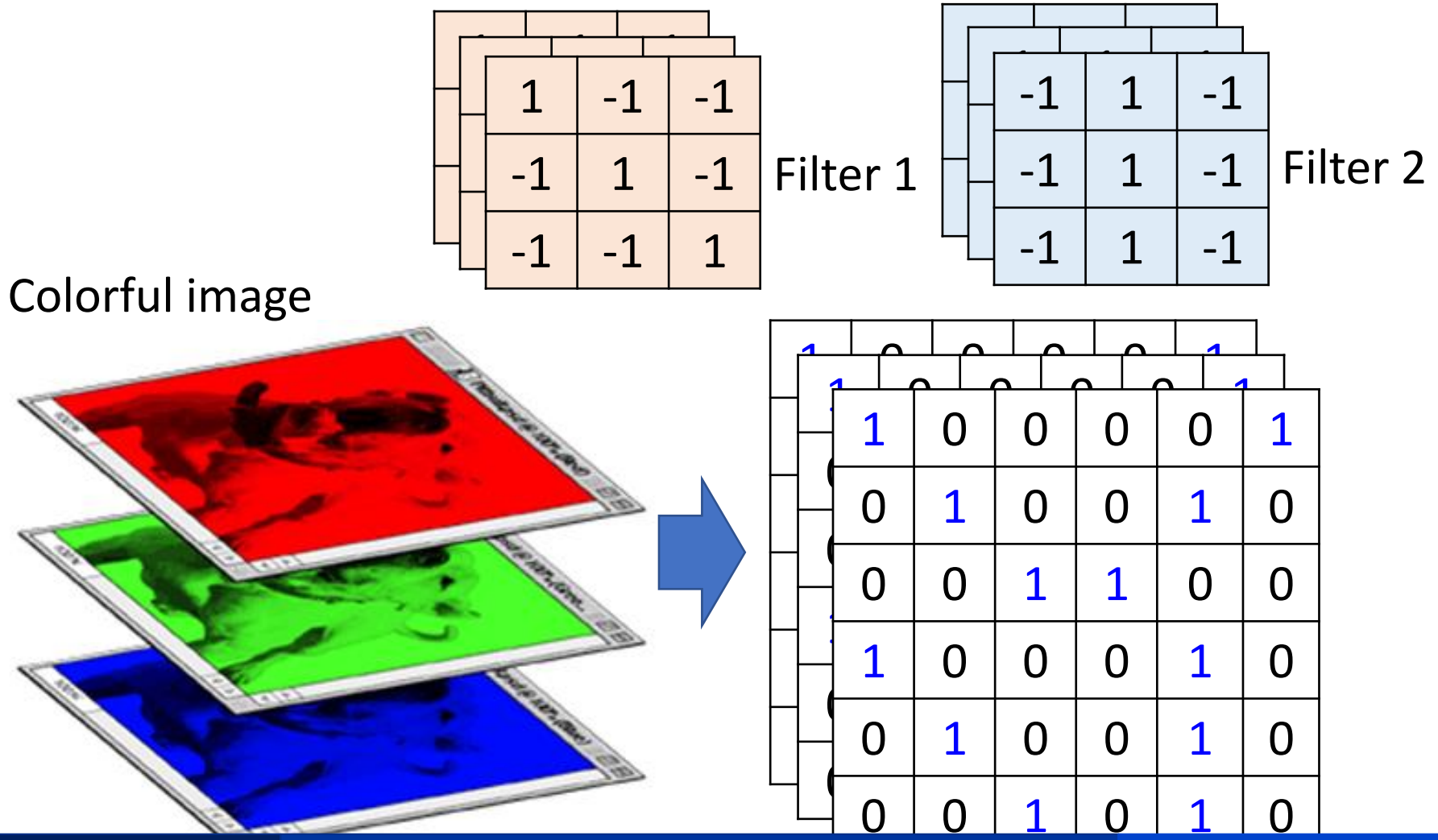
Filter 1



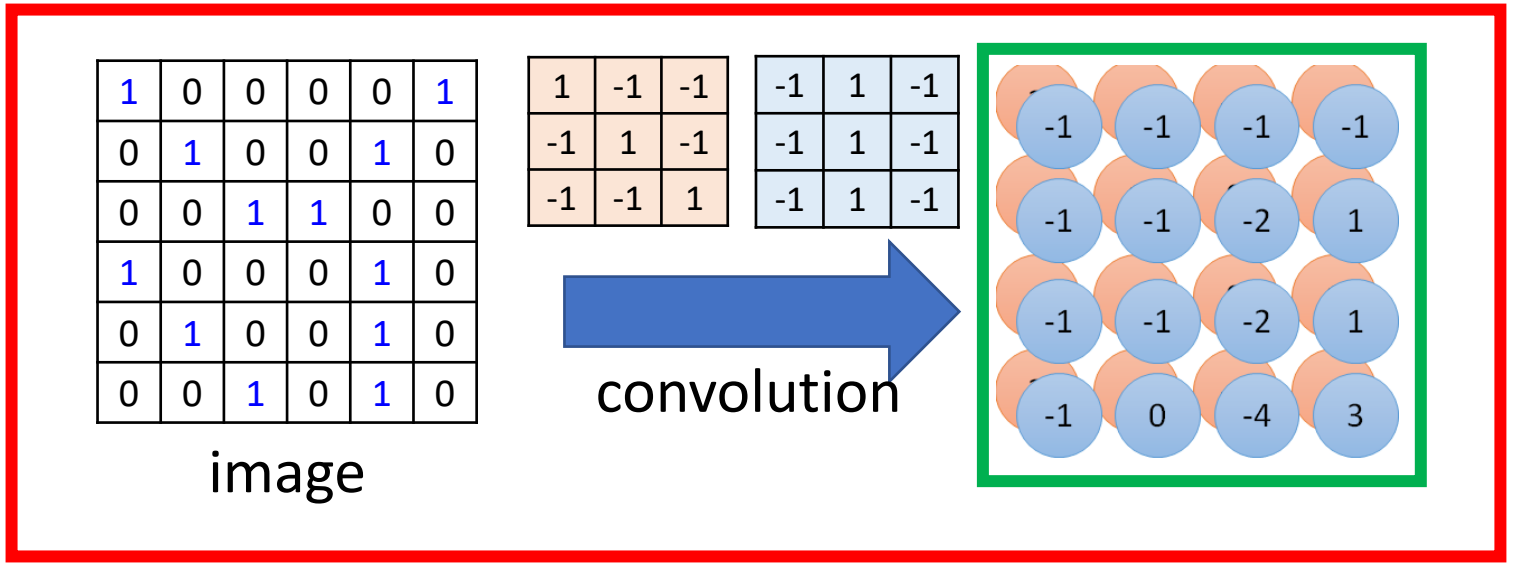
nous obtiendrons ainsi d'autres images 6 x 6

→ Zero padding

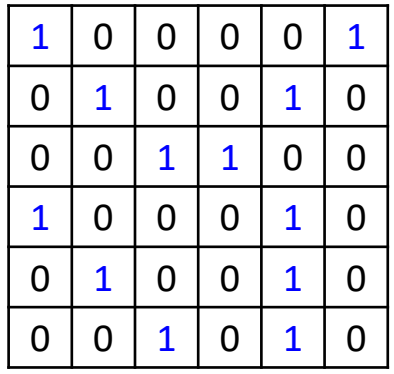
CNN – Colorful image

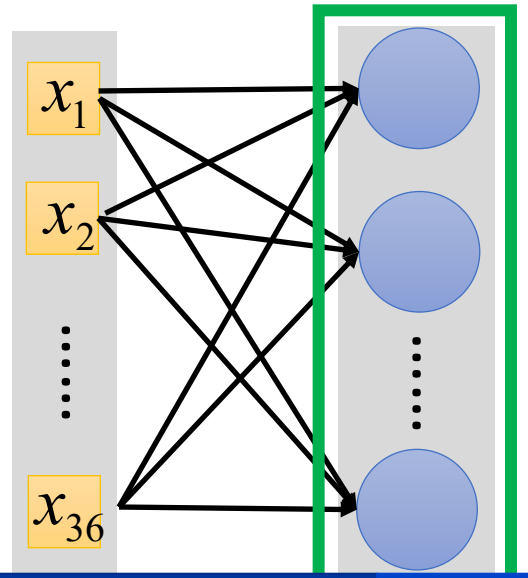


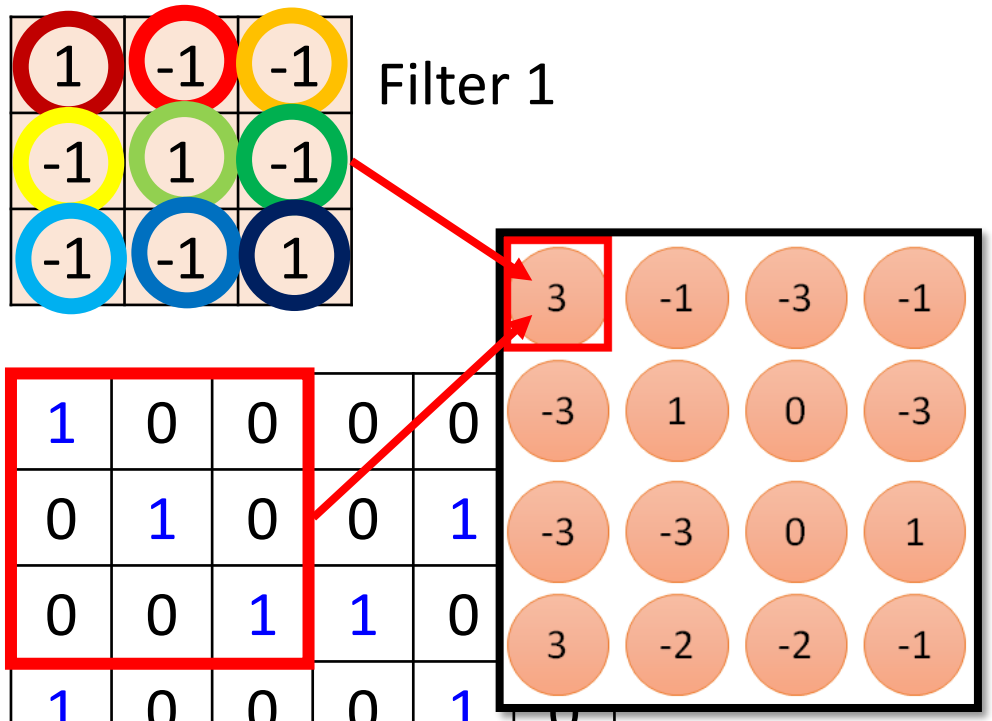
Convolution v.s. Fully Connected



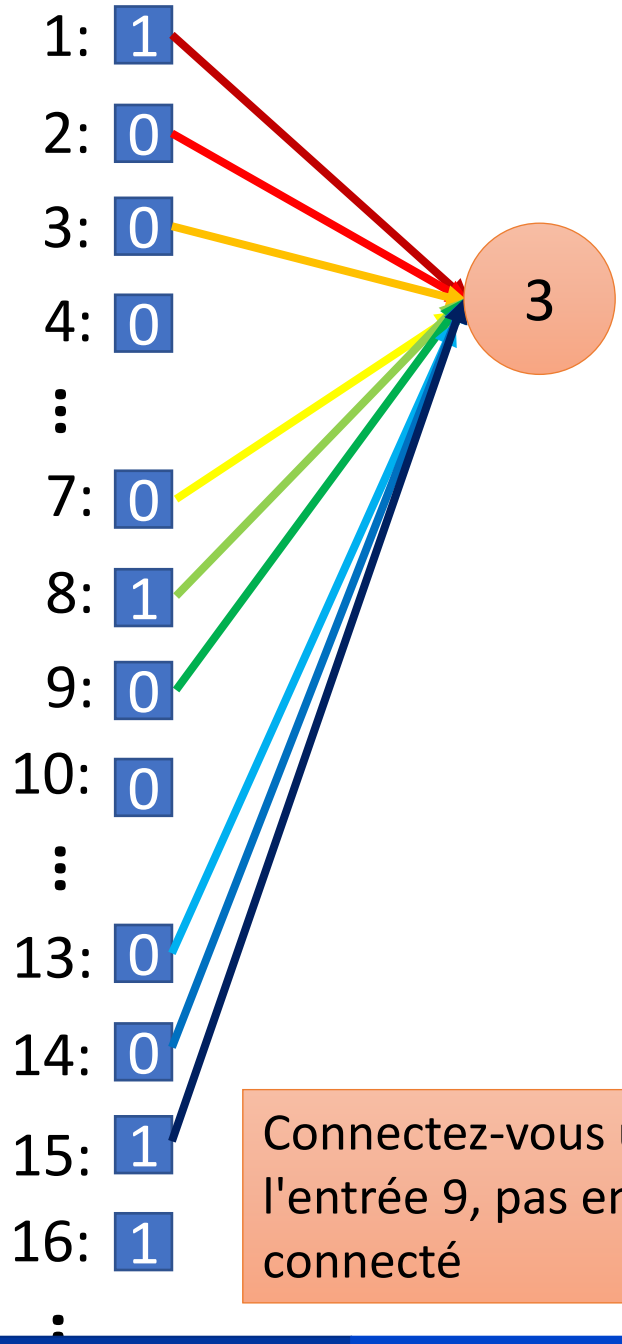
Fully-connected



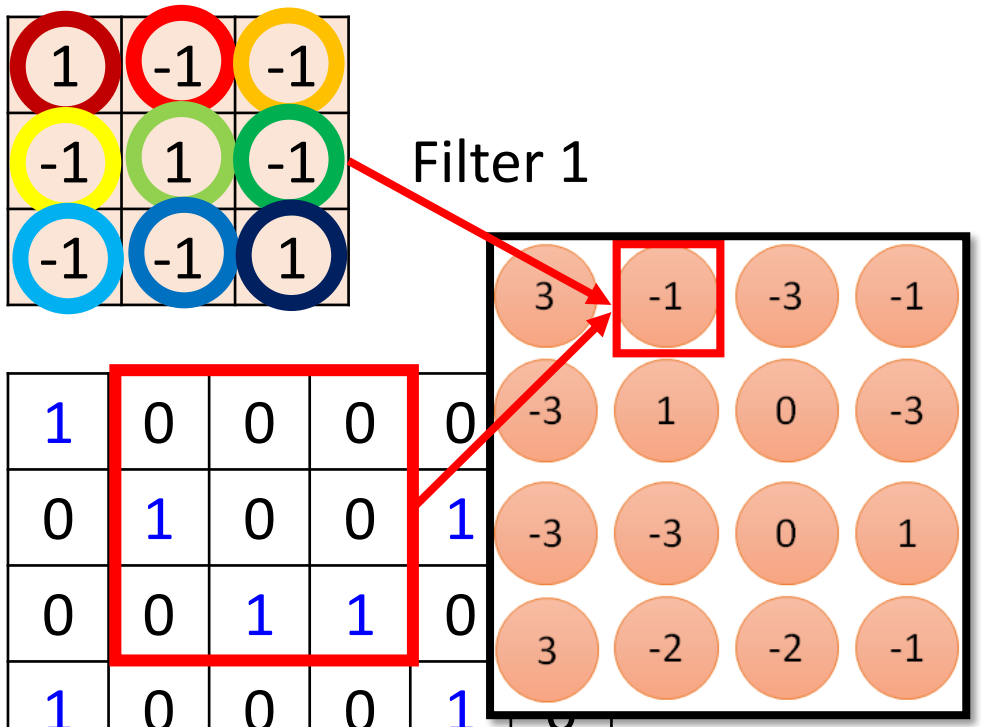




Moins de paramètres!

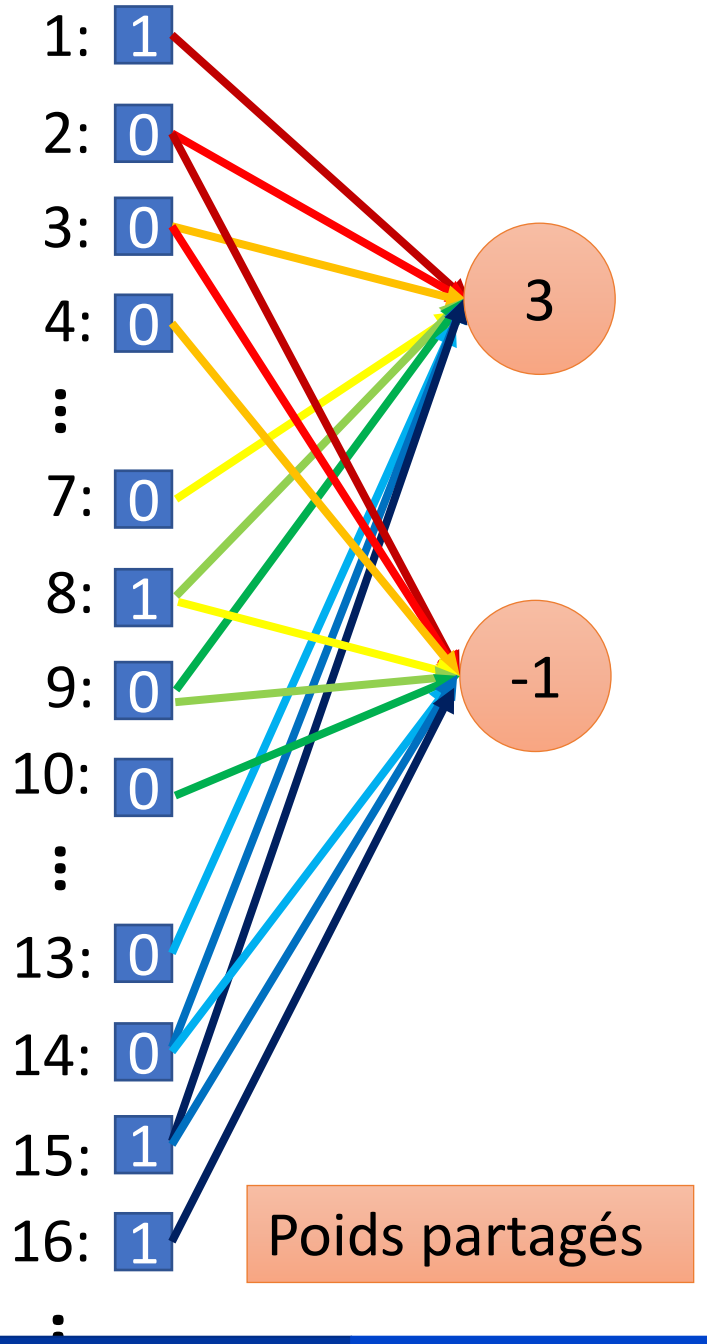


Connectez-vous uniquement à l'entrée 9, pas entièrement connecté

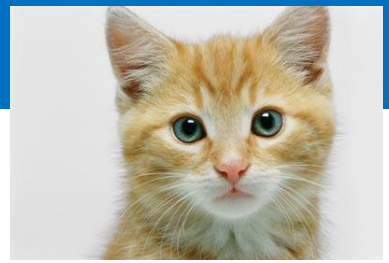


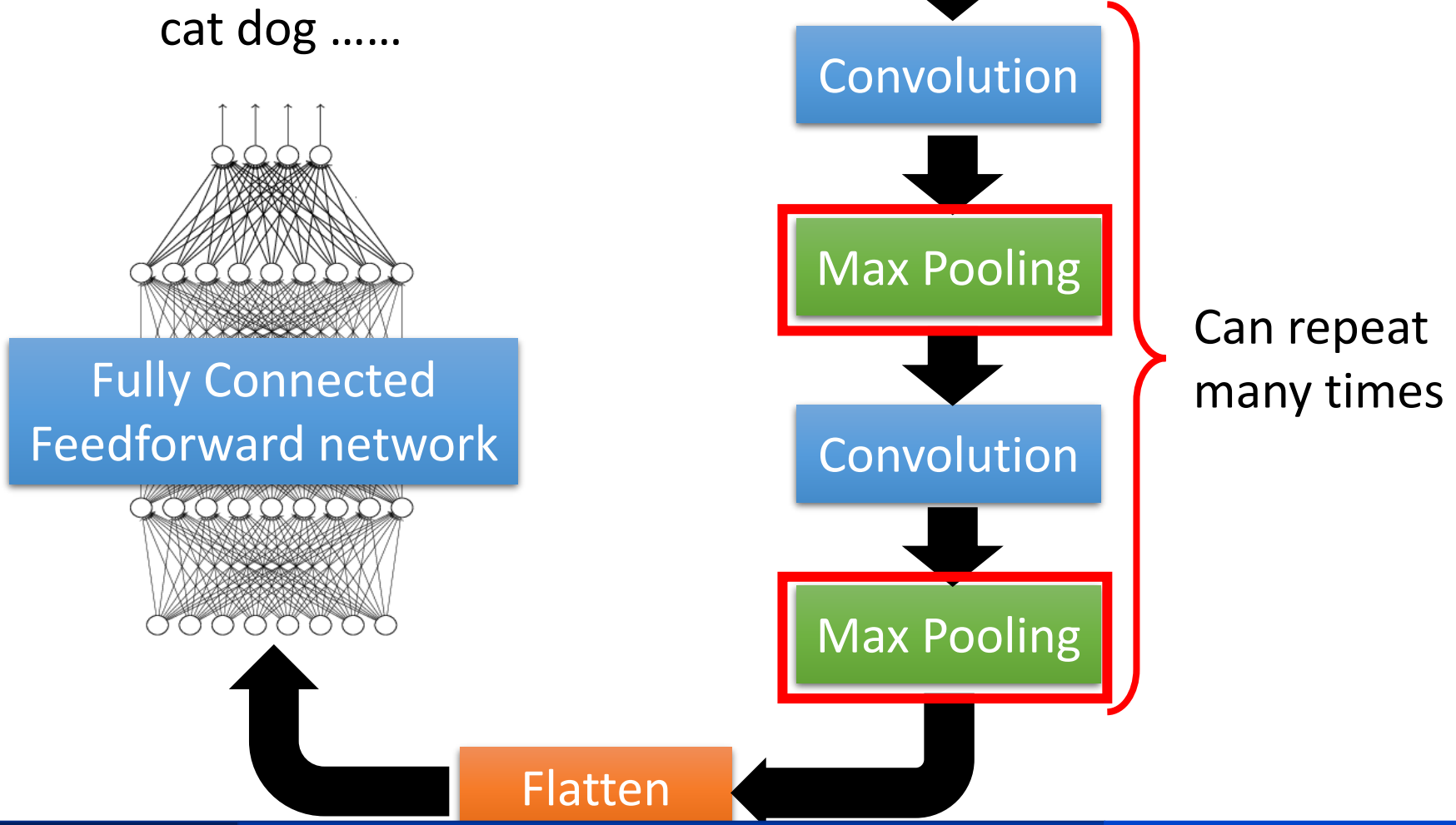
Moins de paramètres!

Encore moins de paramètres !



L'intégralité de CNN





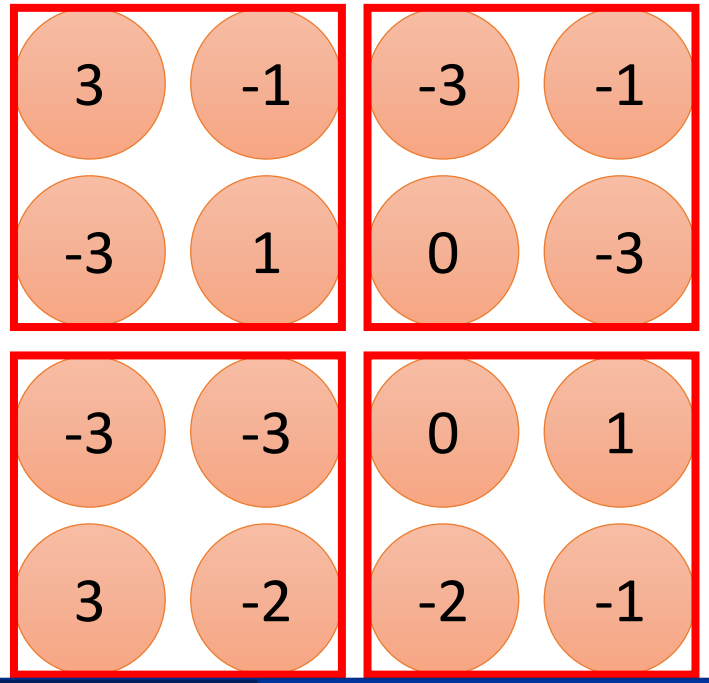
CNN – Max Pooling

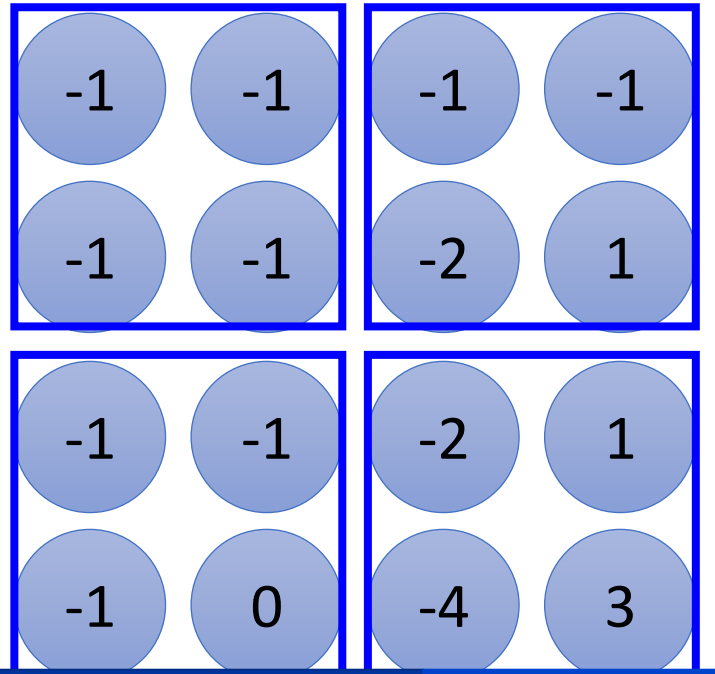
1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

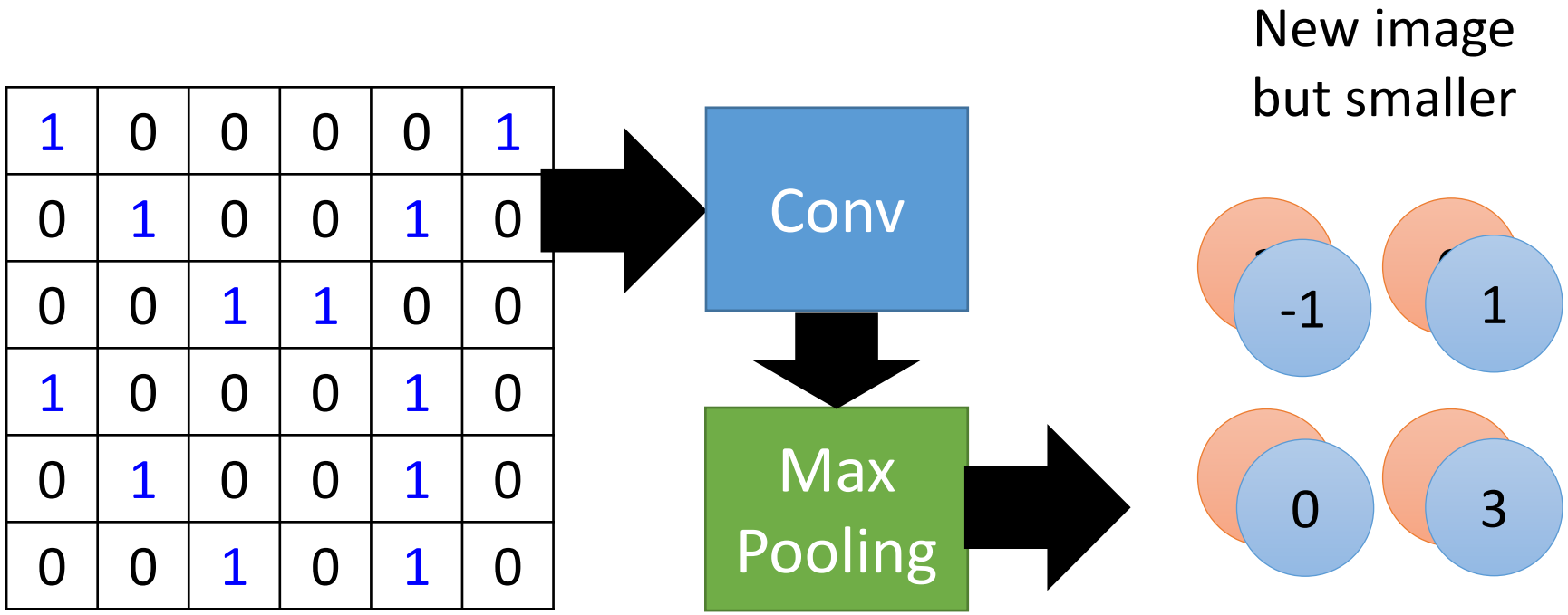
-1	1	-1
-1	1	-1
-1	1	-1

Filter 2

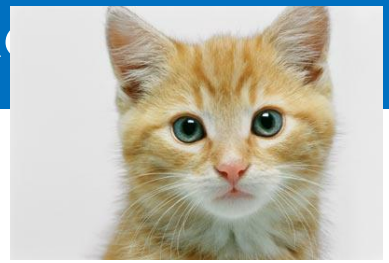


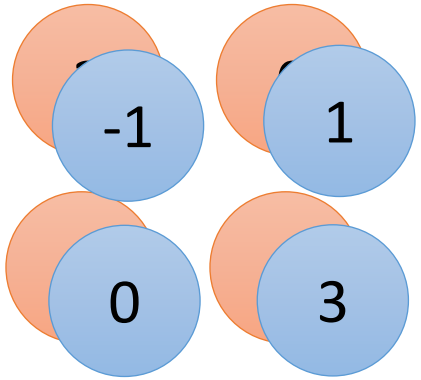


CNN – Max Pooling



L'intégralité d





A new image

Convolution

Max Pooling

Convolution

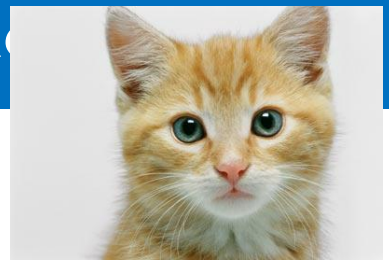
Max Pooling

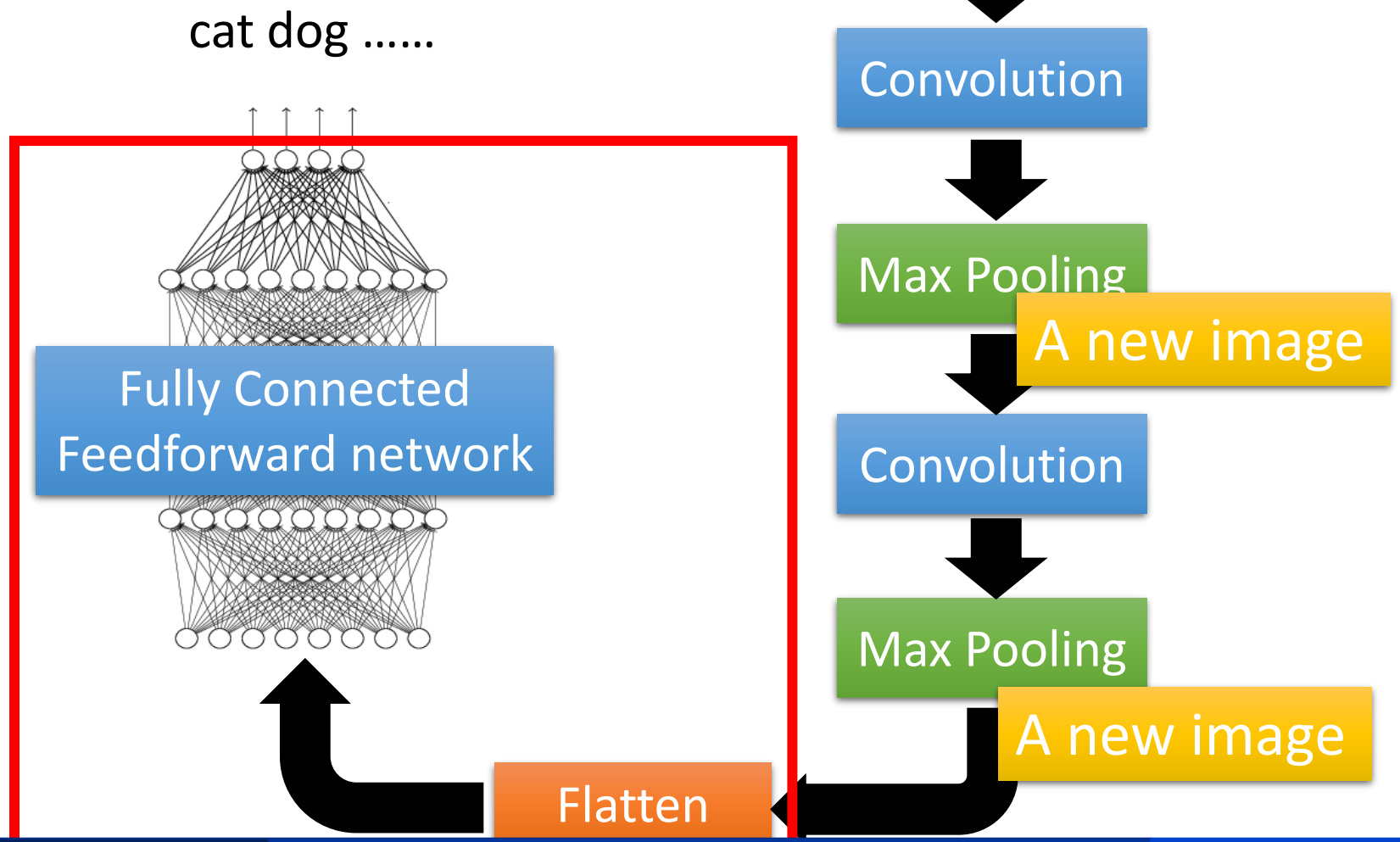
Peut se répéter plusieurs fois

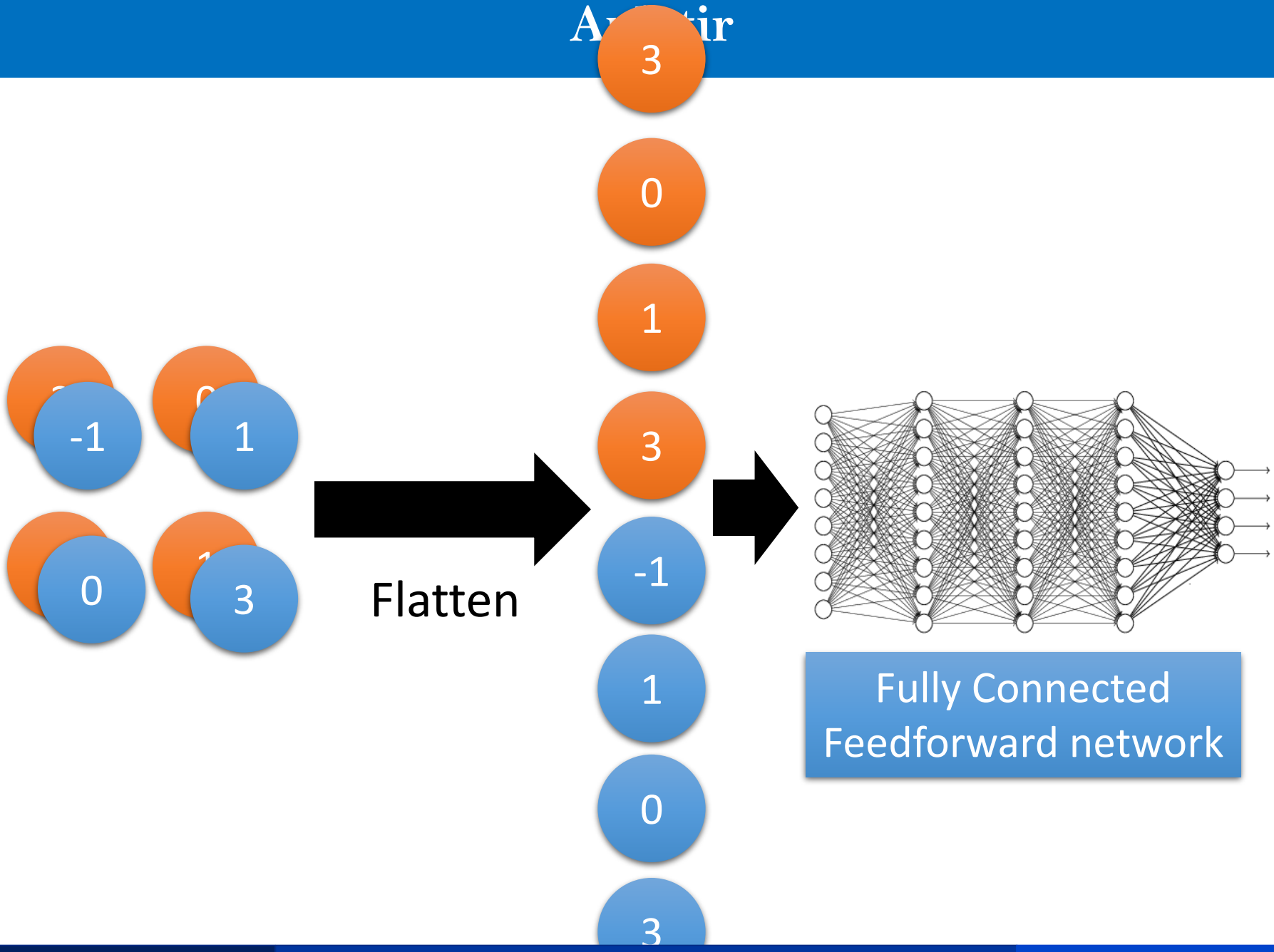
Plus petit que l'image originale

Le nombre du canal est le nombre de filtres

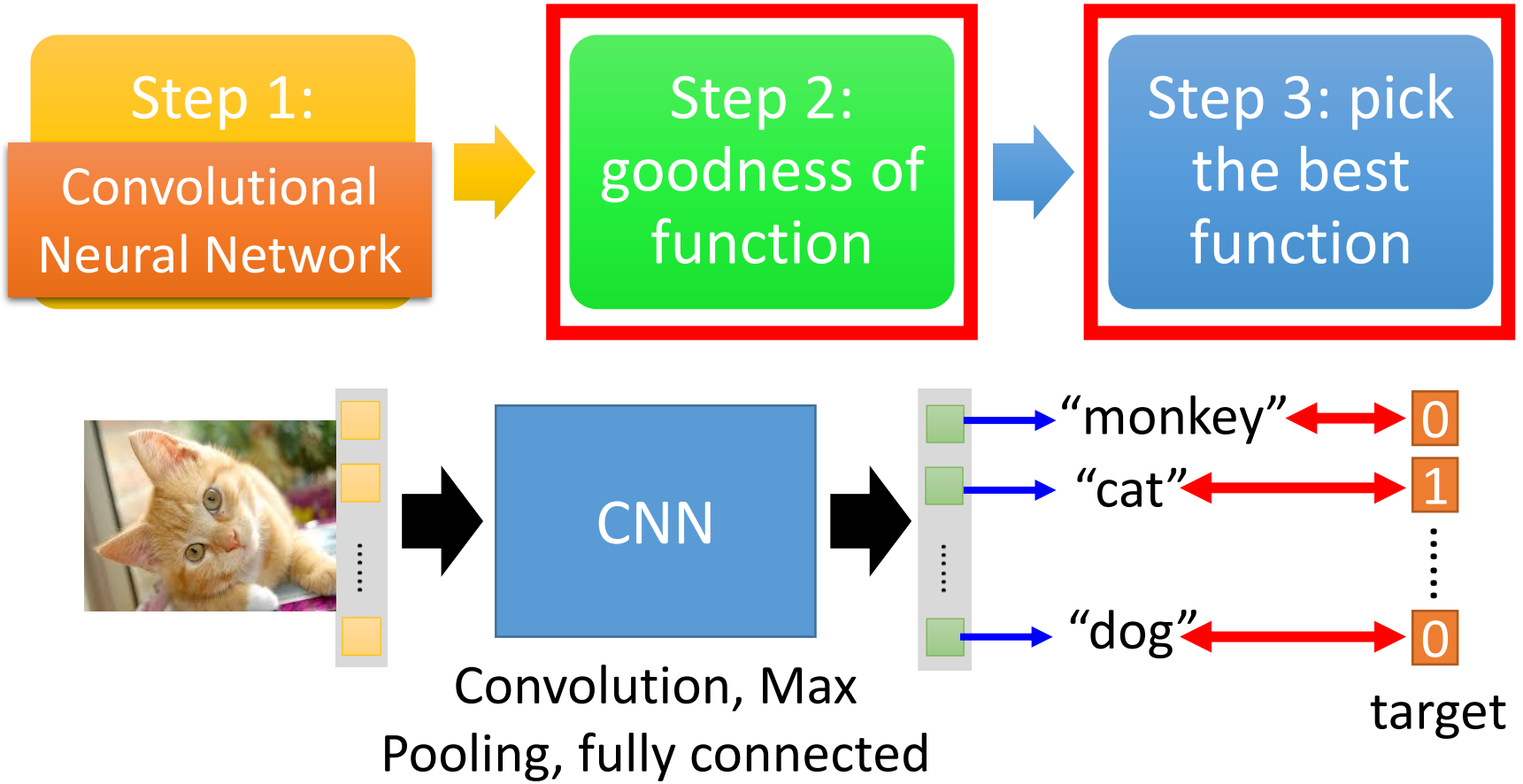
L'intégralité d






-1
1
0
3
Flatten
3
0
1
3
-1
1
0
3
Fully Connected
Feedforward network

Convolutional Neural Network

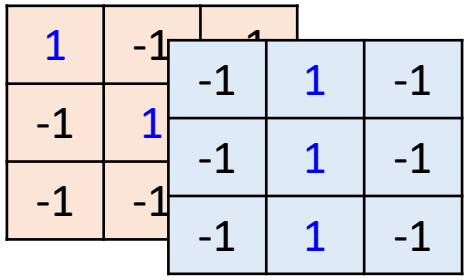


Apprentissage : Rien de spécial, juste une descente de gradient ……

CNN in Keras

Seule la structure du réseau et le format d'entrée (vecteur -> tenseur 3D) ont été modifiés.

```
model2.add( Convolution2D( 25,3,3,
                input_shape=(1,28,28) ) )
```



.....

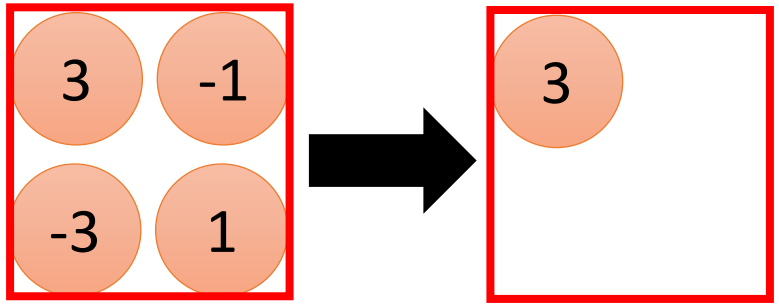
There are 25 3x3 filters.

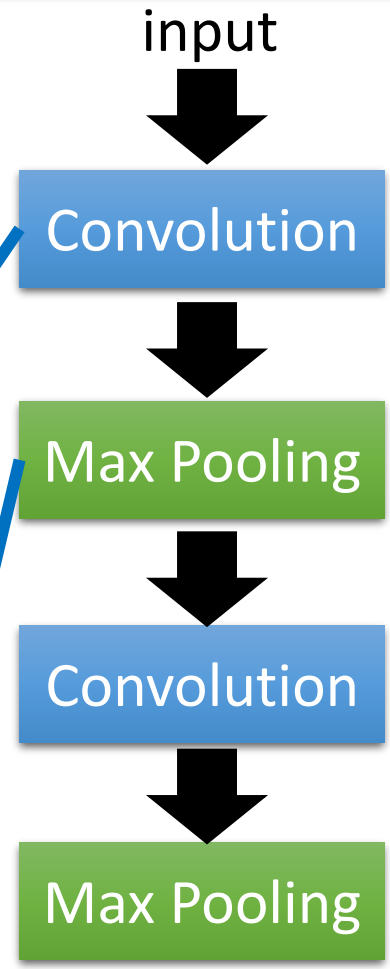
Input_shape = (1 , 28 , 28)

1: black/weight, 3: RGB

28 x 28 pixels

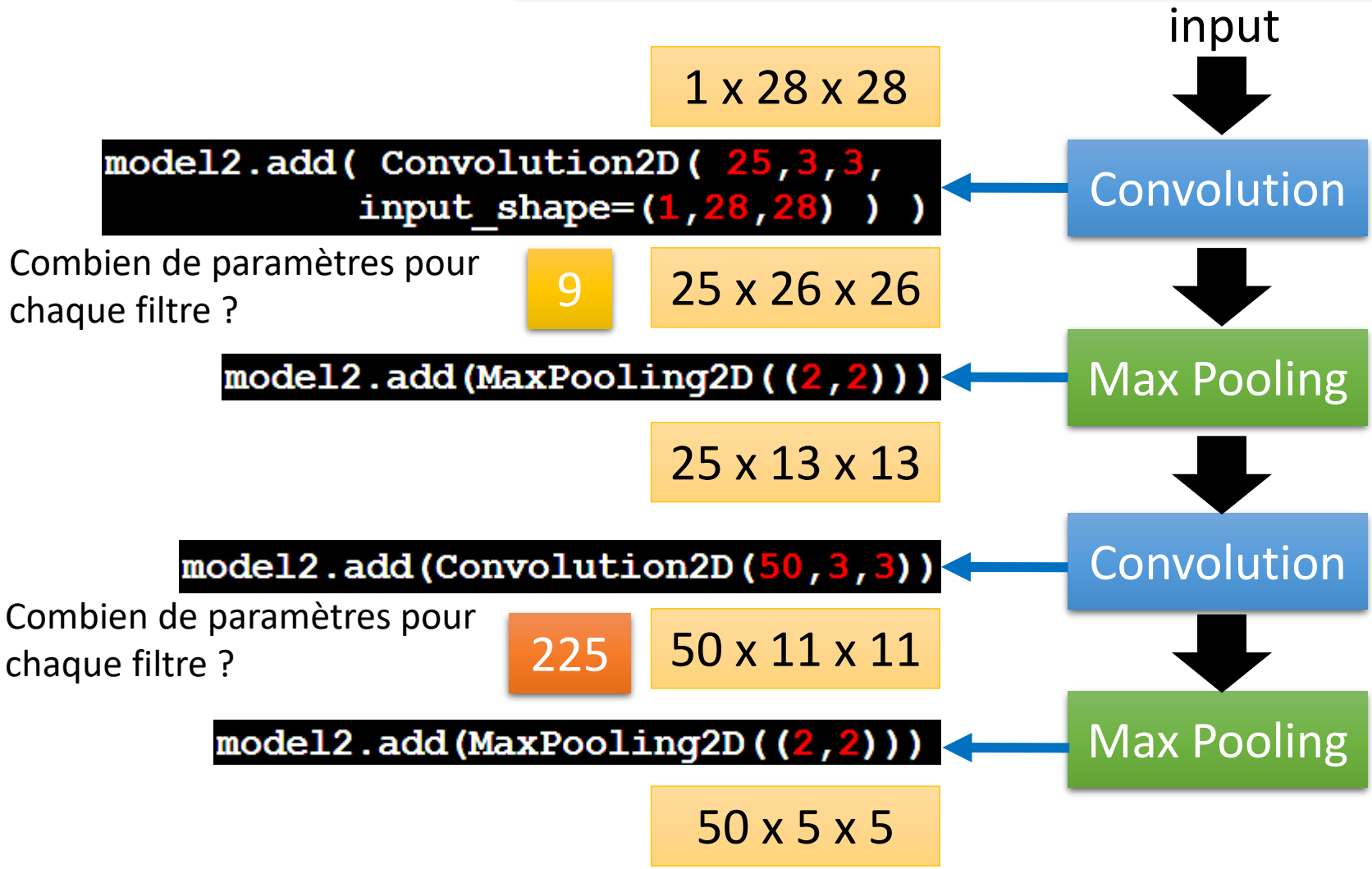
```
model2.add(MaxPooling2D((2,2)))
```





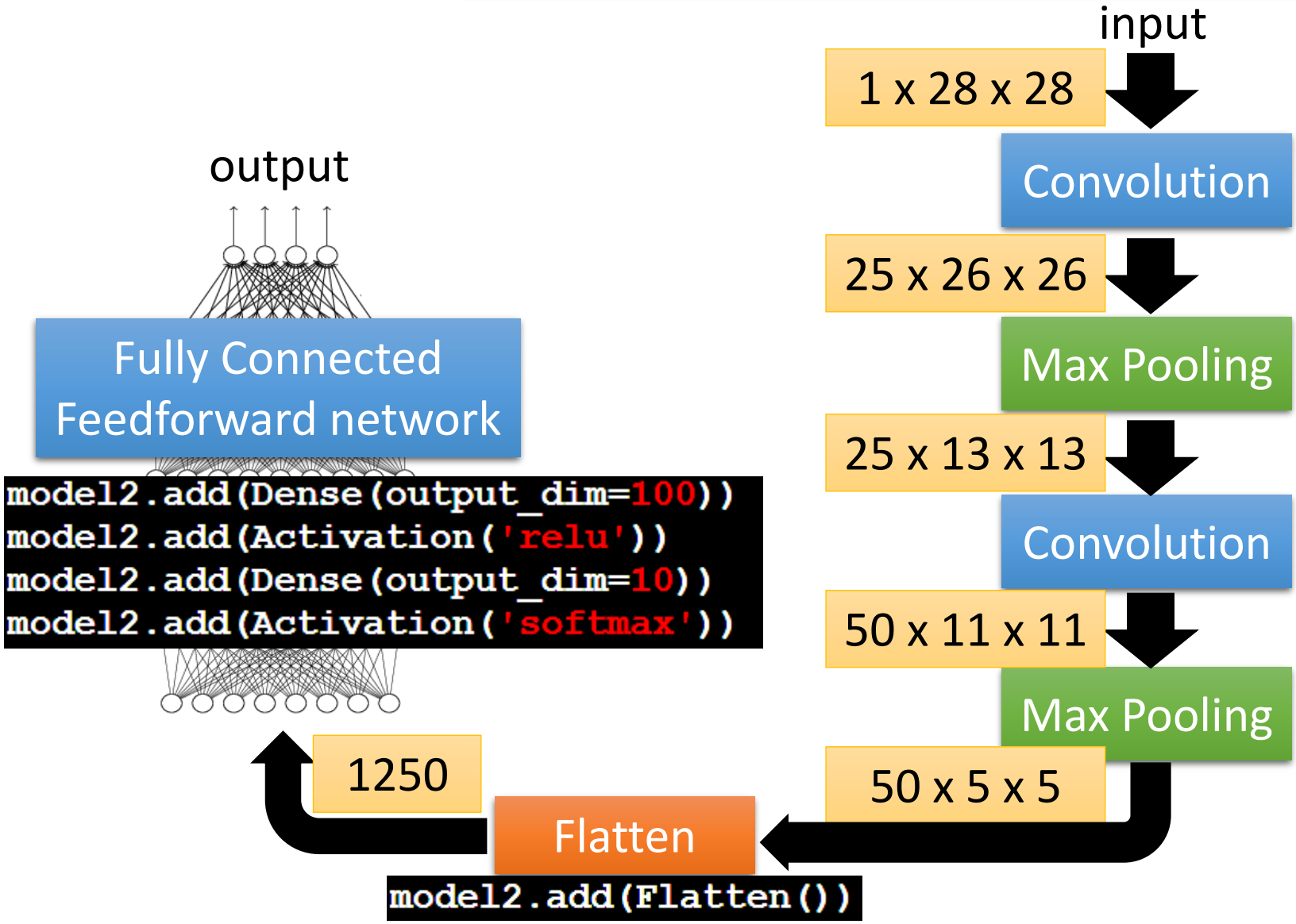
CNN in Keras

Seule la structure du réseau et le format d'entrée (vecteur -> tenseur 3D) ont été modifiés.



CNN in Keras

Seule la structure du réseau et le format d'entrée (vecteur -> tenseur 3D) ont été modifiés.



En résumé …

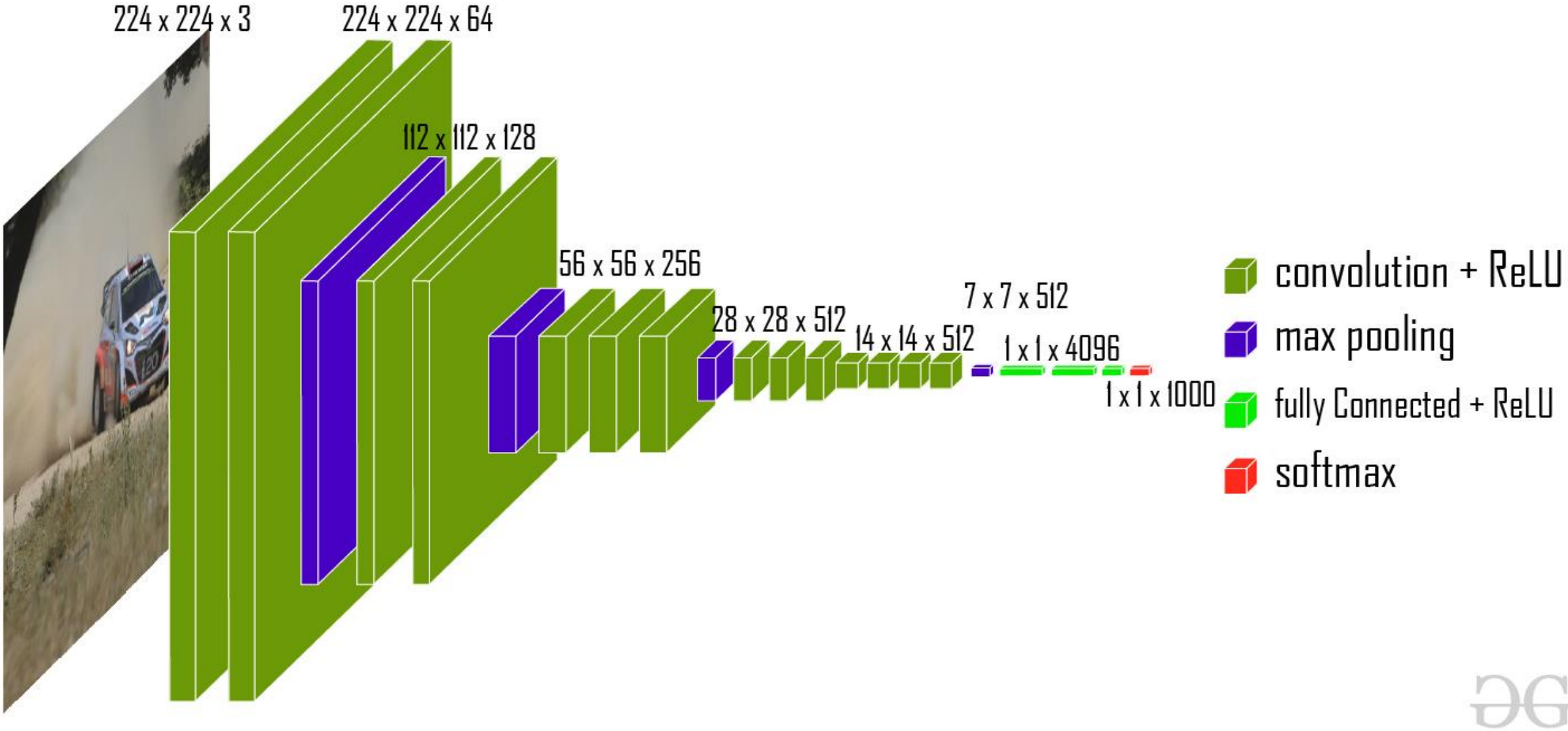
- Les réseaux neuronaux convolutifs (ConvNets ou CNN) sont l'une des principales catégories permettant de reconnaître et de classer des images.
- La détection d'objets, la reconnaissance de visages, etc., sont quelques-uns des domaines dans lesquels les CNN sont largement utilisés.
- Les classifications d'images CNN prennent une image d'entrée, la traitent et la classent dans certaines catégories (par exemple, chien, chat, tigre, lion).

Les architectures CNN

- Une architecture de réseau neuronal convolutif (CNN) est un modèle d'apprentissage profond conçu pour le traitement de données structurées en grille, telles que des images.
- Elle se compose de plusieurs couches, dont des couches convolutives, de pooling et entièrement connectées.
- Exemple:
  - **AlexNet (2012)** – Révolutionne les CNN avec ReLU et GPU.
  - **VGG (2014)** – Empilement de couches 3x3 (VGG-16, VGG-19).
  - **ResNet (2015)** – Skip connections pour réseaux très profonds (ResNet-50, ResNet-101).
  - **EfficientNet (2019)** – Optimisation multi-échelle pour mobiles.
  - **MobileNet (2017)** – Léger pour applications embarquées.
  - **U-Net (2015)** – Architecture pour la segmentation médicale.

Couche	Type	Paramètres	Dimensions Sortie	Nombre de Paramètres
1	Conv3x3	64 filtres, ReLU	224x224x64	(3x3x3)x64 + 64 = **1,792**
2	Conv3x3	64 filtres, ReLU	224x224x64	(3x3x64)x64 + 64 = **36,928**
3	MaxPool2x2	stride=2	112x112x64	**0**
4	Conv3x3	128 filtres, ReLU	112x112x128	(3x3x64)x128 + 128 = **73,856**
5	Conv3x3	128 filtres, ReLU	112x112x128	(3x3x128)x128 + 128 = **147,584**
6	MaxPool2x2	stride=2	56x56x128	**0**
7	Conv3x3	256 filtres, ReLU	56x56x256	(3x3x128)x256 + 256 = **295,168**
8	Conv3x3	256 filtres, ReLU	56x56x256	(3x3x256)x256 + 256 = **590,080**
9	Conv3x3	256 filtres, ReLU	56x56x256	(3x3x256)x256 + 256 = **590,080**
10	MaxPool2x2	stride=2	28x28x256	**0**
11	Conv3x3	512 filtres, ReLU	28x28x512	(3x3x256)x512 + 512 = **1,180,160**
12	Conv3x3	512 filtres, ReLU	28x28x512	(3x3x512)x512 + 512 = **2,359,808**
13	Conv3x3	512 filtres, ReLU	28x28x512	(3x3x512)x512 + 512 = **2,359,808**
14	MaxPool2x2	stride=2	14x14x512	**0**
15	Conv3x3	512 filtres, ReLU	14x14x512	(3x3x512)x512 + 512 = **2,359,808**
16	Conv3x3	512 filtres, ReLU	14x14x512	(3x3x512)x512 + 512 = **2,359,808**
17	Conv3x3	512 filtres, ReLU	14x14x512	(3x3x512)x512 + 512 = **2,359,808**
18	MaxPool2x2	stride=2	7x7x512	**0**
19	FC	4096 neurones, ReLU	4096	(7x7x512)x4096 + 4096 = **102,764,544**
20	FC	4096 neurones, ReLU	4096	4096x4096 + 4096 = **16,781,312**
21	FC	1000 neurones, Softmax	1000	4096x1000 + 1000 = **4,097,000**

l'Architecture VGG-16



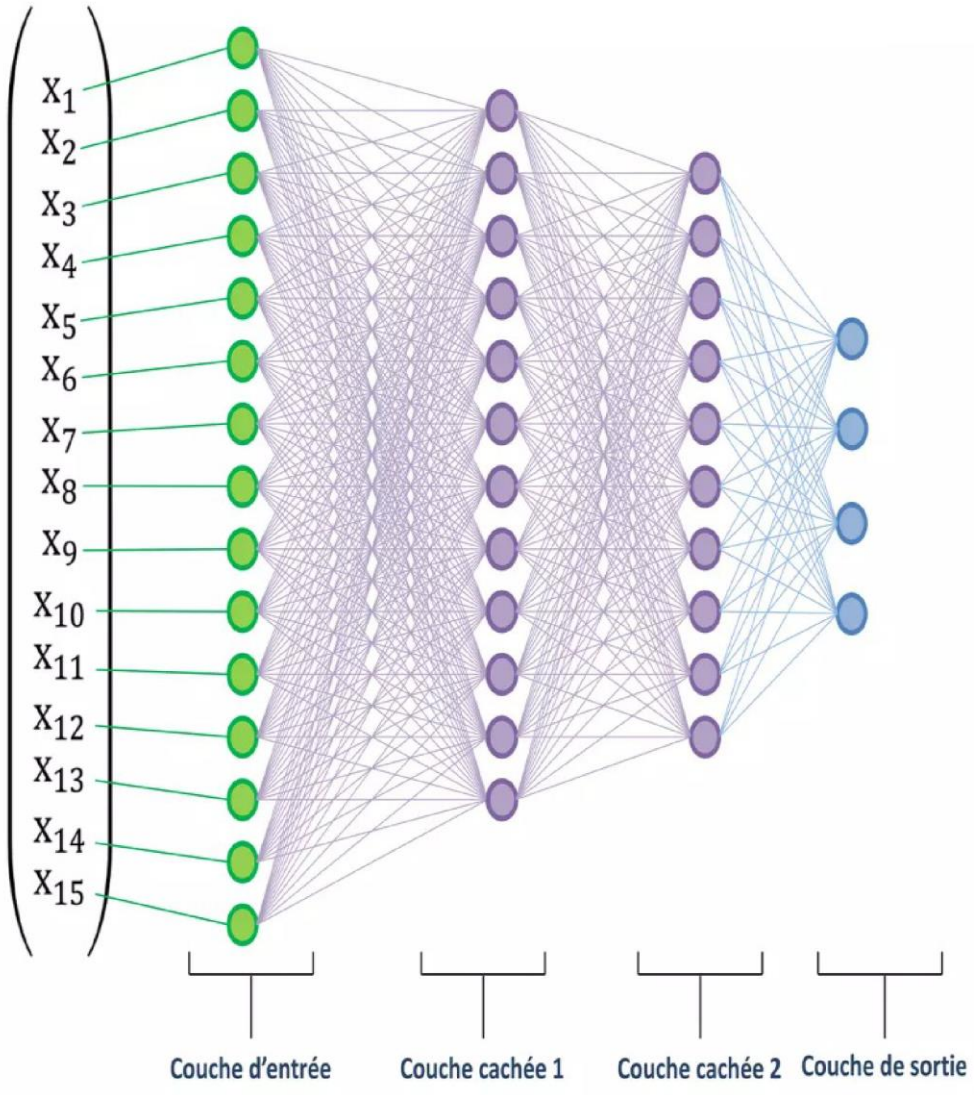
RNN :
Réseaux de neurones Récurrent

Réseaux de neurones Récurrent

- RNN : signifie Récurrent Neural Network est un réseau de neurones qui transmet de l'information de l'étape actuelle à la suivante.
- Utiliser pour traiter les séries temporelles
- Il permet de traiter une information courante selon les précédentes.
- Le terme récurrent désigne le fait que le réseau ne possède qu'une seule couche cachée et qu'à chaque fois que cette couche reçoit une entrée et produit un résultat, celui-ci est retransmis à cette même couche, en plus de l'entrée suivante.

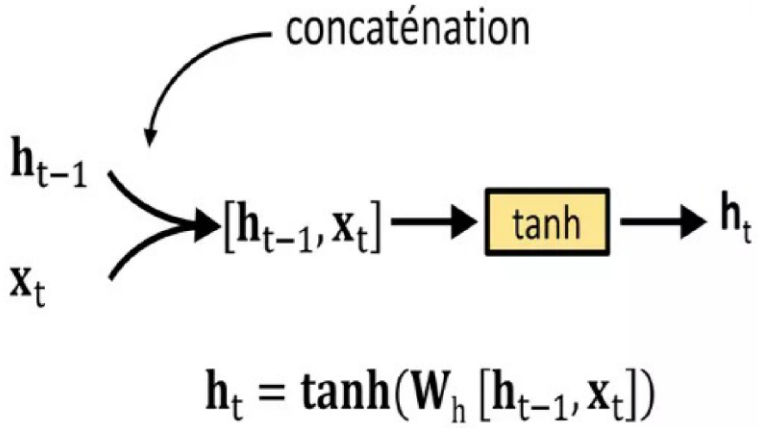
Réseaux de neurones multicouches

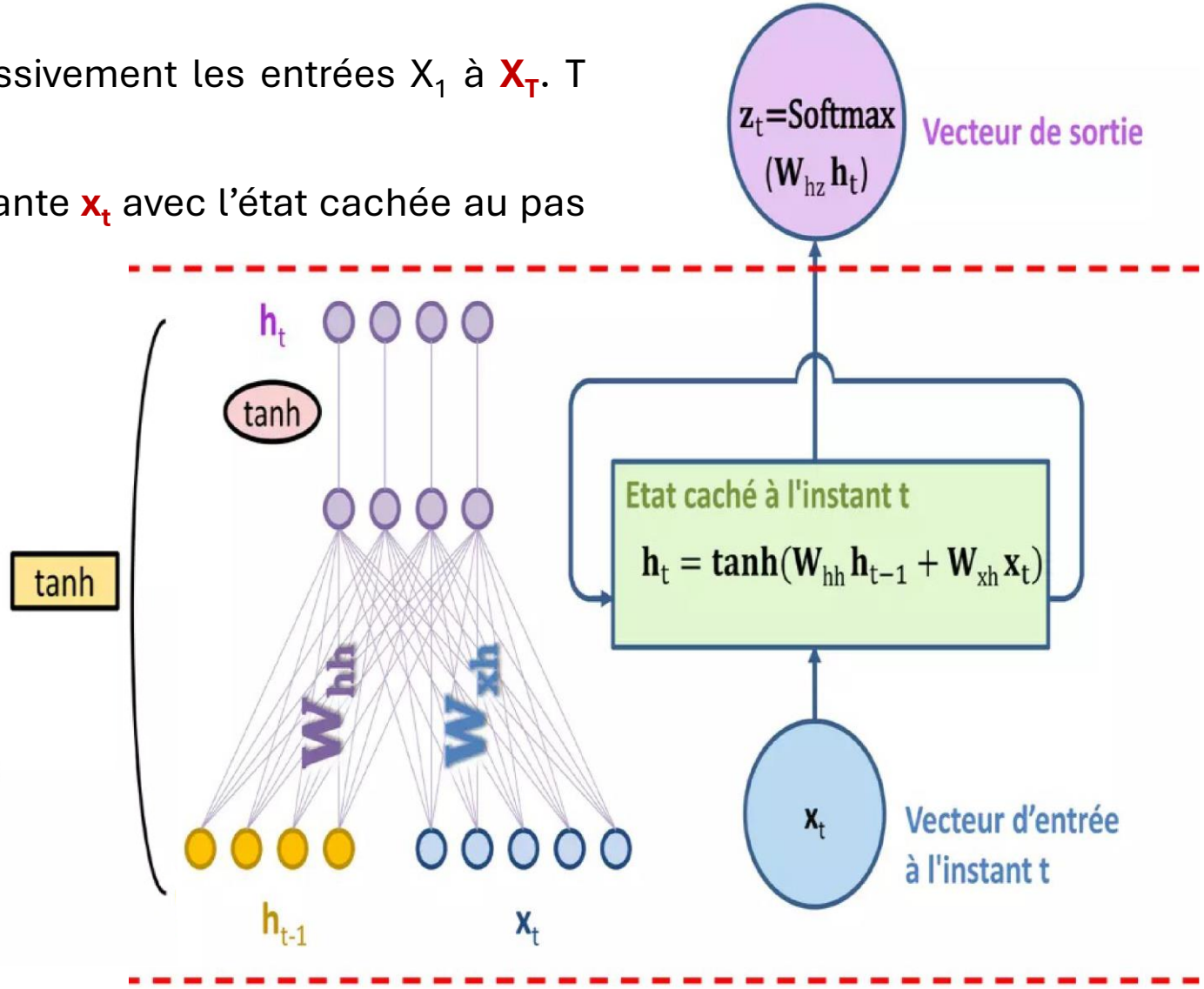
- **Limites des architectures traditionnelles :**
  - Taille fixe de la couche d'entrée
  - Taille fixe de la couche de sortie .
  - Architecture fixe du réseau
  - Ne tiennent pas compte de l'ordre des données.
  - Non adaptées aux séries temporelles



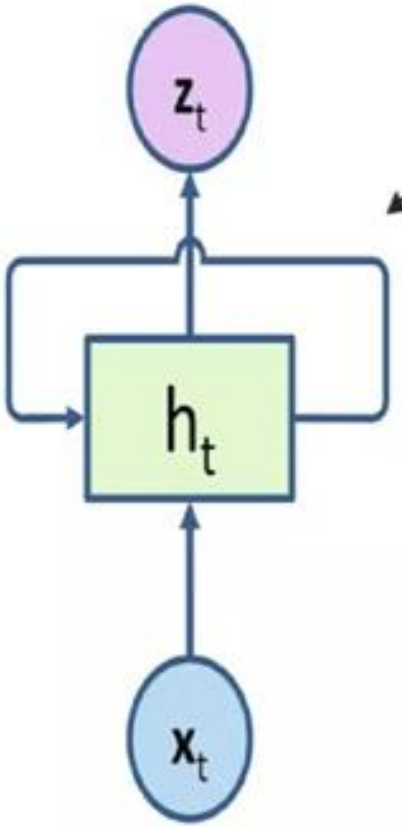
Réseaux de neurones Récurrent

- Soit une suite temporelle : $X_1\ X_2\ X_3\ X_4\ \ldots\ X_T$
- Un réseau de neurone récurrent parcourt successivement les entrées X_1 à **X_T**. T étant la longueur de la séquence.
- À l'instant **t**, la **t^éme** cellule combine l'entrée courante **x_t** avec l'état cachée au pas précédent **h_{t-1}** pour calculer l'état **h_t**.





Réseaux de neurones Récurrent



La ligne qui entoure la couche cachée du réseau de neurones récurrent s'appelle la boucle temporelle.

Elle indique que la couche cachée génère non seulement une sortie, mais que la sortie est renvoyée en tant qu'entrée dans la même couche à l'instant suivant.

Réseaux de neurones Récurrent

L'expression :

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t)$$

est parfois remplacée par :

$$\mathbf{h}_t = \mathbf{f}_W(\mathbf{h}_{t-1}, \mathbf{x}_t) = \tanh(\mathbf{W}_h\,\mathrm{concat}(\mathbf{h}_{t-1}, \mathbf{x}_t))$$

Ou encore

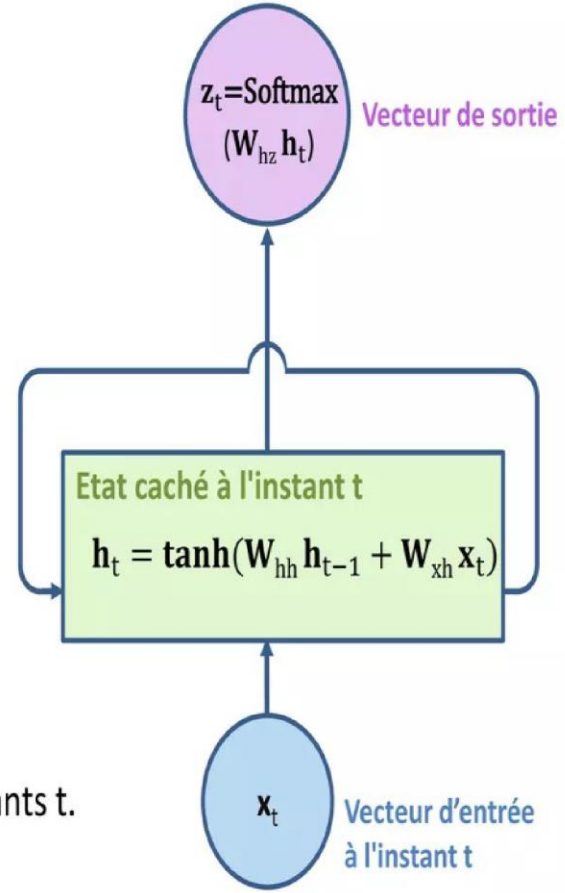
$$\mathbf{h}_t = \mathbf{f}_W(\mathbf{h}_{t-1}, \mathbf{x}_t) = \tanh(\mathbf{W}_h\,[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

A noter que les paramètres $\mathbf{W}_{hh}$ et $\mathbf{W}_{xh}$ (ou $\mathbf{W}_h$) sont les mêmes à tous les instants t.

Pour t=0, on utilise un état caché initialisé aléatoirement.

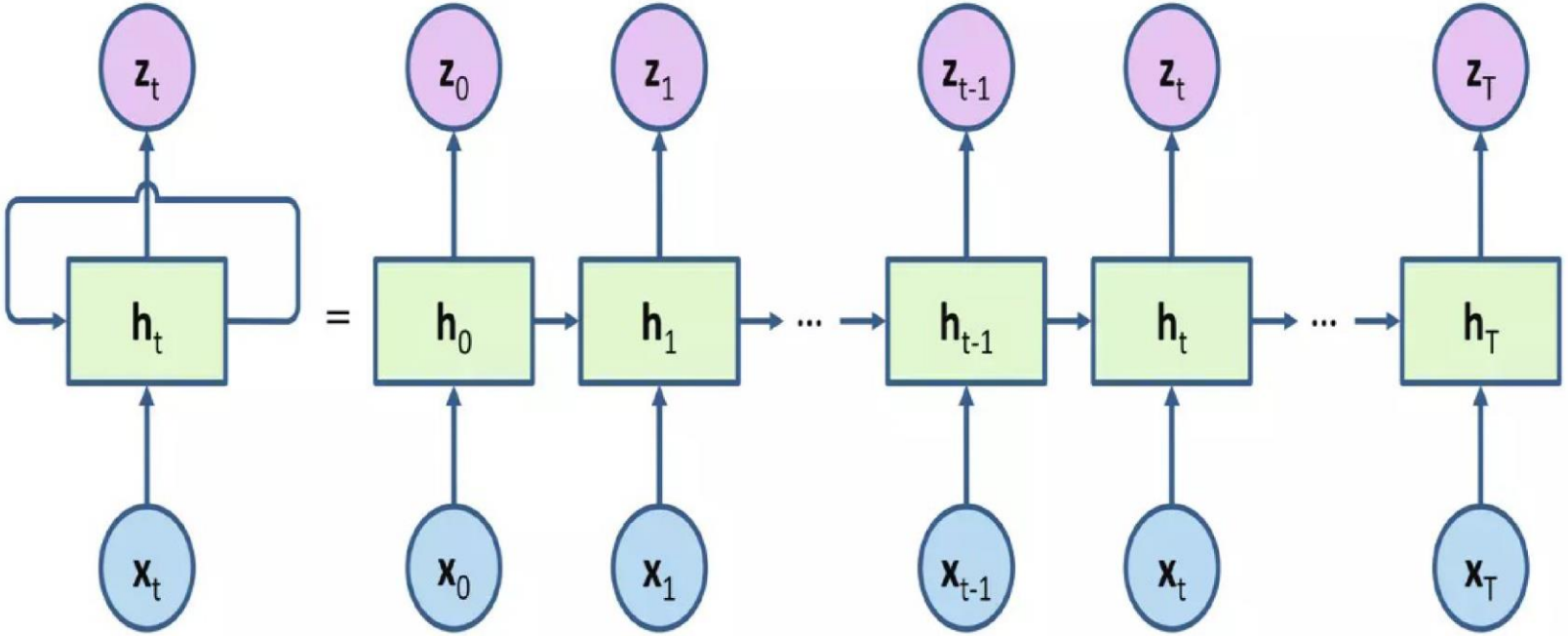
$\mathbf{z}_t$ est une distribution de probabilité sur le vocabulaire elle prédit un mot en sortie.

Parfois on omet la fonction softmax. La sortie devient simplement: $\mathbf{z}_t = \mathbf{W}_{hz}^t\,\mathbf{h}_t$



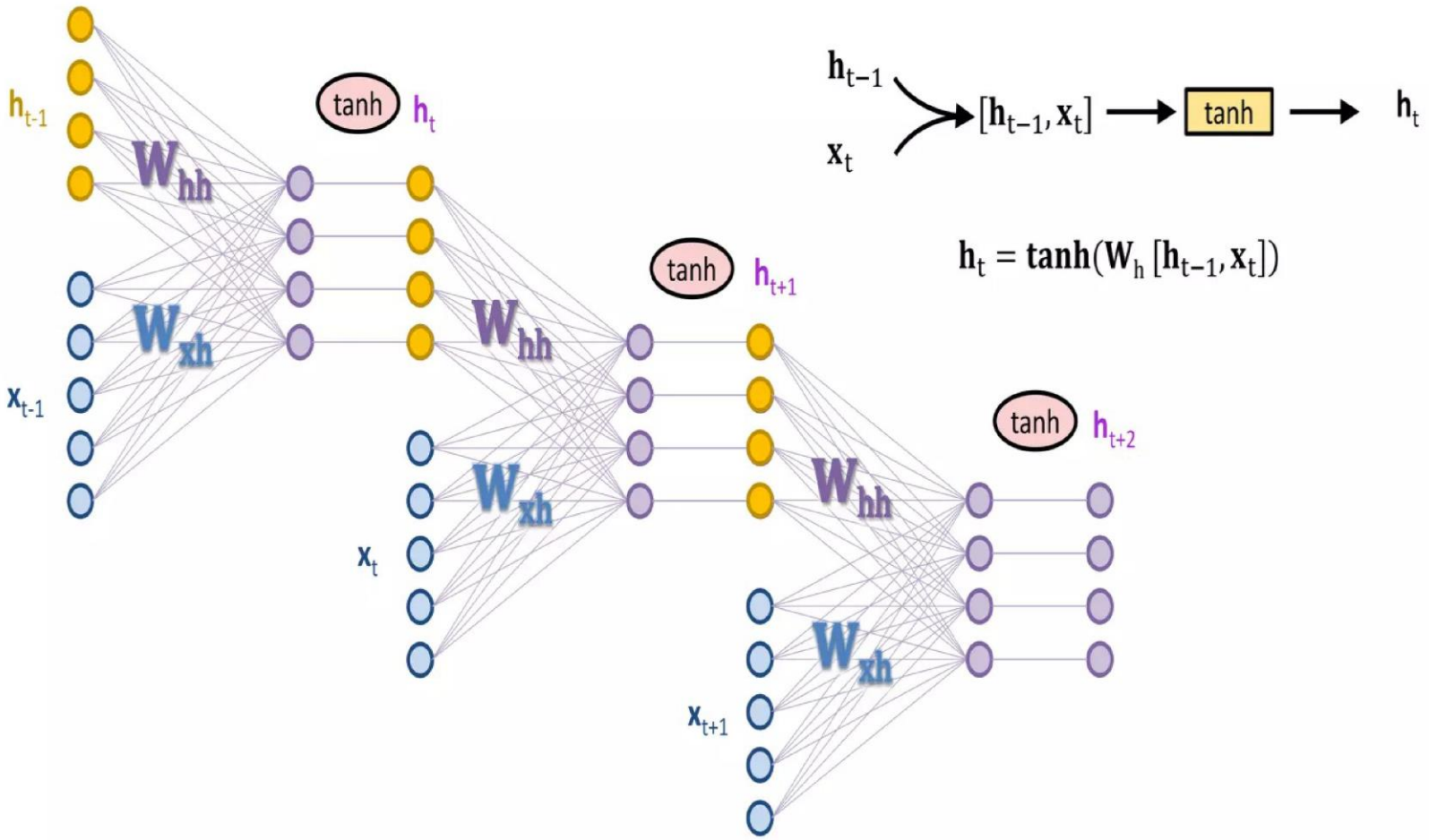
Réseaux de neurones Récurrent

- On propose souvent une présentation du réseau récurrent déroulé pour mieux comprendre son fonctionnement.



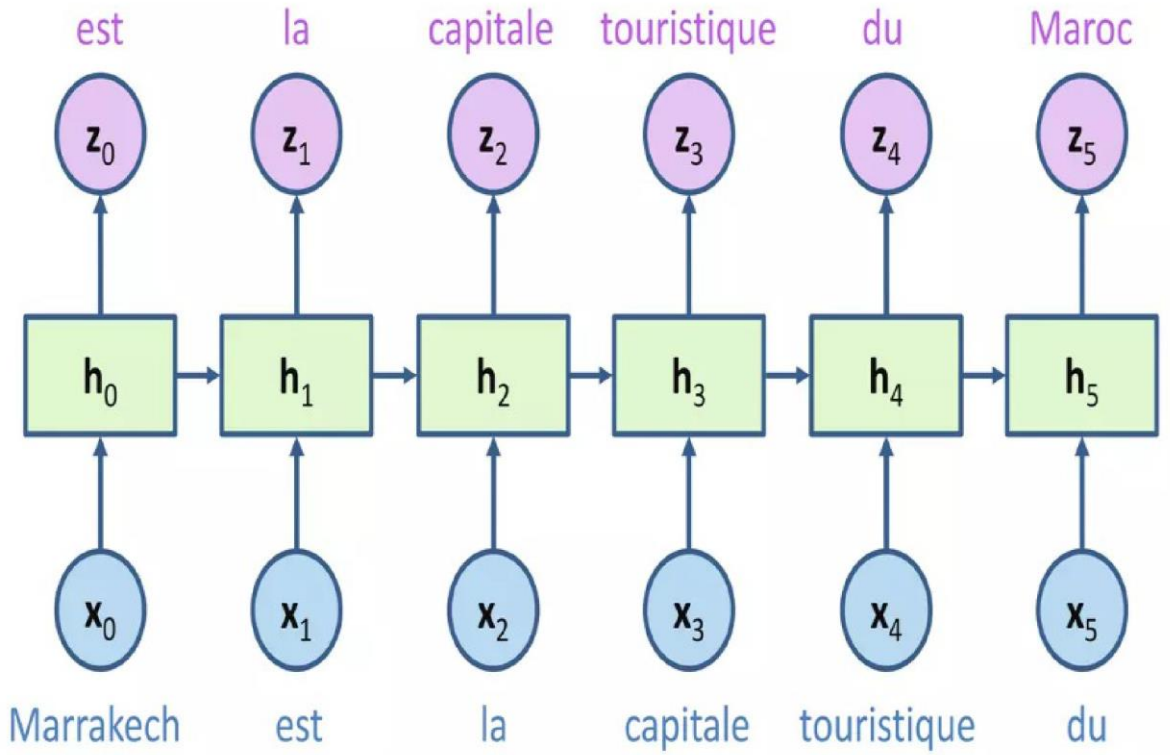
Réseaux de neurones Récurrent

- Un RNN peut ainsi être assimilé à un feedforward neural network comme le montre la figure suivante :



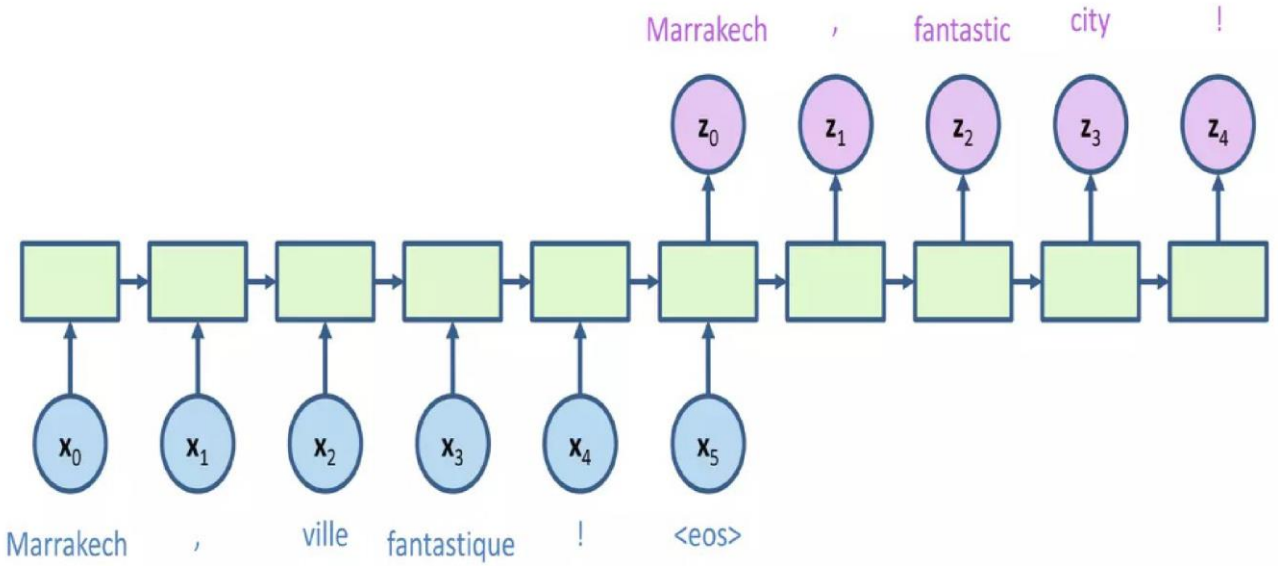
Prédire le mot suivant

- Un réseau récurrent peut par exemple générer une séquence de mots à partir d'un premier mot entré, « Marrakech » dans l'exemple suivant :



Réseau récurrent pour la traduction

- Un réseau récurrent peut aussi être utilisé dans la traduction automatique. Nous présentons ci-dessous une architecture simplifiée basée sur un réseau récurrent. Nous allons revenir à cet exemple avec plus de détails (encodeur/décodeur, modèle de traduction,…) dans de futures présentations.

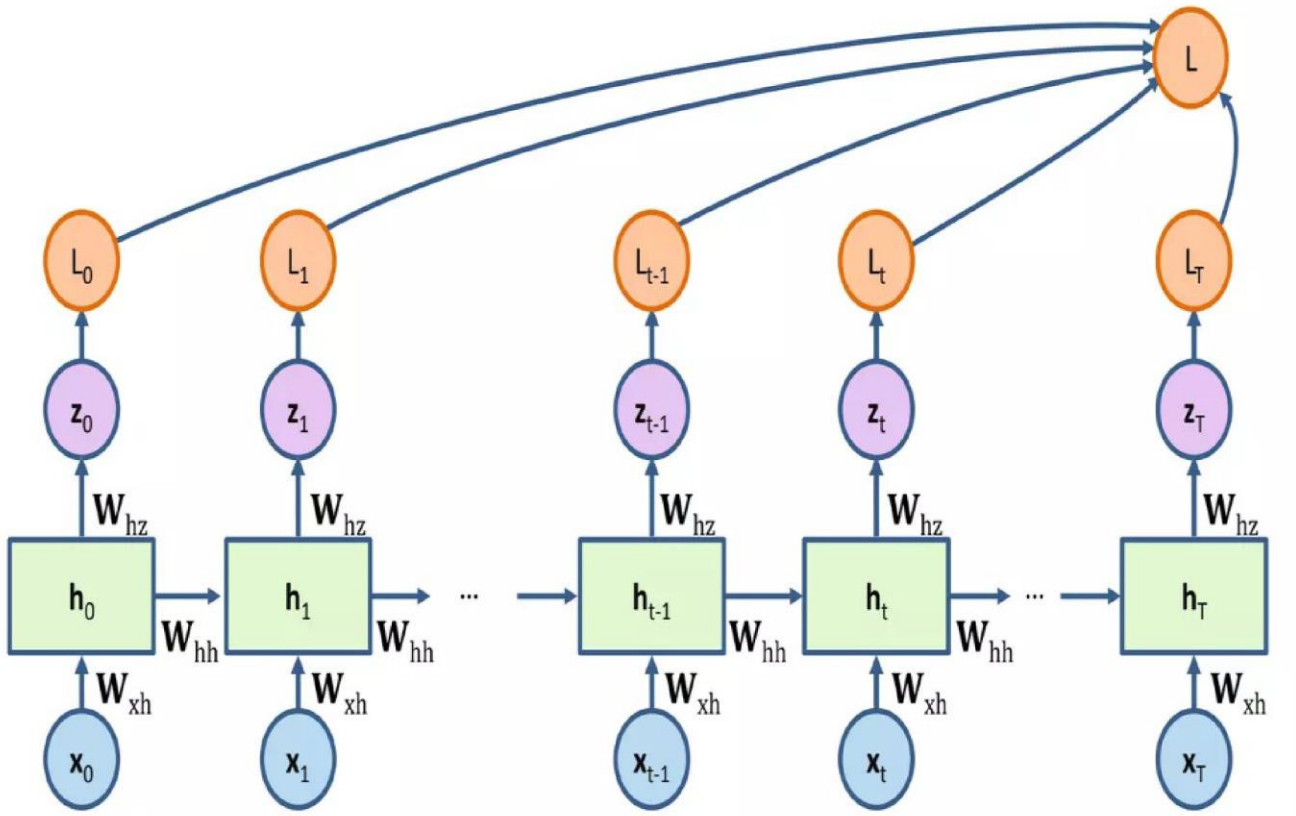


- Les entrées du réseau sont une représentation vectorielle des mots (One-hot encoding, word embedding,…) et non les mots eux-mêmes.

Fonction perte du réseau récurrent

On se place dans un problème de classification de séquences. A chaque pas de temps t, le réseau reçoit une entrée $\mathbf{x}_t$ et retourne une sortie $\mathbf{z}_t$. Soit $\mathbf{y}_t$ la sortie souhaitée. Soit θ le vecteur des paramètres du réseau obtenus à partir de $\mathbf{W}_{hh}$, $\mathbf{W}_{xh}$ et $\mathbf{W}_{hz}$. Le modèle doit minimiser la log-vraisemblance négative :

$$LL(\theta) = -\sum_{t=1}^{T} \log P(\mathbf{y}_t|\mathbf{x}_t)$$

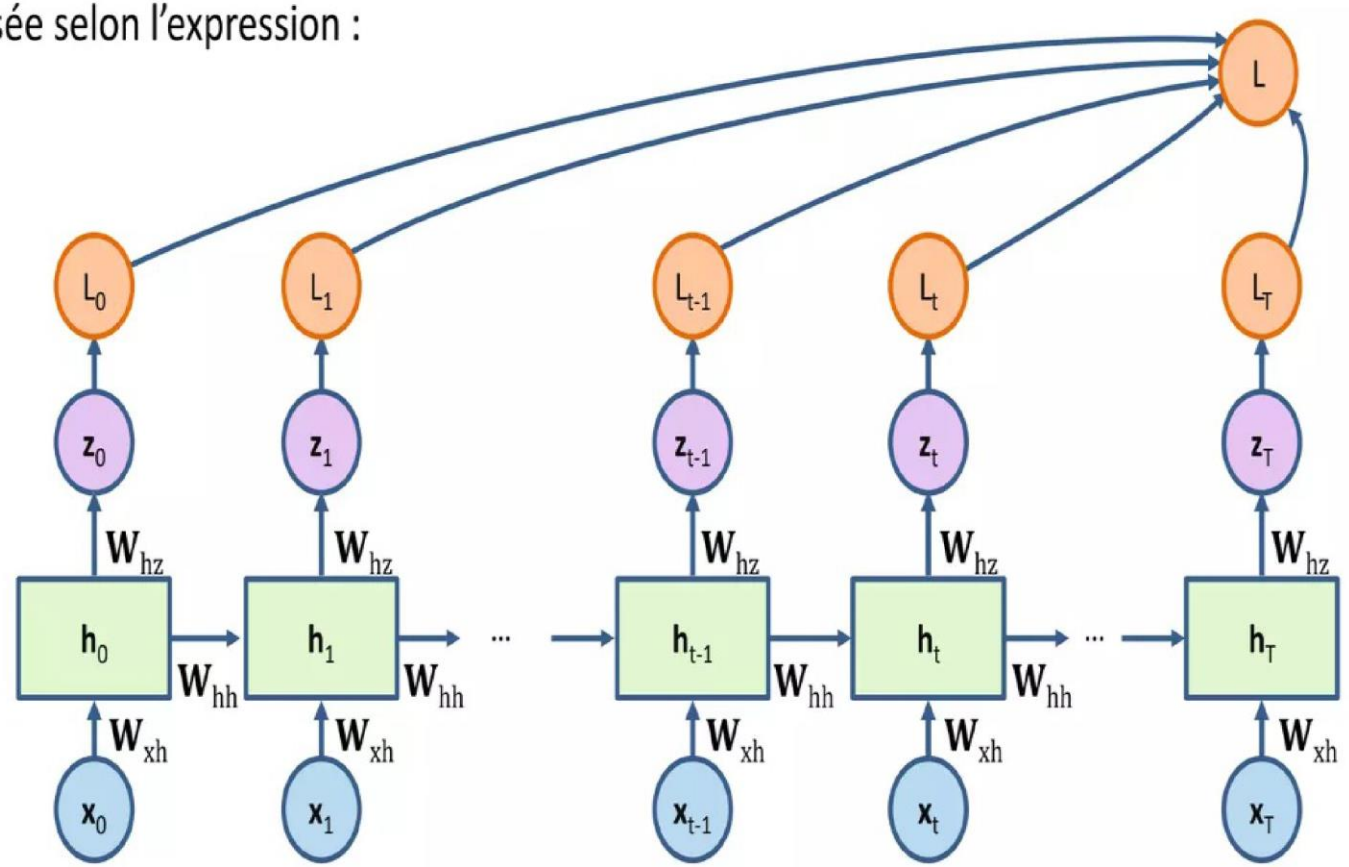


Fonction perte du réseau récurrent

Pour illustrer les concepts introduits, on se place dans un problème de classification de séquences de mots. Pour chaque pas de temps t=1,T, soient $\mathbf{x}_t$ l'entrée du réseau, $\mathbf{z}_t$ le mot retourné et $\mathbf{y}_t$ le mot attendu.

On a $\mathbf{z}_t \in \mathbf{R}^V$ où **V** est la taille du vocabulaire. A chaque pas de temps t=1,T (donc pour chaque mot) on calcule la fonction de perte d'entropie croisée selon l'expression :

$$L_t(\theta) = -\sum_{k=1}^{V} y_{t,k} \log(z_{t,k})$$

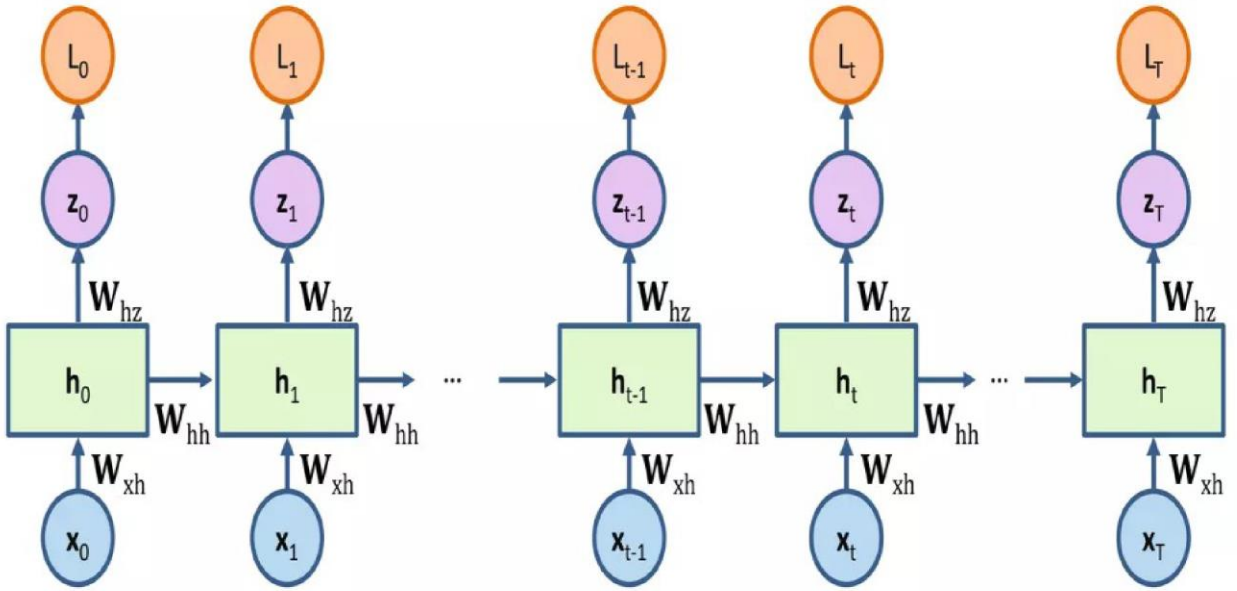


Apprentissage du réseau récurrent

L'apprentissage se fait par rétropropagation du gradient de la fonction de perte sur une séquence ou sur un ensemble de séquences (si taille du batch > 1) :

$$\frac{\partial L(\theta)}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial L_t(\theta)}{\partial \theta}$$

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial h_t}\frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial \mathbf{z}_t}\frac{\partial \mathbf{z}_t}{\partial h_t}\frac{\partial h_t}{\partial \theta}$$



$$L_t(\theta) = -\sum_{k=1}^{V} y_{t,k} \log(z_{t,k})$$

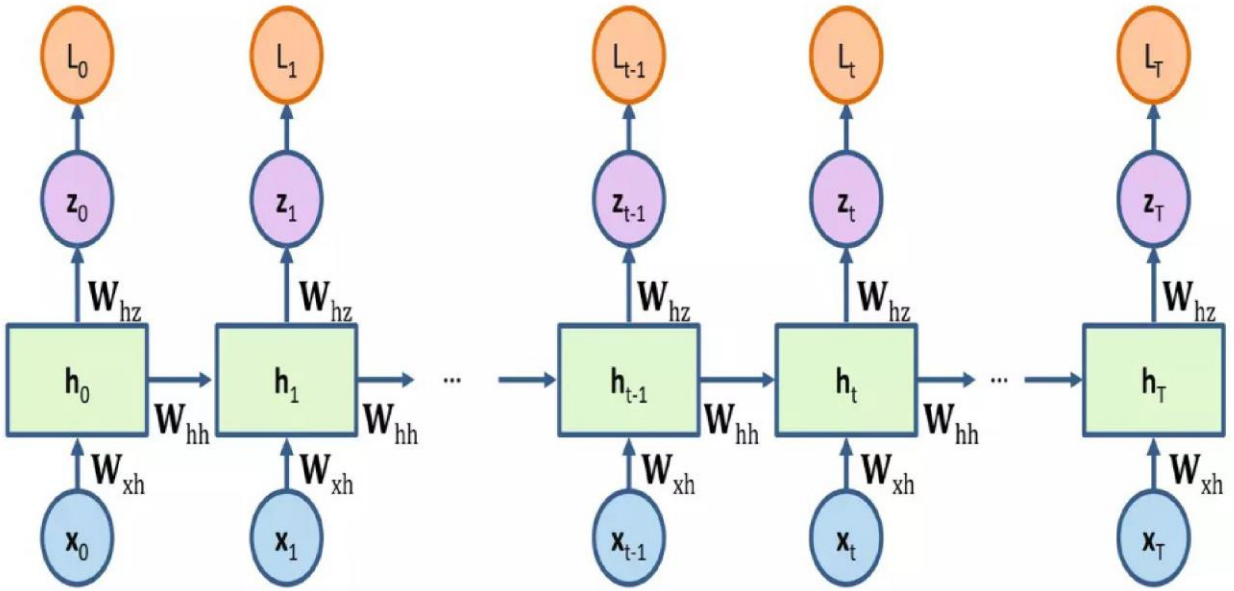
$$z_t = \text{Softmax}(W_{hz} h_t)$$

Apprentissage du réseau récurrent

L'apprentissage se fait par rétropropagation du gradient de la fonction de perte sur une séquence ou sur un ensemble de séquences (si taille du batch > 1) :

$$\frac{\partial L(\theta)}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial L_t(\theta)}{\partial \theta}$$

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial h_t}\frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial \theta}$$



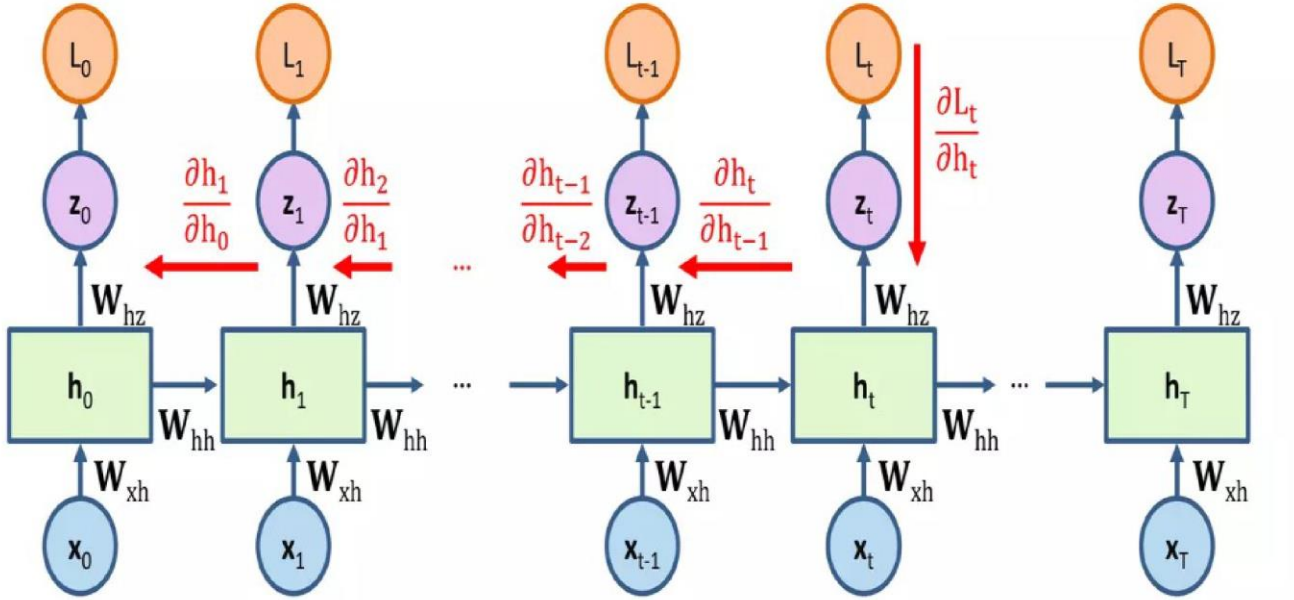
$$\frac{\partial h_t}{\partial \theta} = \frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial h_{t-2}} \cdots \frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial h_0}\frac{\partial h_0}{\partial \theta}$$

Apprentissage du réseau récurrent

L'apprentissage se fait par rétropropagation du gradient de la fonction de perte sur une séquence ou sur un ensemble de séquences (si taille du batch > 1) :

$$\frac{\partial L(\theta)}{\partial \theta} = \sum_{t=1}^{T} \frac{\partial L_t(\theta)}{\partial \theta}$$

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial h_t}\frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial h_{t-2}}\cdots\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial h_0}\frac{\partial h_0}{\partial \theta}$$



On parle alors de rétropropagation à travers le temps (Back prop through time BPTT).

Vanishing Gradients du RNN

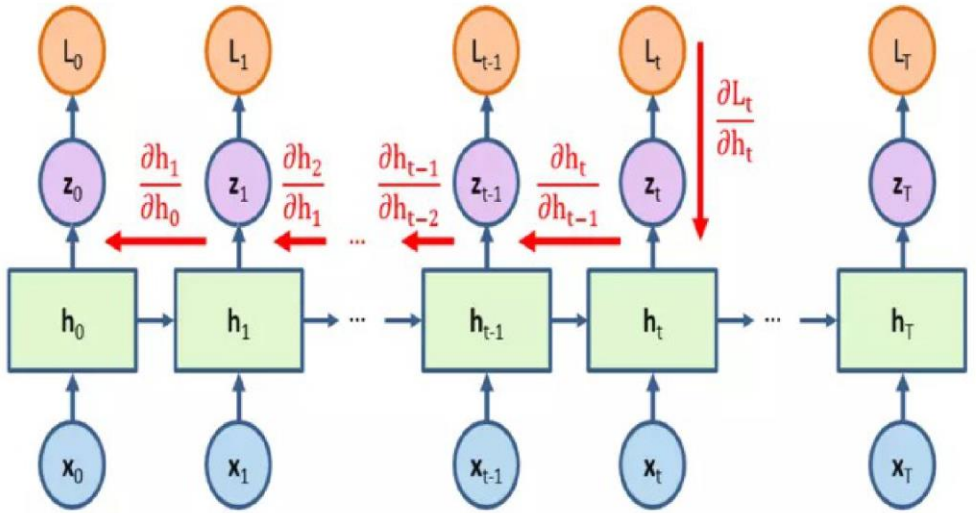
On a :

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t)$$

On pose $\phi_t = \mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t$, ainsi $\mathbf{h}_t = \tanh(\phi_t)$

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} = \frac{\partial \mathbf{h}_t}{\partial \phi_t}\frac{\partial \phi_t}{\partial \mathbf{h}_{t-1}} = \mathrm{diag}(\tanh'(\phi_t))\mathbf{W}_{hh}$$

$\tanh'(x) = 1 - \tanh^2(x)$



est bornée par α=1. Si on avait choisi une sigmoïde σ à la place de tanh, alors σ' serait bornée par α=0.25 par exemple.

On peut ainsi écrire :

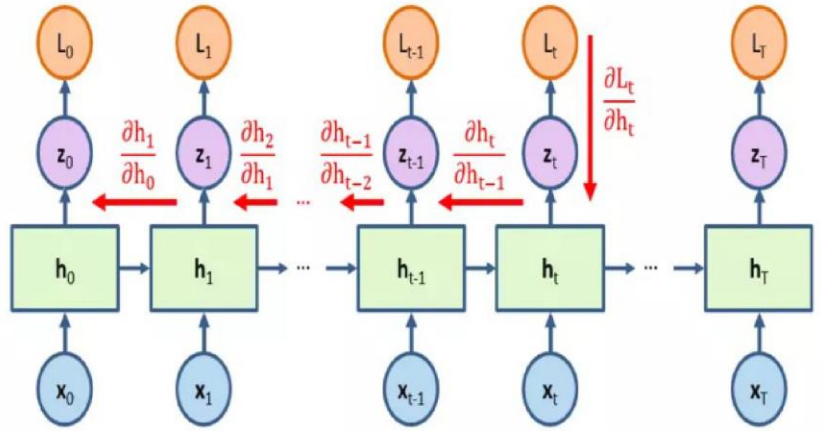
$$\left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_{t-1}} \cdots \frac{\partial \mathbf{h}_1}{\partial \mathbf{h}_0} \right\| \leq (\alpha \|\mathbf{W}_{hh}\|)^t$$

Vanishing Gradients du RNN

On a :

$$\frac{\partial L_t(\theta)}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial h_t}\frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial \theta} = \frac{\partial L_t(\theta)}{\partial z_t}\frac{\partial z_t}{\partial h_t}\frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial h_{t-2}}\cdots\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial h_0}\frac{\partial h_0}{\partial \theta}$$

$$\left\|\frac{\partial h_t}{\partial h_{t-1}}\frac{\partial h_{t-1}}{\partial h_{t-2}}\cdots\frac{\partial h_2}{\partial h_1}\frac{\partial h_1}{\partial h_0}\right\| \leq (\alpha\|W_{hh}\|)^t$$



Deux situations se présentent :

1. $\alpha\|W_{hh}\| < 1$, alors il y a disparition du gradient (vanishing gradient):

Donc $\frac{\partial L_t(\theta)}{\partial \theta}$ devient presque nul surtout lorsque T est grand (séquence longue)

2. $\alpha\|W_{hh}\| > 1$, alors il y a explosion du gradient (exploding gradient).

Dans les deux cas, le réseau récurrent trouve des difficultés à apprendre.

Limites des réseaux RNN

- Les RNN ne permettent pas de mémoriser les dépendances à long ou très long terme.
- Par exemple, dans la phrase suivante, le réseau récurrent risque de ne pas considérer le lien entre les mots « Marrakech » et « rouge » :



Solutions au Pb du vanishing gradient

Pour éviter la disparition ou l'explosion du gradient :

- Arrêter la rétropropagation après K termes, même si cela ne permet pas de mettre à jour tous les poids,
- Maintenir le gradient dans un intervalle fixé à l'avance,
- Pénaliser ou appliquer des techniques pour ajuster le gradient

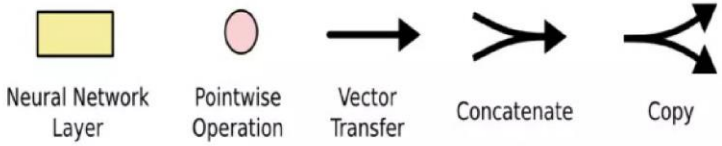
Reste que les LSTM sont considérés comme incontournables pour la mise en œuvre des RNN. Ces réseaux seront décrits dans une future présentation.

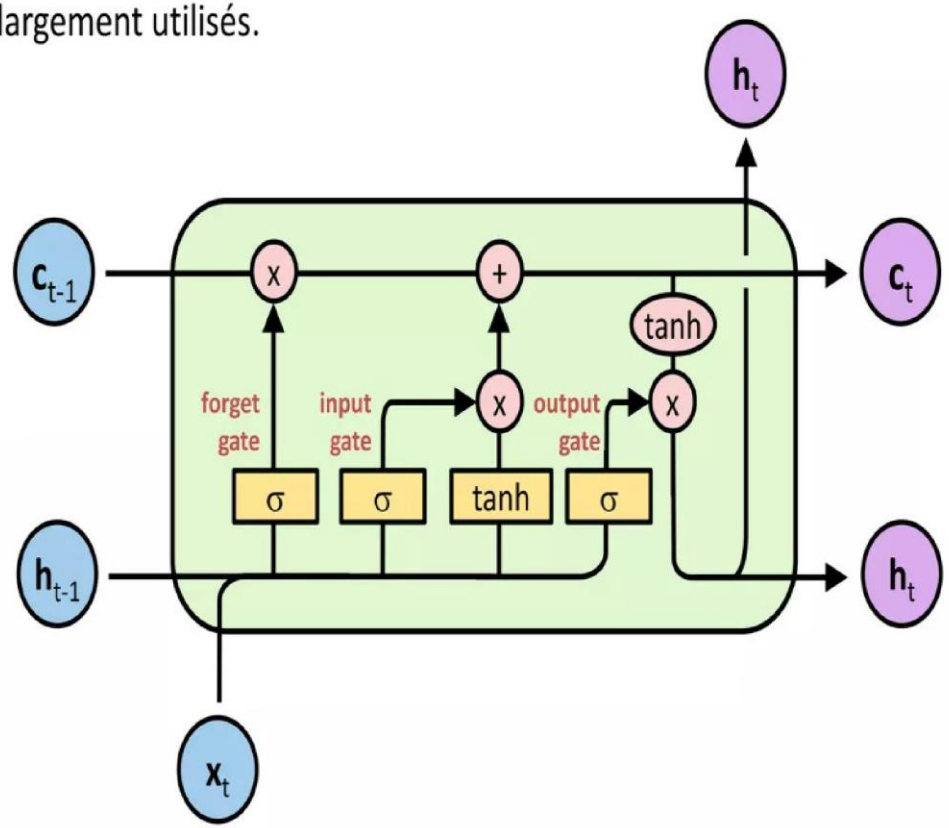
Réseaux LSTM

Les réseaux LSTM (Long Short Term Memory ou mémoire à long terme et à court terme) sont un type spécial de RNN, capable d'apprendre les dépendances à long terme. Ils ont été introduits par Hochreiter et Schmidhuber en 1997, et ont été par la suite affinés et popularisés à travers plusieurs travaux. Ils fonctionnent extrêmement bien sur une grande variété de problèmes et sont maintenant largement utilisés.

Nous allons présenter les LSTM dans une suite de transformations des réseaux RNN standard.

Nous utiliserons alors les symboles suivants(*) :

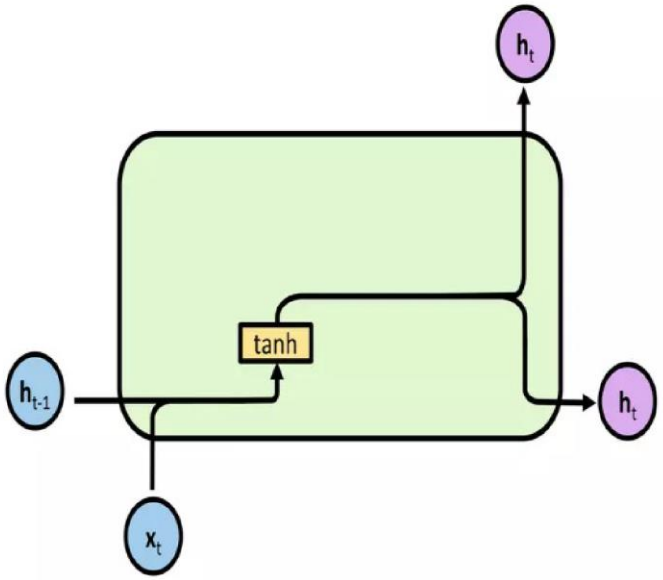




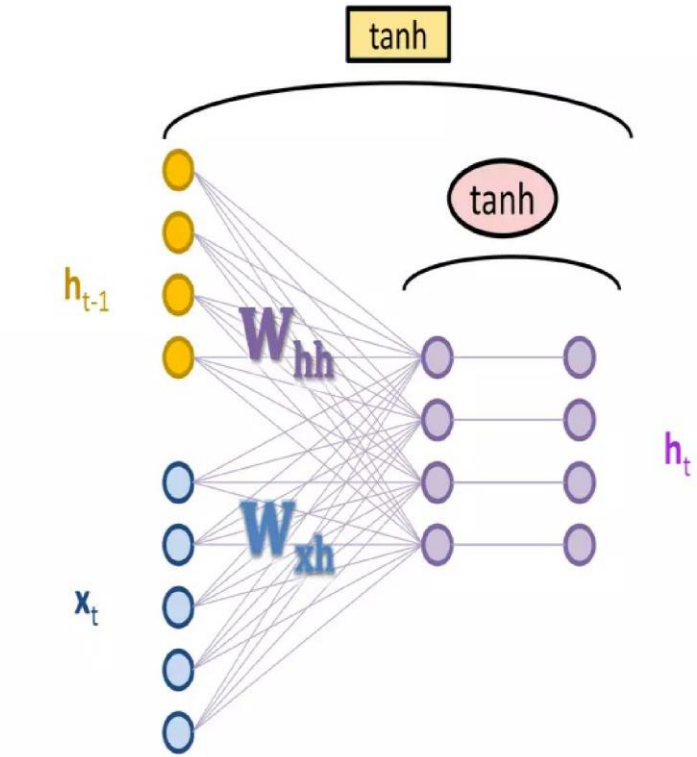
RNN standard

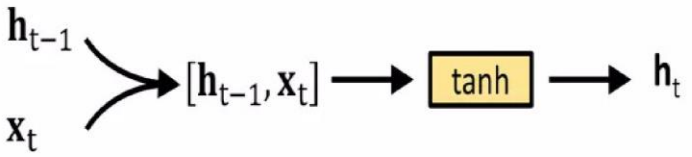
Pour un RNN standard, l'état caché est actualisé par la formule :

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t)$$



tanh est utilisée pour aider à réguler les valeurs circulant à travers le réseau. La fonction tanh écrase les valeurs pour qu'elles soient toujours comprises entre -1 et 1.

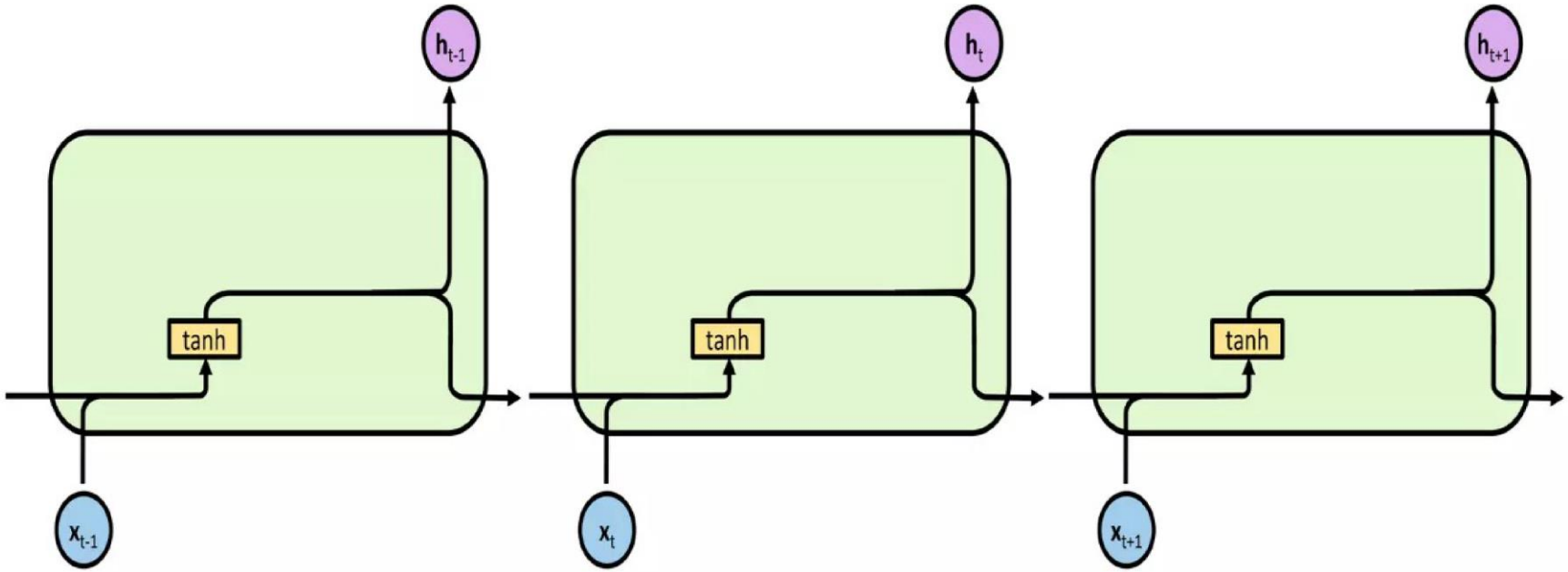




$$\mathbf{h}_t = \tanh(\mathbf{W}_h\,[\mathbf{h}_{t-1}, \mathbf{x}_t])$$

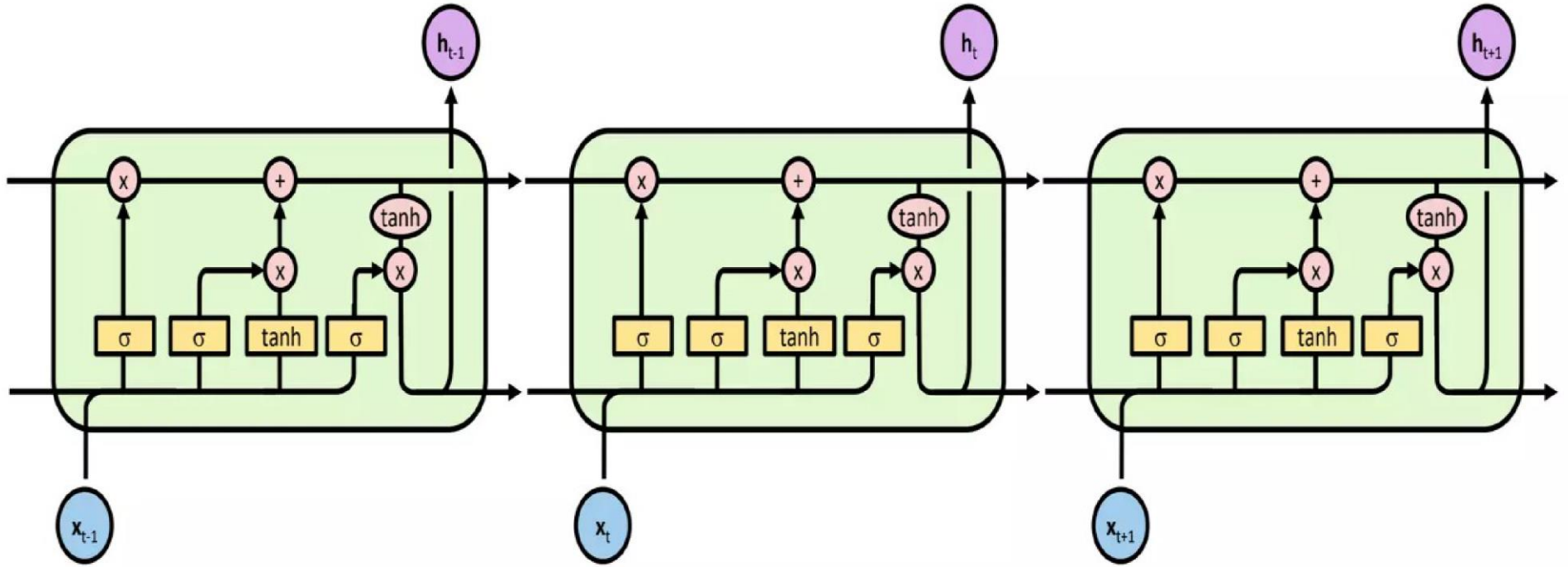
Réseaux RNN

Tous les réseaux de neurones récurrents ont la forme d'une chaîne de modules (ou cellules) répétitifs de réseau de neurones. Dans les RNN standard, ce module répétitif aura une structure très simple, telle qu'une seule couche de tanh.



Réseaux LSTM

Les LSTM ont également cette structure en chaîne, sauf que le module répétitif au lieu d'avoir une seule couche de réseau de neurones dispose de quatre, qui ont des rôles spéciaux que nous allons détailler par la suite.



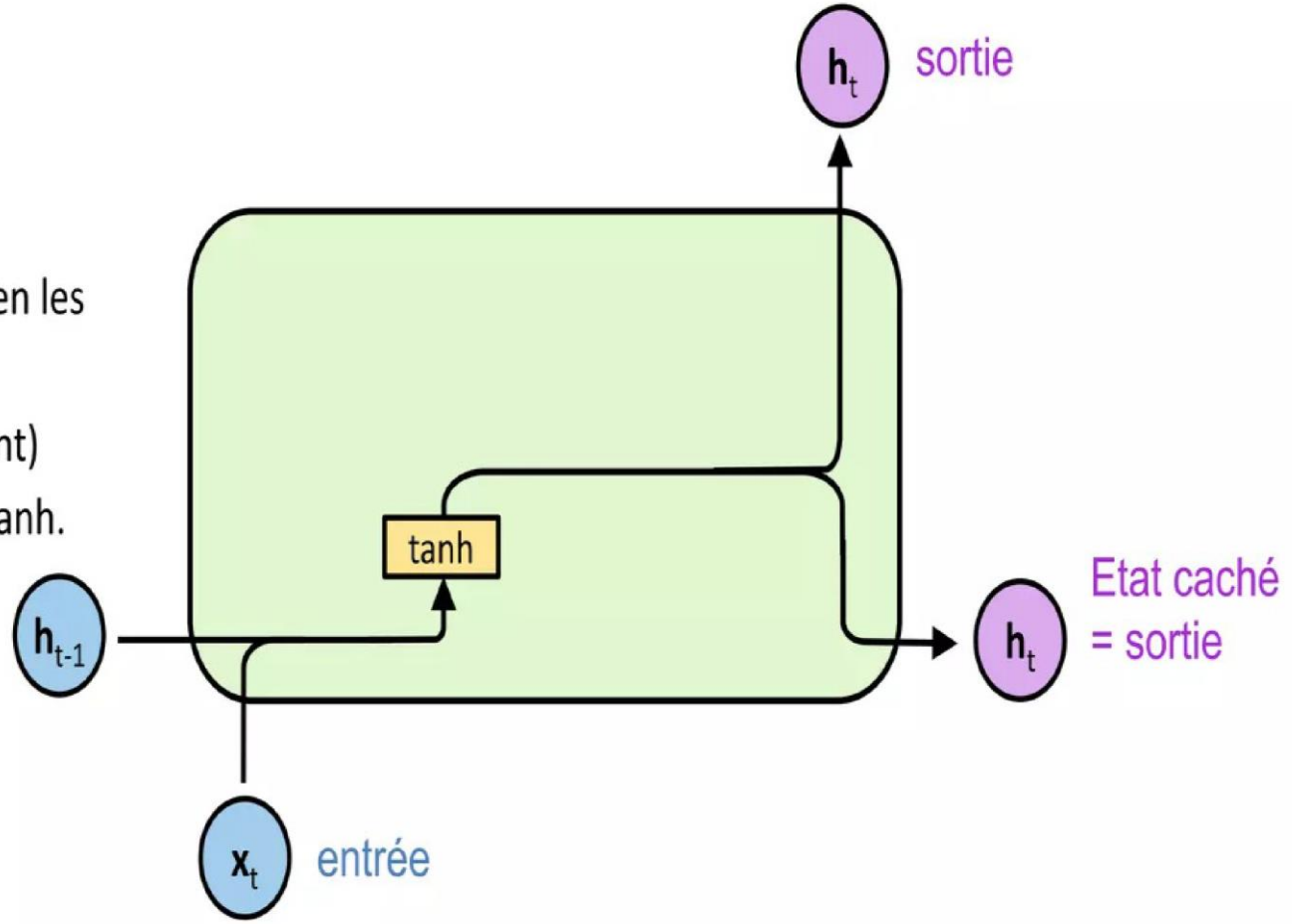
Cellule RNN

Nous allons présenter les LSTM dans une suite de transformations du réseau RNN standard. L'état caché est actualisé par la formule :

Equations

$$\mathbf{h}_t = \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t)$$

Le plus souvent ce réseau ne gère pas bien les dépendances à long terme à cause de la disparition du gradient (vanishing gradient) comme conséquence de l'utilisation de tanh.



Construire la cellule LSTM

Les LSTM sont explicitement conçus pour éviter le problème de dépendance à long terme. On introduit une nouvelle variable $\mathbf{c}_t$ qui permettra de se souvenir des informations pendant de longues périodes.

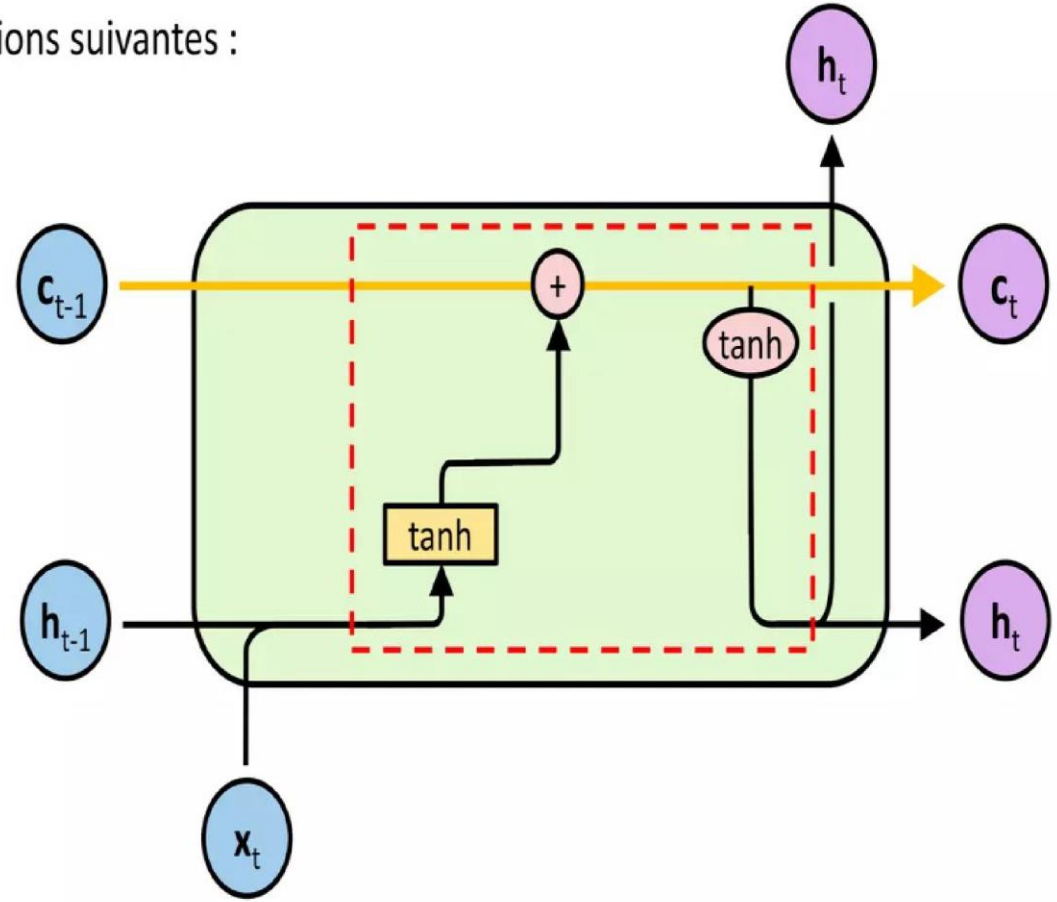
La mise à jour des variables se fera alors selon les équations suivantes :

Equations :

$$\mathbf{c}_t = \mathbf{c}_{t-1} + \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t)$$

$$\mathbf{h}_t = \tanh(\mathbf{c}_t)$$

Cette fois, on est assuré que l'état caché $\mathbf{h}_t$ calculé à l'instant t tient compte de tous les états cachés précédents.



Construire la cellule LSTM

Les LSTM sont explicitement conçus pour éviter le problème de dépendance à long terme. On introduit une nouvelle variable c_t qui permettra de se souvenir des informations pendant de longues périodes.

c_t est appelée état de la cellule.

La mise à jour des variables se fera alors selon les équations suivantes :

Equations :

$$c_t = c_{t-1} + \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t)$$

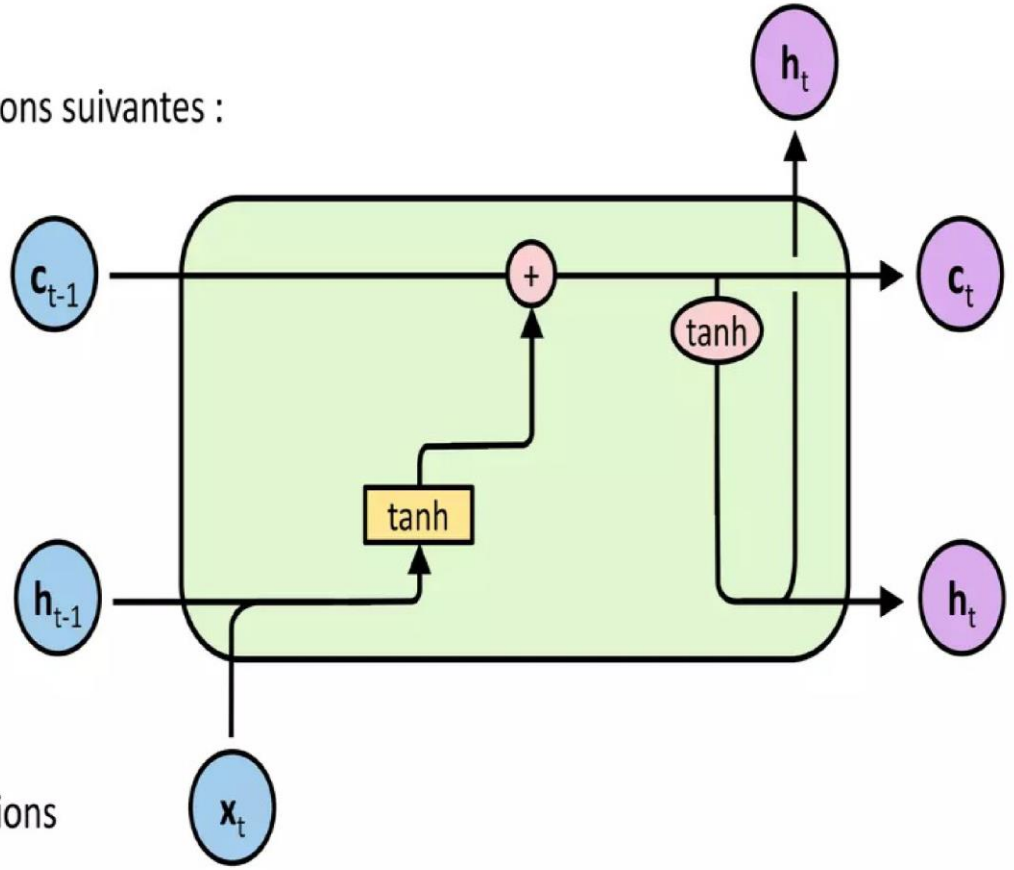
$$h_t = \tanh(c_t)$$

Inconvénient :

Le passé est toujours aussi important que le présent.
Ce qui n'est pas pertinent.

Solution idéale :

Permettre au réseau de garder ou d'oublier les informations du passé selon qu'elles soient déterminantes ou pas.



Construire la cellule LSTM

La solution adoptée est de pondérer la mémoire du passé représentée par $\mathbf{c}_t$ par un réseau de neurones avec une seule couche sigmoïde. Ce réseau va apprendre à oublier les informations non pertinentes. Il est appelé porte d'oubli ou forget gate.

Equations :

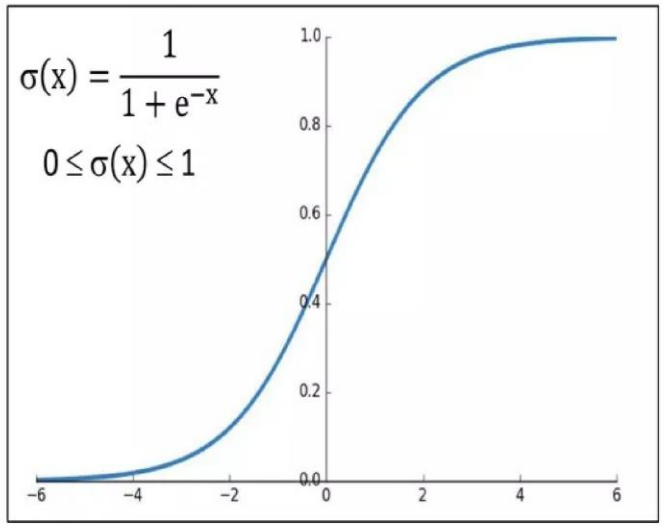
$$f_t = \sigma(W_{hf} h_{t-1} + W_{xf} x_t)$$

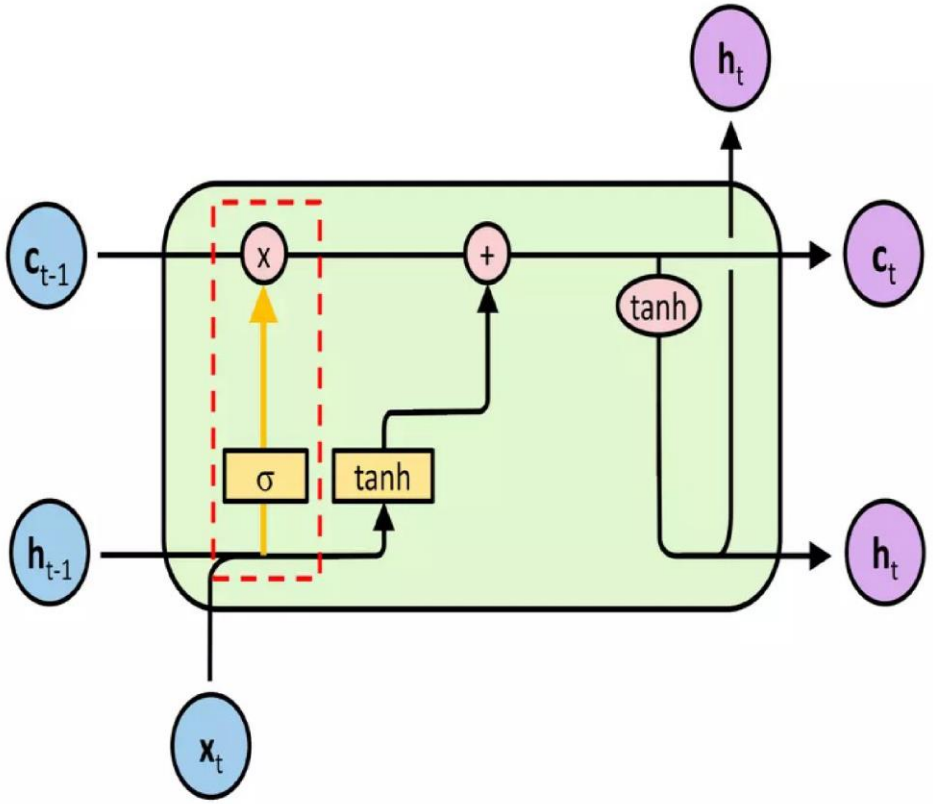
$$c_t = f_t * c_{t-1} + \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$h_t = \tanh(c_t)$$

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

$$0 \leq \sigma(x) \leq 1$$





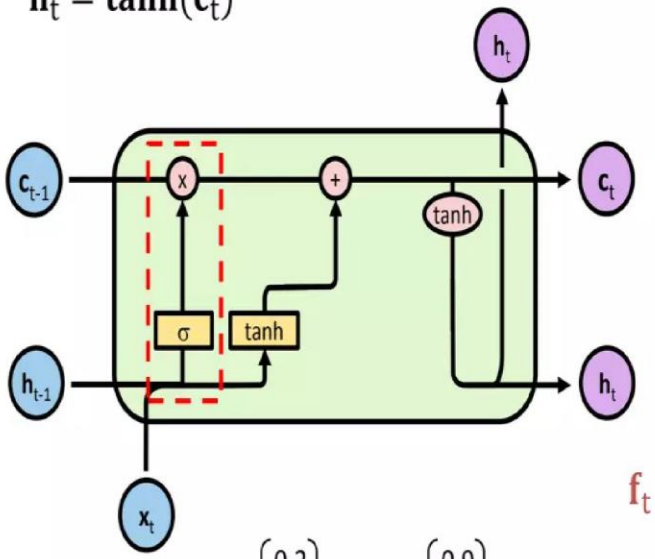
Construire la cellule LSTM

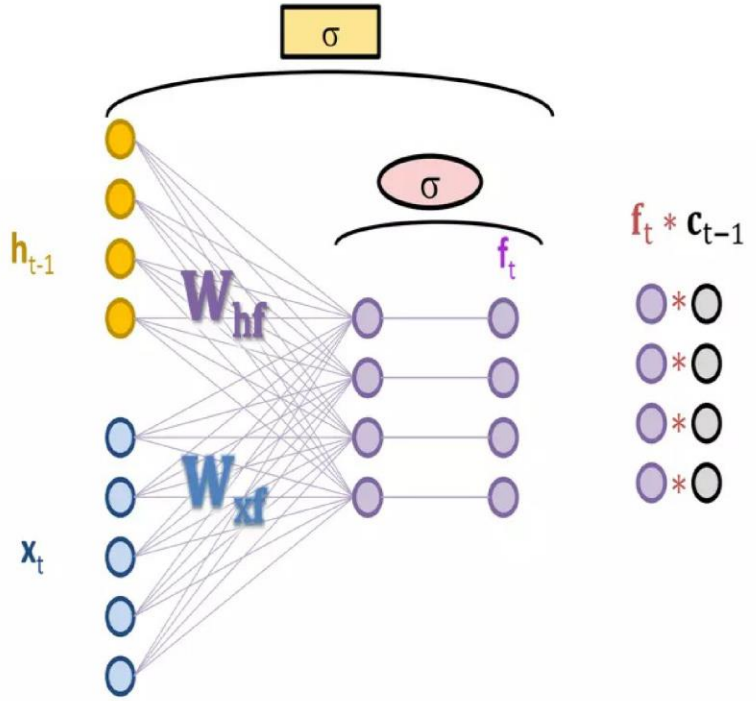
Equations :

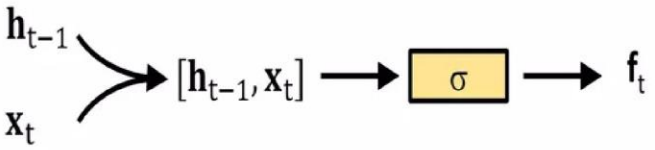
$$f_t = \sigma(W_{hf} h_{t-1} + W_{xf} x_t)$$

$$c_t = f_t * c_{t-1} + \tanh(W_{hh} h_{t-1} + W_{xh} x_t)$$

$$h_t = \tanh(c_t)$$







$$c_{t-1} = \begin{pmatrix} 0.2 \\ -0.4 \\ 0.75 \\ 0.83 \end{pmatrix} \quad f_t = \begin{pmatrix} 0.9 \\ 0.01 \\ 0.3 \\ 0.95 \end{pmatrix} \Rightarrow \quad f_t * c_{t-1} \quad c_t = \begin{pmatrix} 0.18 \\ 0.00 \\ 0.23 \\ 0.79 \end{pmatrix} \leftarrow \text{oubli}$$

Construire la cellule LSTM

Une autre porte est ajoutée pour pondérer la mise à jour (additive) de l'état de la cellule $\mathbf{c}_t$ par la sortie tanh prenant comme entrée $\mathbf{h}_{t-1}$ et $\mathbf{x}_t$. Cette porte qui s'appelle input gate va apprendre à utiliser, à ignorer ou à moduler les informations entrées selon leur importance.

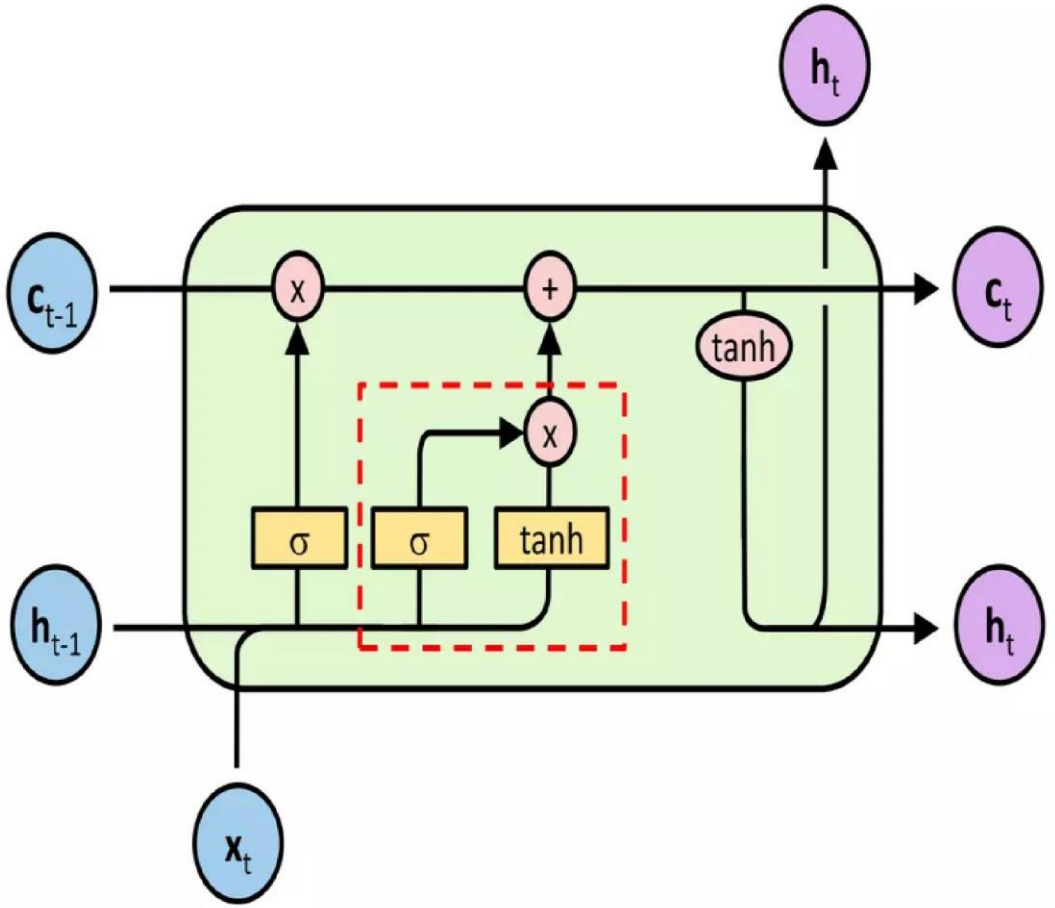
Equations :

$$\mathbf{f}_t = \sigma(\mathbf{W}_{hf}\,\mathbf{h}_{t-1} + \mathbf{W}_{xf}\,\mathbf{x}_t)$$

$$\mathbf{i}_t = \sigma(\mathbf{W}_{hi}\,\mathbf{h}_{t-1} + \mathbf{W}_{xi}\,\mathbf{x}_t)$$

$$\mathbf{c}_t = \mathbf{f}_t * \mathbf{c}_{t-1} + \mathbf{i}_t * \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t)$$

$$\mathbf{h}_t = \tanh(\mathbf{c}_t)$$



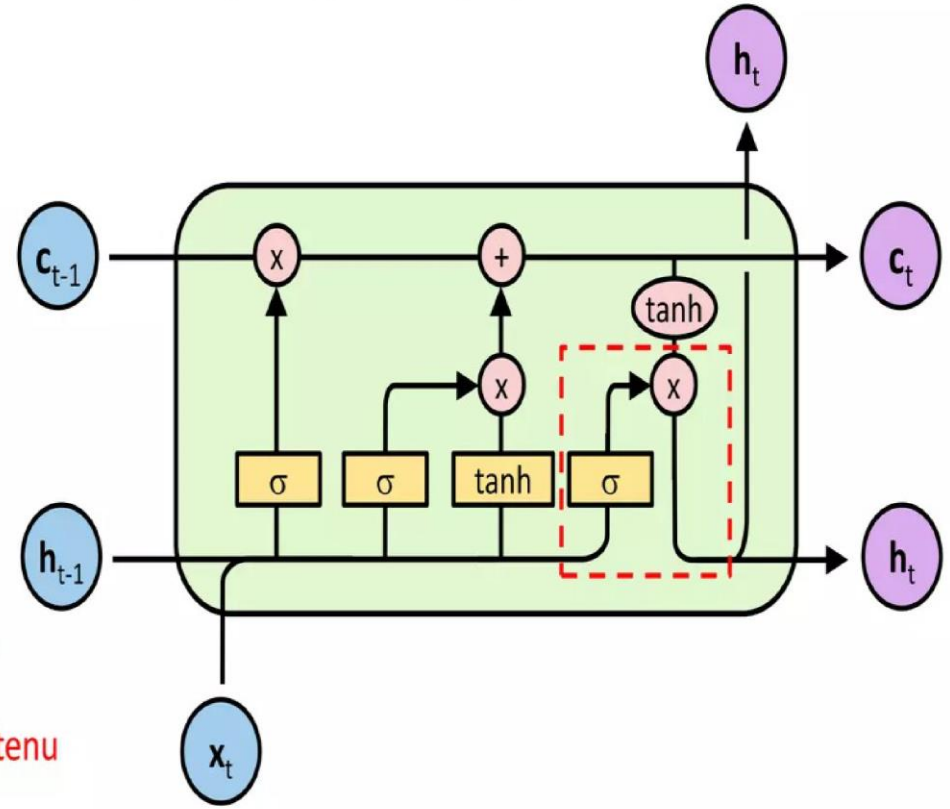
Construire la cellule LSTM

De la même manière, une porte de sortie est ajoutée pour pondérer la mise à jour de l'état caché $\mathbf{h}_t$. Cela permet de décider quelles informations l'état caché $\mathbf{h}_t$ doit porter. Cette porte qui s'appelle output gate.

Equations :

$$\mathbf{f}_t = \sigma(\mathbf{W}_{hf}\,\mathbf{h}_{t-1} + \mathbf{W}_{xf}\,\mathbf{x}_t)$$
$$\mathbf{i}_t = \sigma(\mathbf{W}_{hi}\,\mathbf{h}_{t-1} + \mathbf{W}_{xi}\,\mathbf{x}_t)$$
$$\mathbf{o}_t = \sigma(\mathbf{W}_{ho}\,\mathbf{h}_{t-1} + \mathbf{W}_{xo}\,\mathbf{x}_t)$$

$$\mathbf{c}_t = \mathbf{f}_t * \mathbf{c}_{t-1} + \mathbf{i}_t * \tanh(\mathbf{W}_{hh}\,\mathbf{h}_{t-1} + \mathbf{W}_{xh}\,\mathbf{x}_t)$$
$$\mathbf{h}_t = \mathbf{o}_t * \tanh(\mathbf{c}_t)$$



Etat caché $\mathbf{h}_t$

$\tanh(\mathbf{c}_t) = \begin{pmatrix} -0.8 \\ -0.3 \\ 0.64 \\ -0.01 \end{pmatrix}$ $\mathbf{o}_t = \begin{pmatrix} 0.96 \\ 0.1 \\ 0.01 \\ 0.38 \end{pmatrix}$ ⇒ $\mathbf{h}_t = \begin{pmatrix} -0.77 \\ -0.03 \\ 0.01 \\ 0.00 \end{pmatrix}$ ← retenu ← non retenu

LSTM, une synthèse

Forget gate : $f_t = \sigma(W_{hf}\, h_{t-1} + W_{xf}\, x_t)$

Dans le cas extrême, indique à l'état de la cellule (mémoire à long-terme) les informations à oublier (multiplication par 0) ou à conserver (multiplication par 1).

Input gate : $i_t = \sigma(W_{hi}\, h_{t-1} + W_{xi}\, x_t)$

déterminer quelles informations produite par la couche tanh doivent entrer dans l'état de la cellule (donc à sauvegarder dans la mémoire à long terme).

Output gate : $o_t = \sigma(W_{ho}\, h_{t-1} + W_{xo}\, x_t)$

Indique les informations qui doivent passer au prochain état caché h_t.

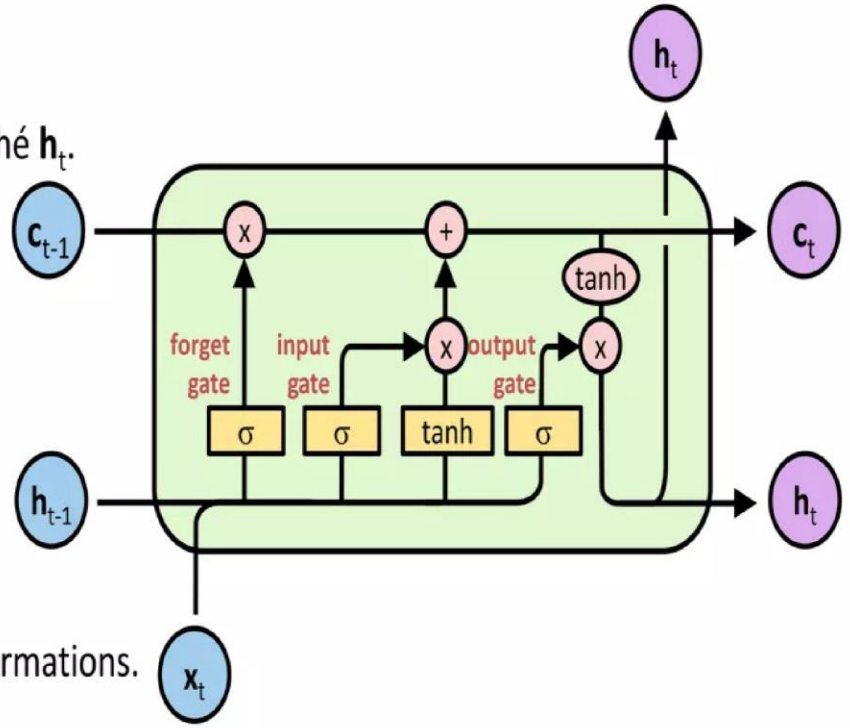
Calcul de c_t et de h_t :

$$c_t = f_t * c_{t-1} + i_t * \tanh(W_{hh}\, h_{t-1} + W_{xh}\, x_t)$$

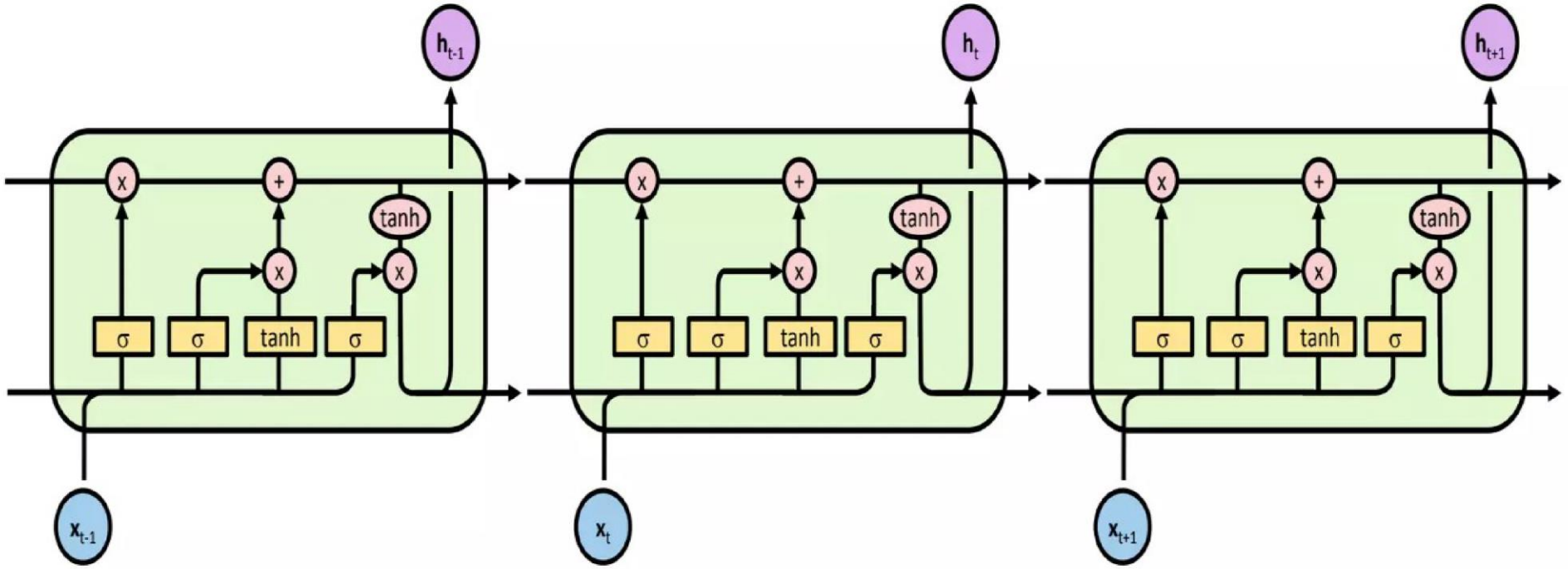
$$h_t = o_t * \tanh(c_t)$$

L'état de la cellule c_t parcourt toute la chaîne du réseau avec des interactions linéaires mineures, ce qui leur permet de transporter efficacement les informations.

Les portes (gates) permettent de réguler intelligemment ces informations.



Réseau LSTM



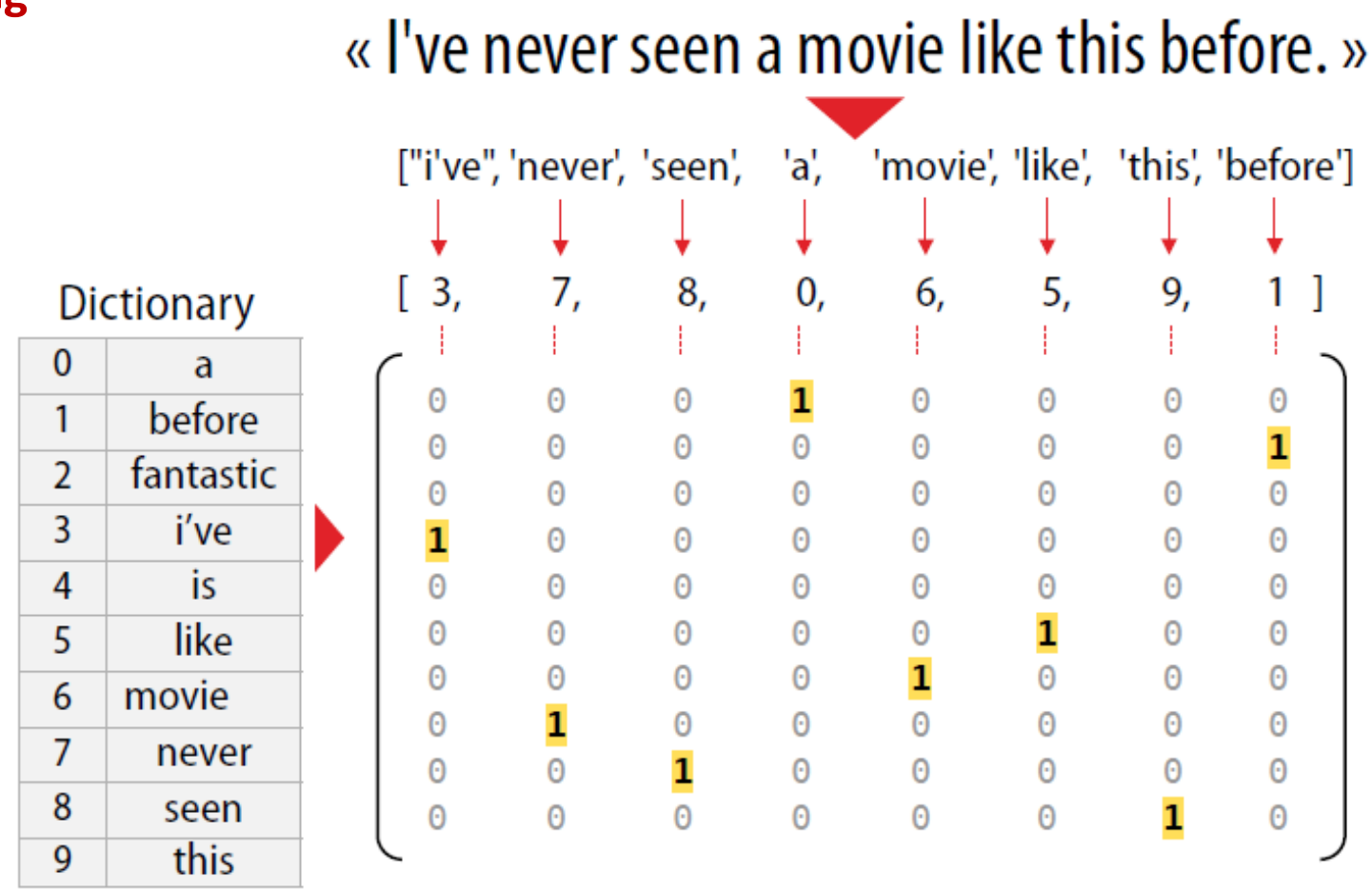
Vectorisation de Texte

- Comment alimenter un réseau neuronal avec du texte

- Text encoding
  - One-hot encoding
  - Word2vec
  - Gloves et FastText
  - TF-IDF
  - Count vectorizer

Vectorisation de Texte

One-hot encoding





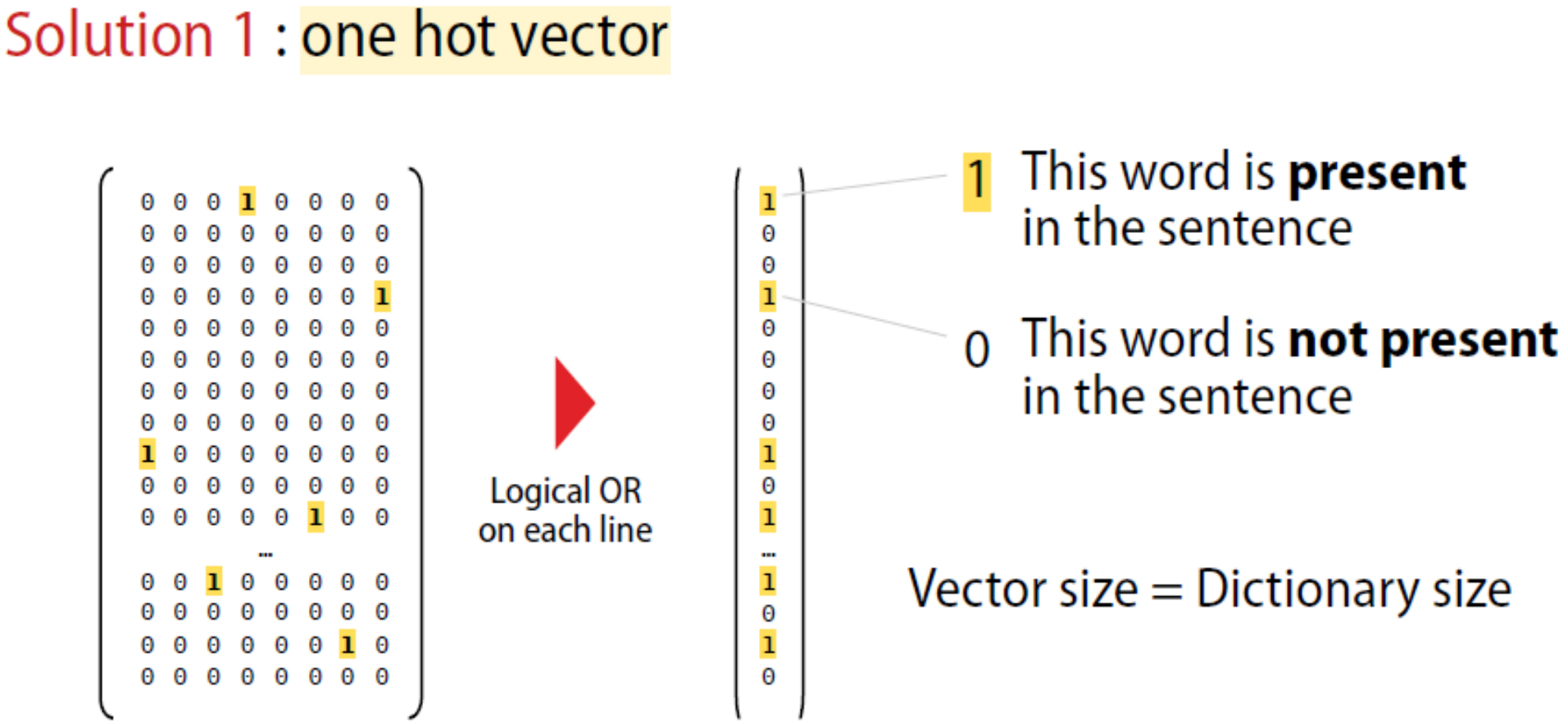




Dictionnaire = 80 000 mots
Phrase = 300 mots

Besoin de 24 10^6 Paramètres

One-hot encoding

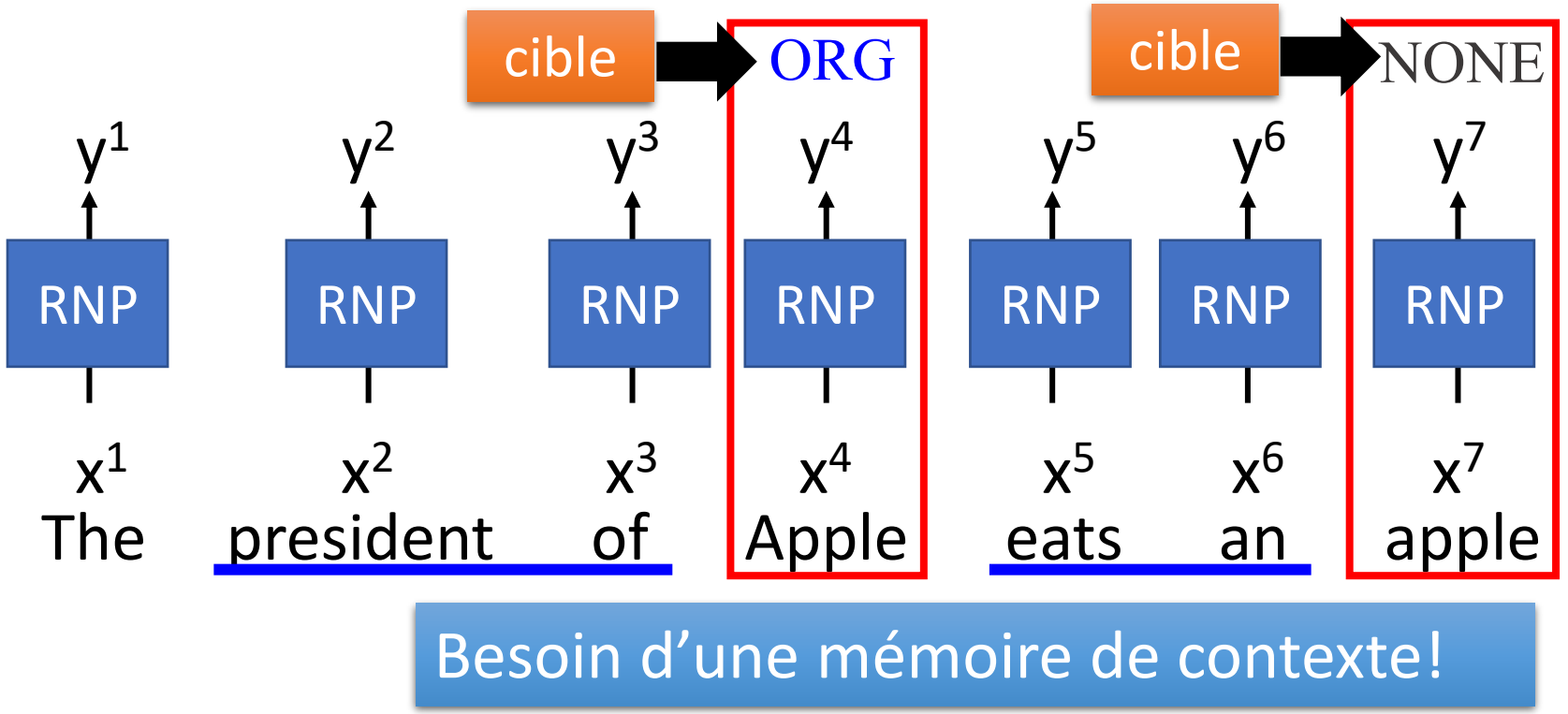


La taille ne dépend plus de la longueur de la phrase.
La taille est drastiquement réduite, mais on perd l'ordre des mots.

Vectorisation de Texte

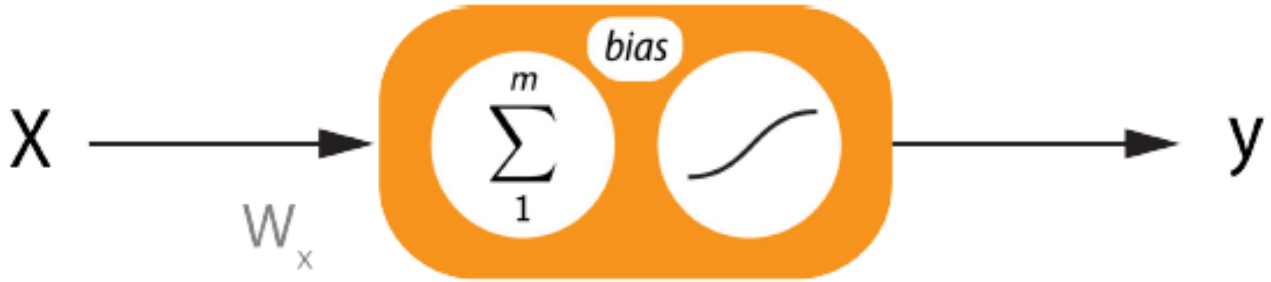
Réseaux de neurones Récurrent

- Par exemple, interprétation de noms d'entités
  - Personnes, lieux, organisations, etc. dans une phrase



Réseaux de neurones Récurrent

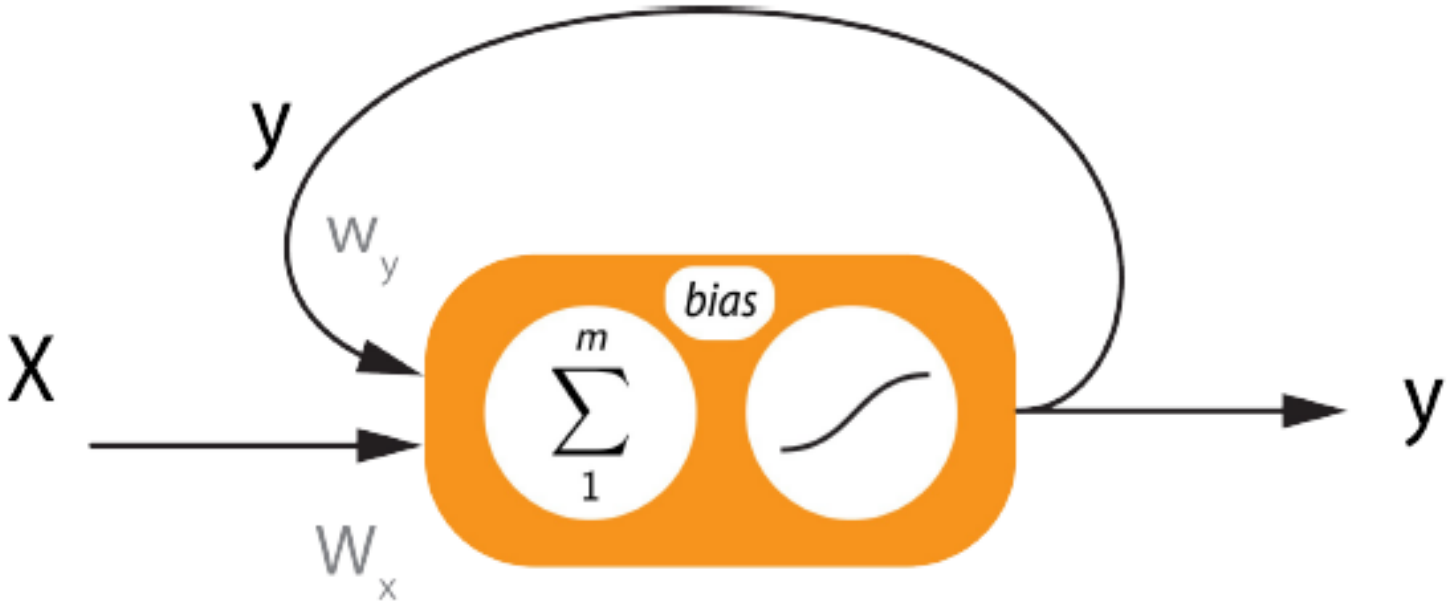
- Un neurone classique



$$y = \sigma(W_x^T \cdot X + b)$$

Réseaux de neurones Récurrent

- Neurone récurrent

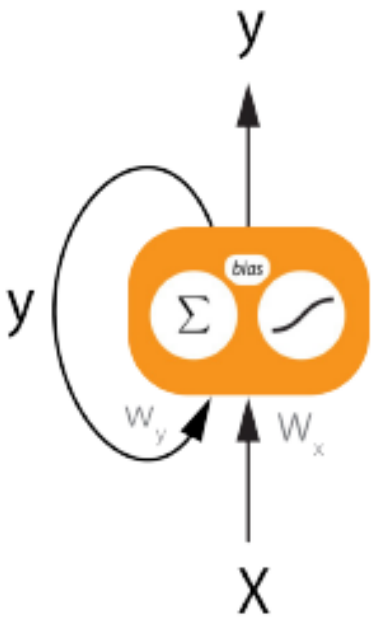


$$y_{(t)} = \sigma(W_x^T \cdot X_{(t)} + w_y \cdot y_{(t-1)} + b)$$

w_y : est un scalaire
W_x : est un vecteur
b : est un scalaire
y : est un scalaire

Réseaux de neurones Récurrent

- Neurone récurrent

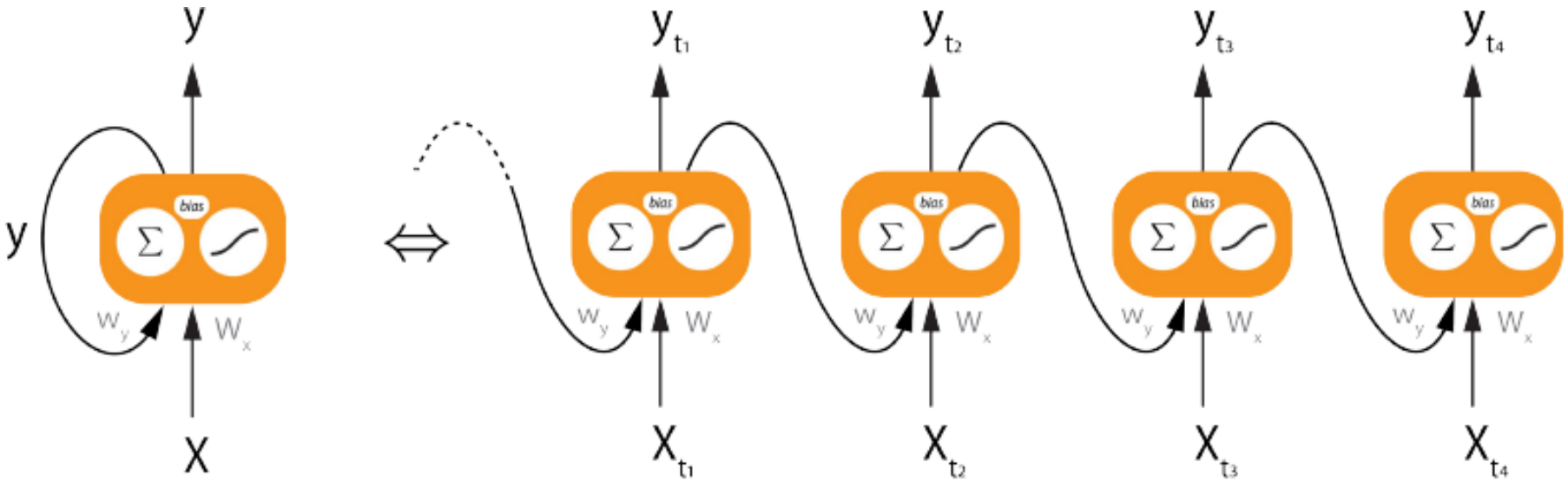


$$y_{(t)} = \sigma(W_x^T \cdot X_{(t)} + w_y \cdot y_{(t-1)} + b)$$

w_y : est un scalaire
W_x : est un vecteur
b : est un scalaire
y : est un scalaire

Réseaux de neurones Récurrent

- Neurone récurrent



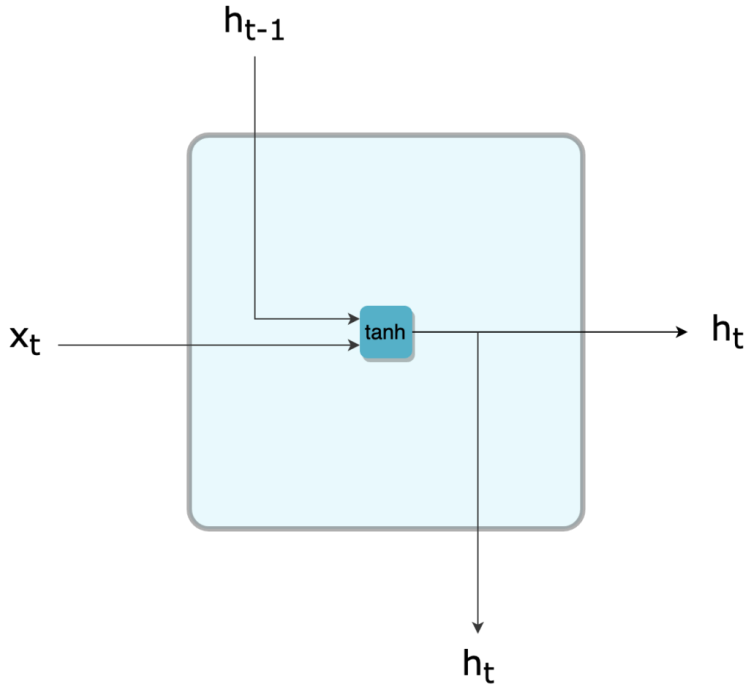
w_y : est un scalaire
W_x : est un vecteur
b : est un scalaire
y : est un scalaire

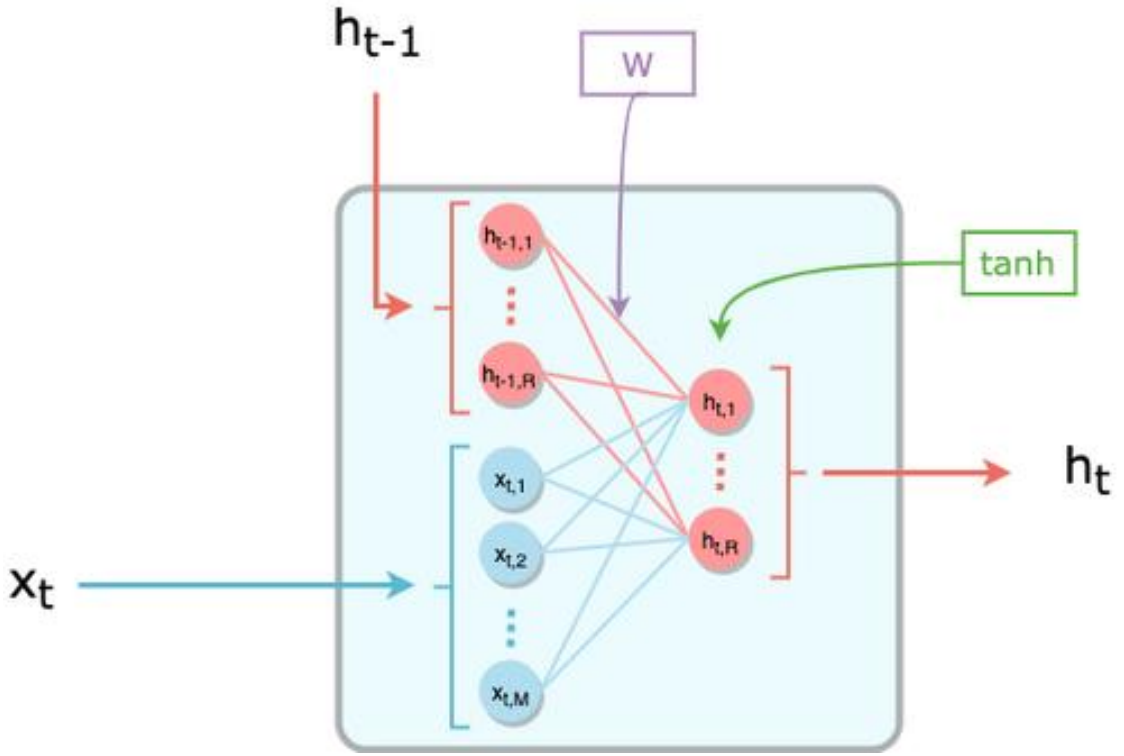
$$y_{(t)} = \sigma(W_x^T \cdot X_{(t)} + w_y \cdot y_{(t-1)} + b)$$

Réseaux de neurones Récurrent

- La t^{ème} cellule n'est rien d'autre qu'une couche dense de taille R, dont l'entrée est la concaténation de $\mathbf{x_t}$ (de taille **M**) et $\mathbf{h_{t-1}}$ (de taille R). La fonction d'activation classique utilisée pour les cellules du RNN est la tangente hyperbolique ***tanh***.

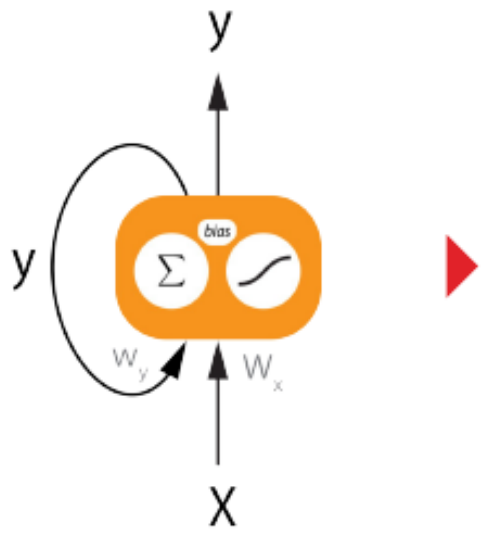
- $h_t = \tanh(W_T * \text{concat}(x_t, h_{t-1}) + b)$



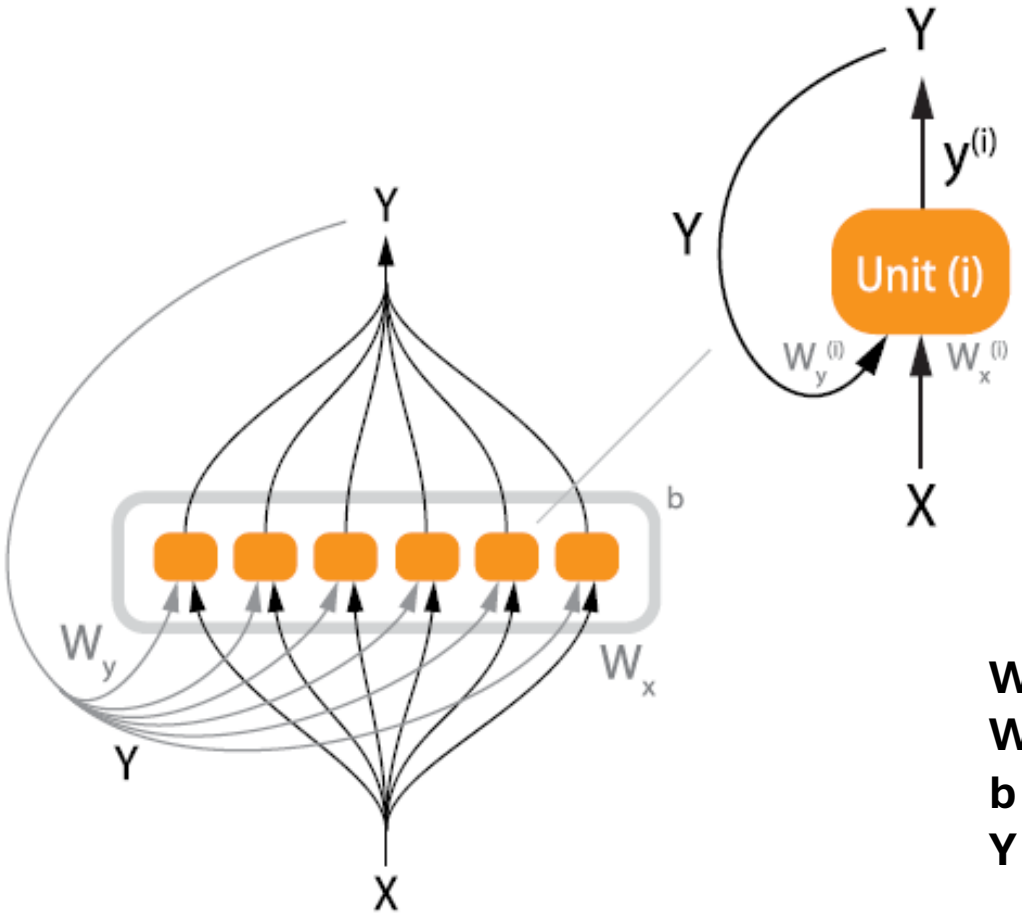


Réseaux de neurones Récurrent

Recurrent **Layer / Cell**



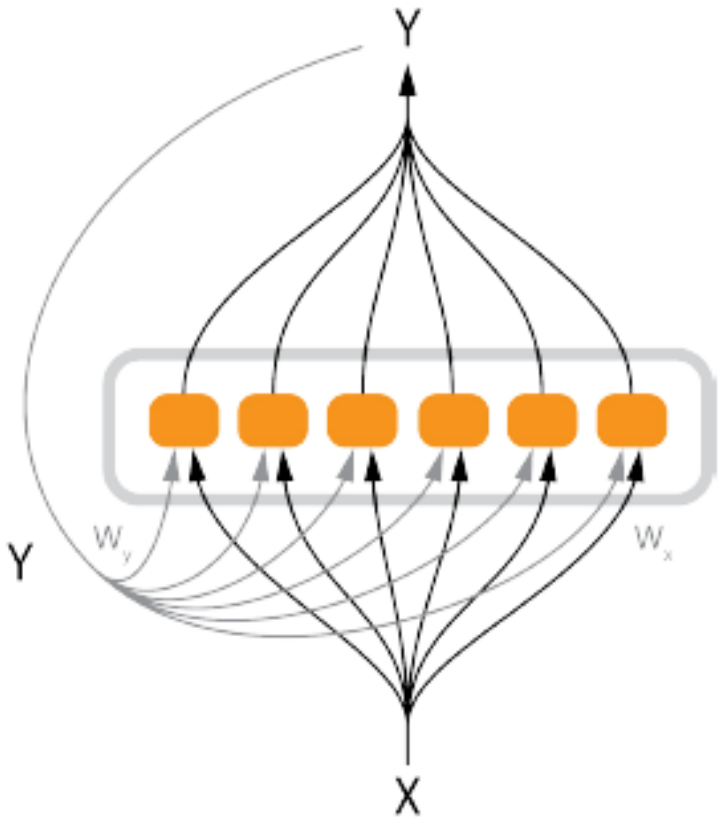
Recurrent **neuron.**



Recurrent **Layer / Cell**

W_y : est un tenseur
W_x : est un tenseur
b : est un vecteur
Y : est un vecteur

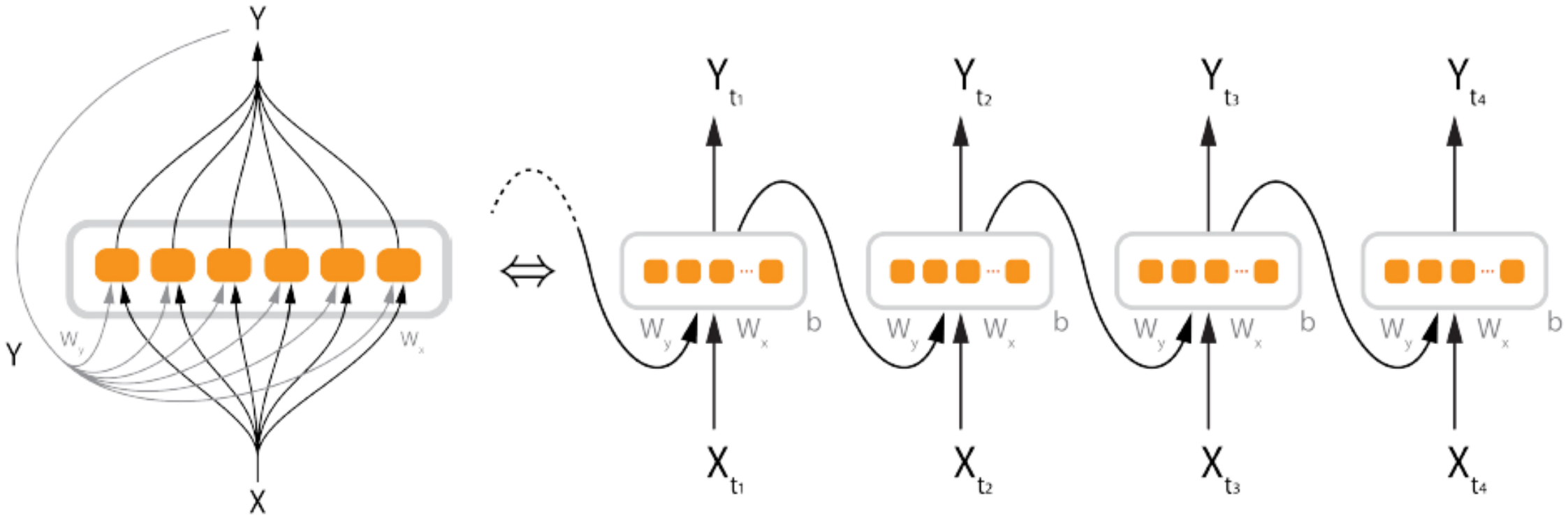
Réseaux de neurones Récurrent



$$Y_{(t)} = \Phi\left(W_x^T \cdot X_{(t)} + W_y^T \cdot Y_{(t-1)} + b\right)$$

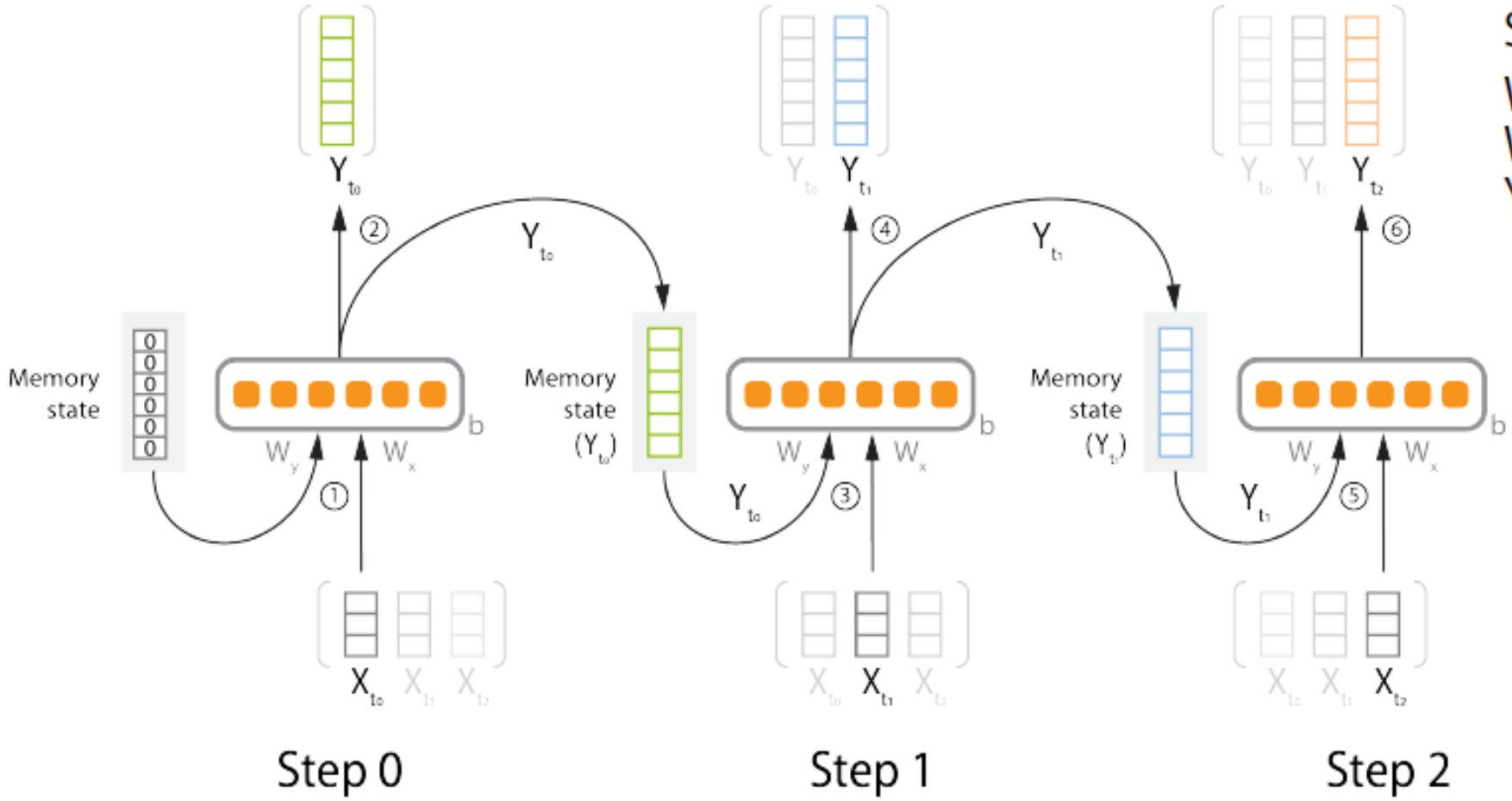
W_y : est un tenseur
W_x : est un tenseur
b : est un vecteur
Y : est un vecteur

Réseaux de neurones Récurrent



$$Y_{(t)} = \Phi\left(W_x^T \cdot X_{(t)} + W_y^T \cdot Y_{(t-1)} + b\right)$$

Réseaux de neurones Récurrent



Shapes :

W_x : (nb_{units}, X size)
W_y : (nb_{units}, nb_{units})
Y : (nb_{units})

$$Y_{(t)} = \Phi\left(W_x^T \cdot X_{(t)} + W_y^T \cdot Y_{(t-1)} + b\right)$$

Example Application

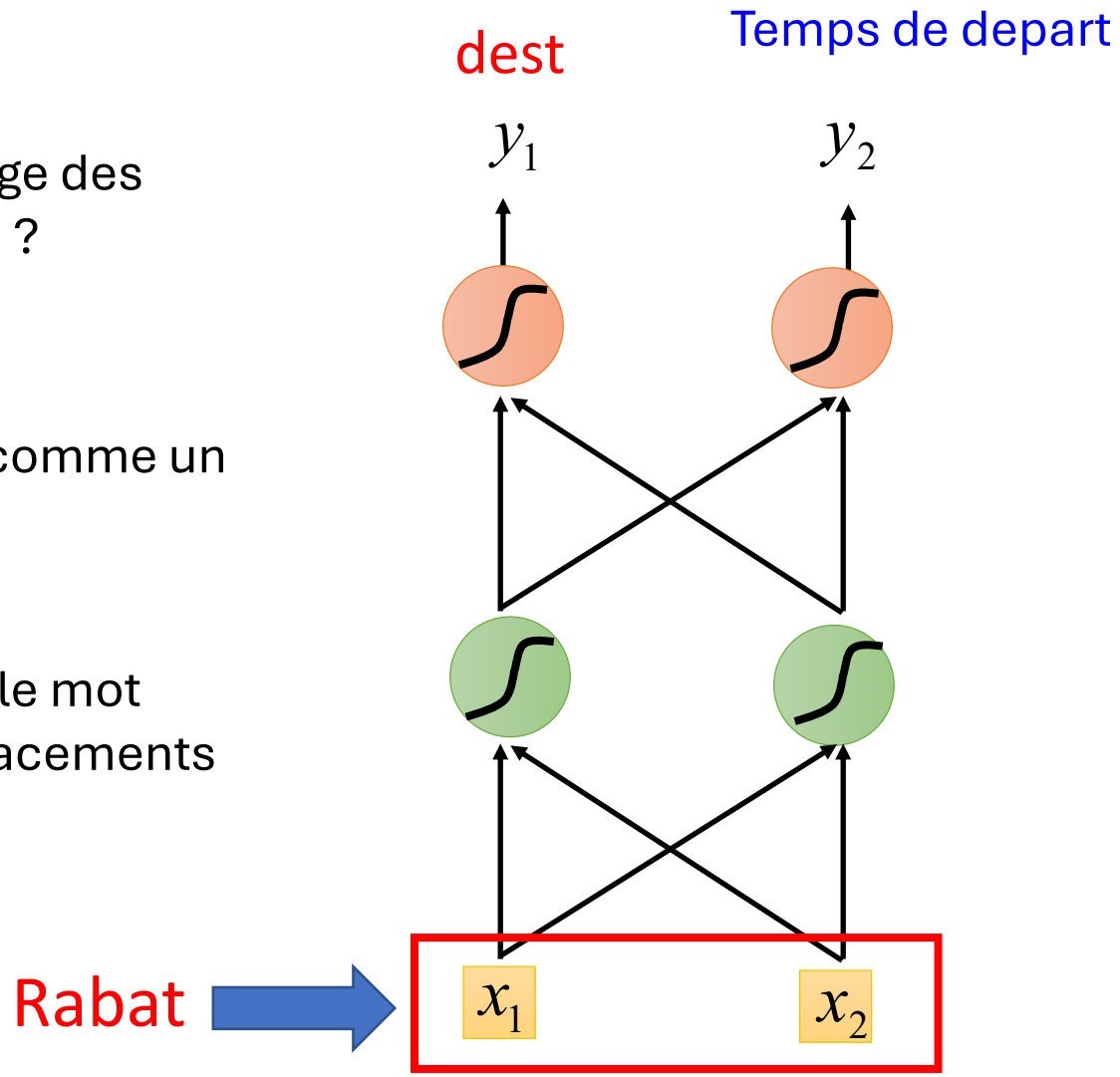
Résoudre le problème du remplissage des créneaux par le réseau Feedforward ?

Input: un mot

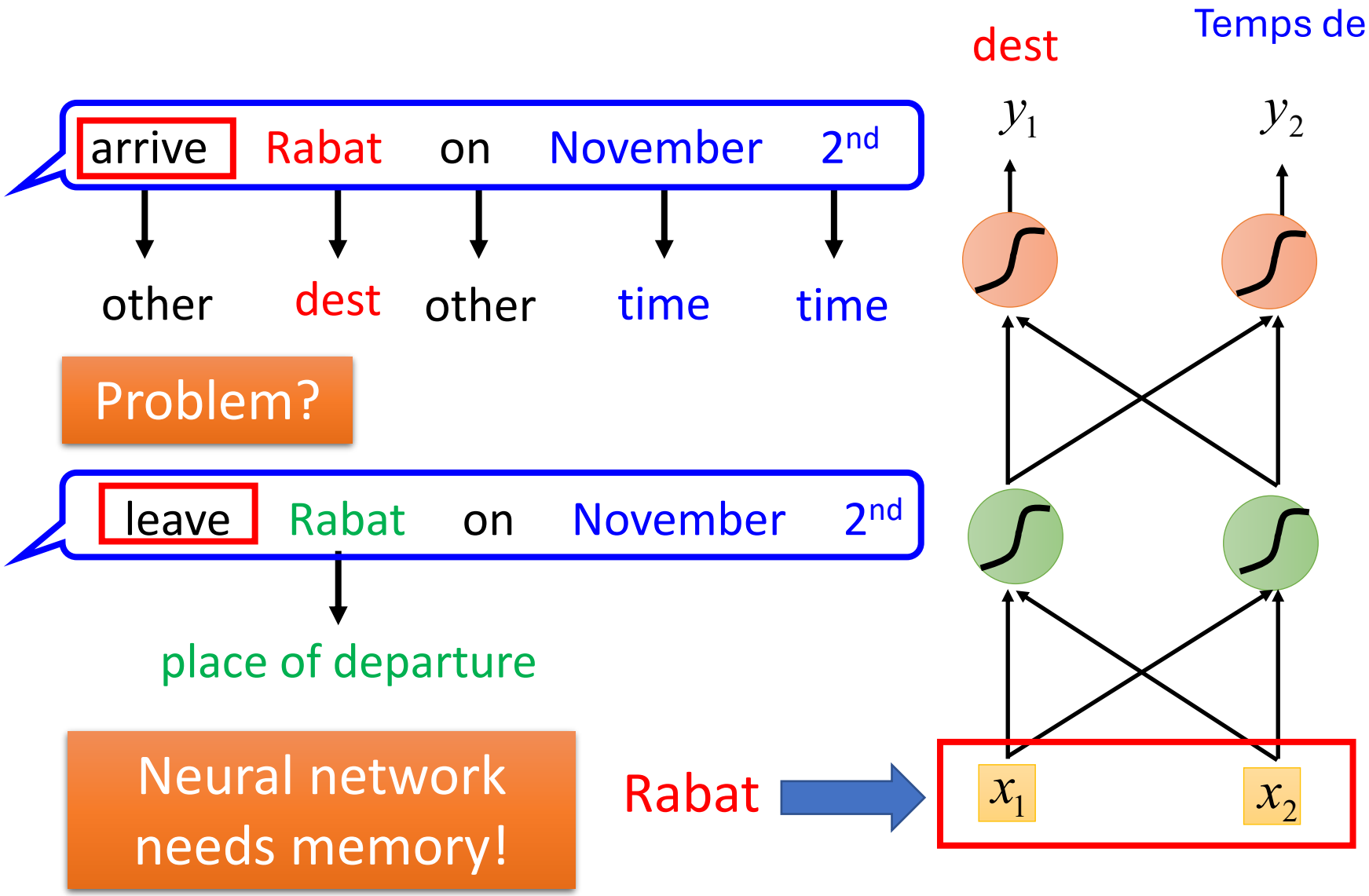
(Chaque mot est représenté comme un vecteur)

Output:

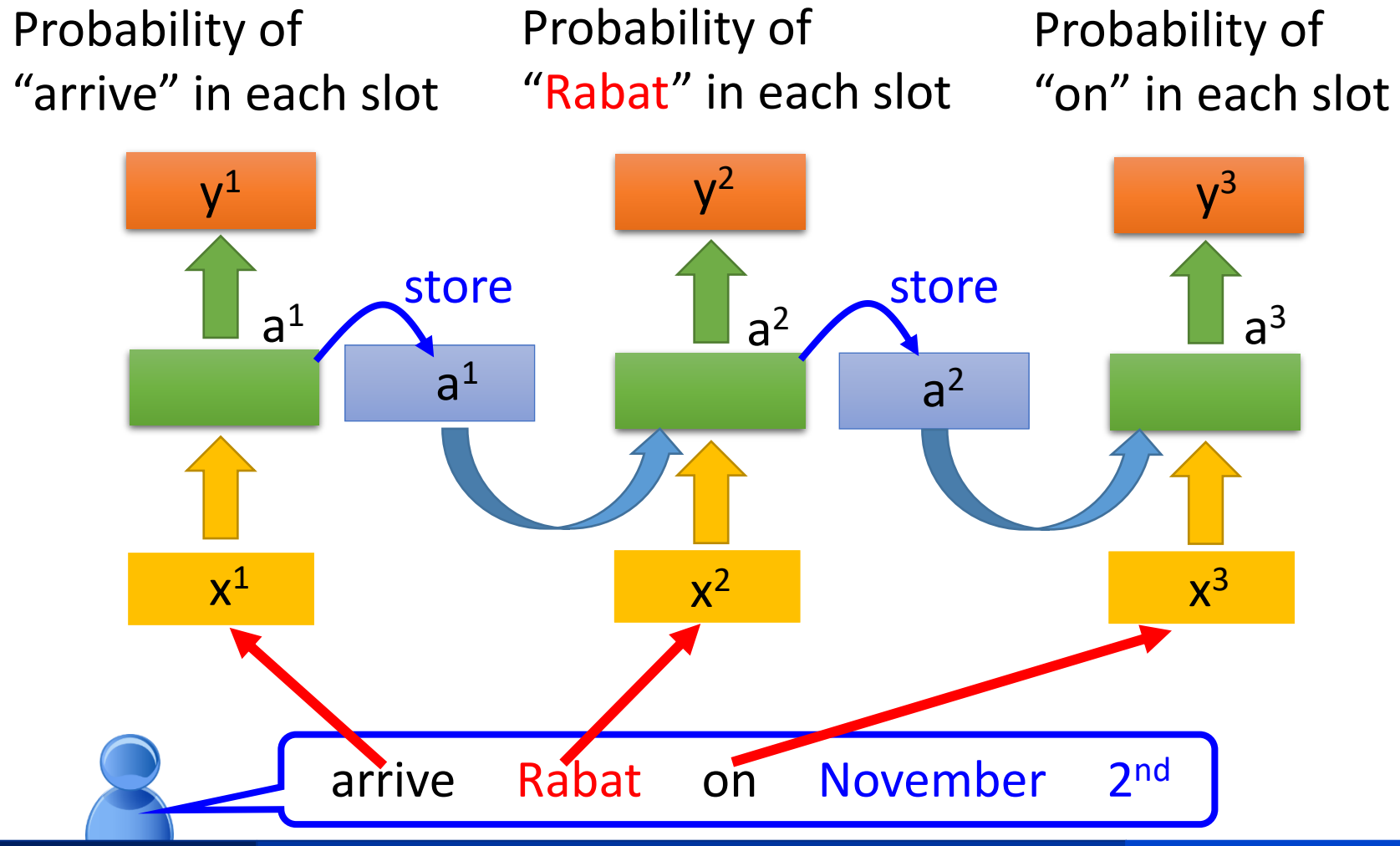
Distribution de probabilité que le mot d'entrée appartienne aux emplacements



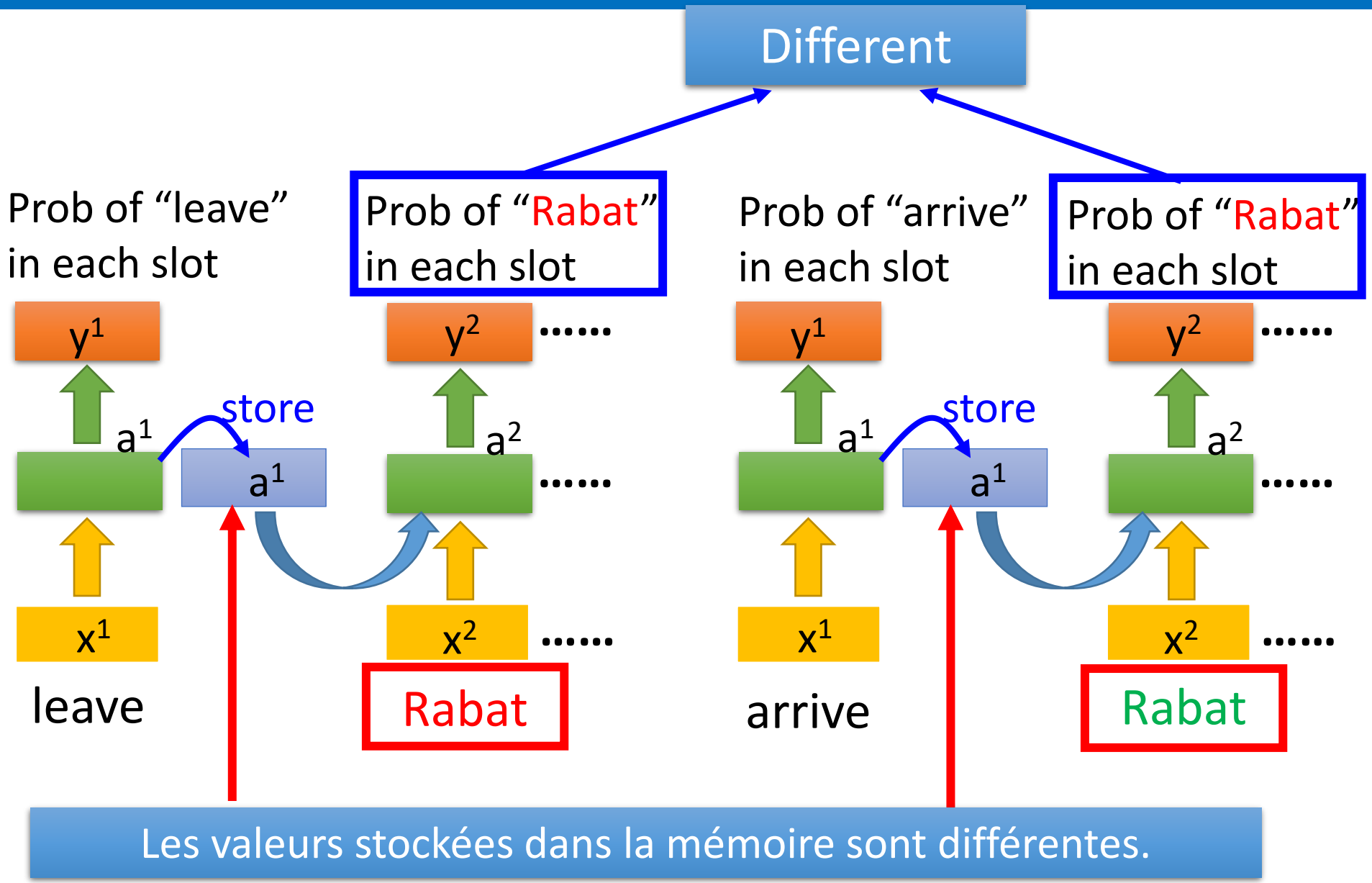
Example Application



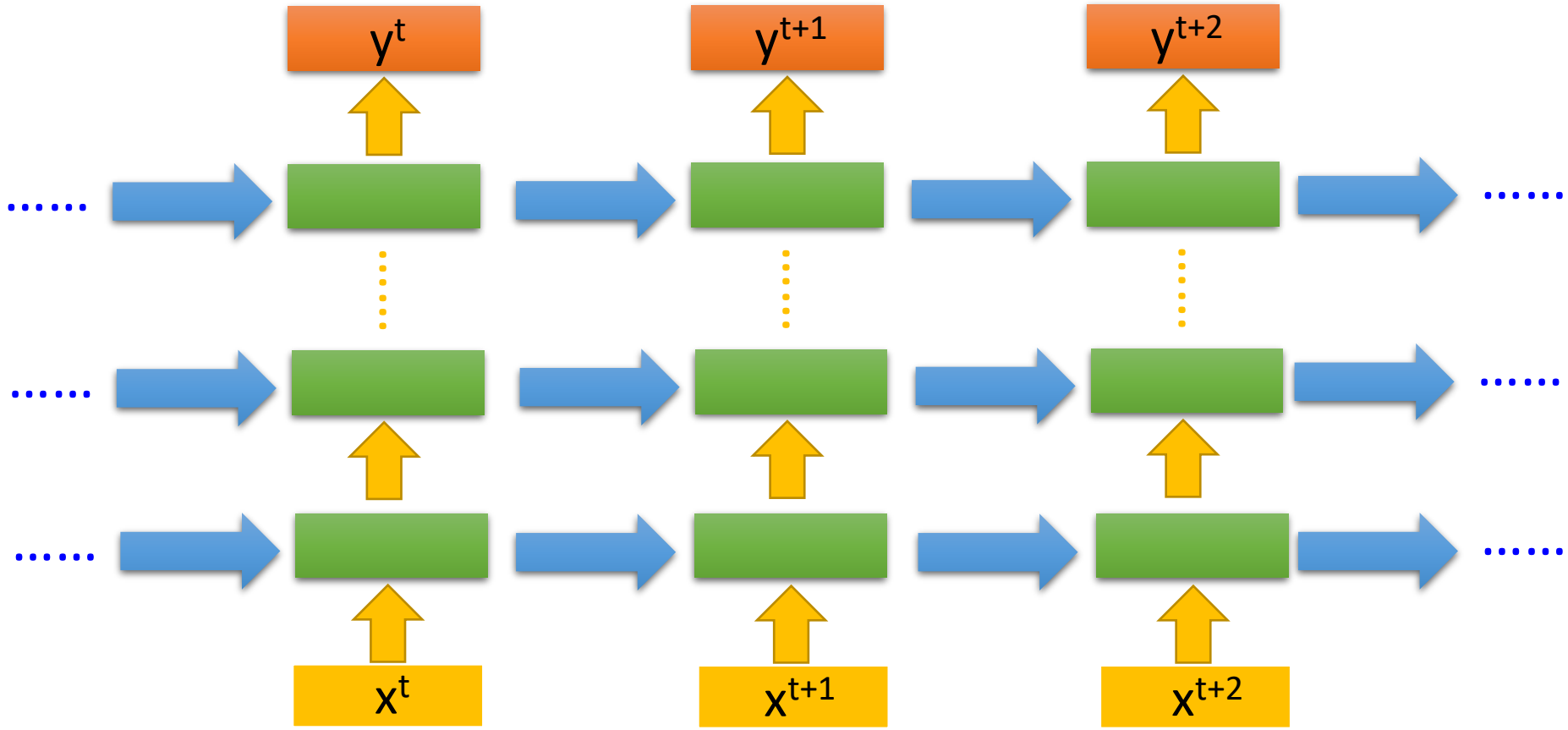
The same network is used again and again.



Réseaux de neurones Récurrent

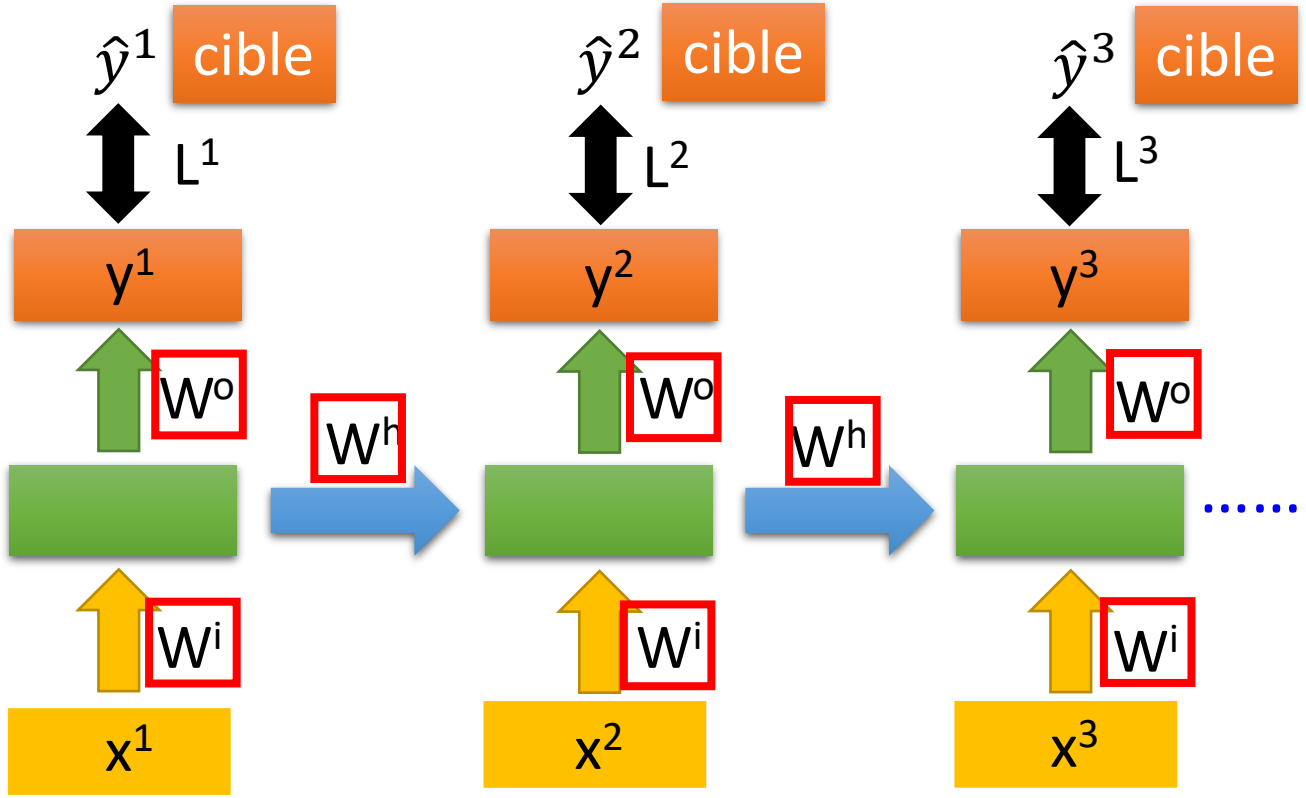


Réseaux de neurones Récurrent



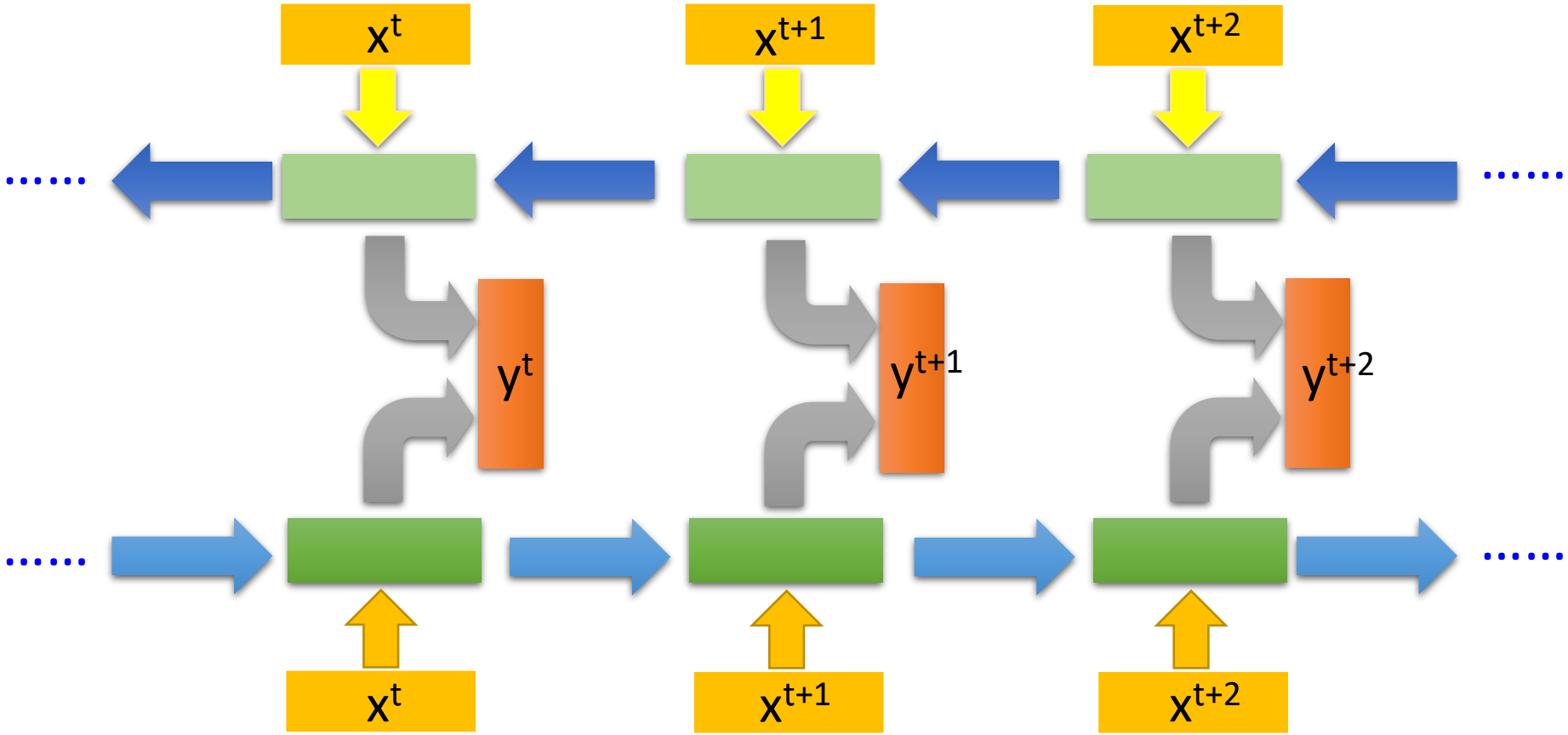
Comment faire l'apprentissage ?

- Comment faire l'apprentissage ?



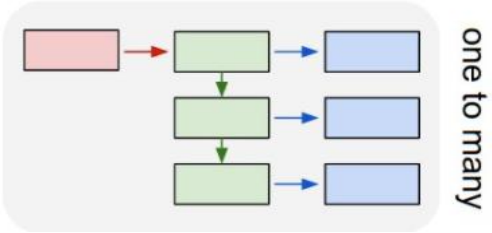
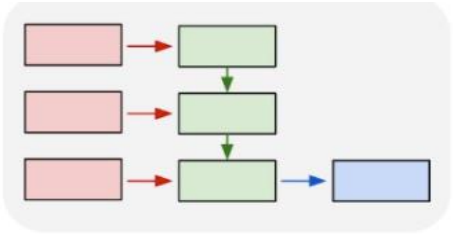
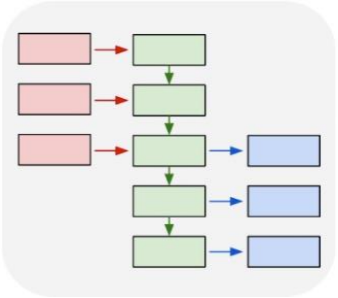
Toruver les valeurs des paramètres qui minimisent le coût

Réseaux de neurones Récurrent Bidirectionnel

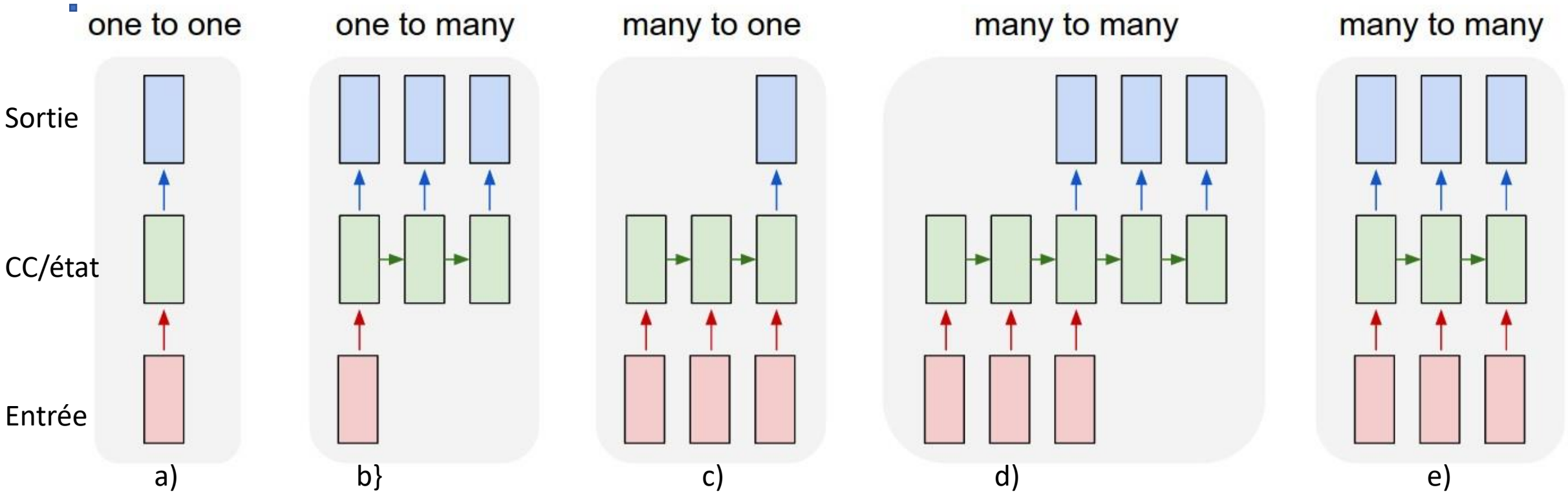


Recurrent Neural Networks (RNNs)

- Les RNN peuvent avoir une ou plusieurs entrées et une ou plusieurs sorties

RNN	Application	Input	Output
	Image Captioning		A person riding a motorbike on dirt road
	Sentiment Analysis	Awesome movie. Highly recommended.	Positive
	Machine Translation	Bonjour	Good Morning

Réseaux de neurones Récurrent



Réseaux de neurones Récurrent

- **one to many** Le RNN reçoit **une unique entrée** et retourne **plusieurs sorties**, l'exemple classique de ce procédé est la légende d'image
- **many to one** On a plusieurs entrées et il y a une unique sortie. Une illustration de ce mode est l'**analyse de sentiment sur des textes**. Cela permet d'identifier un sentiment à partir d'un groupe de mots, déterminer le mot qui manque pour finir la phrase reçue en entrée
- **many to many** , on peut prendre plusieurs entrées et obtenir plusieurs sorties. Nous n'avons pas forcément le même nombre de neurones d'entrée et de sortie. Nous pouvons citer ici, la traduction de texte mais nous pouvons être ambitieux et projeter de finir une œuvre musicale avec son début.

Réseaux de neurones Récurrent

Limitation des RNNs

- Les réseaux neuronaux récurrents souffrent d'un problème appelé **disparition de gradient**. Le problème de la disparition du gradient est le résultat d'un algorithme appelé **rétroropagation** qui permet aux réseaux neuronaux d'optimiser le processus d'apprentissage.

- Le modèle de réseau neuronal compare la différence entre sa sortie et la sortie désirée et renvoie ces informations au réseau pour ajuster des paramètres tels que les poids à l'aide d'une valeur appelée **gradient**. Une plus grande valeur de gradient implique de plus grands ajustements aux paramètres, et vice versa. Ce processus se poursuit jusqu'à ce qu'un niveau satisfaisant de précision soit atteint.

- Les RNNs exploitent l'algorithme de rétroropagation dans le temps (BPTT) où les calculs dépendent des étapes précédentes. Cependant, si la valeur du gradient est trop petite dans une étape pendant la rétroconversion, **la valeur sera encore plus petite dans l'étape suivante**. Cela entraîne une diminution exponentielle des gradients à un point où **le modèle cesse d'apprendre**.

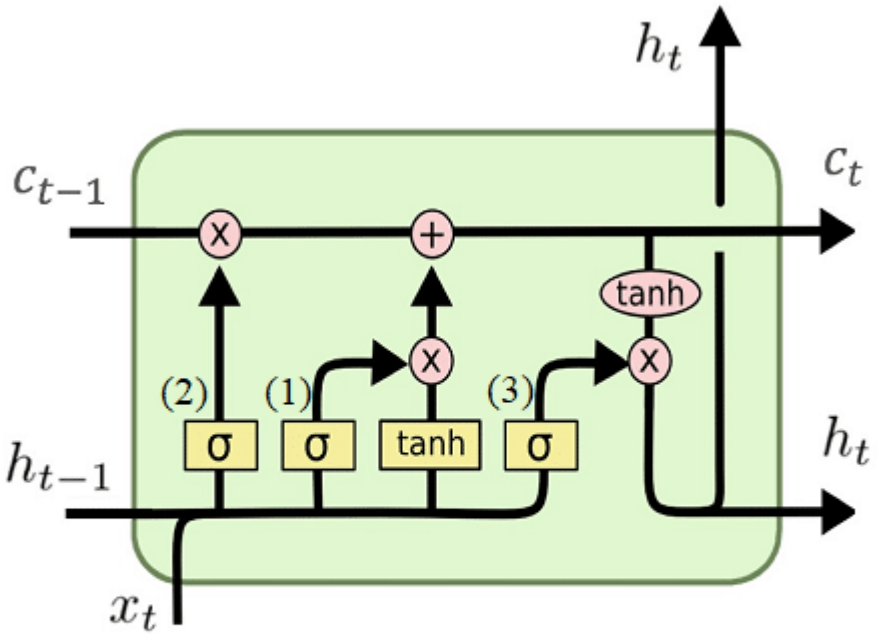
- Ceci est appelé le problème de disparition de gradient et provoque les RNNs à avoir une mémoire à court terme : les sorties antérieures ont de plus en plus peu ou pas d'effet sur la sortie actuelle

LSTM : Long Short Term Memory networks

- Le problème du disparition du gradient « **vanishing gradient** » rencontré lors de l'entraînement des réseaux de neurones récurrents (RNN), a conduit au développement des **réseaux LSTM**, qui utilisent des **mécanismes de régulation** pour éviter le vanishing gradient et capturer efficacement les dépendances à long terme dans les séquences.
- Les LSTM introduisent une nouvelle unité appelée **cellule mémoire**, qui permet au réseau de stocker et d'accéder à des informations sur une période étendue.

LSTM : Long Short Term Memory networks

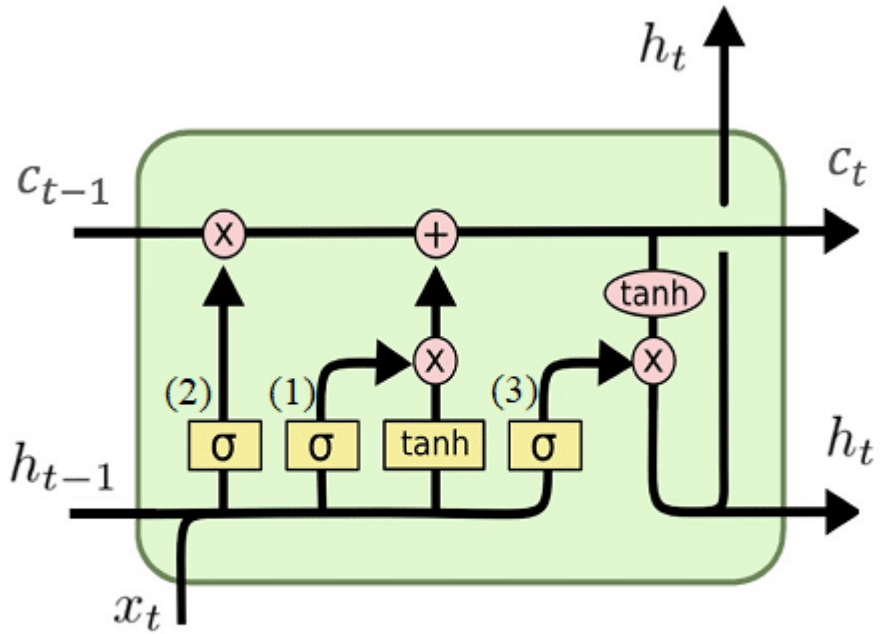
- La cellule mémoire d'un LSTM est composée de **plusieurs portes : une porte d'entrée, une porte de sortie et une porte d'oubli.**
- Ces portes régulent le flux d'informations à l'intérieur de la cellule mémoire, permettant ainsi de **contrôler les informations à retenir et celles à oublier**.
- Cela donne aux LSTM la capacité de mémoriser des informations importantes sur de longues séquences et d'ignorer les éléments moins pertinents.
- h_t est l'état caché habituel des RNN mais dans les réseaux LSTM nous ajoutons un deuxième état appelé c_t.
- Ici, h_t représente la mémoire courte du neurone et c_t représente la mémoire à long terme.



LSTM
(Long-Short Term Memory)

LSTM : Long Short Term Memory networks

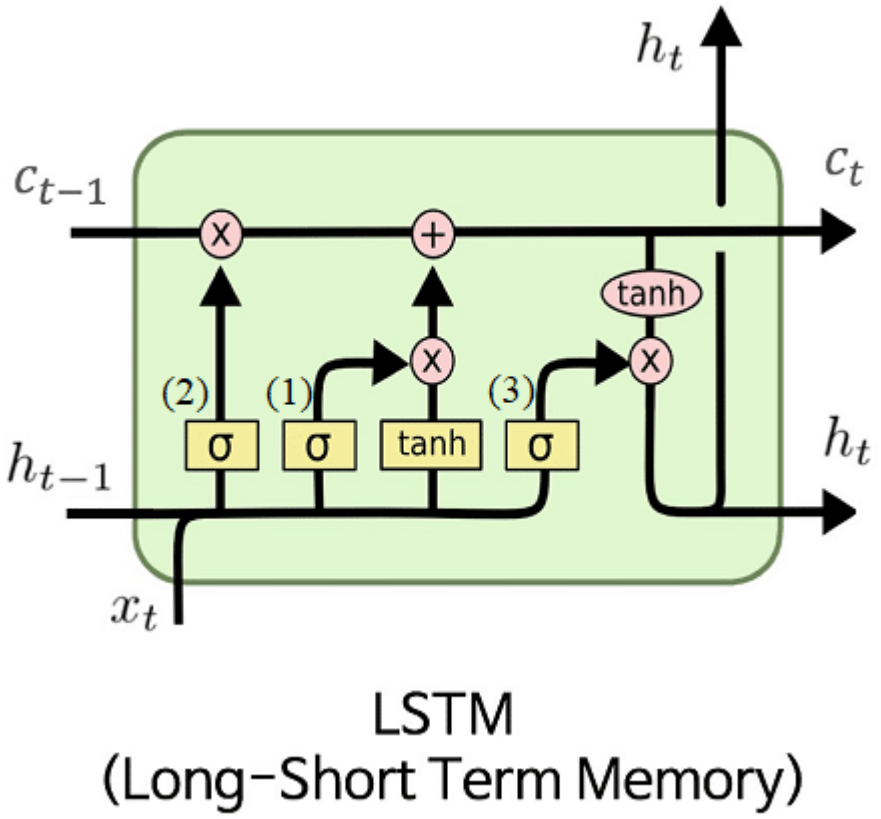
- La porte d'entrée (1) détermine quelles informations doivent être mises à jour dans la cellule mémoire.
- Elle prend en compte l'entrée actuelle x_t ainsi que l'état précédent de la cellule mémoire h_{t-1} et génère **un vecteur d'activation** qui représente les informations à ajouter à la cellule c_{t-1}.
- Cet ajout d'informations se traduit par une opération mathématique effectuée entre ce vecteur d'activation et l'état précédent c_{t-1}.



LSTM
(Long-Short Term Memory)

LSTM : Long Short Term Memory networks

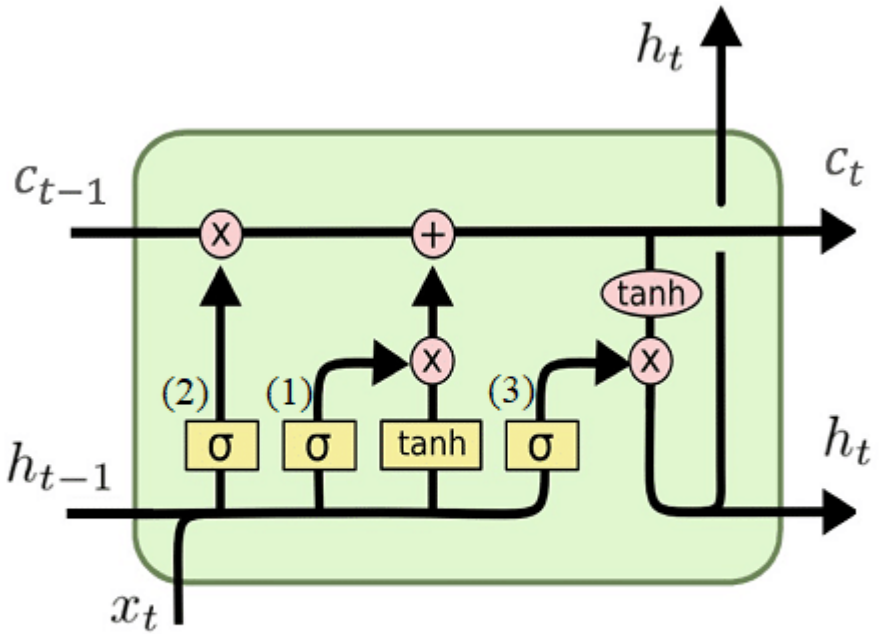
- La porte d'oubli (2) permet au LSTM de supprimer les informations obsolètes de la cellule mémoire.
- Elle utilise à la fois l'entrée actuelle et l'état précédent pour générer un vecteur d'activation qui détermine quelles informations doivent être oubliées. Cela se traduit également par une opération mathématique effectuée entre ce deuxième vecteur d'activation et l'état précédent c_{t-1}. C'est à partir des deux opérations précédentes sur c_{t-1} que l'on obtient l'état actuel c_t.



LSTM
(Long-Short Term Memory)

LSTM : Long Short Term Memory networks

- Enfin, la porte de sortie (3) détermine la sortie du LSTM à un instant donné. Elle utilise l'entrée actuelle x_t et l'état actuel de la cellule mémoire c_t pour générer un vecteur d'activation qui représente la sortie du LSTM. C'est ainsi que l'on obtient h_t.
- La combinaison de ces trois portes permet au réseau LSTM de gérer efficacement les dépendances à long terme. Lors de la rétropropagation du gradient, les LSTM peuvent maintenir **un flux d'informations constant à travers le temps**, évitant ainsi le problème du « vanishing gradient » et permettant un apprentissage plus stable et plus précis.

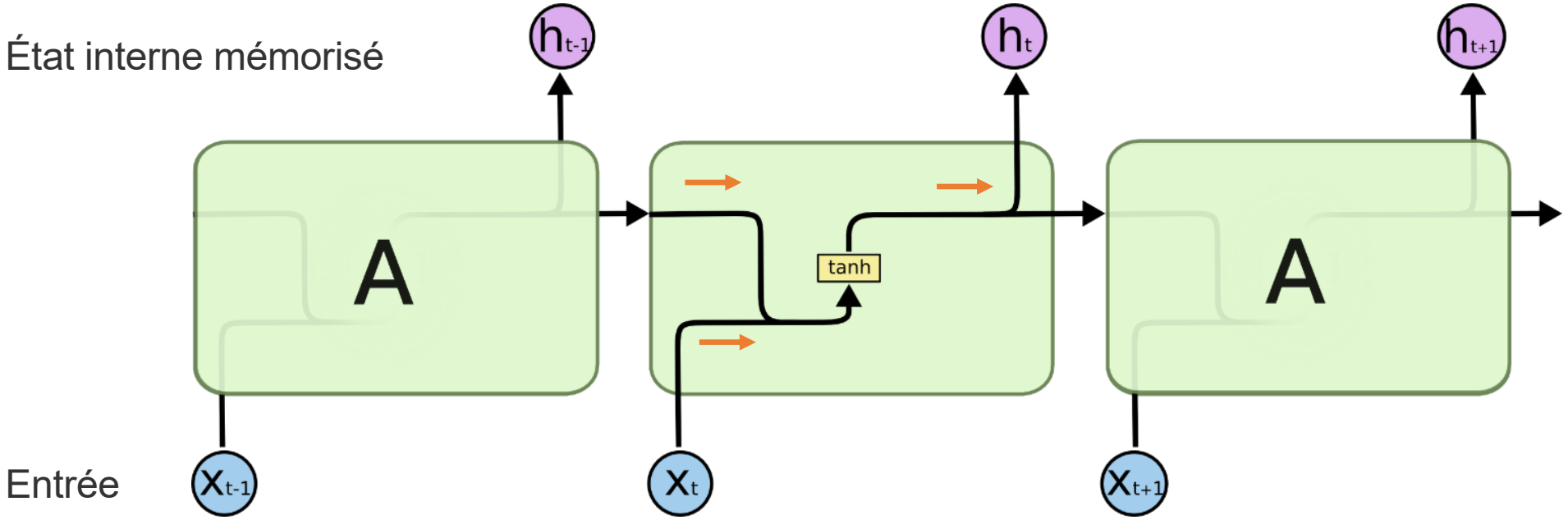


LSTM
(Long-Short Term Memory)

LSTM : Long Short Term Memory networks

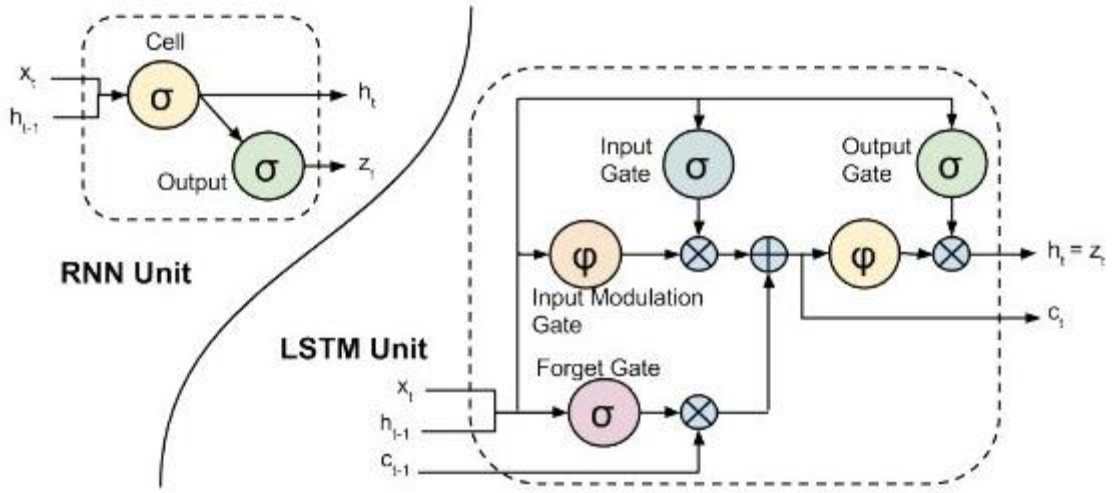
Il faut empêcher le gradient de disparaître…

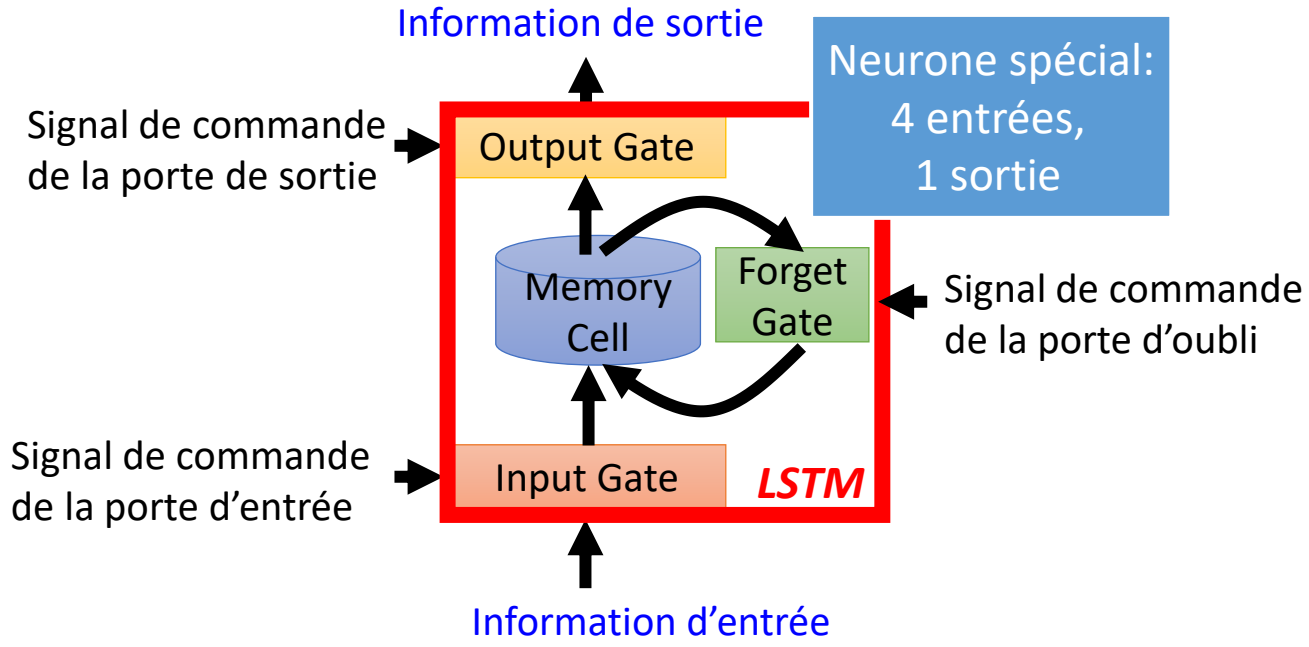
- Normalement, la mémoire du réseau est $h_t = tanh(W \cdot [h_{t-1}, x_t] + b)$ et implique un seul niveau de traitement, créant le risque du gradient évanescent.



LSTM : Long Short Term Memory networks

- Ajoute une mémoire de contexte qui affecte le flux d'information et son traitement (*cell state*)
- Trois portes décident ce qu'une cellule doit oublier de son passé, retenir pour son état courant, et rendre disponible pour le futur

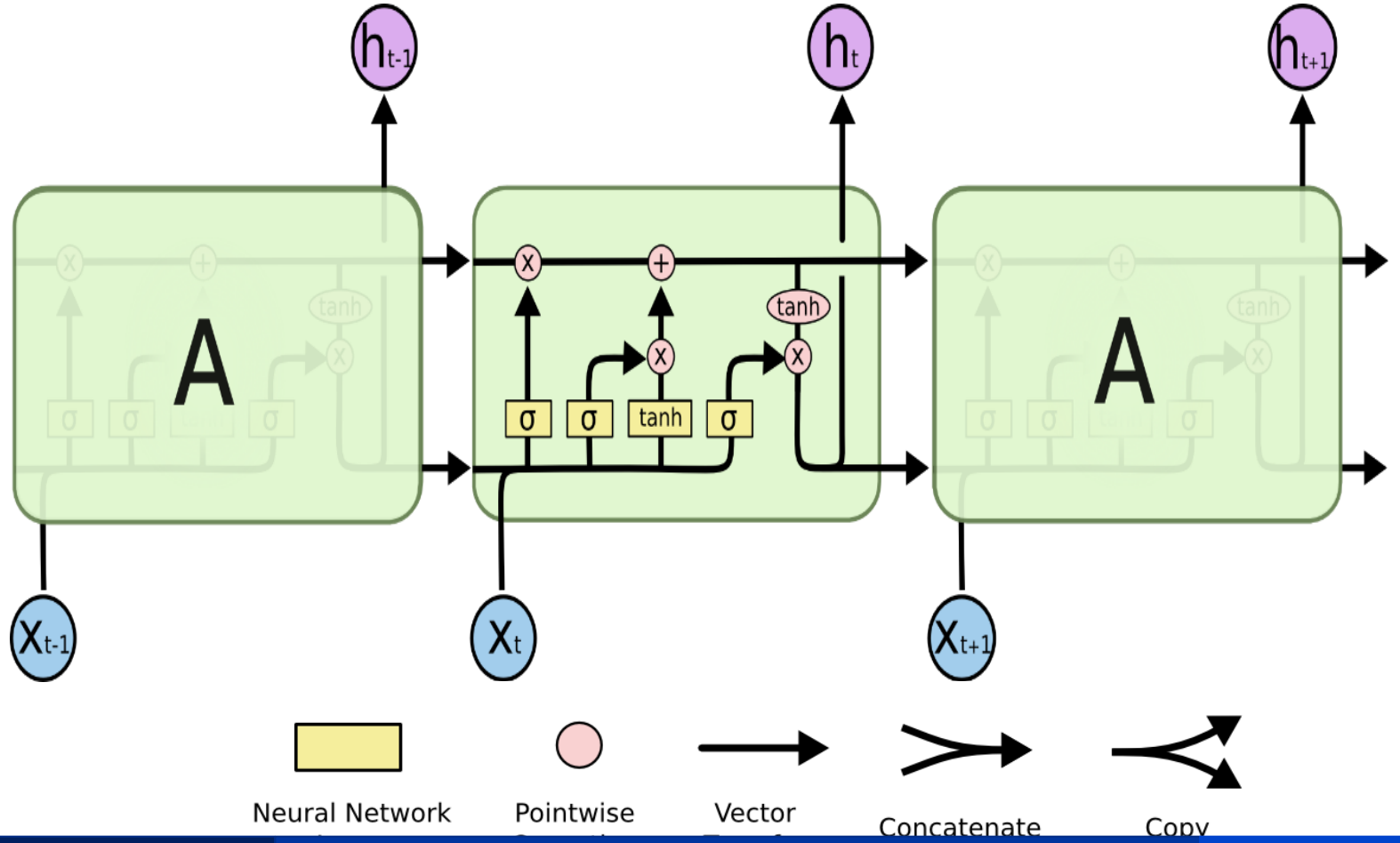




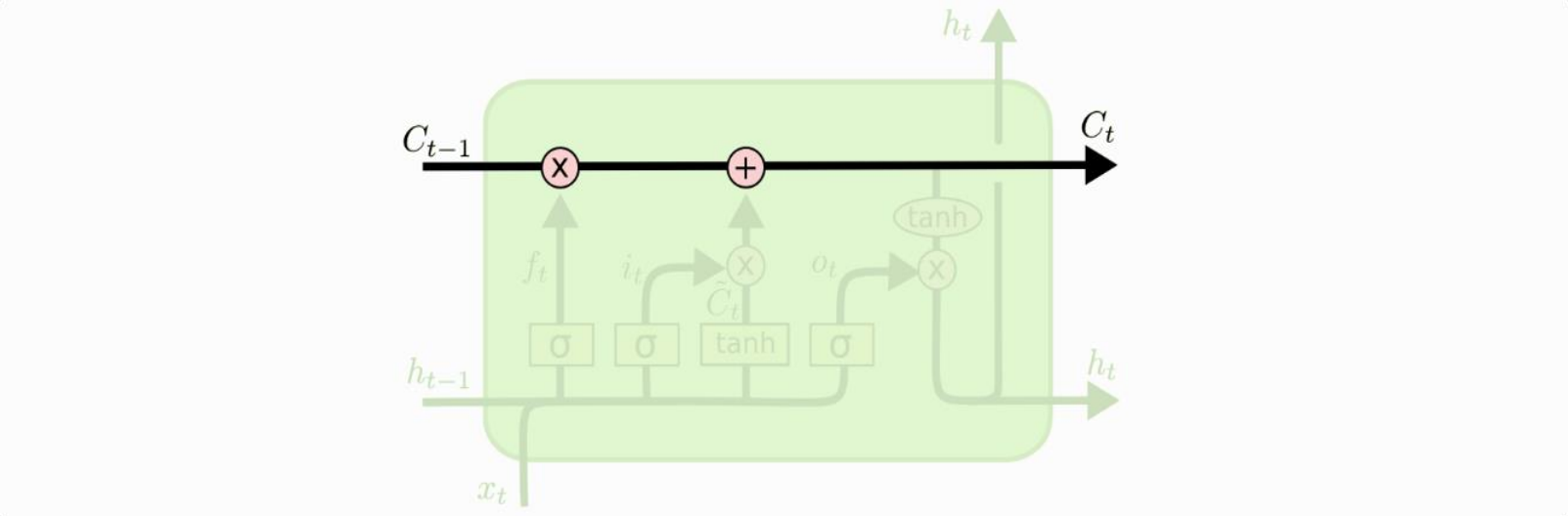
Les signaux de de commande viennent typiquement de perceptrons

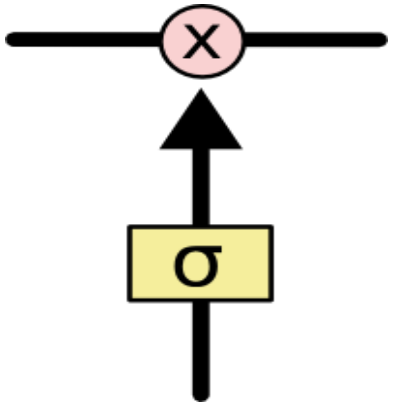
LSTM : Long Short Term Memory networks

Concrètement, la récurrence dans une cellule LSTM implique 4 niveaux de traitement



LSTM : Long Short Term Memory networks



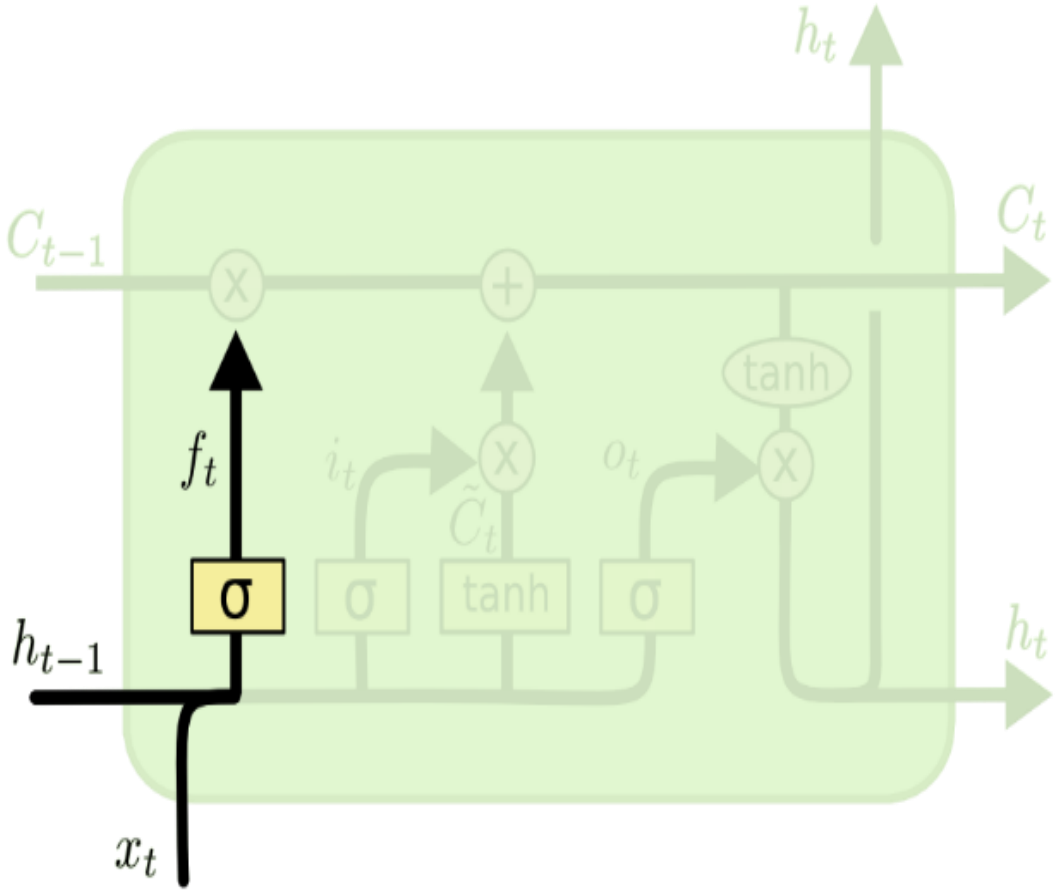


Une cellule STM agit comme un convoyeur d'information contrôlé à l'aide de portes configurables (par entraînement)

Chaque porte est un réseau en soi (perceptron)

LSTM : Long Short Term Memory networks

Porte d'oubli

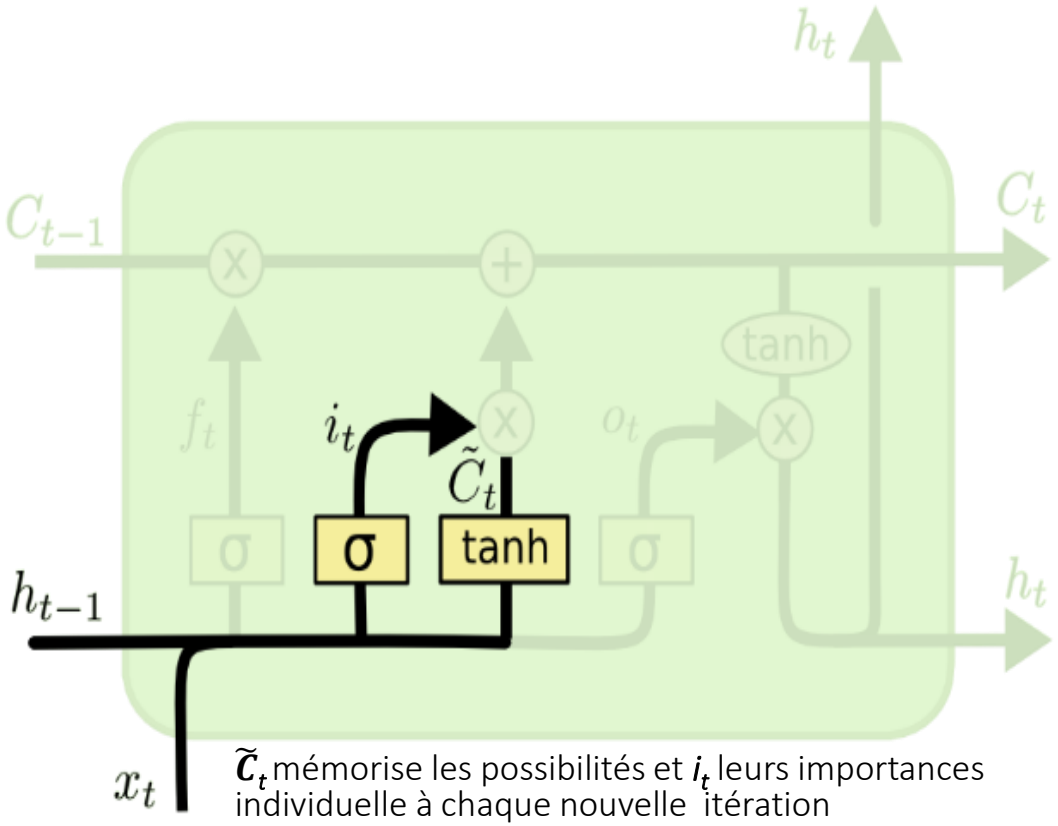


Décide la contribution du contexte précédent à la sortie du réseau

$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

Ex.: dans la prédiction des mots d'une nouvelle phrase, il n'est pas utile de garder le genre du sujet dans la phrase précédente

Porte d'entrée et circuit de mémoire



$\tilde{C}_t$ mémorise les possibilités et i_t leurs importances individuelle à chaque nouvelle itération

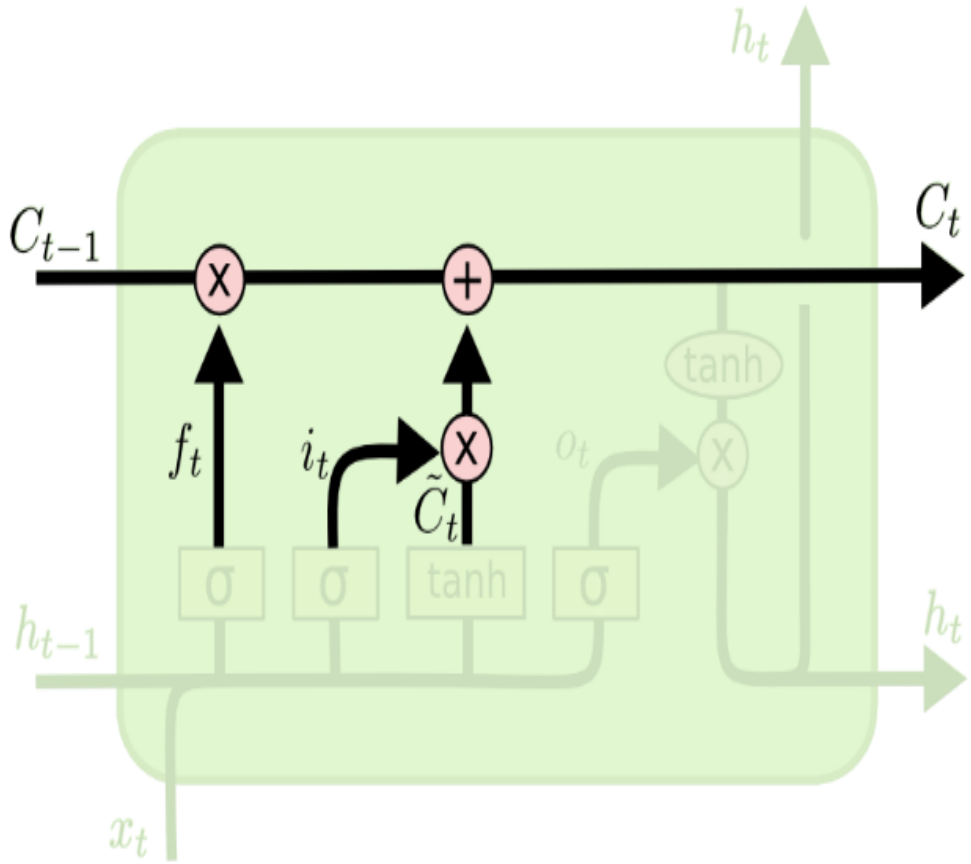
Décide la contribution de la mémoire ($\tilde{C}_t$ est l'équivalent de h_t dans un RRN standard) à la sortie du réseau

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$

Ex.: dans la prédiction des mots d'une nouvelle phrase, il est utile de remplacer le genre du sujet de la phrase précédente par le nouveau.

Mise à jour de la cellule



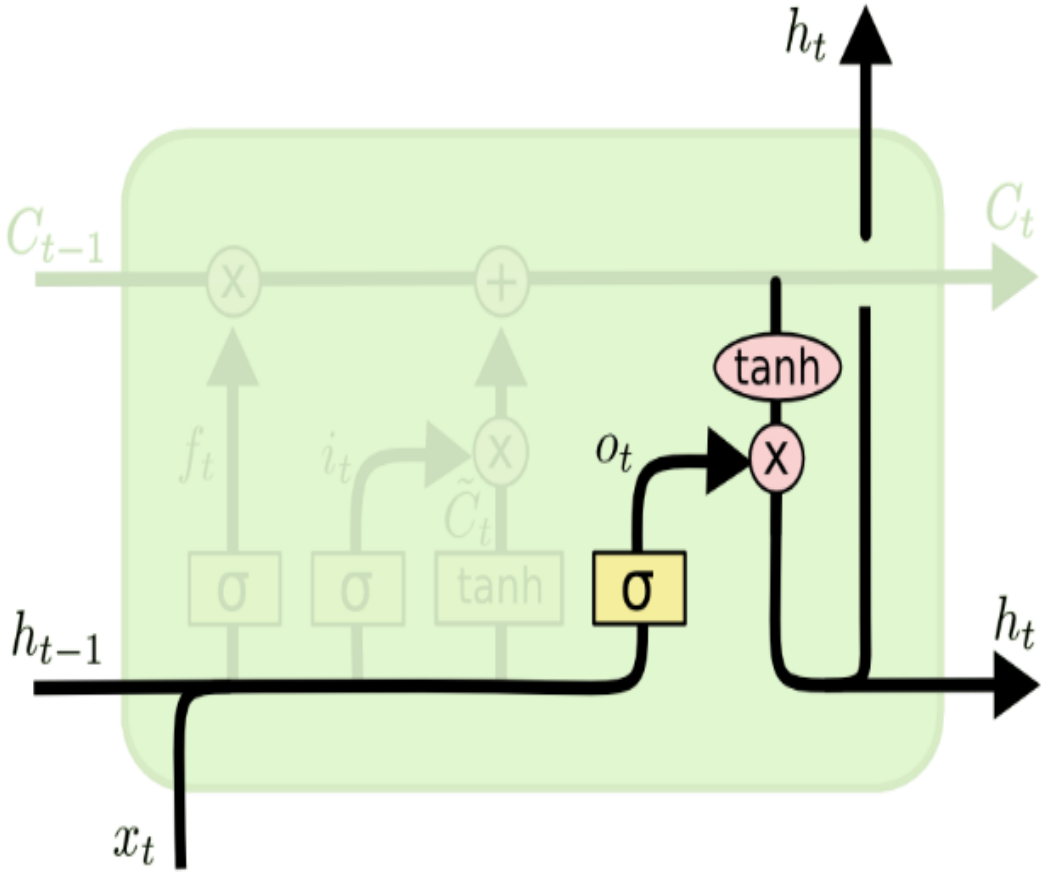
On combine ce qui a été retenu de l'état précédent et ce qui est utilisé de la mémoire courante

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

- Agit comme une nouvelle entrée au réseau adaptée un contexte

LSTM : Long Short Term Memory networks

Porte de sortie



Décide quoi rendre public du nouveau contexte

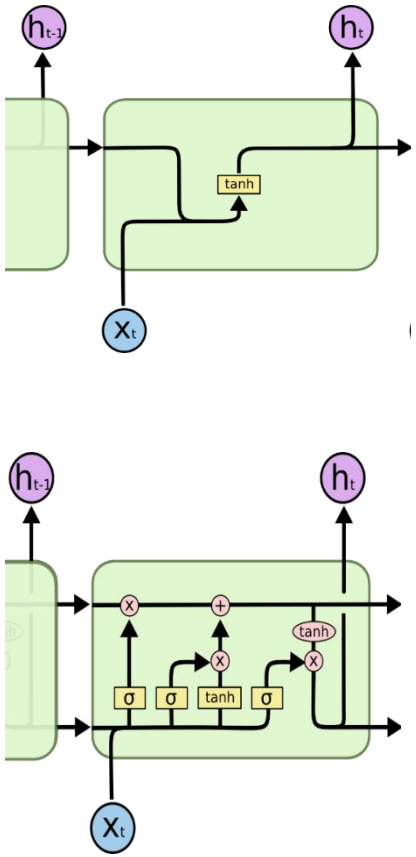
$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$

$$h_t = o_t * \tanh\left(C_t\right)$$

Ex.: dans la prédiction des mots d'une nouvelle phrase, il peut être utile de retenir la pluralité du sujet à la vue d'un verbe, ce qui peut simplifier la conjugaison après.

LSTM : Long Short Term Memory networks

En somme…



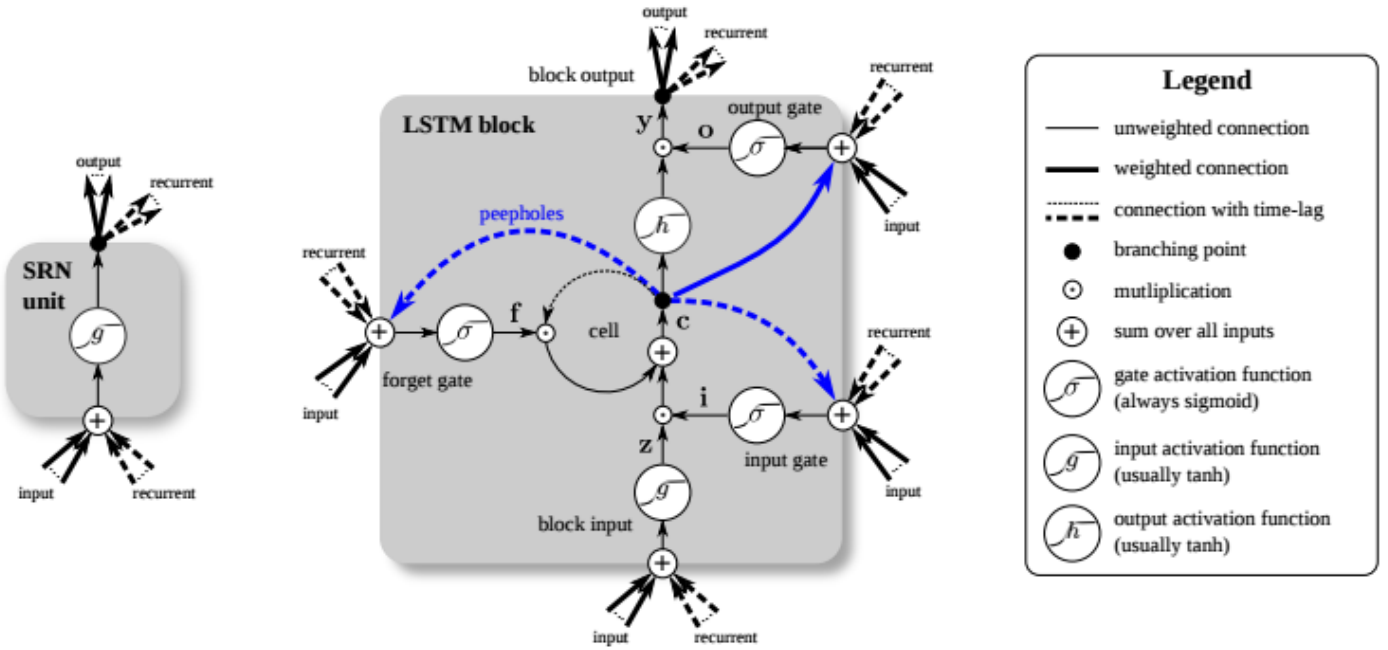
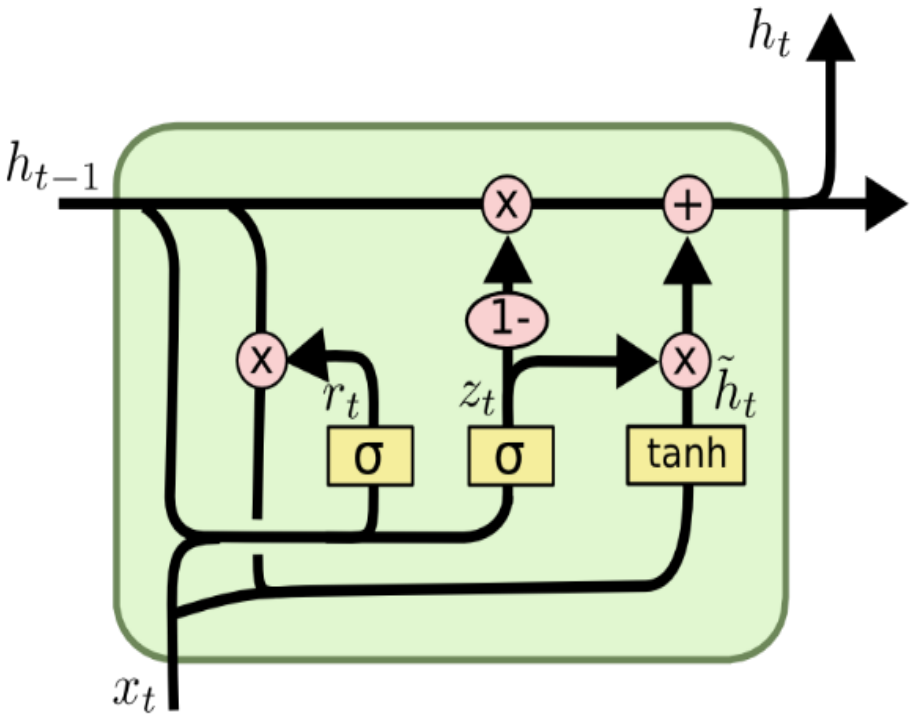


Figure 1. Detailed schematic of the Simple Recurrent Network (SRN) unit (left) and a Long Short-Term Memory block (right) as used in the hidden layers of a recurrent neural network.

Gated recurrent unit



Quoi retenir

$$z_t = \sigma\left(W_z \cdot [h_{t-1}, x_t]\right)$$

Quoi garder

$$r_t = \sigma\left(W_r \cdot [h_{t-1}, x_t]\right)$$

Actualisation du contexte

$$\tilde{h}_t = \tanh\left(W \cdot [r_t * h_{t-1}, x_t]\right)$$

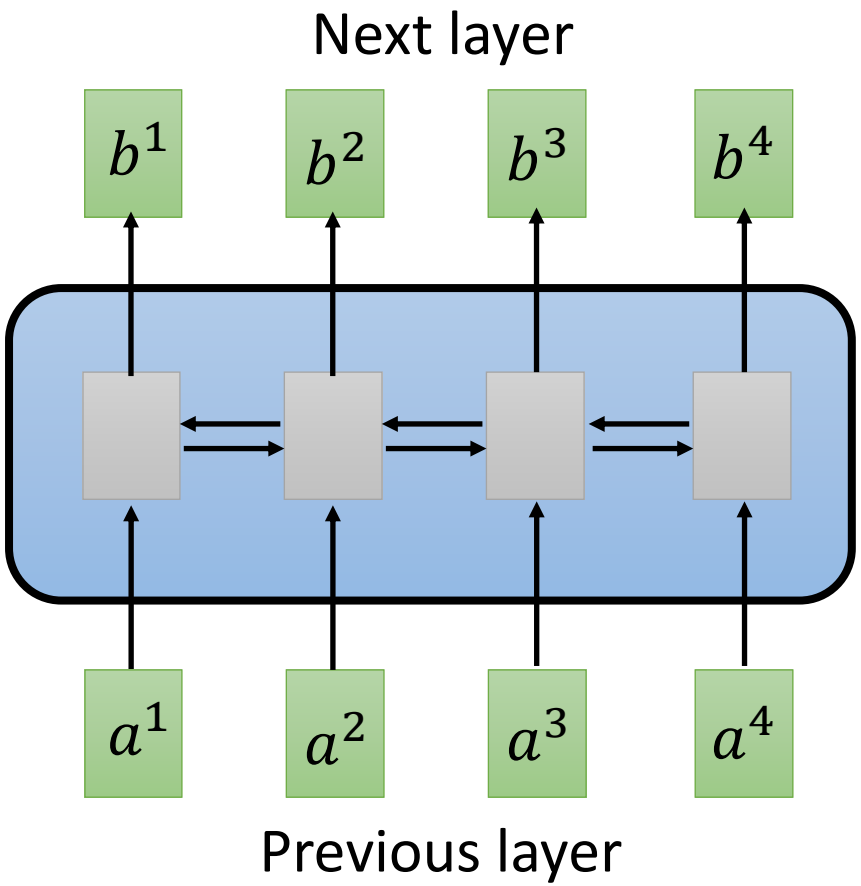
Actualisation de la mémoire

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

Peut être vu comme un LSTM sans porte de sortie et sans mémoire de cellule. Repose sur une porte de remise à zéro $\boldsymbol{r_t}$ qui décide quoi garder de la mémoire précédente et une porte de mise à jour $\boldsymbol{z_t}$ qui décide quoi retenir.

Transformer

Sequence

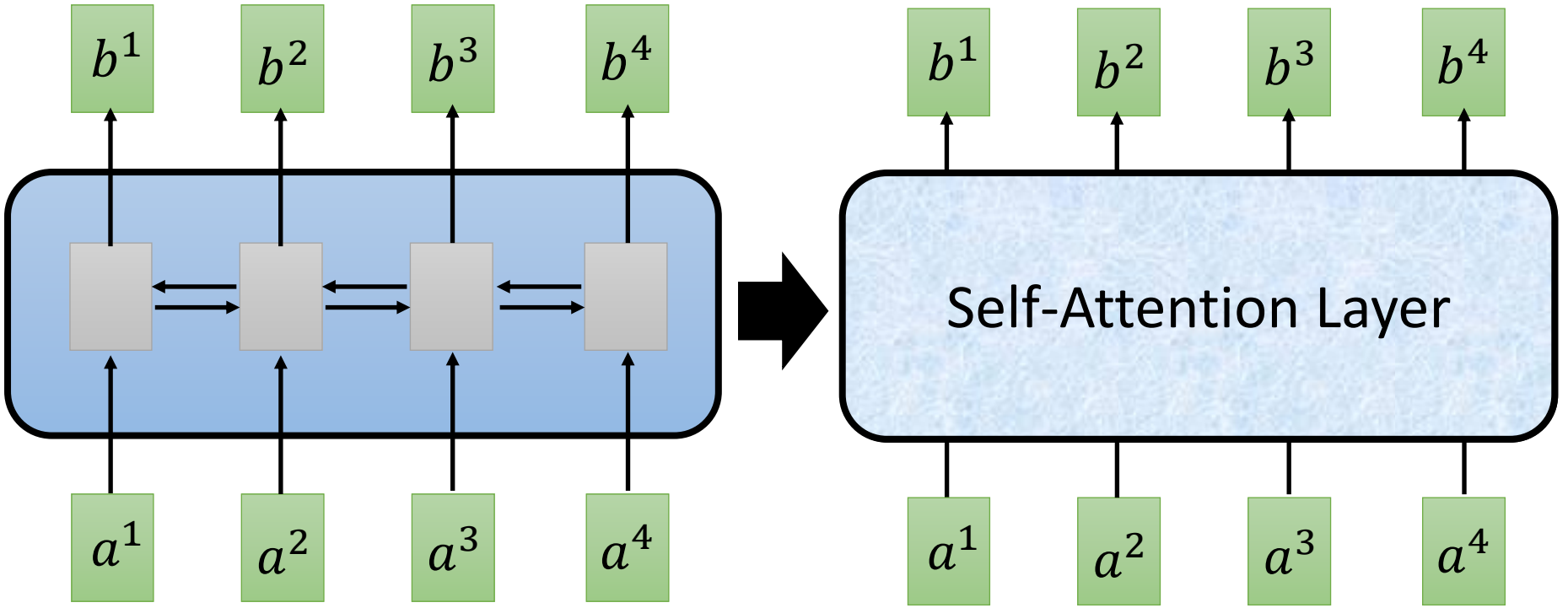


Difficile à mettre en parallèle

Self-Attention

b^i est obtenu sur la base de la séquence d'entrée entière.

b^1, b^2, b^3, b^4 peut être calculé en parallèle.



Nous pouvons remplacer tout ce qui a été fait par RNN par l'auto-attention.

Self-attention

https://arxiv.org/abs/1706.03762

Attention is all you need.

q: requête (pour correspondre à d'autres)

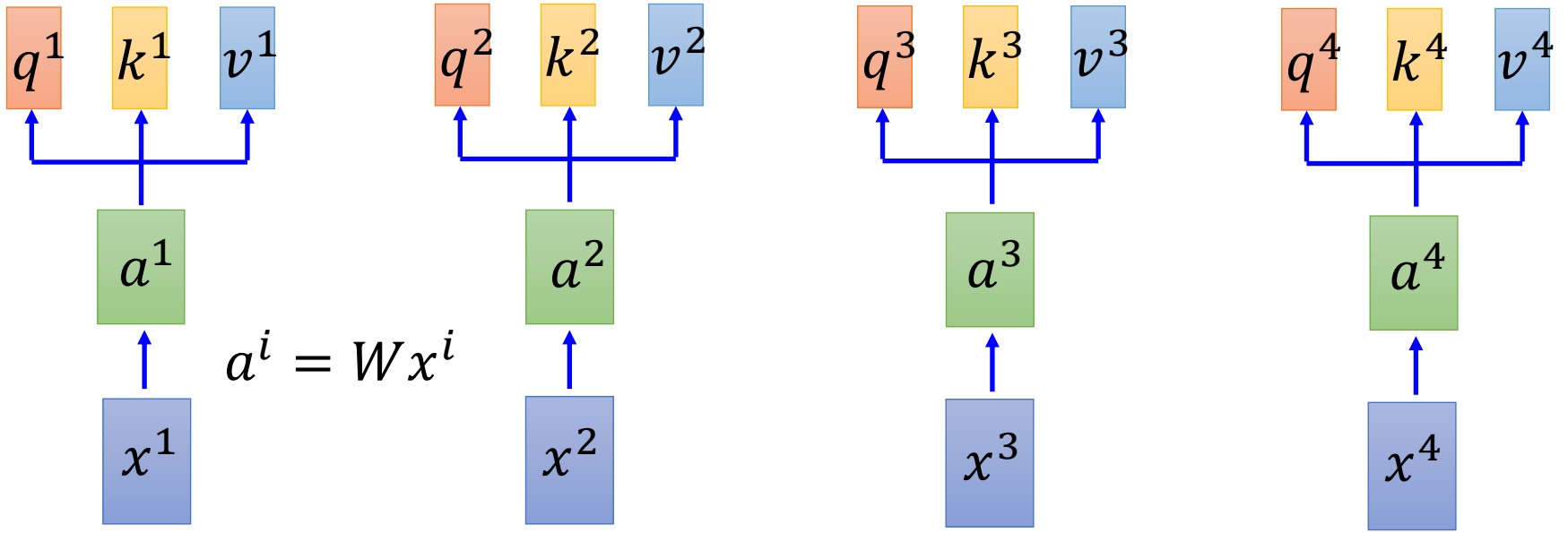
$$q^i = W^q a^i$$

k: clé (à faire correspondre)

$$k^i = W^k a^i$$

v: informations à extraire

$$v^i = W^v a^i$$

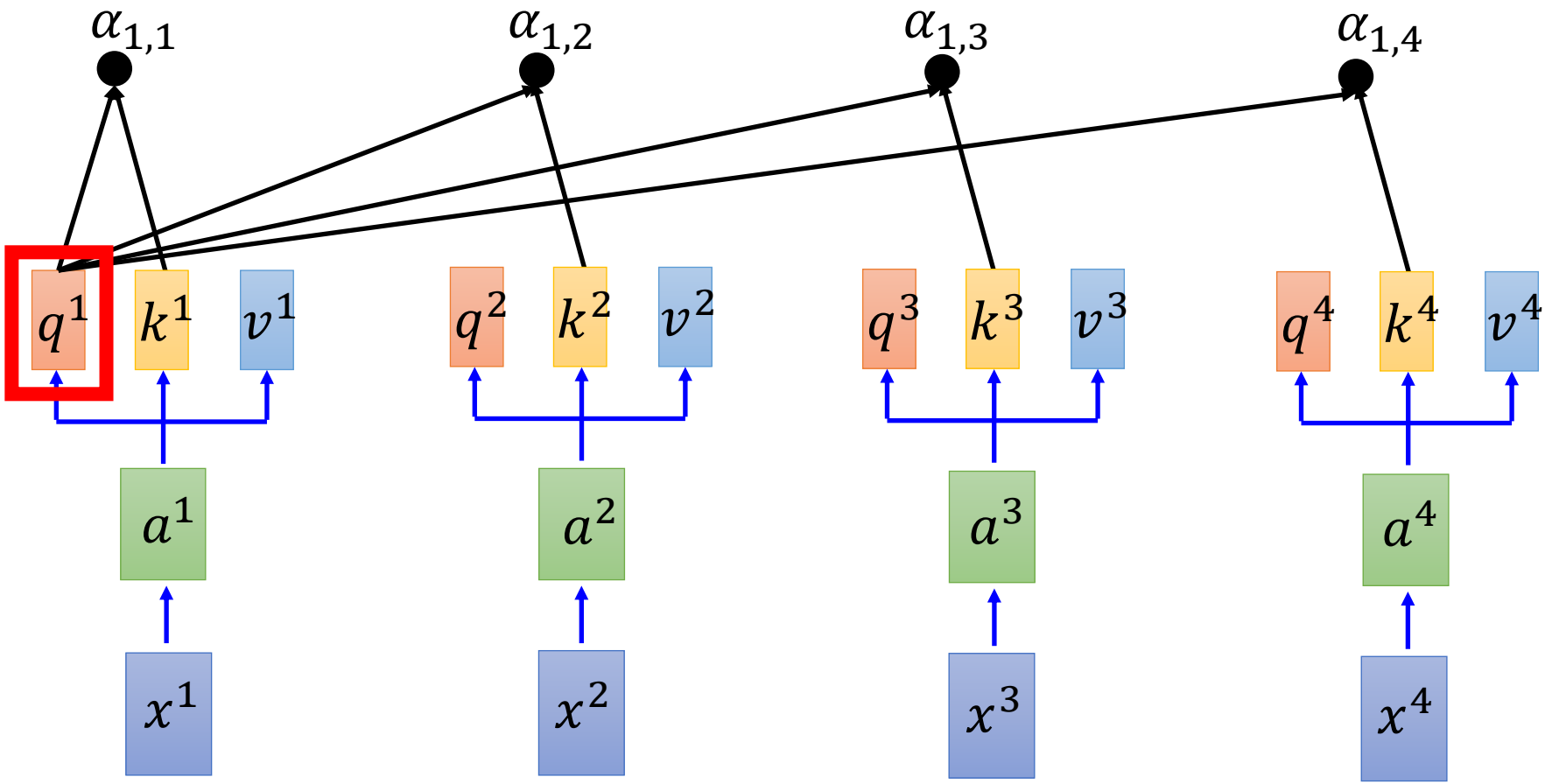


Self-attention

Prenez chaque requête q et faites attention à chaque clé k

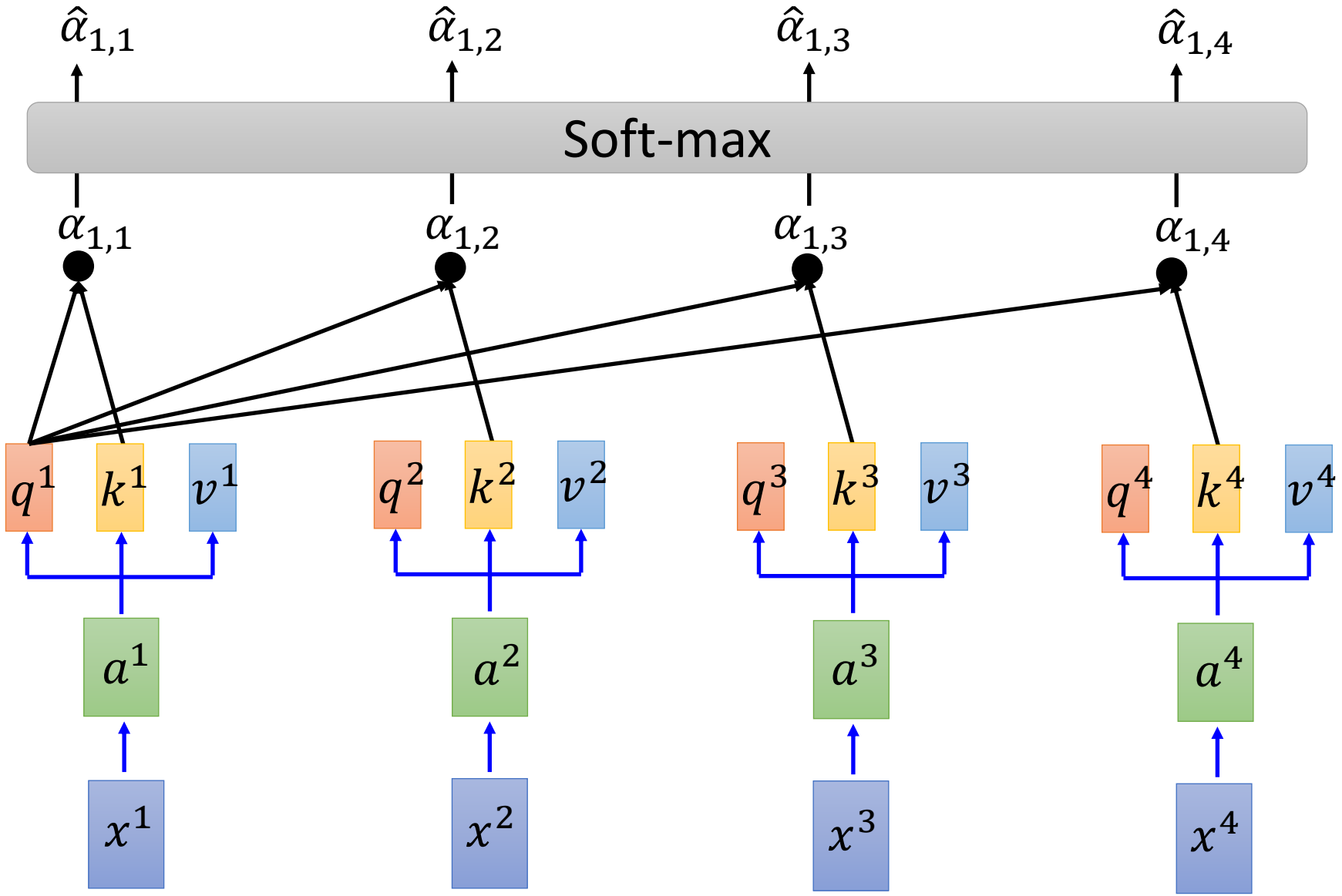
d est la dim de q et k

$$\alpha_{1,i} = \underbrace{q^1 \cdot k^i}_{\text{produit scalaire}} / \sqrt{d}$$



Self-attention

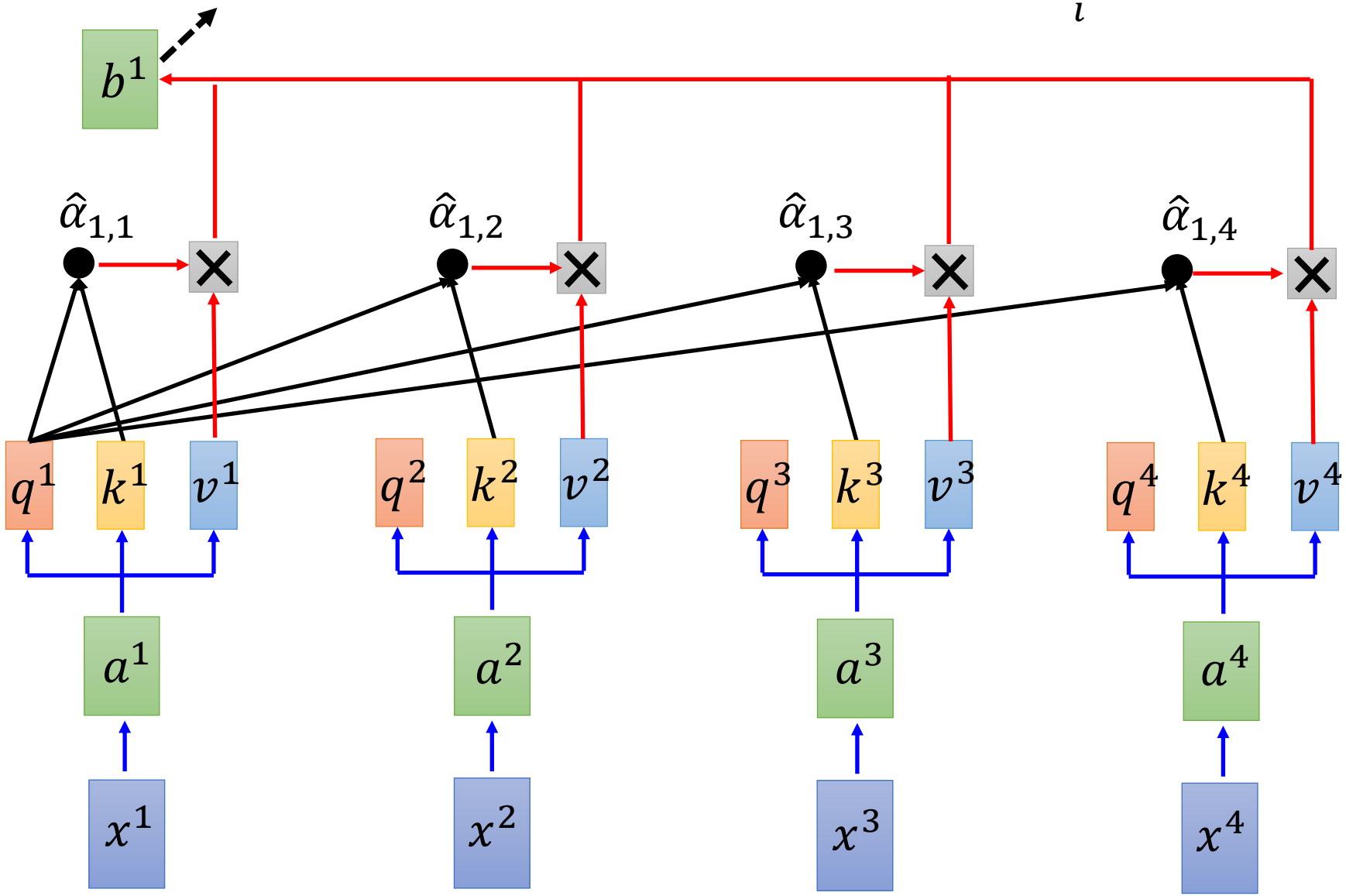
$$\hat{\alpha}_{1,i} = exp(\alpha_{1,i}) / \sum_j exp(\alpha_{1,j})$$



Self-attention

Considérant la séquence entière

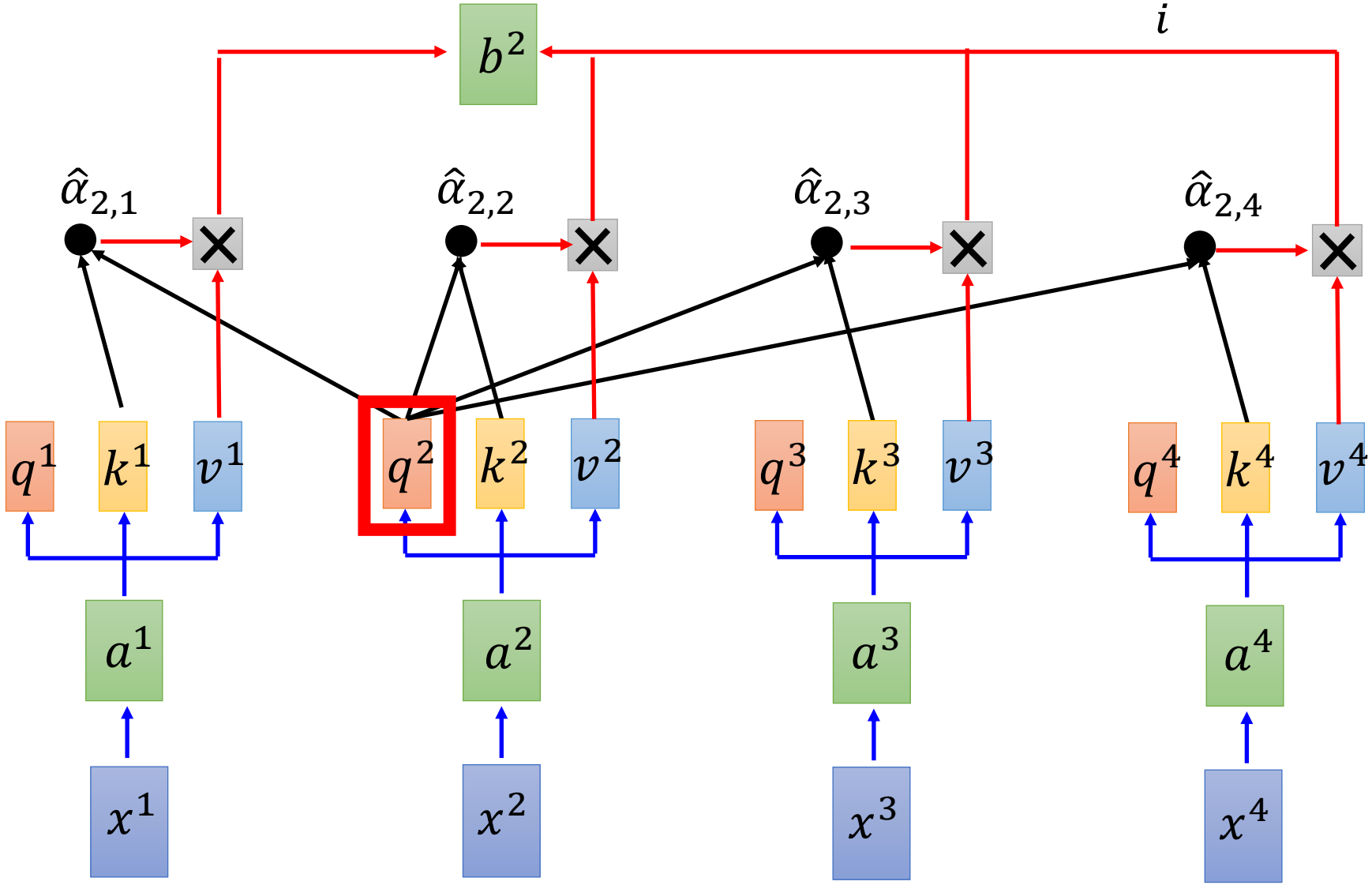
$$b^1 = \sum_i \hat{\alpha}_{1,i} v^i$$



Self-attention

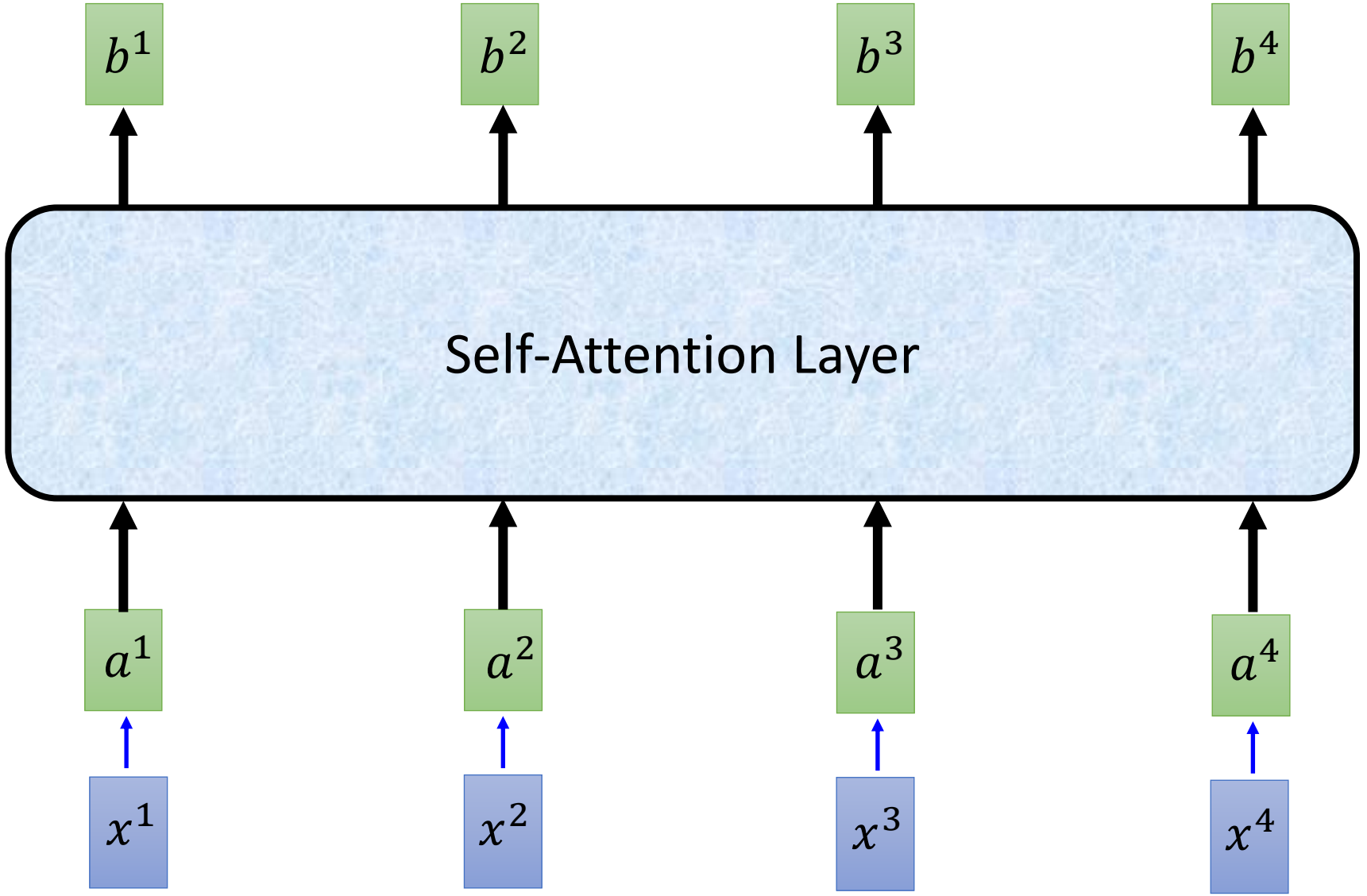
Prenez chaque requête q et faites attention à chaque clé k

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$



Self-attention

b^1, b^2, b^3, b^4 peut être calculé en parallèle.



Self-attention

$$q^i = W^q a^i$$

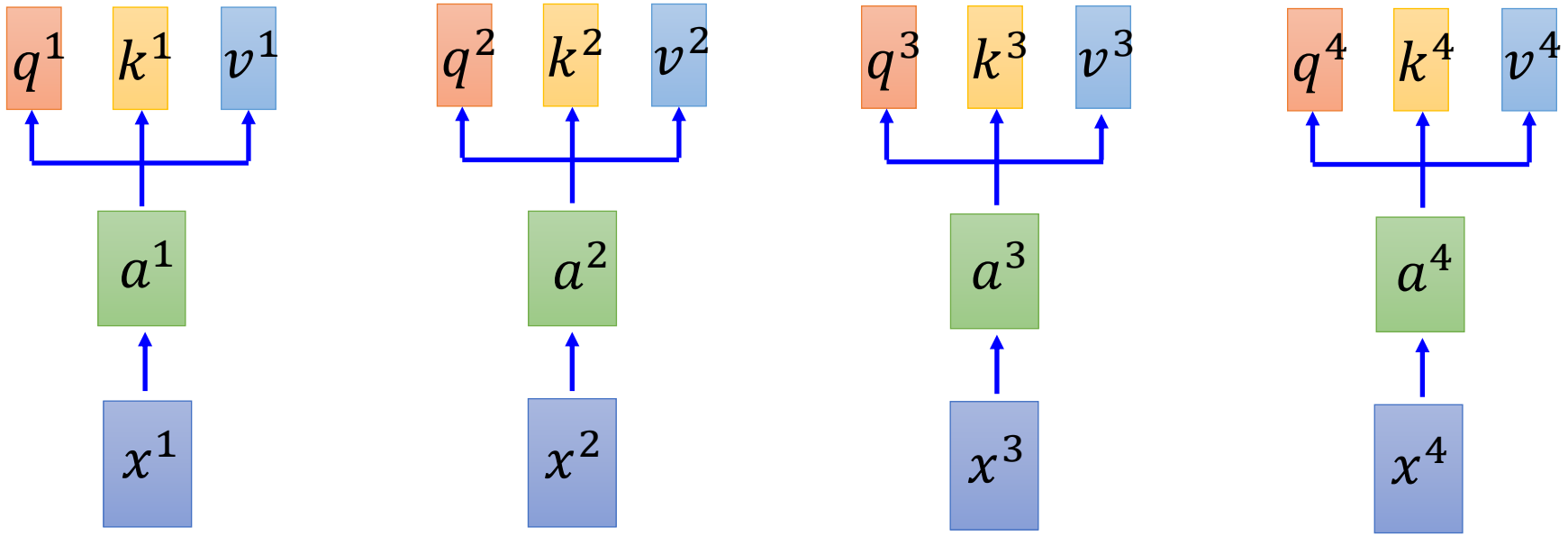
$$k^i = W^k a^i$$

$$v^i = W^v a^i$$

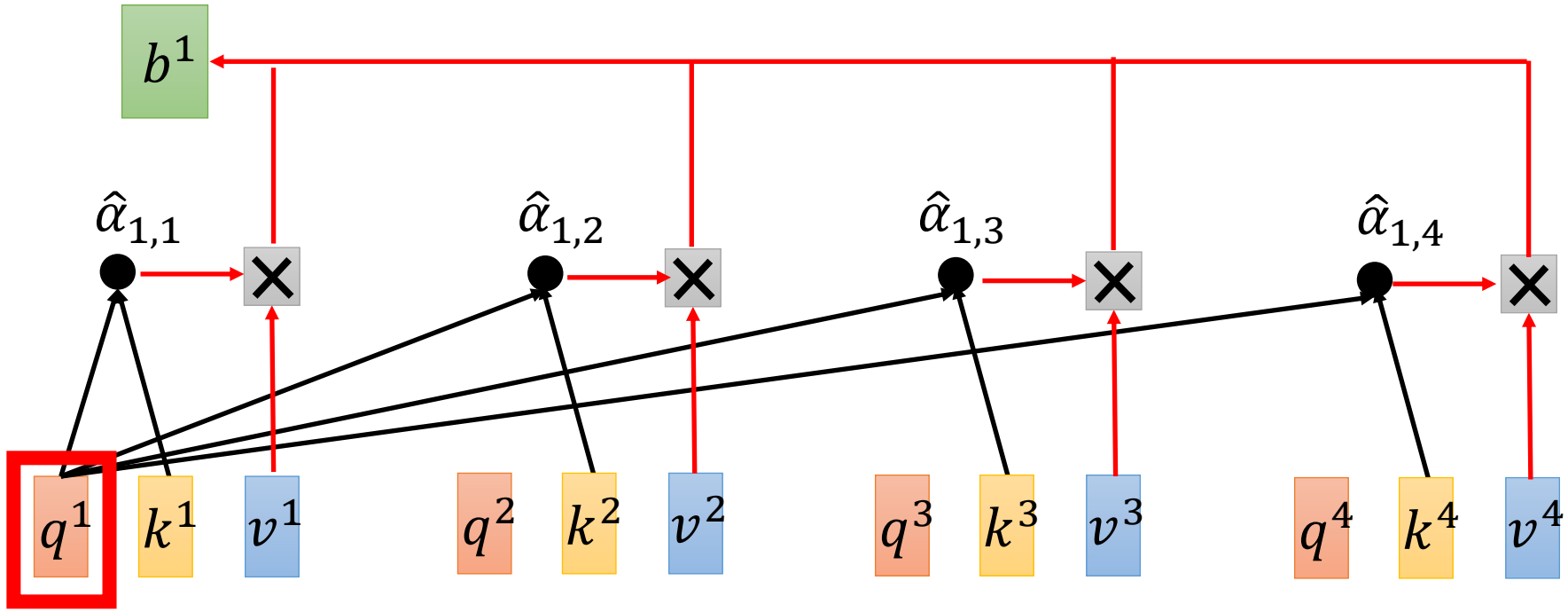
$$\underbrace{q^1\ q^2\ q^3\ q^4}_{Q} = W^q \underbrace{a^1\ a^2\ a^3\ a^4}_{I}$$

$$\underbrace{k^1\ k^2\ k^3\ k^4}_{K} = W^k \underbrace{a^1\ a^2\ a^3\ a^4}_{I}$$

$$\underbrace{v^1\ v^2\ v^3\ v^4}_{V} = W^v \underbrace{a^1\ a^2\ a^3\ a^4}_{I}$$



Self-attention



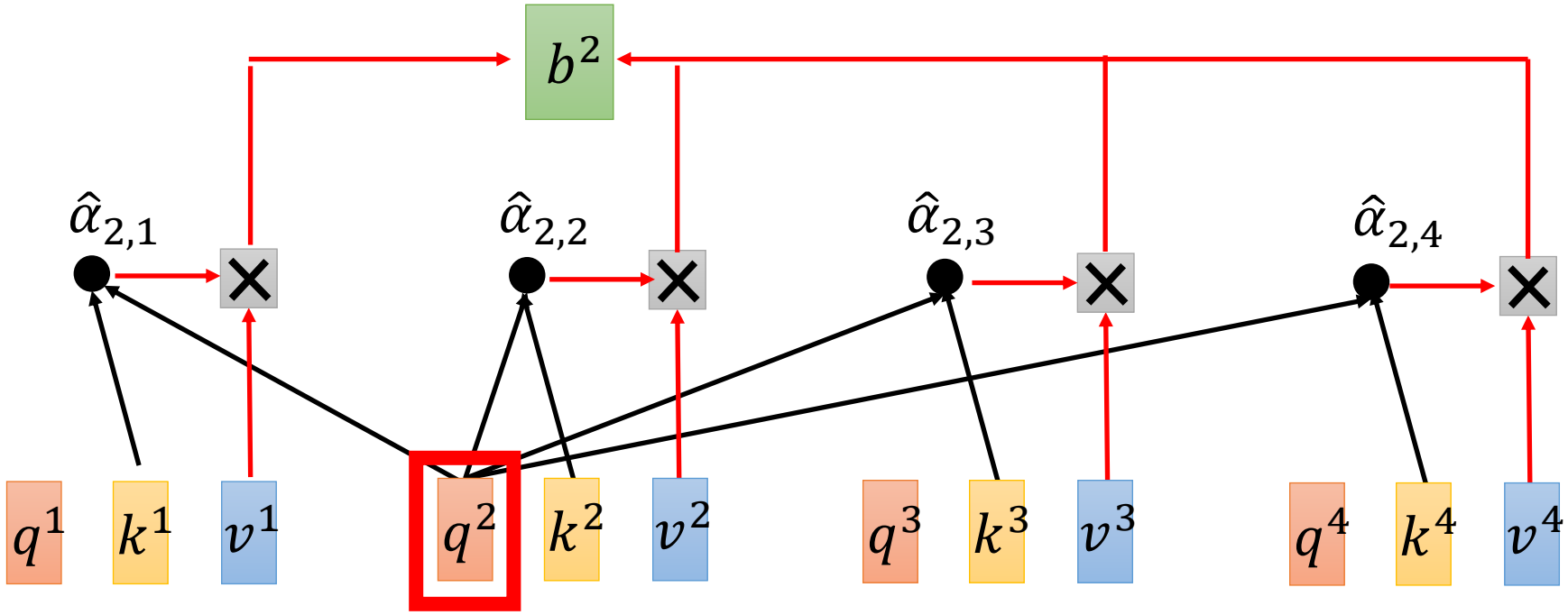
$$\alpha_{1,1} = k^1 \; q^1 \qquad \alpha_{1,2} = k^2 \; q^1$$

$$\alpha_{1,3} = k^3 \; q^1 \qquad \alpha_{1,4} = k^4 \; q^1$$

$$\begin{matrix} \alpha_{1,1} \\ \alpha_{1,2} \\ \alpha_{1,3} \\ \alpha_{1,4} \end{matrix} = \begin{matrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{matrix} \; q^1$$

Self-attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$

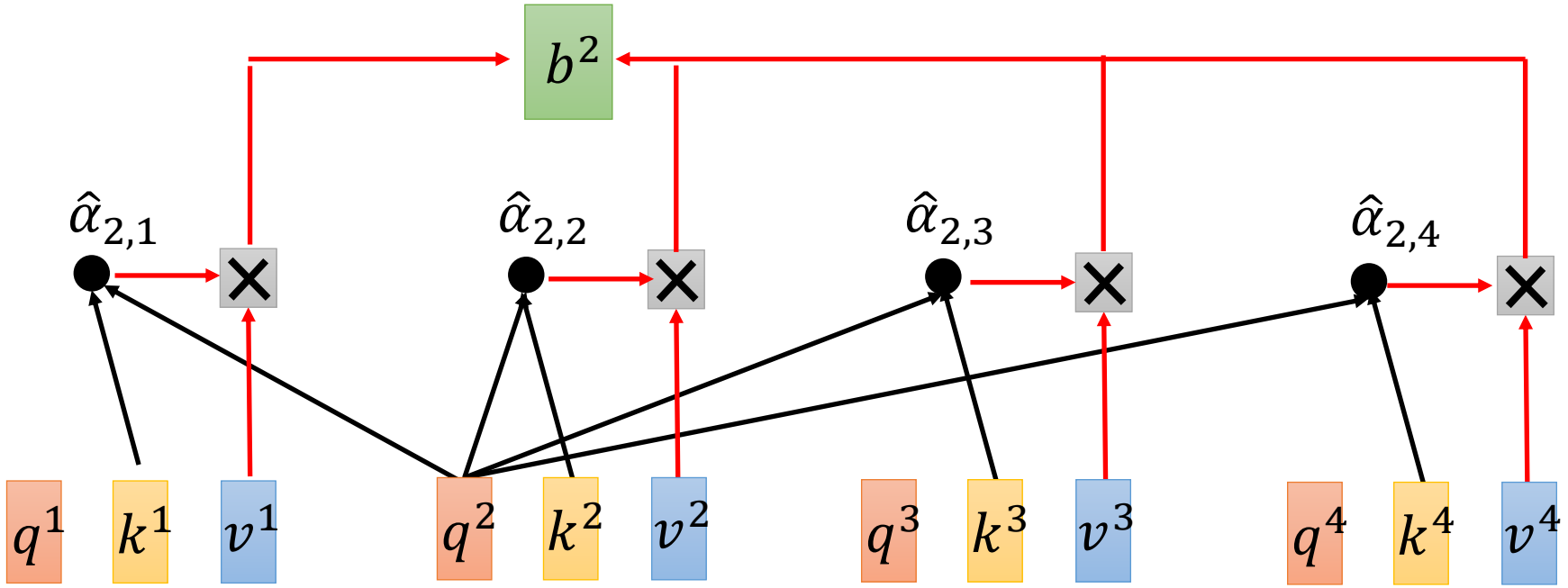


$$\begin{bmatrix} \hat{\alpha}_{1,1} & \hat{\alpha}_{2,1} & \hat{\alpha}_{3,1} & \hat{\alpha}_{4,1} \\ \hat{\alpha}_{1,2} & \hat{\alpha}_{2,2} & \hat{\alpha}_{3,2} & \hat{\alpha}_{4,2} \\ \hat{\alpha}_{1,3} & \hat{\alpha}_{2,3} & \hat{\alpha}_{3,3} & \hat{\alpha}_{4,3} \\ \hat{\alpha}_{1,4} & \hat{\alpha}_{2,4} & \hat{\alpha}_{3,4} & \hat{\alpha}_{4,4} \end{bmatrix} \leftarrow \begin{bmatrix} \alpha_{1,1} & \alpha_{2,1} & \alpha_{3,1} & \alpha_{4,1} \\ \alpha_{1,2} & \alpha_{2,2} & \alpha_{3,2} & \alpha_{4,2} \\ \alpha_{1,3} & \alpha_{2,3} & \alpha_{3,3} & \alpha_{4,3} \\ \alpha_{1,4} & \alpha_{2,4} & \alpha_{3,4} & \alpha_{4,4} \end{bmatrix} = \begin{bmatrix} k^1 \\ k^2 \\ k^3 \\ k^4 \end{bmatrix} \begin{bmatrix} q^1 & q^2 & q^3 & q^4 \end{bmatrix}$$

$\hat{A}$ A K^T Q

Self-attention

$$b^2 = \sum_i \hat{\alpha}_{2,i} v^i$$

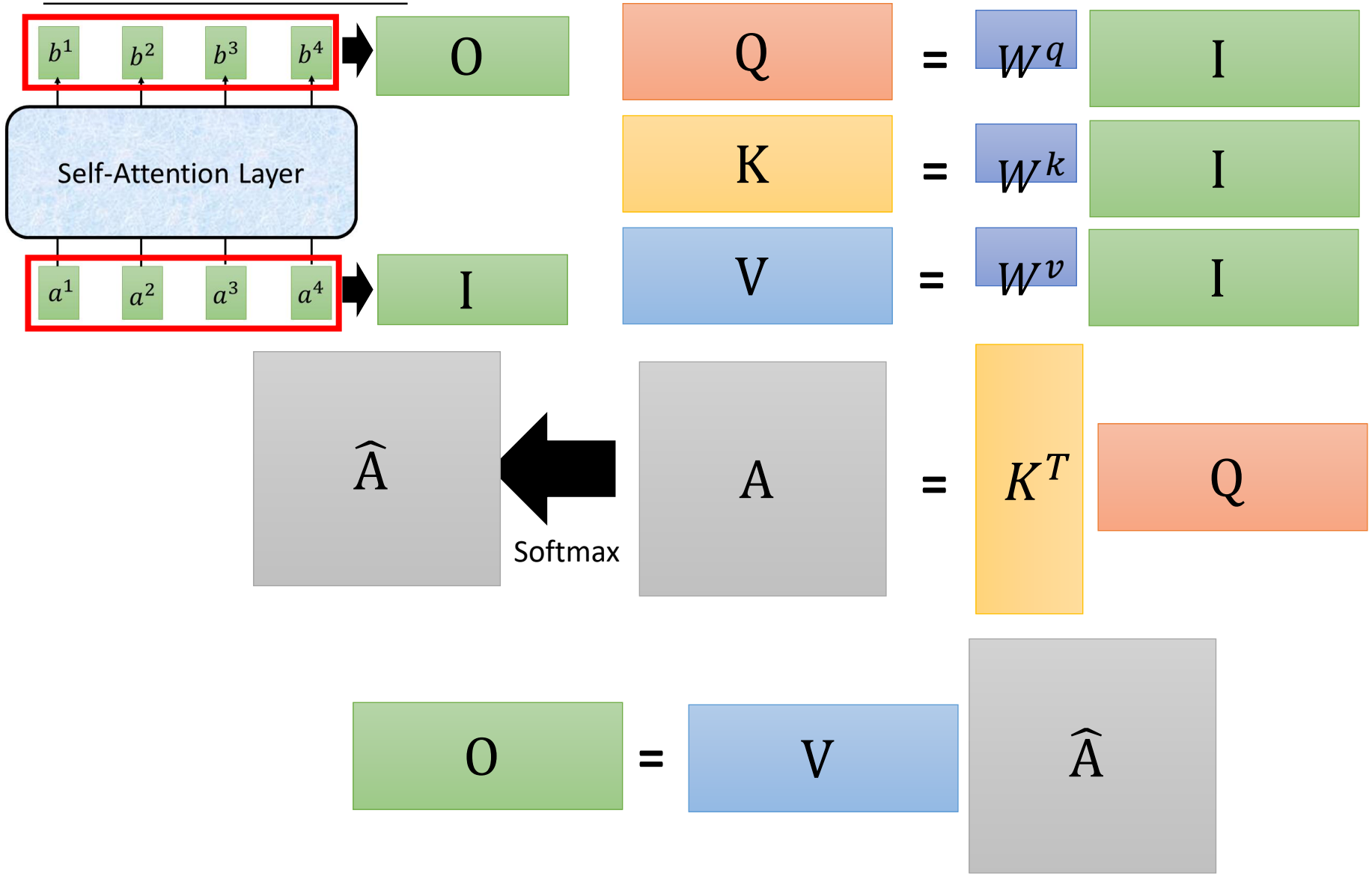


$b^1\ b^2\ b^3\ b^4$ = $v^1\ v^2\ v^3\ v^4$

O V

$$\hat{A} = \begin{bmatrix} \hat{\alpha}_{1,1} & \hat{\alpha}_{2,1} & \hat{\alpha}_{3,1} & \hat{\alpha}_{4,1} \\ \hat{\alpha}_{1,2} & \hat{\alpha}_{2,2} & \hat{\alpha}_{3,2} & \hat{\alpha}_{4,2} \\ \hat{\alpha}_{1,3} & \hat{\alpha}_{2,3} & \hat{\alpha}_{3,3} & \hat{\alpha}_{4,3} \\ \hat{\alpha}_{1,4} & \hat{\alpha}_{2,4} & \hat{\alpha}_{3,4} & \hat{\alpha}_{4,4} \end{bmatrix}$$

Self-attention



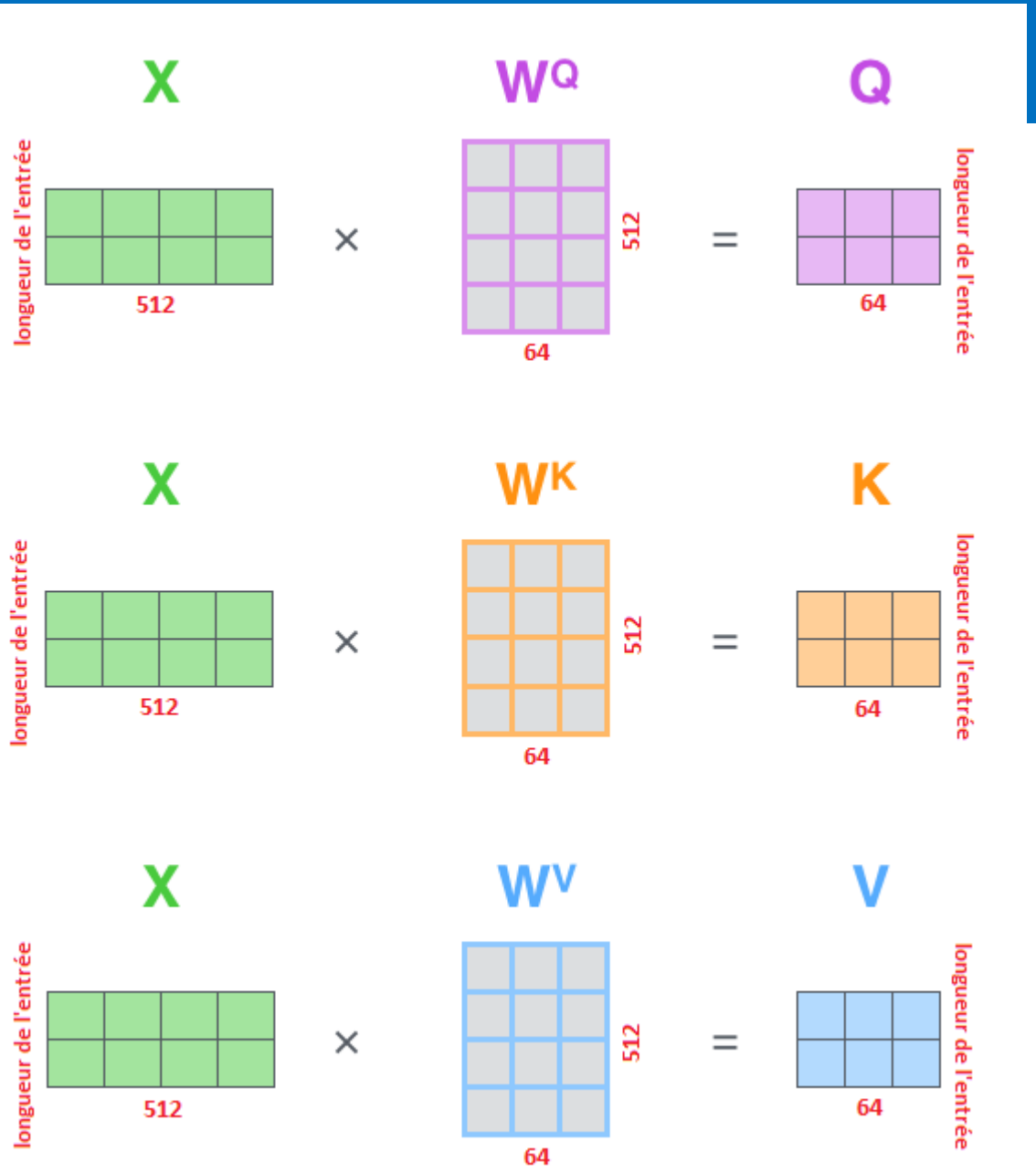
Cela revient à des opérations matricielles, accélérées efficacement par les GPU.

Word	q vector	k vector	v vector	score	score/8	Softmax	Sofmax * v	Sum
BONJOUR	q_1	k_1	v_1	$q_1 . k_1$	$(q_1 . k_1)/8$	x_{11}	$x_{11}.v_1$	z_1
je		k_2	v_2	$q_1 . k_2$	$(q_1 . k_2)/8$	x_{12}	$x_{12} . v_2$	
suis		k_3	v_3	$q_1 . k_3$	$(q_1 . k_3)/8$	x_{13}	$x_{13}.v_3$	

Word	q vector	k vector	v vector	score	score/8	Softmax	Sofmax * v	Sum
Bonjour		k_1	v_1	$q_2 . k_1$	$(q_2 . k_1)/8$	x_{21}	$x_{21}.v_1$	
JE	q_2	k_2	v_2	$q_2 . k_2$	$(q_2 . k_2)/8$	x_{22}	$x_{22} . v_2$	z_2
suis		k_3	v_3	$q_2 . k_3$	$(q_2 . k_3)/8$	x_{23}	$x_{23}.v_3$	

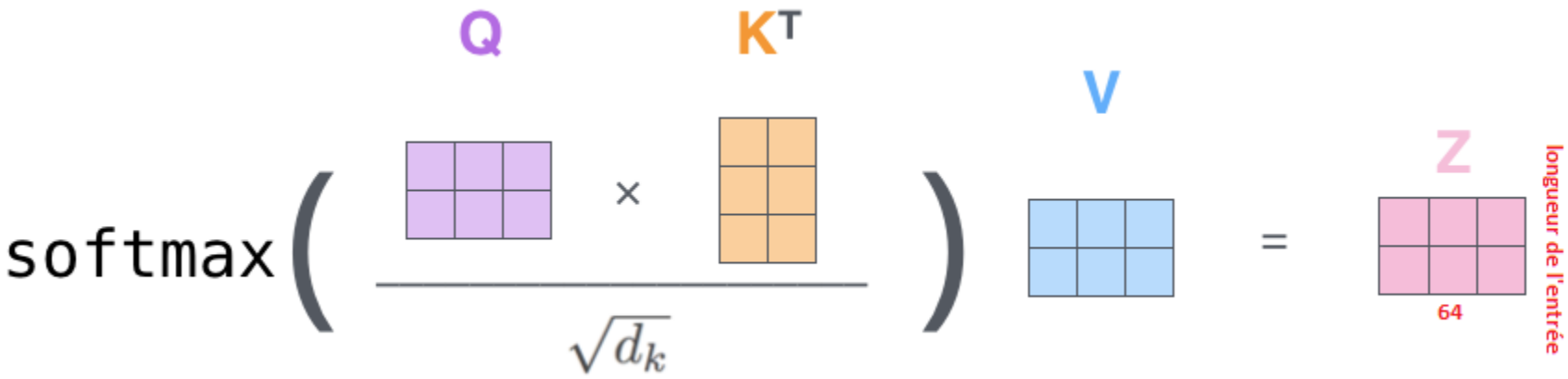
Word	q vector	k vector	v vector	score	score/8	Softmax	Sofmax * v	Sum
Bonjour		k_1	v_1	$q_3 . k_1$	$(q_3 . k_1)/8$	x_{31}	$x_{31}.v_1$	
je		k_2	v_2	$q_3 . k_2$	$(q_3 . k_2)/8$	x_{32}	$x_{32} . v_2$	
SUIS	q_3	k_3	v_3	$q_3 . k_3$	$(q_3 . k_3)/8$	x_{33}	$x_{33}.v_3$	z_3

- La première étape consiste à calculer les matrices Requête, Clé et Valeur.
- Pour ce faire, nous concaténons nos entrées dans une matrice X et nous la multiplions par les matrices de poids que nous avons entraînés (W^Q, W^K, W^V).
- chaque ligne de la matrice X correspond à un mot de la phrase d'entrée. Nous voyons à nouveau la différence de taille entre le vecteur d'entrée(512, ou 4 cases dans la figure), et les vecteurs q/k/v (64, ou 3 cases dans la figure).



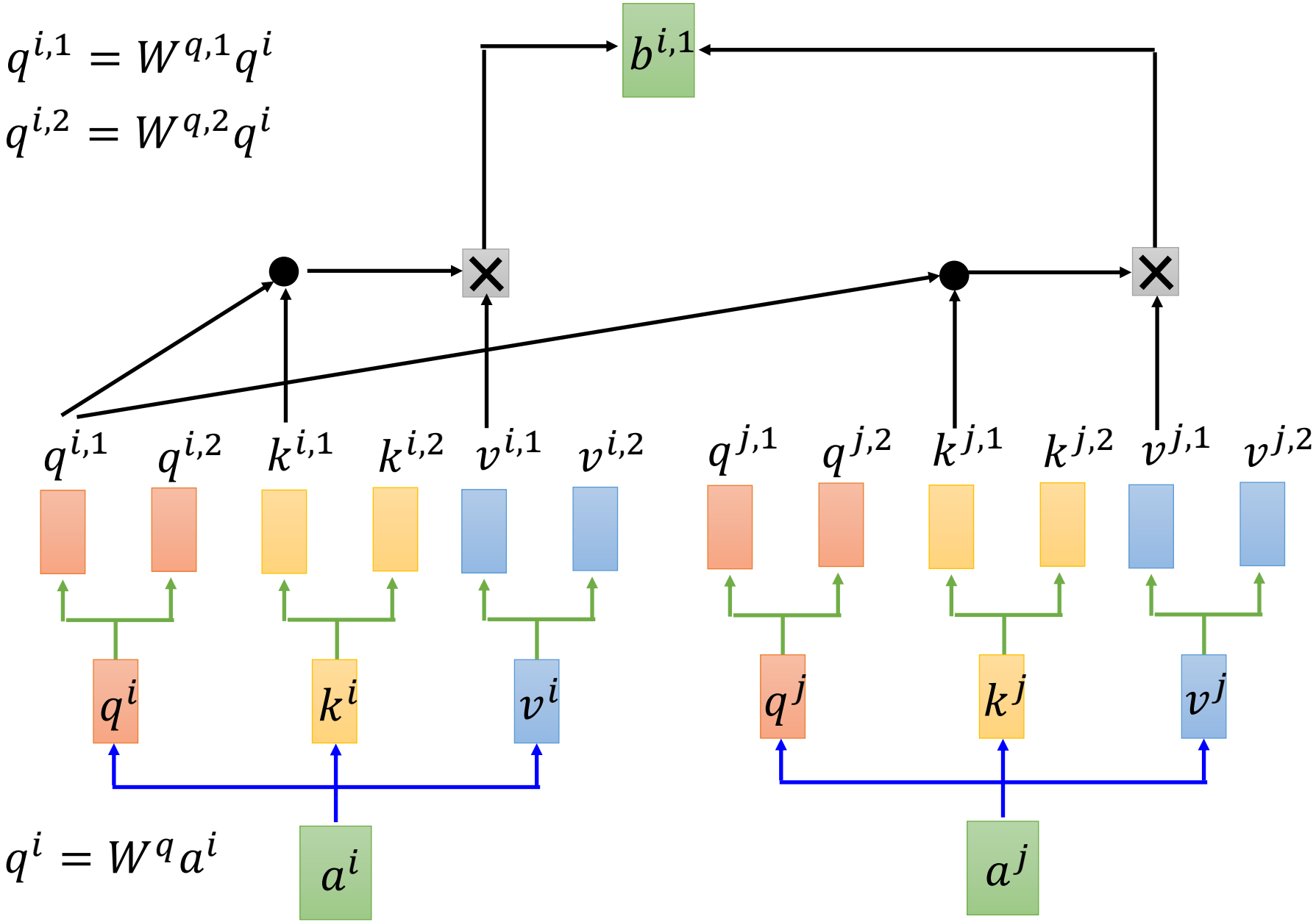
Self Attention

- Enfin, puisqu'il s'agit de matrices, nous pouvons condenser les étapes 2 à 6 en une seule formule pour calculer les sorties de la couche d'auto-attention.



Multi-head Self-attention

(2 têtes à titre d'exemple)

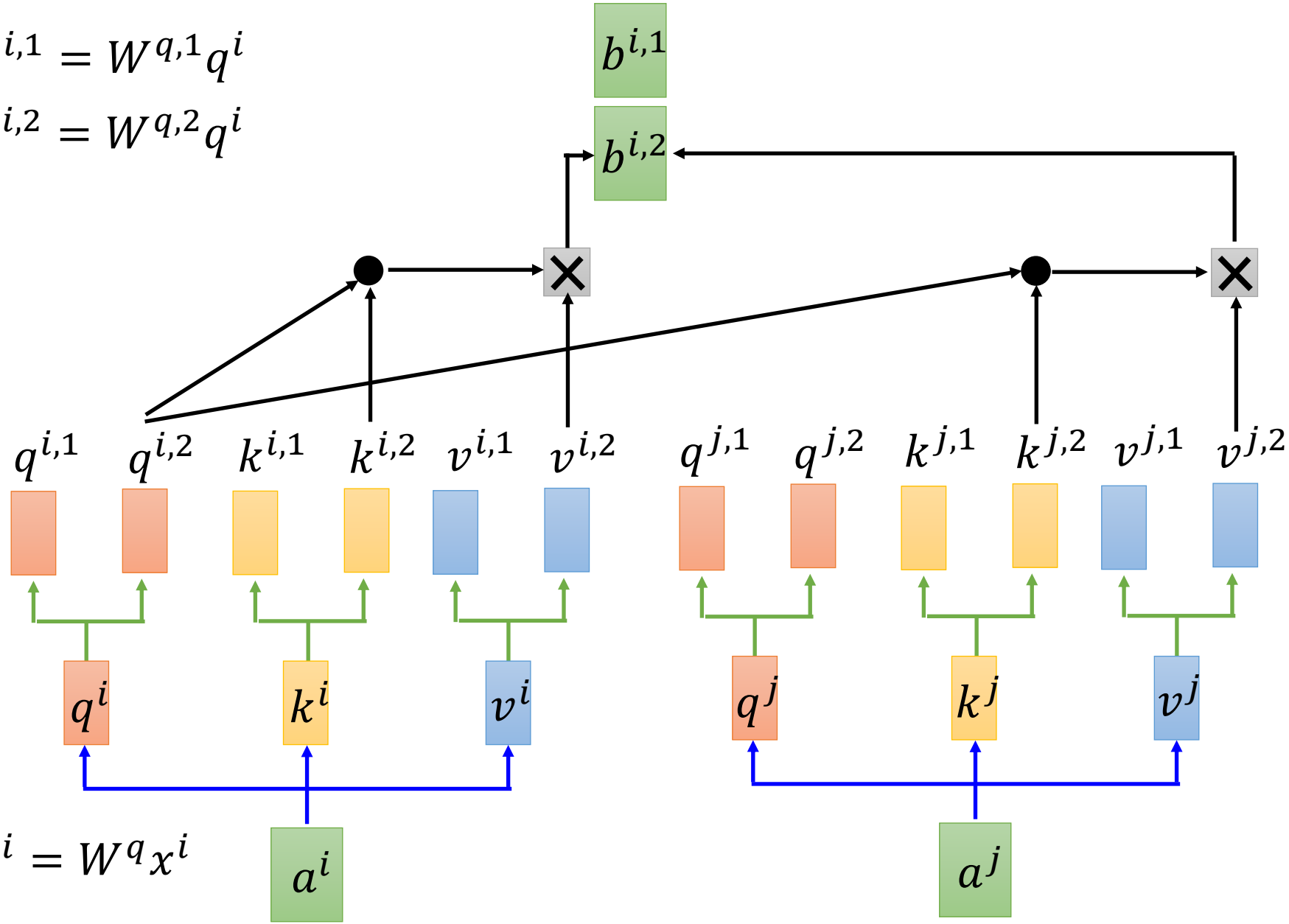


Multi-head Self-attention

(2 têtes à titre d'exemple)

$$q^{i,1} = W^{q,1} q^i$$

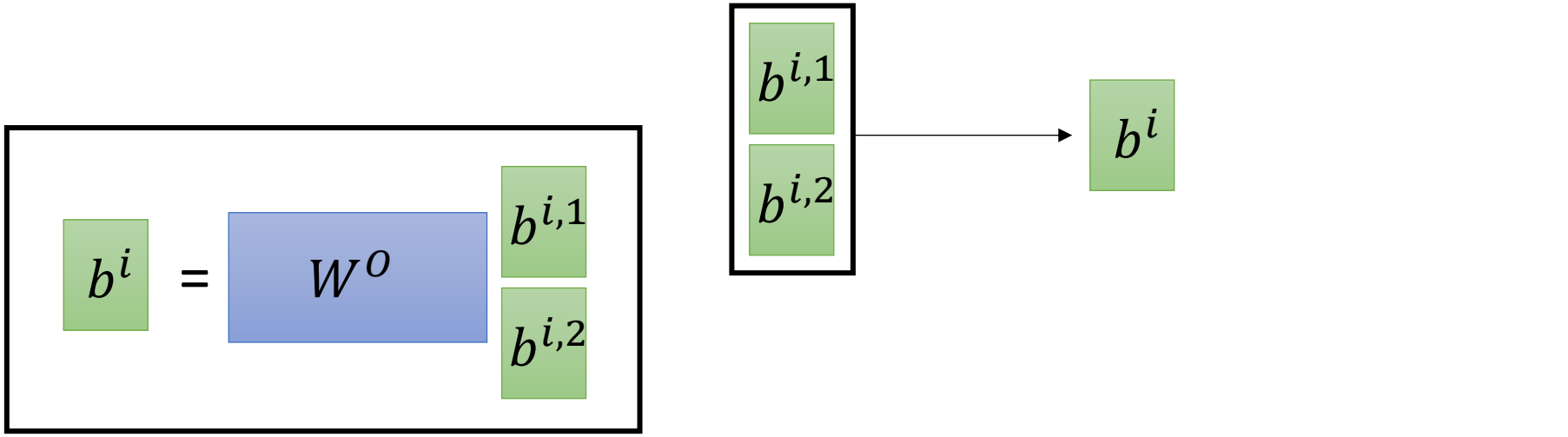
$$q^{i,2} = W^{q,2} q^i$$

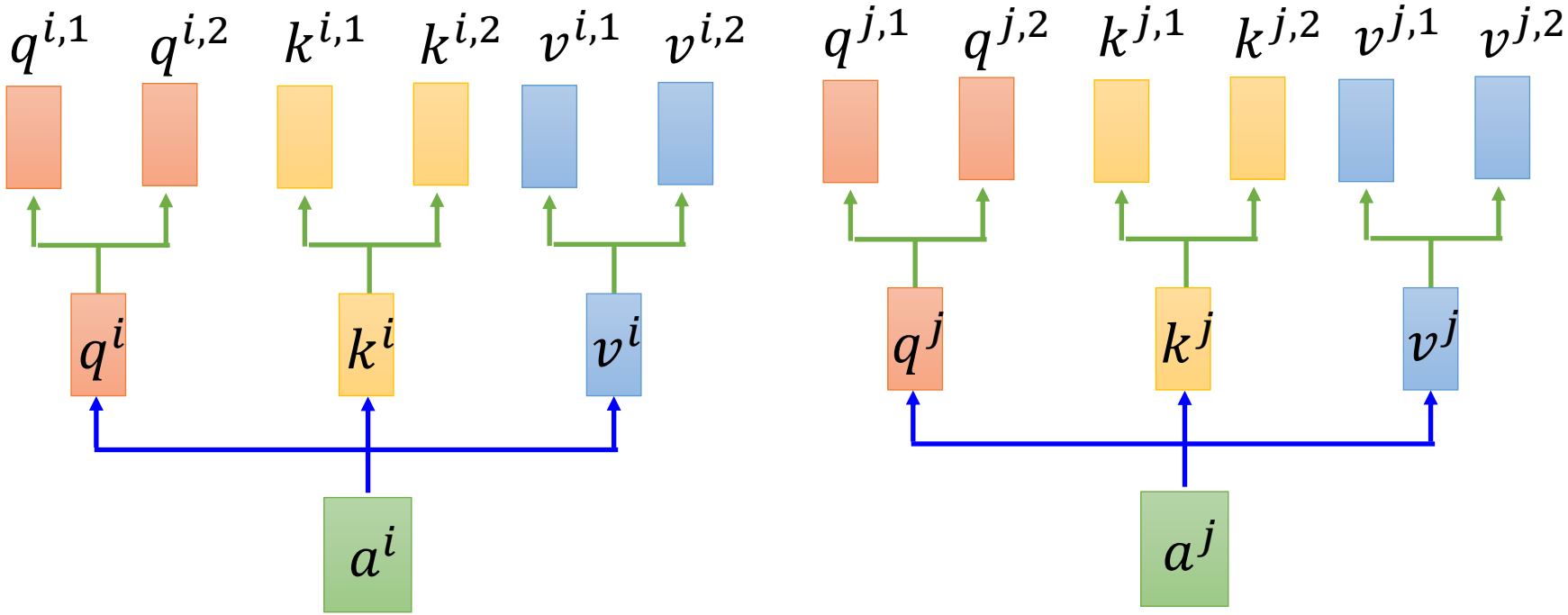


$$q^i = W^q x^i$$

Multi-head Self-attention

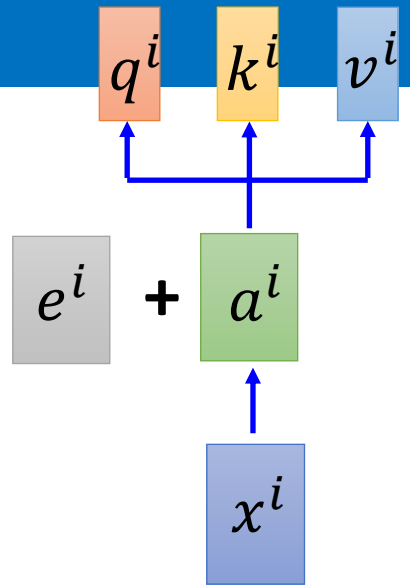
(2 têtes à titre d'exemple)

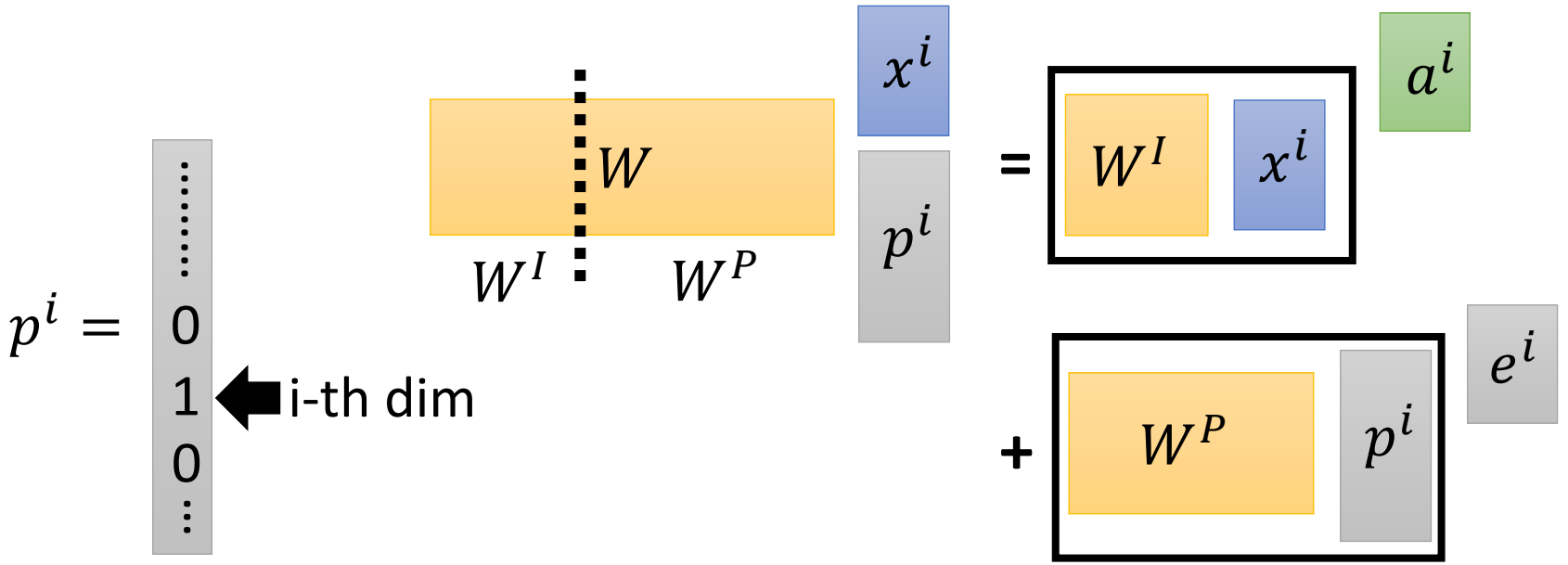




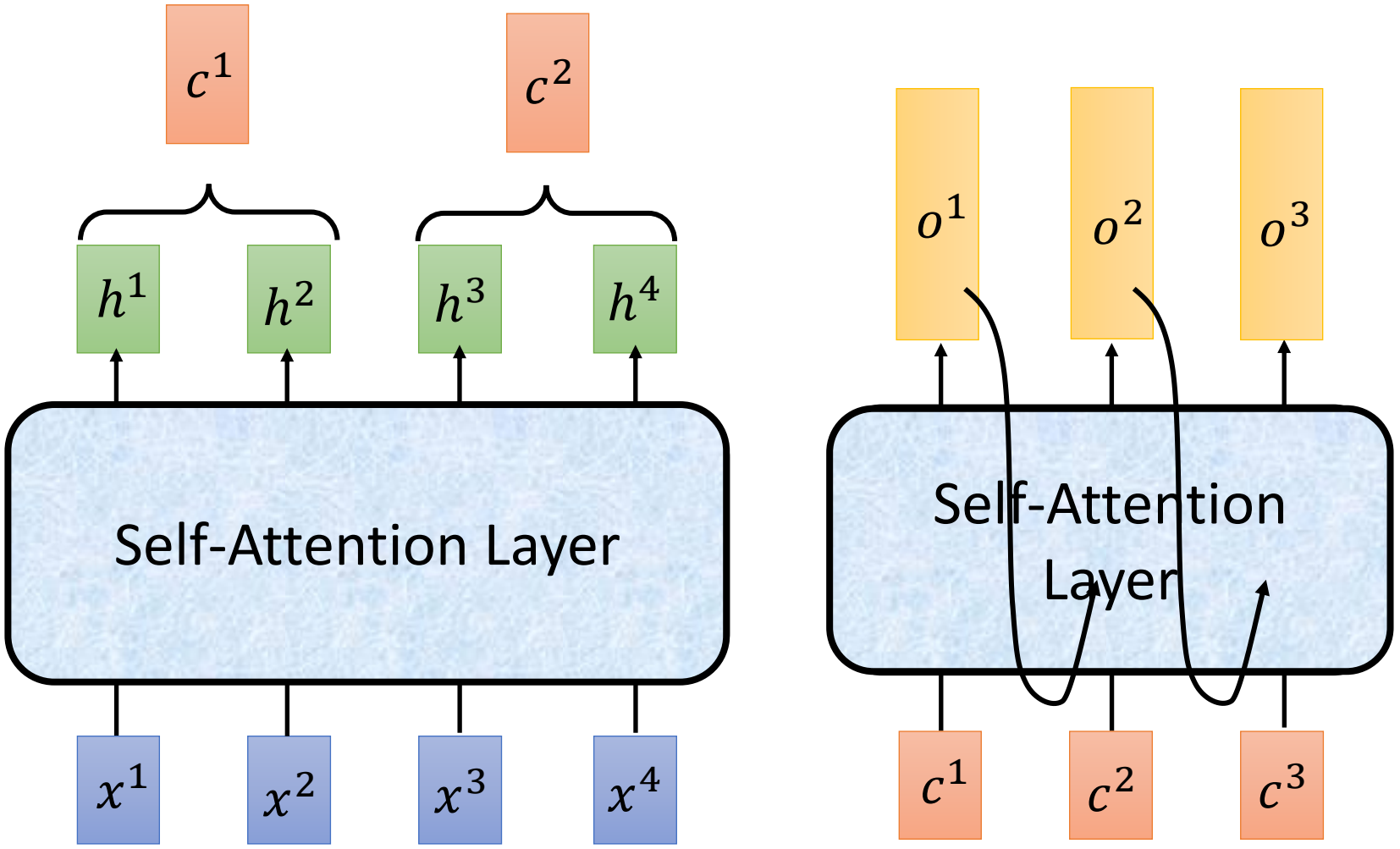
Positional Encoding

- Aucune information de position dans l'auto-attention.
- Article original : chaque position possède un vecteur de position unique e^i (non appris à partir des données)
- En d'autres termes : a chaque x^i on ajoute un vecteur one-hot p^i





Seq2seq with Attention

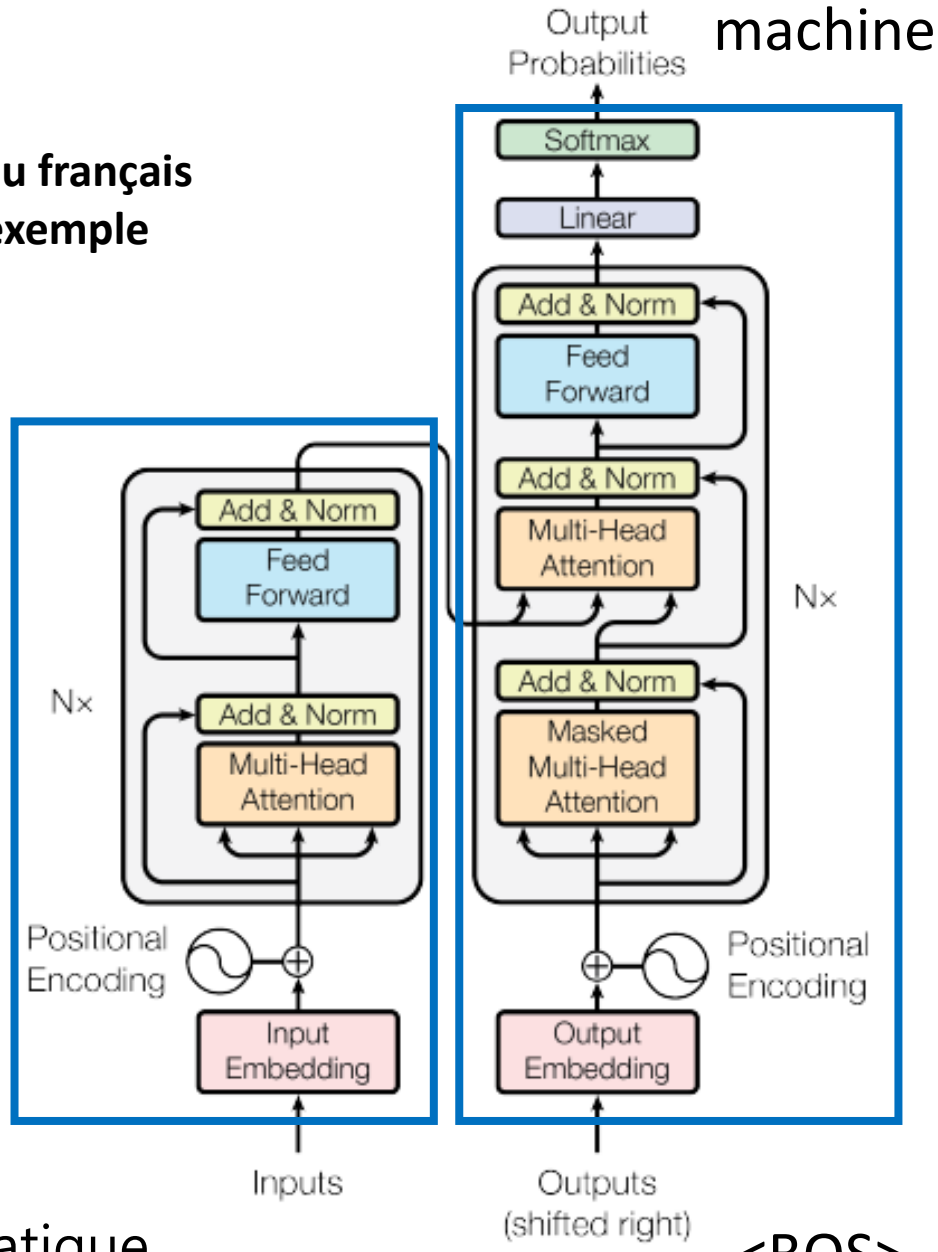


https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html

Transformer

Utiliser la traduction du français vers l'anglais comme exemple

machine learning



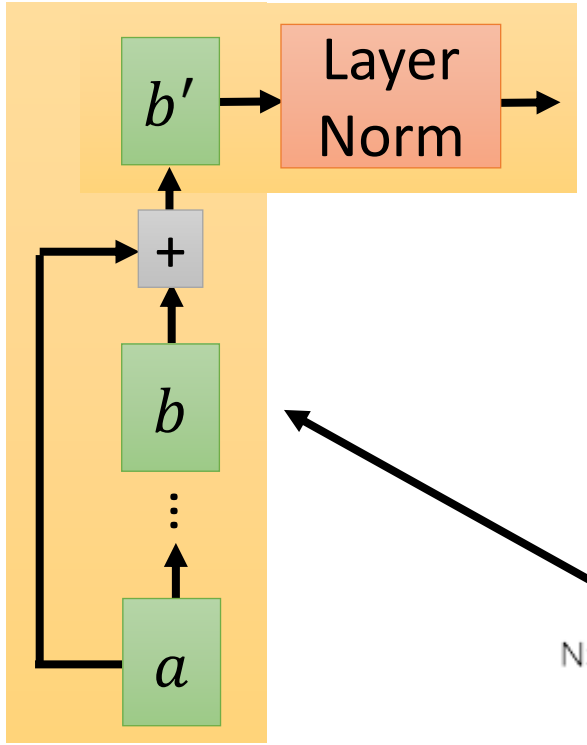
Encoder

Decoder

apprentissage automatique

<BOS> machine

Transformer

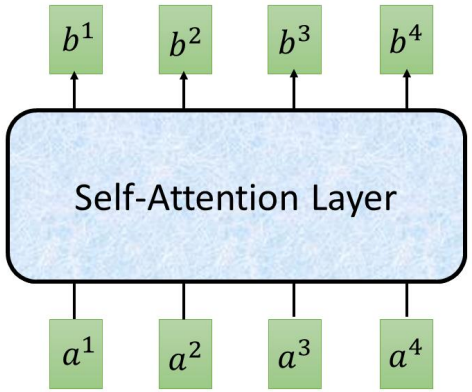


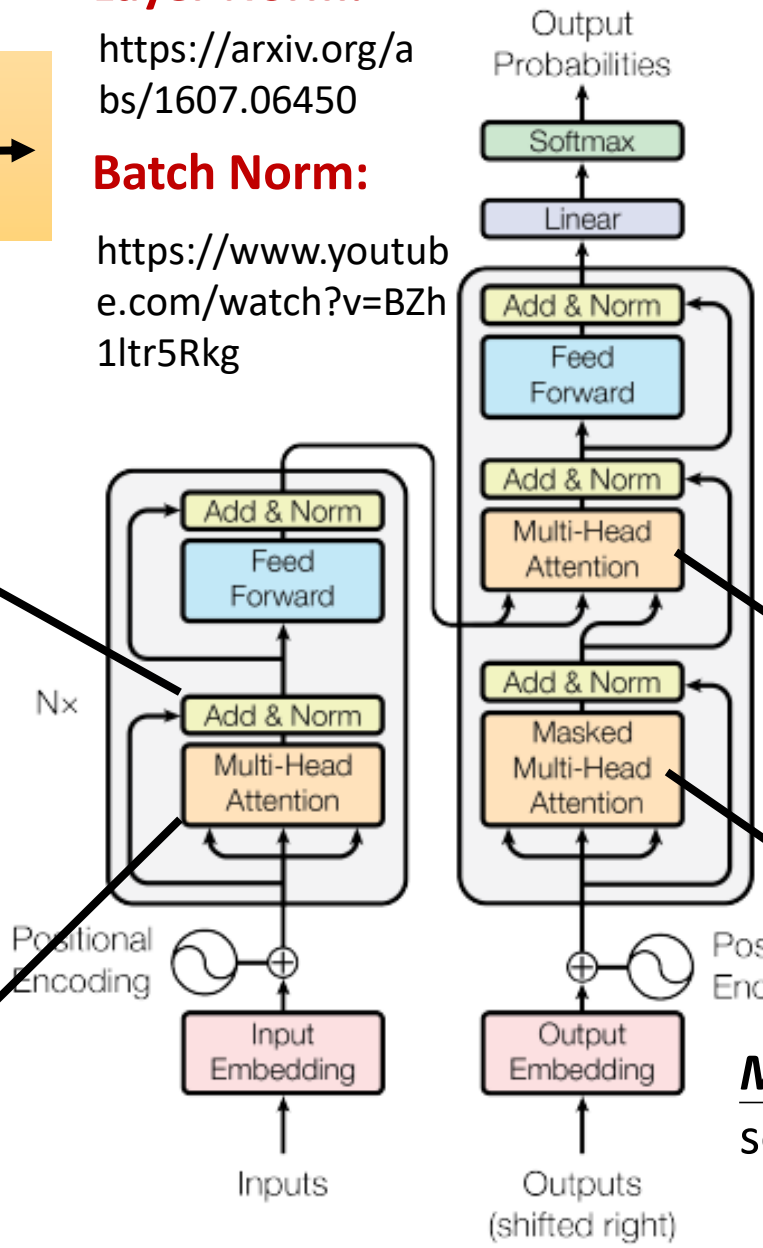
Layer Norm:

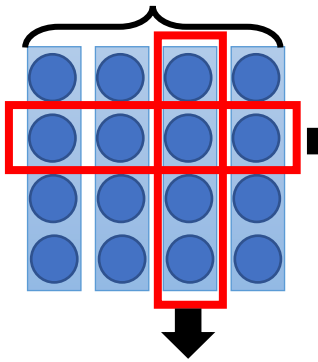
https://arxiv.org/abs/1607.06450

Batch Norm:

https://www.youtube.com/watch?v=BZh1ltr5Rkg







Batch Size

$\mu = 0, \sigma = 1$ Batch

$\mu = 0, \sigma = 1$ Layer

attend on the input sequence

Masked: assister à la séquence générée

Batch Normalization (BatchNorm)

Principe :

- Normalise les **activations d'une couche** en utilisant la **moyenne et la variance calculées sur un batch** (lot d'exemples).
- Formule pour un neurone :

$$\hat{x}_i = \frac{x_i - \mu_{\text{batch}}}{\sqrt{\sigma^2_{\text{batch}} + \epsilon}}$$

où :

- μ_{batch} = moyenne du batch,
- σ^2_{batch} = variance du batch,
- ϵ = petite constante pour la stabilité numérique.

Problèmes dans les Transformers :

- **Dépendance au batch** : Performance dégradée avec des petits batches (ex: en inference).
- **Incompatible avec les séquences de longueur variable** (les Transformers gèrent des inputs de tailles différentes).

Utilisation :

- Peu adaptée aux architectures basées sur l'attention (rare dans les Transformers).

Layer Normalization (LayerNorm)

Principe :

- Normalise les **activations pour chaque exemple individuellement** (calcule moyenne/variance sur **toutes les features** de l'exemple).
- Formule pour un token donné :

$$\hat{x}_i = \frac{x_i - \mu_{\text{layer}}}{\sqrt{\sigma^2_{\text{layer}} + \epsilon}}$$

où :

- μ_{layer} = moyenne des features du token,
- σ^2_{layer} = variance des features du token.

Avantages dans les Transformers :

- **Indépendante du batch** : Fonctionne bien même avec batch_size=1 (critique pour l'inférence).
- **Stabilise l'apprentissage** : Réduit les problèmes de gradients (exploding/vanishing) dans les réseaux profonds.
- **Adaptée aux séquences** : Gère naturellement les longueurs variables (car normalise token par token).

Exemple

- Imaginons que la phrase **"Je mange une pomme"** passe à travers un Transformer (par exemple, pour une tâche comme la traduction ou la génération de texte). Voici comment **Layer Normalization (LayerNorm)** et **Batch Normalization (BatchNorm)** interviendraient :

1. **Représentation des Tokens (Embeddings) :** La phrase est tokenisée en :
   - ["Je", "mange", "une", "pomme"]
   - Chaque token est converti en un vecteur (embedding) de dimension d=512 (par exemple) :
     - Je → [0.2, -0.5, 0.7, ..., 1.1] (vecteur de 512 valeurs)
     - mange → [-0.3, 0.8, 0.1, ..., -0.4]
     - etc.

Batch Normalization

- Calcul : Moyenne et variance sont calculées sur tous les tokens du batch (ex: si batch=8 phrases, on normalise sur 8×4=32 tokens).
- **Problème :**
  - Si le batch contient des phrases de longueurs différentes, la normalisation est faussée. En inference (batch_size=1), les stats sont instables.
- **Exemple :**
  - Pour le token "Je", BatchNorm utiliserait la moyenne/variance de tous les "Je", "mange", "une", etc. du batch.
- **Risque** : Si un autre batch a des mots très différents, la normalisation est incohérente.

→ Peu adapté aux séquences de texte !

Layer Normalization (LayerNorm)

- Calcul : Moyenne et variance sont calculées pour chaque token individuellement (sur ses 512 features).
- Avantage :
  - Indépendant du batch → fonctionne même avec un seul token.
  - Stable pour les séquences de longueur variable.
- Exemple pour le token "mange" :
  1. Soit son embedding : [-0.3, 0.8, 0.1, ..., -0.4] (512 valeurs).
  2. LayerNorm calcule :
     - Moyenne (μ) = moyenne des 512 valeurs.
     - Variance (σ^2) = variance des 512 valeurs.
  3. Normalisation :
     - Chaque feature est ajustée : $$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$$
  4. Résultat : Un vecteur recentré/réduit, **sans dépendre des autres tokens**.

→ **Stabilise l'apprentissage pour chaque mot individuellement**

Dans un Bloc Transformer

- Pour chaque token ("Je", "mange", etc.) :

1. Self-Attention : Les tokens interagissent (ex: "mange" dépend de "pomme").
2. LayerNorm appliquée après :
    - Normalise les features du token après attention.
    - Exemple pour "mange" :
        - Input avant normalisation : [0.6, -1.2, 0.9, ...] (sortie de l'attention).
        - Après LayerNorm : [0.1, -0.3, 0.4, ...] (échelle cohérente).

Masked Multi-Head Attention

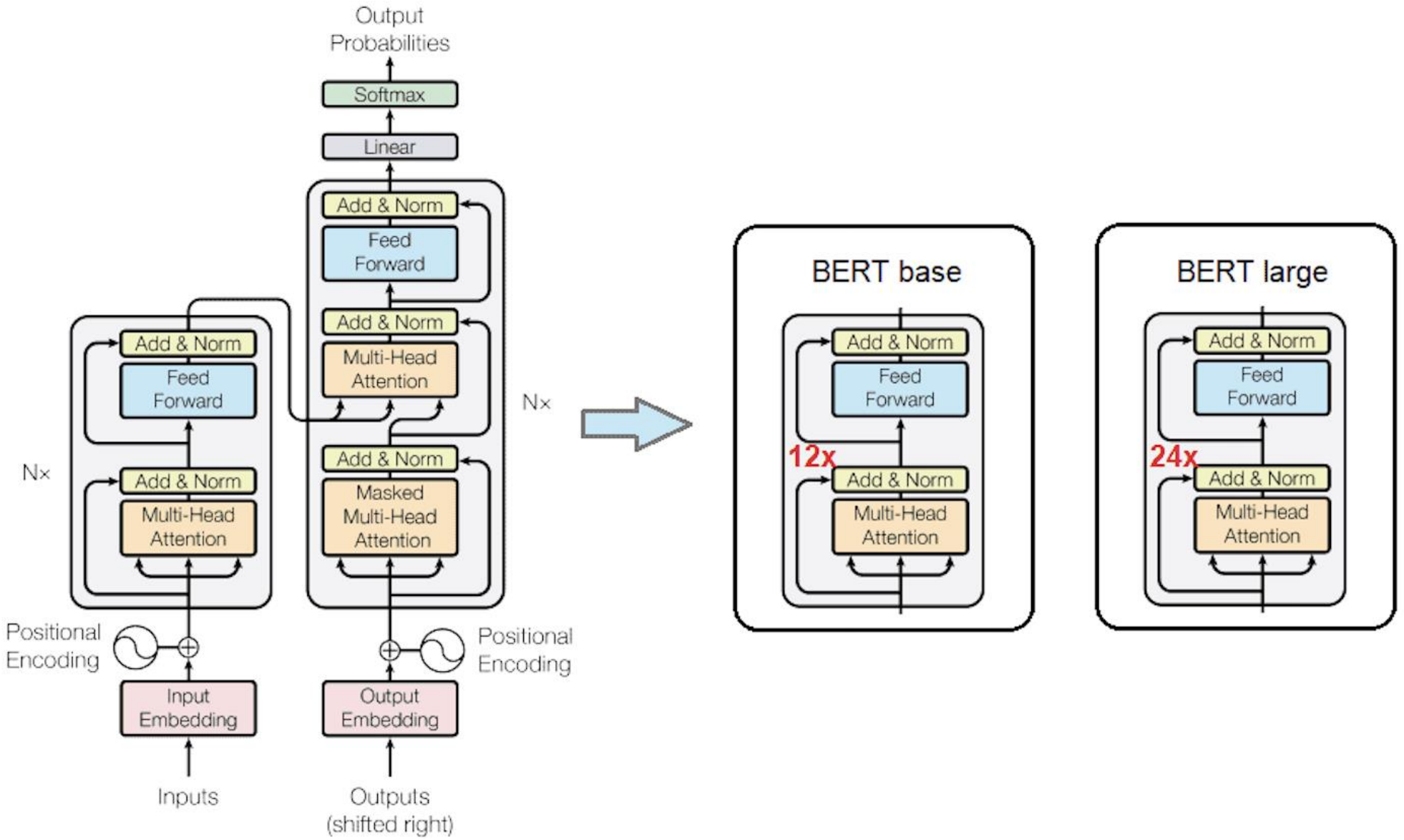
- Le **Masked Multi-Head Attention** est un mécanisme clé dans les modèles **Transformer** (comme GPT), utilisé pour empêcher le modèle de "tricher" en regardant les tokens futurs lors de l'entraînement ou de la génération de texte.

Définition Simple

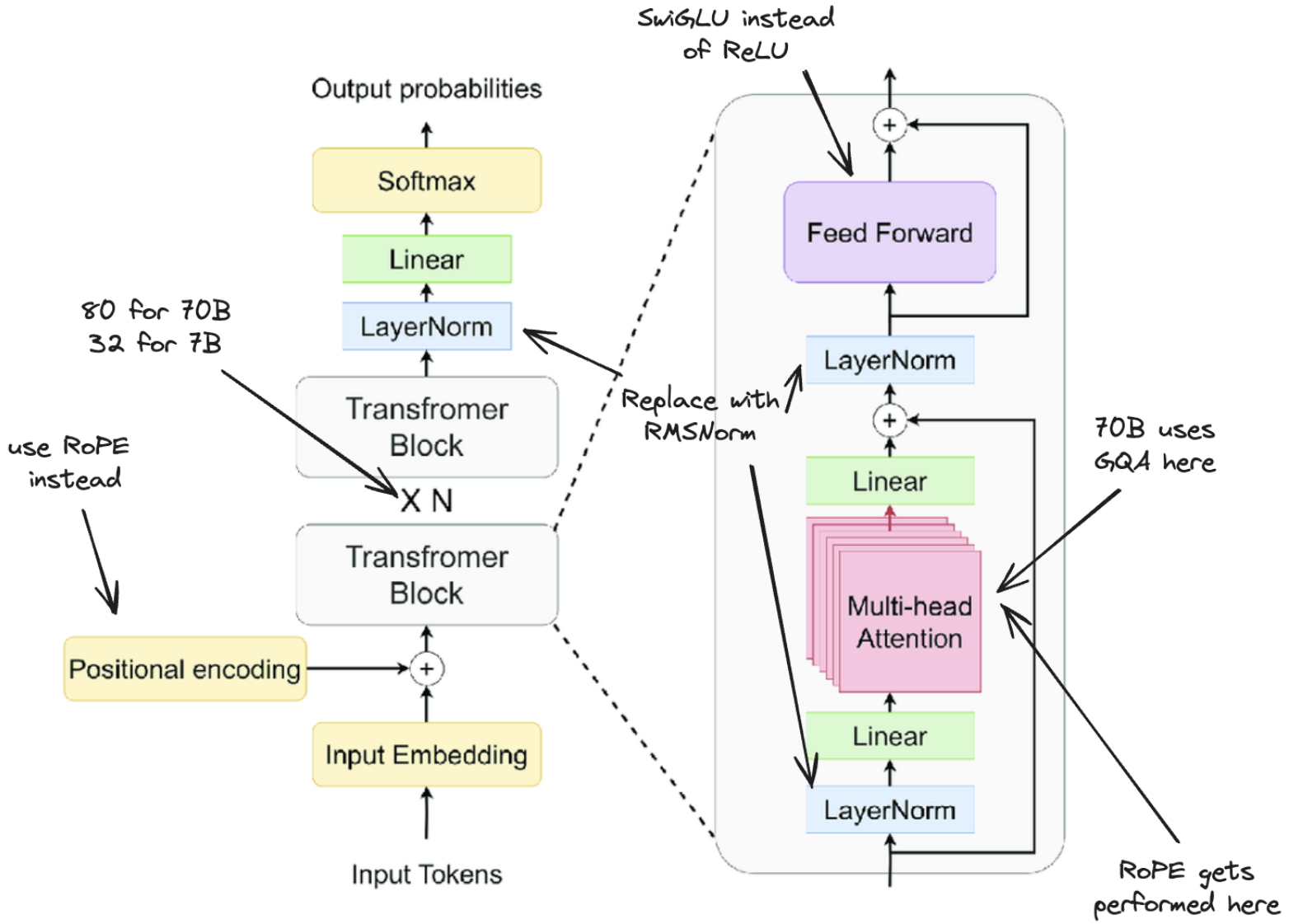
  - **Multi-Head Attention** : Mécanisme où le modèle utilise plusieurs "têtes" d'attention pour capturer différents types de dépendances entre les mots (ex: relations syntaxiques, sémantiques).
  - **Masked** : Un masque est appliqué pour **cacher les tokens futurs**, forçant le modèle à ne dépendre que des tokens précédents.

- **But** : Éviter que le modèle utilise des informations des mots "futurs" (qu'il ne devrait pas encore connaître).
  - **Exemple** : Pour prédire le mot "mange" dans "Je mange une pomme", le modèle ne doit pas voir "une pomme".
- **Applications :**
  - GPT (génération de texte).
  - Décodeur des Transformers (traduction automatique).

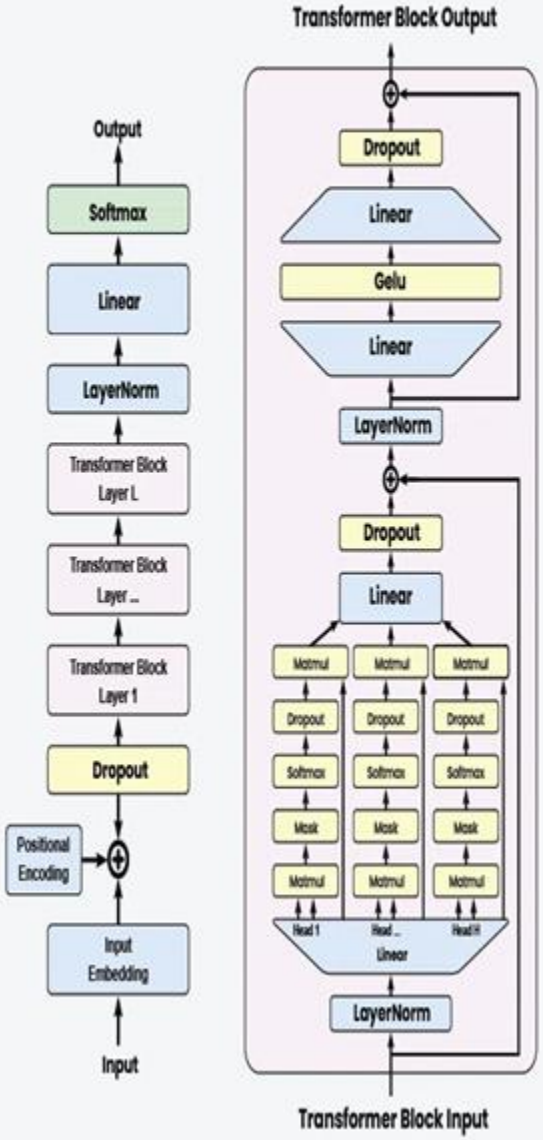
BERT


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
Nx
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Nx
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)
BERT base
12x
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
BERT large
24x
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention

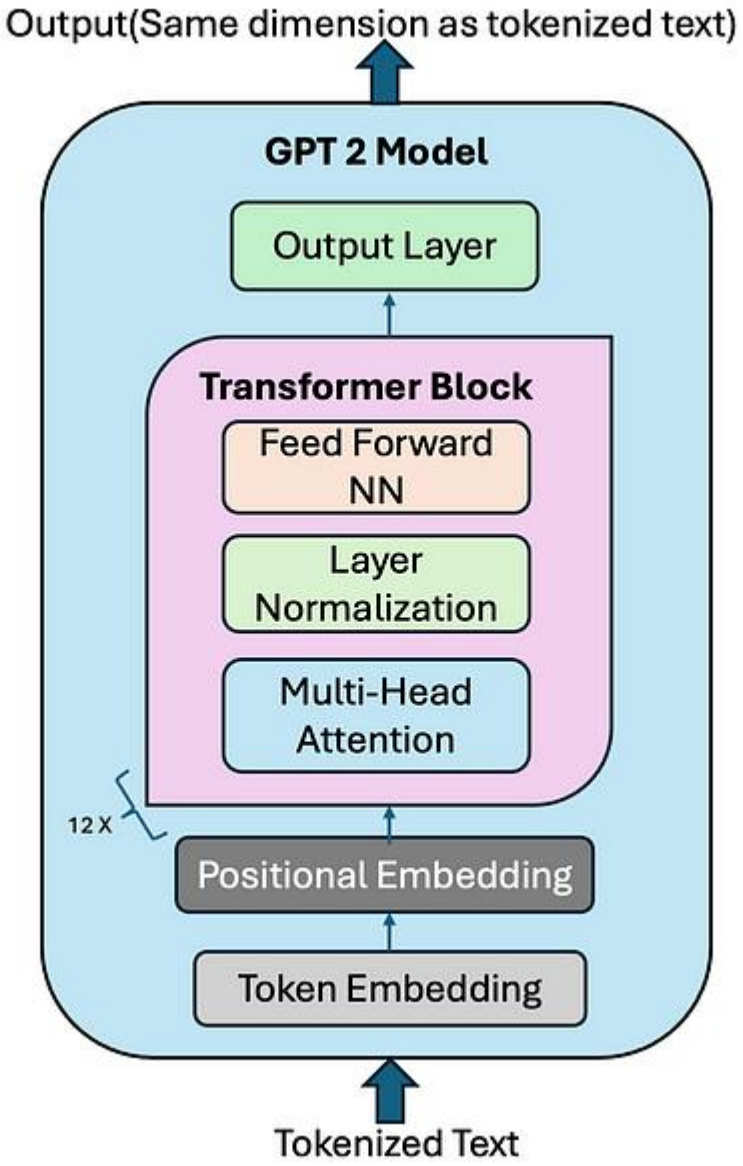
LLAMA


SwiGLU instead of ReLU
Output probabilities
Softmax
Linear
LayerNorm
80 for 70B
32 for 7B
Transfromer Block
X N
use RoPE instead
Transfromer Block
Positional encoding
Input Embedding
Input Tokens
Replace with RMSNorm
Feed Forward
LayerNorm
Linear
Multi-head Attention
Linear
LayerNorm
70B uses GQA here
RoPE gets performed here

GPT Arcihtecture



GPT-2


Output(Same dimension as tokenized text)
GPT 2 Model
Output Layer
Transformer Block
Feed Forward NN
Layer Normalization
Multi-Head Attention
12 X
Positional Embedding
Token Embedding
Tokenized Text

Résumé

- **Qu’est-ce qu’un Transformer ?**

- Les Transformers sont des modèles de deep learning introduits par l’article “Attention is All You Need” en 2017. Contrairement aux réseaux de neurones traditionnels et aux LSTM (Long Short-Term Memory), les Transformers peuvent traiter un texte entier en parallèle, grâce à un mécanisme appelé self-attention. Cela permet aux Transformers de capturer des relations complexes dans le texte, même entre des mots éloignés.

Résumé

- **Qu’est-ce qu’un Tokenizer ?**

- Un tokenizer est un outil qui segmente le texte brut en unités plus petites appelées tokens. Ces tokens peuvent être des mots, des sous-mots, ou même des caractères individuels.

  Chaque token est ensuite mappé à un identifiant numérique unique, basé sur un vocabulaire préétabli que le modèle connaît.

  Exemple :

  Supposons que nous avons la phrase **“Je veux apprendre le NLP”**.

- Le tokenizer pourrait **la segmenter en tokens : [‘Je’, ‘veux’, ‘apprendre’, ‘le’, ‘NLP’].**

  Ensuite, chaque token est converti en un identifiant unique, par exemple, **[‘Je’ -> 345, ‘veux’ -> 786, ‘apprendre’ -> 232, ‘le’ -> 154, ‘NLP’ -> 876]**.

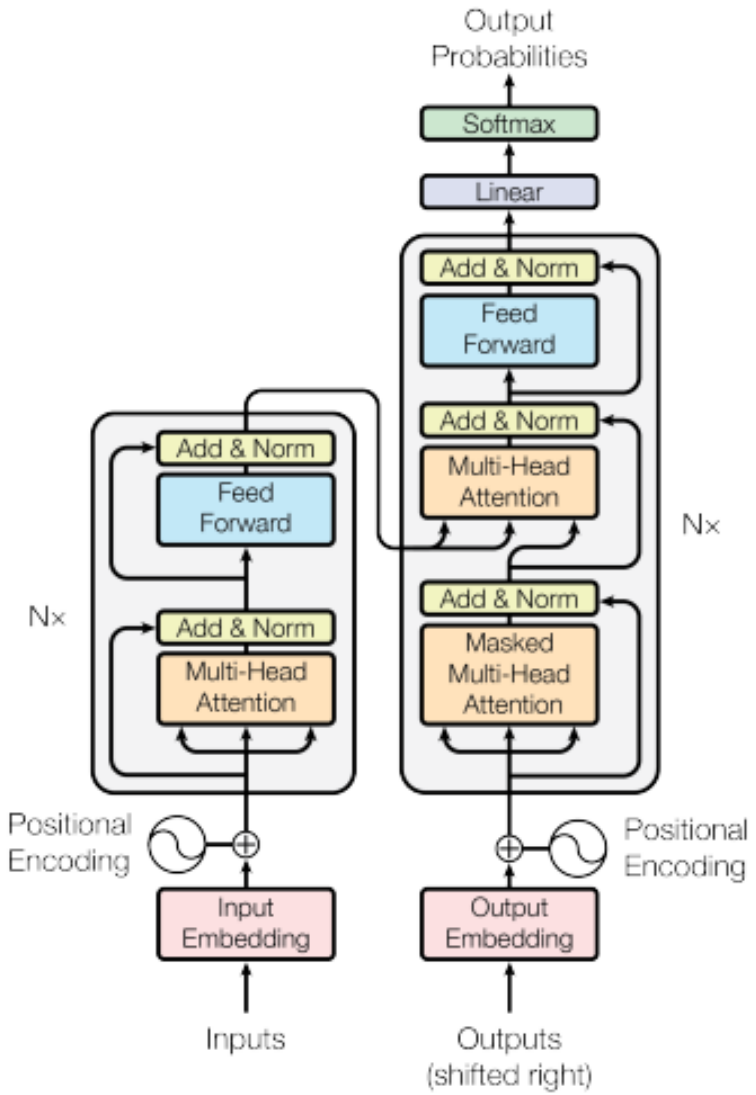
  Ces identifiants numériques peuvent maintenant être utilisés comme entrée pour un modèle Transformer.

Résumé

- **Qu’est-ce qu’un Embedding ?**
- Un embedding est une représentation vectorielle dense d’un token. Plutôt que de représenter un mot ou un sous-mot par un simple identifiant, un embedding le représente sous forme de vecteur de nombres réels, généralement de haute dimension (comme 512 ou 768 dimensions dans le cas de BERT).
  Ces vecteurs sont conçus pour capturer les relations sémantiques entre les tokens. Par exemple, les mots “roi” et “reine” auront des embeddings similaires, car ils partagent des significations connexes.
  Comment sont générés les Embeddings ?
- **Après la tokenization, le modèle utilise ses couches internes pour transformer les identifiants de tokens en embeddings.**
  Ces embeddings sont ensuite utilisés par les couches supérieures du modèle pour effectuer des tâches spécifiques, comme la classification ou la génération de texte.

Résumé

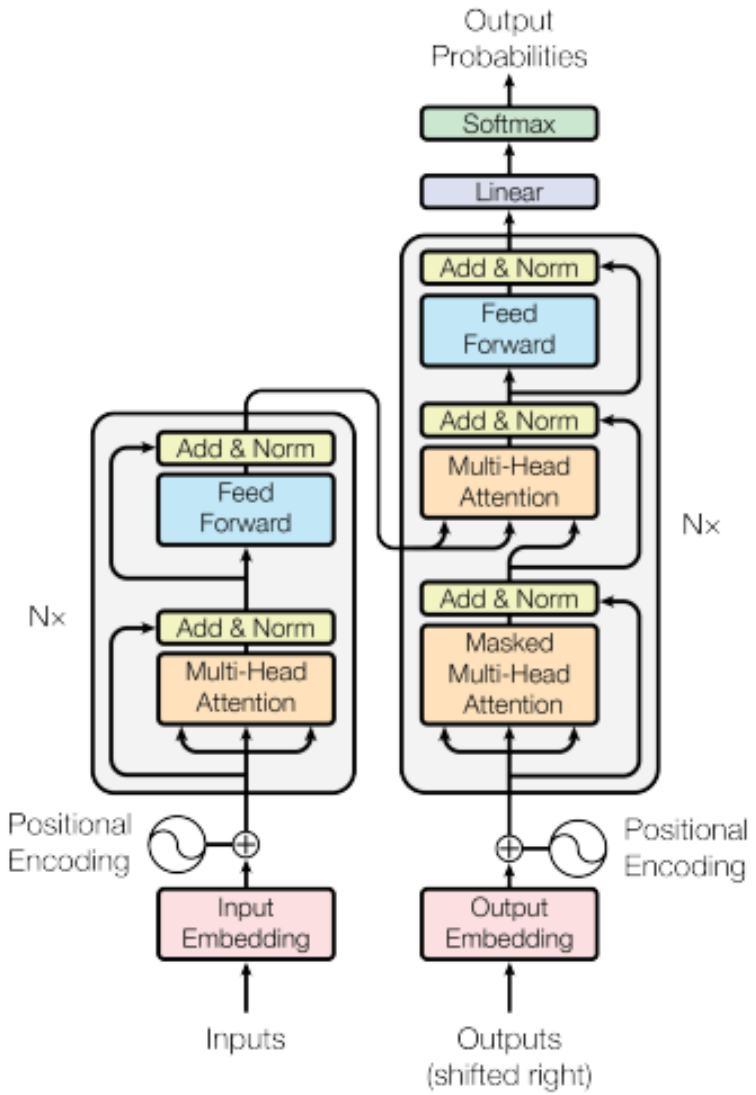
- Le diagramme est tiré de l'article "Attention Is All You Need" et illustre l'architecture complète d'un modèle Transformer, incluant à la fois **l'encodeur (à gauche)** et **le décodeur (à droite)**.



Résumé

1. Input Embedding et Positional Encoding (Gauche)

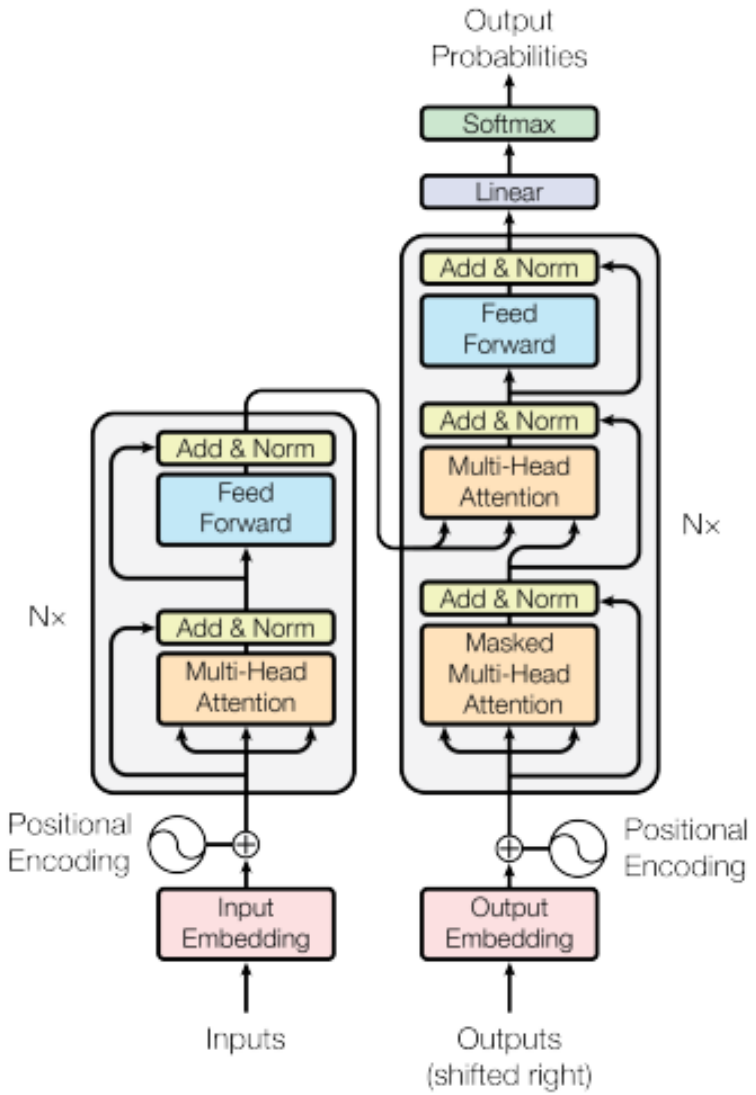
- **Input Embedding** : Les tokens de la séquence d'entrée (par exemple, des mots) sont convertis en vecteurs denses via une couche d'embedding.
- **Positional Encoding** : Ces vecteurs d'embedding sont enrichis avec des informations de position pour indiquer l'ordre des tokens dans la séquence, car les Transformers traitent les tokens en parallèle et n'ont pas de notion de séquence par défaut.



Résumé

2. L'Encodeur (Encoder)

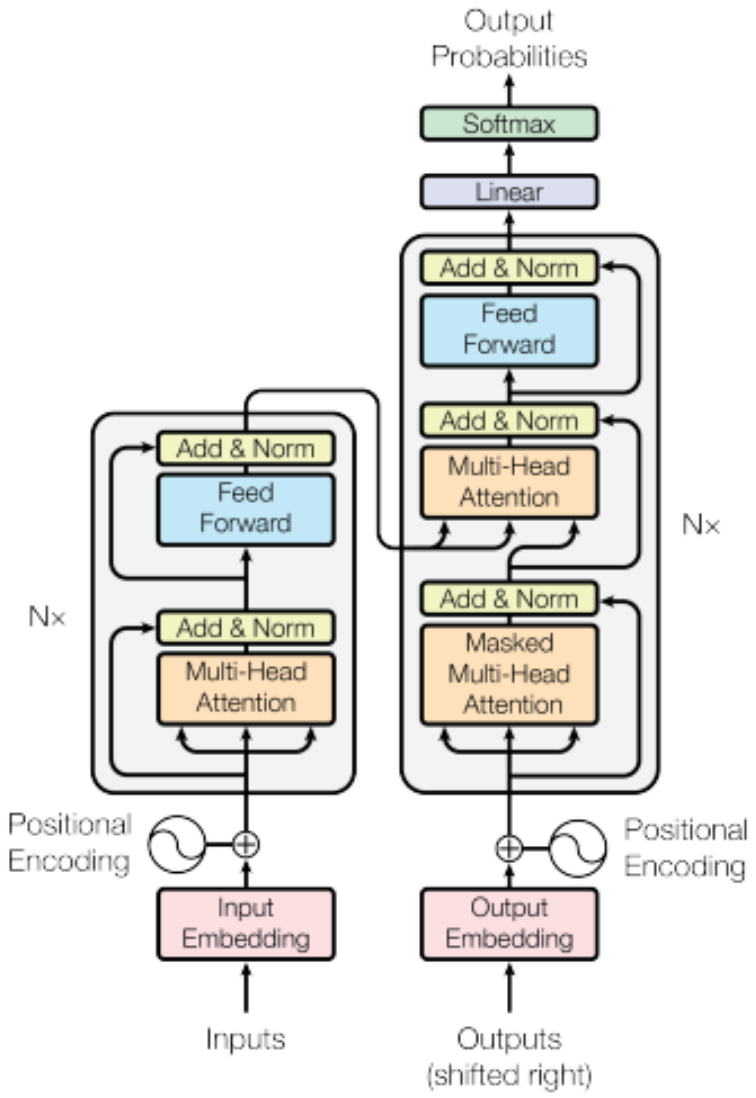
- **Multi-Head Attention** : Chaque token "prête attention" aux autres tokens de la séquence. Ce mécanisme de **Multi-Head Attention** est appliqué avec plusieurs têtes d'attention, chacune capturant des aspects différents du contexte.
- **Add & Norm** : Après la **Multi-Head Attention**, une connexion résiduelle (Add) ajoute la sortie de la couche d'attention à son entrée d'origine, suivie d'une normalisation (Norm) pour stabiliser l'entraînement.
- **Feed-Forward Neural Network** : Chaque vecteur est ensuite passé à travers un réseau de neurones dense pour une transformation supplémentaire.
- **Add & Norm** : Comme précédemment, une connexion résiduelle est appliquée après le réseau feed-forward, suivie d'une normalisation.
- **Nx** : L'ensemble des étapes (Multi-Head Attention → Add & Norm → Feed-Forward → Add & Norm) est répété plusieurs fois (par exemple, 6 ou 12 fois) pour approfondir la capacité du modèle à capturer les relations contextuelles.



Résumé

3. Le Décodeur (Decoder) (Droite)

- **Masked Multi-Head Attention** : Le décodeur commence par une attention masquée. Cette attention masquée permet au modèle de prédire le mot suivant tout en empêchant le modèle de regarder les futurs tokens lors de la génération de la séquence de sortie.
- **Add & Norm** : Comme dans l'encodeur, une connexion résiduelle est appliquée après la **Masked Multi-Head Attention**, suivie d'une normalisation.
- **Multi-Head Attention (Cross-Attention)** : Cette attention permet au décodeur de se concentrer sur la séquence d'entrée (produite par l'encodeur). Cela relie les représentations du décodeur aux représentations de l'encodeur, permettant au décodeur d'incorporer des informations sur l'entrée pour générer la sortie.
- **Add & Norm** : Connexion résiduelle et normalisation après l'étape de cross-attention.
- **Feed-Forward Neural Network** : Comme dans l'encodeur, chaque vecteur est passé à travers un réseau de neurones dense pour une transformation supplémentaire.
- **Add & Norm** : Connexion résiduelle et normalisation après l'étape feed-forward.
- **Nx** : Ce bloc complet (Masked Multi-Head Attention → Add & Norm → Cross-Attention → Add & Norm → Feed-Forward → Add & Norm) est répété plusieurs fois.



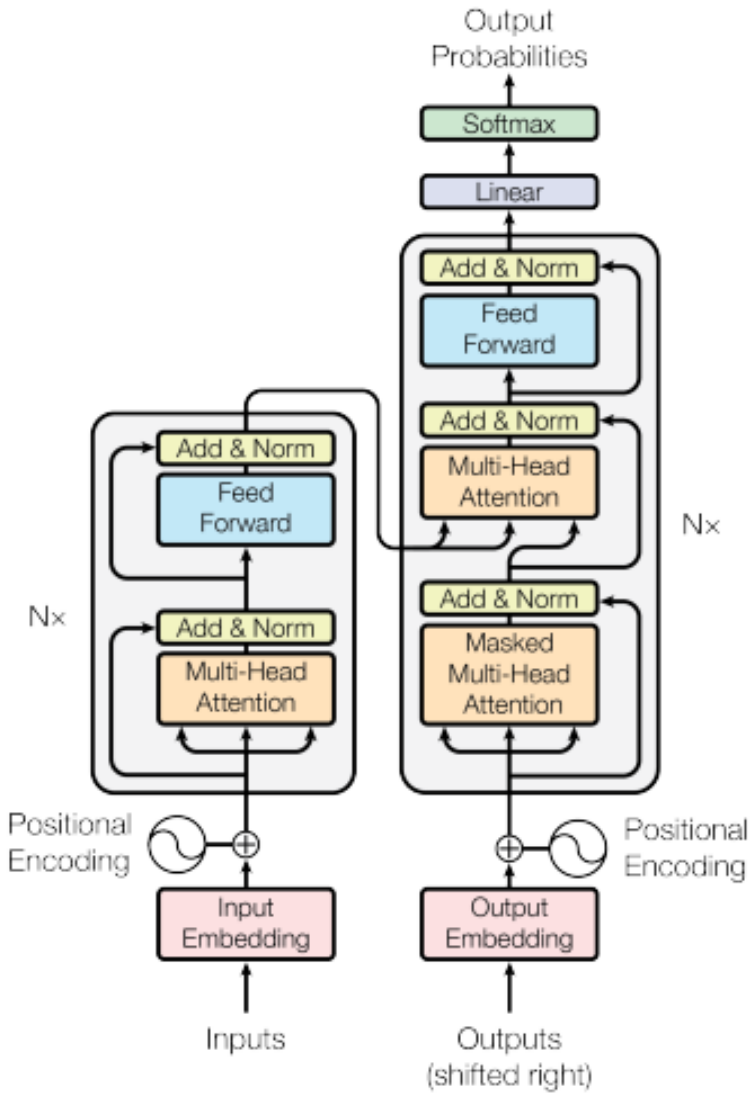
Résumé

4. Output Layer (Softmax)

- **Linear Layer** : Les sorties finales du décodeur sont passées par une couche linéaire.
- **Softmax** : Cette couche génère une distribution de probabilités sur le vocabulaire, permettant au modèle de prédire le prochain token de la séquence de sortie.

5. Connexion entre Encodeur et Décodeur

- Le décodeur reçoit les représentations de l'encodeur via le **Multi-Head Attention** (parfois appelé **Cross-Attention** dans ce contexte), ce qui permet au décodeur de prendre en compte la séquence d'entrée lorsqu'il génère la séquence de sortie.



Exemple

- Exemple avec plus de détails et des calculs précis :
- Prenons une phrase simple : **"Le chat dort"** et expliquons étape par étape la transformation, y compris **Self-Attention** et **Multihead Attention** avec des calculs.

Exemple

Étape 1 : **Embedding des mots**

Chaque mot est converti en un vecteur via un embedding appris, par exemple :

- "Le" → $[1, 0, 1]$
- "chat" → $[0, 1, 1]$
- "dort" → $[1, 1, 0]$

Ensuite, on ajoute des **encodages positionnels** pour inclure la position dans la phrase. Par exemple :

- "Le" → $[1 + 0.1, 0 + 0.2, 1 + 0.3] = [1.1, 0.2, 1.3]$
- "chat" → $[0 + 0.1, 1 + 0.2, 1 + 0.3] = [0.1, 1.2, 1.3]$
- "dort" → $[1 + 0.1, 1 + 0.2, 0 + 0.3] = [1.1, 1.2, 0.3]$

Étape 2 : **Création des matrices Q (Query), K (Key), et V (Value)**

On transforme les embeddings à l'aide de matrices de poids appris (W_Q, W_K, W_V).

Exemple :

Si $W_Q = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{bmatrix}$, calculons Q pour "Le" :

$$Q(\text{"}Le\text{"}) = [1.1, 0.2, 1.3] \cdot W_Q = [1.1 + 0 + 1.3, 0 + 0.2 + 1.3, 1.1 + 0.2 + 0] = [2.4, 1.5, 1.3]$$

De manière similaire :

- $K(\text{"}chat\text{"}) = [0.1, 1.2, 1.3] \cdot W_K$
- $V(\text{"}dort\text{"}) = [1.1, 1.2, 0.3] \cdot W_V$

Exemple

Étape 3 : Calcul des scores d'attention

Les scores mesurent la similarité entre les mots en prenant le produit scalaire entre Q et K. Par exemple, si $Q(\text{"}Le\text{"}) = [2.4, 1.5, 1.3]$ et $K(\text{"}chat\text{"}) = [0.5, 1.0, 0.5]$:

$$\text{Score}(\text{"}Le\text{"}, \text{"}chat\text{"}) = Q(\text{"}Le\text{"}) \cdot K(\text{"}chat\text{"}) = (2.4 \cdot 0.5) + (1.5 \cdot 1.0) + (1.3 \cdot 0.5) = 1.2 + 1.5 + 0.65 = 3.$$

On répète ce calcul pour toutes les paires de mots, créant une matrice de scores. Par exemple :

	Le	chat	dort
Le	3.0	3.35	2.7
chat	3.35	4.0	2.8
dort	2.7	2.8	3.5

Exemple

Étape 4 : **Application de Softmax**

Pour chaque mot (ligne), on applique une fonction Softmax pour convertir les scores en probabilités :

$$\text{Softmax}(x_i) = \frac{\exp(x_i)}{\sum_j \exp(x_j)}$$

Exemple pour la ligne de "Le" :

$$\text{Poids} = \frac{\exp([3.0, 3.35, 2.7])}{\exp(3.0) + \exp(3.35) + \exp(2.7)} = [0.30, 0.44, 0.26]$$

Exemple

Étape 5 : Mise à jour des représentations (Self-Attention)

Les poids obtenus sont utilisés pour combiner les valeurs (V) des mots. Par exemple, pour "Le" :

- Si $V(\text{"}Le\text{"}) = [1.0, 0.5, 0.8]$, $V(\text{"}chat\text{"}) = [0.9, 0.7, 0.6]$, et $V(\text{"}dort\text{"}) = [0.3, 0.4, 0.5]$

  ,

  La nouvelle représentation est :

$$\text{Nouvelle "Le"} = (0.30 \cdot V(\text{"}Le\text{"})) + (0.44 \cdot V(\text{"}chat\text{"})) + (0.26 \cdot V(\text{"}dort\text{"}))$$

$$\text{Nouvelle "Le"} = [0.30, 0.15, 0.24] + [0.396, 0.308, 0.264] + [0.078, 0.104, 0.13] = [0.774, 0.562, 0.634]$$

Exemple

Étape 6 : **Multihead Attention**

1. On applique plusieurs Self-Attention en parallèle (chaque tête utilise des matrices W_Q, W_K, et W_V différentes).
2. Les résultats sont concaténés et transformés à nouveau pour fusionner les informations.

Étape 7 : **Résiduelles et normalisation**

La sortie est ajoutée à l'entrée originale ($X + \text{Attention}$) puis normalisée pour stabiliser l'entraînement.

Exemple

- **Pourquoi l' étape 7 est importante ?**

1. **Connexion résiduelle** :
    1. En ajoutant l’entrée originale aux résultats de l’attention ou du réseau feed-forward, cette étape permet d’atténuer le problème de gradient vanishing, particulièrement dans les réseaux profonds.
    2. Cela aide également à conserver une partie de l’information d’origine, même après plusieurs transformations.
2. **Normalisation de couche (Layer Normalization)** :
    1. Cette normalisation standardise les activations pour chaque mot (chaque vecteur), ce qui accélère la convergence et améliore la stabilité de l’entraînement.

Le future

- **IA générale (AGI) :** Intelligence artificielle capable de comprendre, apprendre, et appliquer des connaissances de manière générale, similaire à l'intelligence humaine.
- **IA superintelligente :** Une IA qui dépasse de loin les capacités intellectuelles humaines.
- **Neurotechnologie :** Interfaces cerveau-ordinateur avancées permettant une interaction directe entre le cerveau humain et les machines.
- **IA éthique :** Développement de systèmes d'IA conçus pour être transparents, équitables et éthiques.
- **IA adaptative :** Systèmes capables de s'adapter et d'évoluer de manière autonome en fonction des changements environnementaux et des nouvelles données.
- **IA collaborative :** IA travaillant de manière symbiotique avec les humains, améliorant la collaboration homme-machine.